

Continuum: Master Implementation Plan

Unified Comprehensive Dossier & Specification

Working name: Continuum **Document date:** 2026-07-24 / 2026-07-28 **Status:** Revision 3 Master Implementation Architecture & Dossier **Primary Language:** Rust | **Concrete Runtime:** asupersync | **Proof Authority:** Lean 4

Contents

Master Implementation Plan	6
Continuum: Revision 3 Master Implementation Plan	6
Reference Documents: Specifications & Architecture Documents	45
Document: docs/00_EXECUTIVE_SPEC.md	45
Document: docs/01_ARCHITECTURE.md	47
Document: docs/02_SEMANTICS.md	50
Document: docs/03_ASSURANCE_AND_TCB.md	54
Document: docs/04_IMPLEMENTATION_ROADMAP.md	59
Document: docs/05_RESEARCH_LANDSCAPE.md	60
Document: docs/06_RESEARCH_AGENDA.md	64
Document: docs/07_BENCHMARKS_AND_EVALUATION.md	71
Document: docs/08_RISK_REGISTER.md	76
Document: docs/09_THREAT_MODEL.md	81
Document: docs/10_INTEROP_AND_MIGRATION.md	86
Document: docs/11_LANGUAGE_AND_DX.md	89
Document: docs/12_GOVERNANCE_AND_ENGINEERING.md	92
Document: docs/13_BIBLIOGRAPHY.md	95
Document: docs/14_GLOSSARY.md	109
Document: docs/15_COMPETITIVE_MATRIX.md	110
Document: docs/16_PROOF_OBLIGATIONS.md	112
Document: docs/17_DOMAIN_PACK_CONTRACT.md	115
Document: docs/18_CLAIMS_MATRIX.md	118
Document: docs/19_TEST_STRATEGY.md	122
Document: docs/20_ADVERSARIAL_DESIGN_REVIEW.md	124
Document: docs/21_NOVEL_CONTRIBUTIONS.md	127
Document: docs/22_TLA_EXAMPLES_COMPATIBILITY_PROGRAM.md	129
Document: docs/23_LEAN4_FORMALIZATION_PROGRAM.md	131
Document: docs/24_REVISION_2_ARCHITECTURE.md	134
Document: docs/25_SEMANTIC_FEATURE_MATRIX.md	136
Document: docs/26_RELEASE_GATES_REV2.md	138
Document: docs/27_SPIKE_FINDINGS_REV2.md	140
Document: docs/28_THEORY_OF_SUCCESS.md	141

Continuum: Master Implementation Plan & Comprehensive Dossier

Document: docs/29_PROOF_ENGINEERING_STRATEGY.md	143
Document: docs/30_AGENT_NATIVE_MICRO_REVOLUTION.md	144
Document: docs/31_FALSIFICATION_AND_KILL_CRITERIA.md	146
Document: docs/32_CORPUS_PORTING_PLAYBOOK.md	147
Document: docs/33_REVISION_3_ARCHITECTURE.md	148
Document: docs/34_DEVELOPER_EXPERIENCE_PRODUCT_CONTRACT.md	151
Document: docs/35_CONTINUUMD_WORKBENCH_DAEMON.md	154
Document: docs/36_AGENT_PROTOCOL_AND_TOOL_CONTRACTS.md	157
Document: docs/37_HUMAN_WORKFLOWS.md	159
Document: docs/38_COUNTEREXAMPLE_EXPERIENCE.md	161
Document: docs/39_VERIFICATION_DEBUGGER.md	163
Document: docs/40_SEMANTIC_DIFF_AND_IMPACT.md	165
Document: docs/41_REPAIR_TRANSACTIONS.md	167
Document: docs/42_INCREMENTAL_VERIFICATION.md	169
Document: docs/43_CONTINUUM_FORGE.md	171
Document: docs/44_MULTI_AGENT_EVIDENCE_GRAPH.md	174
Document: docs/45_CONTINUUMBENCH.md	175
Document: docs/46_IDE_CLI_DAP_MCP_AND_SARIF.md	179
Document: docs/47_ONBOARDING_AND_PROGRESSIVE_DISCLOSURE.md	181
Document: docs/48_EXPLANATION_SCIENCE.md	183
Document: docs/49_SECURITY_FOR_AUTONOMOUS_AGENTS.md	185
Document: docs/50_AGENT_EVALUATION_AND_REWARD_HACKING.md	186
Document: docs/51_PRODUCT_WALKTHROUGHS.md	188
Document: docs/52_RELEASE_GATES_REV3.md	190
Document: docs/53_SPIKE_FINDINGS_REV3.md	192
Document: docs/54_THEORY_OF_AGENTIC_ACCELERATION.md	195
Document: docs/55_AGENT_API_REFERENCE_SKETCH.md	197
Reference Documents: Architecture Decision Records (ADRs)	203
Document: adr/0001-asupersync-execution-substrate.md	203
Document: adr/0002-causal-intermediate-representation.md	203
Document: adr/0003-no-ambient-nondeterminism.md	204
Document: adr/0004-standalone-models-and-zoomable-refinement.md	204
Document: adr/0005-partial-order-primary-semantics.md	205
Document: adr/0006-typed-assurance-lattice.md	205
Document: adr/0007-verification-engine-portfolio.md	206
Document: adr/0008-proof-carrying-results.md	206
Document: adr/0009-production-partial-order-conformance.md	207
Document: adr/0010-cancellation-calculus.md	207
Document: adr/0011-semantic-domain-packs.md	208
Document: adr/0012-dual-authoring-surfaces.md	208
Document: adr/0013-exact-state-identity.md	209
Document: adr/0014-explicit-fairness-and-ranking-liveness.md	209
Document: adr/0015-strong-refinement-for-hyperproperties.md	210
Document: adr/0016-typed-timed-and-probabilistic-extensions.md	210
Document: adr/0017-solver-and-tcb-policy.md	211
Document: adr/0018-semantic-versioning-and-replay.md	211
Document: adr/0019-adapter-isolation.md	212
Document: adr/0020-experimental-mathematics-governance.md	212
Document: adr/0021-tla-examples-corpus-contract.md	213
Document: adr/0022-lean4-metatheory-and-certificate-authority.md	214
Document: adr/0023-semantic-triptych-and-independent-paths.md	215
Document: adr/0024-observer-indexed-independence.md	215
Document: adr/0025-stratified-model-language-fragments.md	216
Document: adr/0026-proof-producing-reductions.md	217

Document: adr/0027-nominal-orbit-finite-symmetry.md	218
Document: adr/0028-assumption-synthesis-as-games.md	218
Document: adr/0029-semantic-equivalence-not-tla-source-cloning.md	219
Document: adr/0030-semiring-valued-analysis.md	219
Document: adr/0031-corpus-derived-language-governance.md	220
Document: adr/0032-weak-memory-execution-graph-lane.md	221
Document: adr/0033-checked-projection-and-generated-rust-interfaces.md	221
Document: adr/0034-owned-temporal-semantics-with-leanltl-bridge.md	221
Document: adr/0035-proof-receipts-and-axiom-manifests.md	222
Document: adr/0036-authoritative-workbench-daemon.md	222
Document: adr/0037-explicit-content-addressed-handles.md	222
Document: adr/0038-machine-contracts-not-terminal-prose.md	223
Document: adr/0039-protected-intent-contract.md	223
Document: adr/0040-typed-context-packs.md	224
Document: adr/0041-verification-debugger-and-dap.md	224
Document: adr/0042-mcp-adapter-not-authority.md	225
Document: adr/0043-repair-transactions.md	225
Document: adr/0044-incremental-semantic-database.md	225
Document: adr/0045-progressive-disclosure-with-lossless-expansion.md	226
Document: adr/0046-semantic-diff-gates.md	226
Document: adr/0047-forge-sandbox-and-nonvacuity.md	227
Document: adr/0048-proof-service-isolation.md	227
Document: adr/0049-explicit-budgets-and-resumable-search.md	227
Document: adr/0050-proof-oriented-model-program-lenses.md	228
Document: adr/0051-assurance-envelope.md	228
Document: adr/0052-multi-agent-evidence-graph.md	229
Reference Documents: Requests for Comments (RFCs)	230
Document: rfcs/0001-causal-intermediate-representation.md	230
Document: rfcs/0002-controlled-effects-and-domain-packs.md	233
Document: rfcs/0003-continuum-model-language.md	236
Document: rfcs/0004-exploration-dpor-and-unfoldings.md	239
Document: rfcs/0005-certificates-and-independent-kernel.md	242
Document: rfcs/0006-production-trace-conformance.md	244
Document: rfcs/0007-storage-and-crash-semantics.md	246
Document: rfcs/0008-liveness-fairness-and-progress.md	248
Document: rfcs/0009-views-and-refinement.md	250
Document: rfcs/0010-assurance-results-and-claims.md	252
Document: rfcs/0011-tla-examples-tribunal.md	254
Document: rfcs/0012-lean-metatheory-and-reflective-certificates.md	256
Document: rfcs/0013-semantic-triptych.md	259
Document: rfcs/0014-observer-indexed-dpor.md	260
Document: rfcs/0015-temporal-fairness-and-hyperproperties.md	262
Document: rfcs/0016-parameterized-and-nominal-verification.md	263
Document: rfcs/0017-assumption-and-protocol-synthesis-games.md	264
Document: rfcs/0018-agent-native-proof-and-repair-loop.md	266
Document: rfcs/0019-corpus-port-manifest.md	267
Document: rfcs/0020-proof-producing-transformations.md	268
Document: rfcs/0021-corpus-oracle-protocol.md	270
Document: rfcs/0022-weak-memory-local-refinement.md	270
Document: rfcs/0023-checked-choreographic-projection.md	271
Document: rfcs/0024-proof-receipt-format.md	271
Document: rfcs/0025-property-directed-abstraction-loop.md	271
Document: rfcs/0026-continuumd-native-protocol.md	272
Document: rfcs/0027-agent-tool-protocol.md	275

Document: rfcs/0028-context-pack-format.md	277
Document: rfcs/0029-causal-verification-debugger.md	279
Document: rfcs/0030-incremental-semantic-query-engine.md	280
Document: rfcs/0031-semantic-and-intent-diff.md	281
Document: rfcs/0032-repair-transaction-protocol.md	283
Document: rfcs/0033-continuum-forge.md	285
Document: rfcs/0034-continuum-bench.md	286
Document: rfcs/0035-isolated-lean-proof-service.md	286
Document: rfcs/0036-proof-oriented-correspondence.md	287
Document: rfcs/0037-intent-contract.md	287
Document: rfcs/0038-multi-agent-evidence-graph.md	290
Document: rfcs/0039-explanation-engine.md	291
Document: rfcs/0040-protocol-adapters.md	291
Reference Documents: Research Programs & Notes	292
Document: research/01-true-concurrency.md	292
Document: research/02-symbolic-inductive-and-parameterized.md	294
Document: research/03-refinement-composition-and-sheaves.md	297
Document: research/04-liveness-progress-and-fairness.md	299
Document: research/05-runtime-conformance-and-observability.md	301
Document: research/06-timed-probabilistic-and-hyperproperties.md	304
Document: research/07-alien-mathematics.md	306
Document: research/08-agentic-verification.md	309
Document: research/09-cancellation-and-obligation-calculus.md	311
Document: research/10-combinatorial-geometry-of-concurrency.md	314
Document: research/11-tla-examples-corpus-gap-analysis.md	317
Document: research/12-lean-reflection-and-proof-certificates.md	319
Document: research/13-observer-indexed-independence.md	321
Document: research/14-nominal-sets-and-orbit-finite-verification.md	322
Document: research/15-games-and-assumption-synthesis.md	323
Document: research/16-causal-abstract-interpretation.md	325
Document: research/17-proof-producing-reductions-and-translation-validation.md	326
Document: research/18-coalgebra-modal-logic-and-behavioral-interfaces.md	327
Document: research/19-semiring-provenance-and-weighted-exploration.md	328
Document: research/20-rust-concurrency-verification-frontier-2026.md	330
Document: research/21-choreographies-and-session-types.md	331
Document: research/22-weak-memory-execution-graphs.md	332
Document: research/23-compositional-proof-algebra.md	333
Document: research/24-automatic-abstraction-and-property-directed-modeling.md	335
Document: research/25-agent-computer-interfaces-for-formal-systems.md	336
Document: research/26-causal-and-contrastive-counterexample-explanations.md	338
Document: research/27-incremental-verification-and-proof-reuse.md	339
Document: research/28-agentic-proof-repair-and-context-compilation.md	341
Document: research/29-protocol-invariant-ranking-and-abstraction-cosynthesis.md	343
Document: research/30-quality-diversity-for-verified-algorithm-discovery.md	344
Document: research/31-proof-oriented-bidirectional-transformations.md	345
Document: research/32-semantic-context-compilation-and-information-theory.md	347
Document: research/33-agent-benchmarks-and-reward-hacking.md	349
Document: research/34-human-factors-of-formal-systems-workbenches.md	350
Document: research/35-security-of-agentic-verification-workbenches.md	352
Document: research/36-active-diagnosis-and-experiment-design.md	353
Reference Documents: Executable Spike Reports	355
Document: spikes/R3_SPIKE_REPORT.md	355
Document: spikes/SPIKE_REPORT.md	357
Reference Documents: Implementation Sequence & G0 Matrix	360

Continuum: Master Implementation Plan & Comprehensive Dossier

Document: notes/G0_SPIKE_MATRIX.md	360
Document: notes/START_HERE_IMPLEMENTATION.md	362
Reference Documents: ContinuumBench Specifications	370
Document: benchmarks/README.md	370
Document: benchmarks/continuum-bench/README.md	372

Master Implementation Plan

Continuum: Revision 3 Master Implementation Plan

Working name: Continuum

Document date: 2026-07-24

Status: implementation architecture and research program

Primary implementation language: Rust

Concrete runtime: asupersync

Proof authority: Lean 4

Compatibility tribunal: TLA+ Examples, TLC, Apalache, Quint, P, Loom/Shuttle, and selected proof systems

Product thesis: verification-native development and invention for concurrent/distributed Rust systems

0. Declaration of intent

Continuum exists to make formally grounded concurrent and distributed systems engineering the normal way humans and advanced coding agents work—not a specialist ritual performed after design decisions have hardened.

The project is not a Rust rewrite of TLC. It is not a deterministic simulator with formal-methods branding. It is not an LLM wrapper around TLA+, Lean, or a pile of subprocesses. It is not a magical claim that one language or algorithm defeats undecidability.

Continuum is a coherent semantic environment in which:

- an abstract model can be written before production code exists;
- real asupersync Rust code runs under production and controlled execution semantics;
- model and implementation are connected by explicit refinement;
- safety, liveness, fairness, cancellation, durability, faults, time, and weak memory are represented honestly;
- counterexamples are reduced to causal explanations;
- successful results carry independently checkable evidence;
- production observations can be checked without inventing a false total order;
- humans receive progressive, comprehensible tooling;
- agents receive a compact, typed Agent–Computer Interface rather than terminal sludge;
- repairs cannot game properties, assumptions, bounds, observers, or assurance policy;
- protocol and algorithm synthesis happens inside a fixed intent and proof envelope.

The north star is:

A capable coding agent should be able to invent or repair a concurrent Rust algorithm, receive exact semantic feedback, iterate autonomously, and produce a patch whose behavior, assumptions, coverage, and proof obligations are legible to a human and checkable independently.

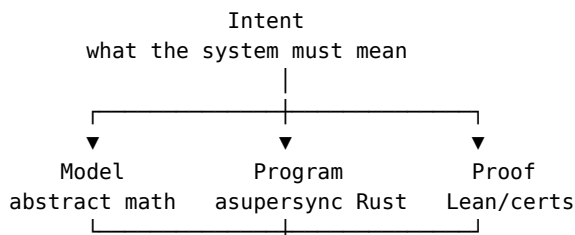
That requires more than a strong verifier. It requires an impeccable workbench.

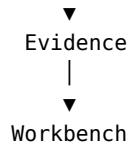
0.1 Revision 3: the product becomes the proof-guided loop

Revision 2 established the semantic triptych:

Model ↔ Program ↔ Proof

Revision 3 protects the loop with **Intent** and makes **Evidence** the sole currency of progress:





Intent

Intent includes:

- property ASTs and observer scopes;
- environment assumptions;
- weak/strong fairness declarations;
- model bounds and cutoff claims;
- fault and recovery envelope;
- storage/network/time semantics;
- trusted and opaque boundaries;
- assurance requirements;
- optimization objectives and non-vacuity constraints;
- security and privacy policy.

Intent is content-addressed and immutable by default. Any change receives a semantic diff and cannot be smuggled inside an ordinary repair.

Evidence

Evidence includes:

- exact counterexamples and replay records;
- causal cores and counterfactual branches;
- finite closure certificates;
- SAT/SMT/CHC/PDR proof artifacts;
- Lean theorem receipts and axiom manifests;
- refinement witnesses;
- coverage and reduction certificates;
- mutation sensitivity;
- production trace verdicts;
- explicit Inconclusive explanations.

A claim without evidence is a hypothesis, regardless of whether it came from a human, an agent, or a sophisticated engine.

Workbench

The workbench is not an interface veneer. It determines whether the semantic system can be used correctly. It owns:

- immutable workspace snapshots;
- incremental query execution;
- typed verification tasks;
- Context Packs;
- the causal debugger;
- semantic and intent diff;
- repair transactions;
- proof and solver isolation;
- multi-agent evidence coordination;
- Continuum Forge;
- evaluation and anti-reward-hacking gates.

0.2 The product theorem

Continuum should satisfy the following engineering theorem:

For every accepted operation, the user can determine what snapshot was analyzed, what intent was protected, what semantic engines ran, what was explored or proved, what was omitted, what assumptions were used, what changed, and how to reproduce or independently check the result.

The universal operation shape is:

```
(snapshot, intent, operation, budget, policy)
  → verdict
  + evidence references
  + new snapshot or continuation
  + explicit omissions/unknowns
  + allowed next operations, warnings, cost, and epoch identities
```

This shape applies equally to:

- check;
- explore;
- prove;
- explain;
- debug;
- repair;
- synthesize;
- observe;
- review.

No semantic state is hidden in a terminal process, editor connection, chat session, or MCP transport.

0.3 Claim status of this plan

This plan uses the dossier claim lattice (README.md). Unless marked otherwise, sections here are DESIGN (accepted architecture, not evidence). The following are explicitly weaker:

- HYPOTHESIS: §6 general context compilation, §8.3 neighborhood adequacy, §9 sub-file-granularity incremental trust, §12 causal explanation science, §14 Forge co-synthesis and quality-diversity, §16 bidirectional lenses, production partial-order conformance (Phase F). Each is owned by a research lane with kill criteria (see §24.5).
 - Evidence boundary: no Lean source in this dossier has been parsed, elaborated, or kernel-checked; no theorem may be described as machine-checked (VALIDATION_REPORT.md, docs/18 C035). The Revision 3 spikes are finite Python reference experiments that validate artifact shapes and interaction contracts, not engines, scale, concurrency, persistence, or soundness (docs/53, “What remains unproven”).
 - G0 status: 8 of 15 G0 items have spike evidence. DX-10, DX-13, and DX-14 are open and freeze-blocking (Phase A). DX-06, DX-09, DX-11, and DX-15 are re-homed to the gates owning their machinery (G4, G8, G6, G9 respectively — see §22 G0); the re-homing is their recorded decision. Statuses live in notes/G0_SPIKE_MATRIX.md, from which these counts derive.
-

1. Twenty-five design bets

B1 — Intent integrity is a verification property

A tool that proves a weakened property has failed. Continuum versions intent and treats unauthorized changes as violations.

B2 — Agent interface quality is part of verifier capability

An agent forced to scrape prose, track cursor positions, or reconstruct task state from logs will make more invalid moves and consume more context. Machine users receive concise typed operations and stable handles.

B3 — continuumd is the authority

One daemon owns the semantic query graph, CAS, task lifecycle, evidence ledger, and snapshots. CLI, Cargo, IDE, DAP, MCP, SARIF, TUI, and web clients are projections.

B4 — Explicit handles dominate implicit sessions

Workspace, proof state, debugger branch, task, continuation, crashpack, and synthesis archive are explicit opaque handles. This enables resumability, sharing, isolation, caching, and deterministic handoff.

B5 — Context is compiled, not dumped

A Context Pack is a property-directed program slice over causal, proof, source, state, and assumption graphs. It contains an omission manifest and expandable references.

B6 — Counterexamples are causal objects

The primary failure artifact is a minimal or near-minimal causal explanation with abstract/concrete state deltas, obligation flow, missing order, counterfactual repairs, and replay—not a chronological log.

B7 — Debugging follows the partial order

The debugger steps among enabled semantic events, not merely source lines. It supports reverse causal stepping, alternate schedule branching, and observer-level state views.

B8 — Repairs are transactions

A patch proposal, hypothesis, semantic diff, replay, neighborhood campaign, proof impact, and promotion decision form one atomic evidence-bearing workflow.

B9 — Clean verification audits incremental verification

Incremental reuse is essential for interactive work but dangerous if invalidation is unsound. Every reuse edge has a class and is statistically/deterministically audited against clean recomputation.

B10 — Model/program synchronization is proof-oriented

Bidirectional assistance can propose edits, but ambiguity produces a conflict and obligations rather than a guessed rewrite. Round-trip laws are not enough; semantic preservation matters.

B11 — Assurance is an envelope, not a badge

Every result describes dimensions such as bounds, faults, fairness, values, schedules, weak-memory model, observer, proof status, and unknowns. “Verified” alone is prohibited in machine output.

Every envelope dimension names its producing engine or carries a typed `Unsupported` value; a dimension is never silently omitted. Until the weak-memory lane ships (ADR-0032), every envelope’s memory dimension reads `Unsupported(sequential-consistency-only)`. Until the timed and probabilistic extensions ship (ADR-0016), Continuum does not advertise timed or probabilistic proof support in any machine output, and timing fields in Intent Contracts are declarative assumptions, not checked semantics.

B12 — Agents are untrusted search procedures

Agents may propose models, invariants, proofs, rankings, abstractions, patches, and algorithms. Only engines and independent checkers promote evidence states.

B13 — Multi-agent coordination uses an evidence graph

Conversation is ephemeral coordination. Claims, dependencies, conflicts, counterexamples, patches, proof goals, and receipts are durable typed nodes.

B14 — Search must preserve behavioral diversity

Forge retains a quality-diversity archive of semantically distinct correct candidates rather than collapsing immediately to one syntactic optimum.

B15 — Synthesis is constrained by intent and non-vacuity

A synthesized protocol must satisfy safety, progress, implementability, observer obligations, and explicit optimization objectives. Disabling all behavior is not a solution.

B16 — Verification and synthesis share counterexamples

CEGIS, invariant inference, ranking synthesis, repair, and protocol synthesis consume the same normalized counterexample and proof-obligation artifacts.

B17 — Proof context is a product

Lean goals are accompanied by relevant declarations, proof slices, failed attempts, countermodels, candidate lemmas, source correspondence, and version identity. Proof workers need not reconstruct context.

B18 — Interactive results require budget semantics

Every long task accepts wall, CPU, memory, state, solver, token, and proof budgets and returns a resumable continuation with monotonic evidence.

B19 — Cancellation correctness applies to verification itself

Search, proof, synthesis, indexing, and debugging tasks run under asupersync regions and publish only committed artifacts. Cancellation cannot leave false finality or orphan work.

B20 — Security boundaries are semantic boundaries

Agent permissions, opaque effects, foreign solvers, proof services, production traces, and domain packs are capability-scoped and visible in receipts.

B21 — Humans need progressive disclosure, not simplification by omission

The default view says what failed and why. Every simplification has a direct path to exact state, causal events, formulae, proof obligations, and raw artifacts.

B22 — Benchmarks must test governance, not just bug fixing

ContinuumBench includes attempts to weaken properties, hide events, shrink bounds, overfit traces, exploit stale snapshots, inject prompts through source, and forge evidence.

B23 — The TLA+ corpus is both language coverage and interaction coverage

Each port includes authoring, checking, explanation, deliberate failure, proof/refinement where applicable, and an agent task—not merely semantic execution.

B24 — Formal-methods UX requires empirical science

Human diagnosis accuracy, time, confidence calibration, and error patterns are measured. Agent success, token cost, invalid actions, and expensive failures are measured.

B25 — The final product is an invention accelerator

Once the trust loop is closed, Continuum should search beyond known implementations: new synchronization schemes, storage protocols, replication algorithms, resource schedulers, and cancellation protocols—while emitting proofs and executable Rust candidates.

2. Constitutional invariants

INV-001 — Protected intent

Ordinary tasks may not mutate intent. Intent changes require a new Intent Contract, semantic diff, policy decision, and invalidation of dependent evidence.

INV-002 — No hidden semantic state

Every stateful workflow uses explicit handles. A dropped connection or restarted client does not change meaning.

INV-003 — No prose-only machine interfaces

Human text may accompany a result, but agents and integrations consume versioned schemas and enums.

INV-004 — No self-certification

Search code does not check its own strongest claims. Certificates cross an independent checker; foundational theorems cross Lean.

INV-005 — No ambient nondeterminism

Controlled code accesses scheduling, time, entropy, I/O, faults, and cancellation through explicit capabilities.

INV-006 — Replay stability

A failure advertised as replayable must reproduce under its pinned semantic epoch or be downgraded to an engine defect.

INV-007 — Omission transparency

Any bounded Context Pack, explanation, slice, or visualization names what was omitted and how to retrieve it.

INV-008 — Typed inconclusiveness

Timeout, unsupported semantics, insufficient telemetry, abstraction ambiguity, and incomplete proof search are distinct outcomes.

INV-009 — Monotonic task evidence

Resuming a task may add evidence or refine an unknown; it may not silently replace prior artifacts under the same identity.

INV-010 — Incremental parity

An incremental result claiming exactness must match clean evaluation for the same snapshot and semantic epoch.

INV-011 — Intent-preserving repair

A repair transaction cannot be promoted if it changes protected intent unless explicitly reclassified as an intent revision.

INV-012 — Non-vacuous synthesis

Forge objectives include required progress/availability behaviors and mutation challenges. Safety by disabling the system is rejected.

INV-013 — Property-scoped reduction

Independence, symmetry, abstraction, slicing, and quotienting are justified relative to named observers/properties and fairness obligations.

INV-014 — Version-explicit proof

Lean version, library closure, theorem hashes, axioms, certificate schema, and checker identity are recorded.

INV-015 — Agent least authority

Agents cannot alter evidence status, sign receipts, access ungranted production traces, or execute unrestricted host effects.

INV-016 — Source is untrusted data

Comments, logs, docs, model strings, and production payloads cannot issue instructions to the workbench or proof service.

INV-017 — Semantic atomicity of publication

Artifacts become visible only after their content, provenance, and references are durably committed.

INV-018 — Corpus honesty

Compatibility claims include unsupported features, configured bounds, expected verdict, state/trace parity, and proof status.

3. Product experience

3.1 Five-minute human path

A developer adds:

```
[dev-dependencies]
continuum = "..."
```

```
[package.metadata.continuum]
entry = "crate::protocol"
profile = "dev"
```

```
intent = "in_7c2f91"          # registry identity, never a file path
intent_bundle = "inb_09aa41"  # §4.2.1: CI fails closed without it
```

Then:

```
cargo continuum init
cargo continuum check
```

This Cargo metadata is the single configuration surface for Rust projects; model-only projects use the same schema in a standalone `continuum.toml`. Configuration references intent by registry identity (`in_*`), never by a workspace file path (§4.2).

`cargo continuum init` also proposes draft Intent Contracts from property templates, the domain-pack library, and observed effect footprints. Drafts enter the registry at status `Proposed`; they gain protection (INV-001) only on explicit acceptance. Continuum never silently promotes an inferred intent to protected status, and never claims a generated model is the intended abstraction.

First-run contract: while every governing intent is `Proposed`, `cargo continuum check` runs the drafts and reports each verdict as `Hypothesis(unaccepted-intent)` — structurally distinct from `FAILURE` and `PASS` in every surface — alongside intent-free findings that need no contract at all: effect footprint, uncontrolled-nondeterminism sites (INV-005), and unmapped opaque effects. This output is the specified zero-configuration value of the tool, not a degraded mode.

The first result is concise:

```
FAILURE AckImpliesDurable          intent in_7c2... (unchanged)
assurance  bounded: ≤3 nodes · ≤2 faults · exhaustive schedules · SC memory
```

Ack became observable before the corresponding write became durable.

Causal core: 4 events · 2 tasks · 1 cancellation

Abstract mismatch: acked +1, durable unchanged

Unknown: production storage profile assumed (contractual)

```
replay  crash_7m3...
debug   continuum debug crash_7m3...
explain continuum explain crash_7m3... --level causal
repair  continuum repair begin crash_7m3...
```

No Java installation, model-config archaeology, or megabytes of state dumps.

3.2 Agent path

An agent does not invoke arbitrary shell text. It calls:

```
{
  "operation": "verification.start",
  "idempotency_key": "b1946ac9...",
  "snapshot": "ws_4f...",
  "intent": "in_91...",
  "budget": {"states": 100000, "wall_ms": 30000},
  "arguments": {
    "target": {"kind": "property", "id": "AckImpliesDurable"}
  },
  "output_policy": {"context_pack": true, "max_tokens": 6000}
}
```

The result contains typed verdict, task/continuation handles, evidence roots, and a Context Pack. The agent may request expansion by graph node, source span, proof goal, alternate branch, or assumption.

3.3 Design-before-code path

```
continuum model new lease-service
continuum model check lease-service --property no-overlapping-valid-leases
continuum forge lease-service --holes quorum,renewal_guard
```

The model is independent of Rust. Later:

```
cargo continuum bind lease-service crate::lease
cargo continuum refine --from program:lease --to model:lease-service
```

3.4 Repair path

```
continuum repair begin crash_7m3...
continuum repair apply --patch patch.diff --hypothesis "publish only after sync"
continuum repair evaluate rt_2c...
continuum repair promote rt_2c...
```

Promotion returns a receipt only after all policy gates close.

3.5 Invention path

```
continuum forge synthesize models/broadcast.ctm \
  --holes delivery_rule,ack_rule,state_summary \
  --objective latency,persistent-writes \
  --diversity behavior \
  --assurance bounded-proof
```

Forge returns a Pareto/quality-diversity archive, not a single opaque answer.

4. Authoritative workbench architecture

4.1 continuumd

continuumd is a long-lived local or remote service composed of:

Protocol gateway
Snapshot/CAS service
Intent registry
Incremental semantic database
Task scheduler and budget manager
Engine orchestrator
Evidence graph and receipt ledger
Context compiler
Debugger service
Repair transaction manager
Forge coordinator
Proof/solver isolation gateways
Audit and policy engine

It runs its own work as cancel-correct asupersync regions. A task has explicit lifecycle:

Created → Running → Suspended | Completed | Failed | Cancelled
 |
 └ continuation + committed partial evidence

The scheduler defines priority classes — interactive > CI > background/swarm — with preemption of suspendable tasks (safe under B18 continuations). Every capability grant carries concurrency and resource quotas. The daemon operates under a declared memory budget with class-aware eviction across the incremental cache, overlay snapshots, and materialized debugger branches. G5 latency compliance is measured with a saturating background swarm present, not on an idle daemon.

4.2 Workspace snapshots

A workspace snapshot contains content identities for:

- source files;
- CML modules;
- Rust semantic extraction;
- domain-pack manifests;
- dependency lockfiles;
- toolchain and semantic epochs;
- the content identity of the governing Intent Contract (a reference, not the contract itself);
- generated correspondence;

- proof environment;
- configuration.

Snapshots form a Merkle DAG. A tool call never means “whatever is currently on disk”; it means a named snapshot. Clients may create a snapshot from a working tree, overlay an in-memory editor buffer, or fork an existing snapshot.

Intent Contracts are stored and versioned only in the intent registry, outside every writable or forkable snapshot. `workspace.fork` preserves the intent binding by identity; rebinding a snapshot lineage to a different intent is a privileged operation that produces a semantic intent diff and invalidates dependent evidence (INV-001).

4.2.1 Intent distribution and convergence

Registries converge through signed, content-addressed **intent bundles** (`inb_*`): an export of one or more contracts with their acceptance records and policy tables. Bundles may be vendored in the repository or fetched by identity; import is idempotent and never changes protection status — an imported Proposed contract stays Proposed, and an imported acceptance is honored only if its signature chain satisfies the local policy. Divergent branches are reconciled by semantic three-way merge: each head’s §5.3 diff against the common ancestor; two revisions merge automatically only when both diffs are classified independent within supported fragments — otherwise the merge is a Conflict node requiring the revise-intent capability. CI fails closed when the bundle referenced by the workspace configuration is absent or its acceptance chain does not verify, and §17.6’s intent diff ships with a PR-reviewable projection (SARIF note plus rendered semantic diff).

4.3 Native protocol

The native protocol uses strongly typed request/response definitions over local IPC or authenticated HTTP/QUIC. Requirements:

- idempotency keys;
- explicit snapshot and intent handles;
- capability negotiation;
- pagination and continuation;
- deterministic ordering;
- structured error taxonomy;
- asynchronous task handles;
- cancellation and budget updates;
- evidence subscriptions;
- OpenTelemetry context;
- protocol and semantic version separation.

The daemon serves protocol majors N and $N-1$. Evidence and receipt schemas are readable by every future verifier for their declared schema epoch: artifact readability is decoupled from protocol majors and covered by the kernel crates’ reproducible-build covenant (§20).

MCP, LSP, DAP, and SARIF translate to this protocol. None define core semantics.

4.4 Content-addressed artifacts

Artifact classes include:

```
ws_* workspace snapshot
in_* intent contract
inb_* signed intent bundle
model_* elaborated model
cir_* causal execution graph
crash_* crashpack
ctx_* Context Pack
proof_* proof artifact
ps_* proof state
task_* task
ev_* evidence node or edge
dbg_* debugger branch
receipt_* signed/checked receipt
rt_* repair transaction
```

```
forge_* synthesis archive
cont_* resumable task continuation
cap_* capability
diff_* semantic/intent diff
defect_* engine-defect report
```

Handles carry a kind prefix but are otherwise structureless. Content- addressed identities are derivable by anyone holding the content — the §4.5 existence-oracle rule exists because of this — so handles are identifiers, not secrets, and confer no authority. Capability tokens (cap_*) are minted randomly and do confer authority. Authorization is always checked independently of handle possession.

4.5 Operational contract of continuumd

The daemon is part of the trust spine; its operational behavior is specified, verified, and gated (G1), not left to implementation:

- **Crash safety.** Publication commits content before index; a crash leaves unreachable content eligible for GC, never a stale index entry. On restart, Running tasks resume from their last committed continuation or transition to Failed with a typed reason — never to a silently reconstructed state. An index verifier (fsck) ships with the daemon.
- **Storage lifecycle.** Artifacts are garbage-collected by reachability from named roots, receipts, and retention policy. The daemon reports storage attribution by artifact class. Disk exhaustion during publication aborts atomically (INV-017); it never truncates. Campaign-class evidence (neighborhood, mutation) is summarizable: after promotion, raw per-class results may be rolled up into a coverage certificate attested by a kernel-covenant checker; the receipt references the summary, raw classes become GC-eligible under retention policy, and later access yields Redacted(summarized, commitment) with the standard §18.4 downgrade. Summarization is the default after promotion and opt-out per retention policy.
- **Purge without breaking receipts.** Sensitive artifact classes are encrypted at rest per artifact; purge shreds the key and replaces content with a typed Redacted(reason, commitment) stub. Receipts referencing purged content remain structurally verifiable and report the redaction; claims requiring the hidden data downgrade per §18.4.
- **Durability and restore.** The CAS, evidence ledger, intent registry, and audit log support backup and verified restore; restore runs the index verifier, and a receipt whose referenced content was lost reports Redacted(lost, commitment) rather than disappearing. Audit logs are exportable as append-only streams.
- **Multi-user baseline.** Remote mode requires an identity model; capabilities are minted, scoped, delegated, and revoked through daemon operations recorded in the audit log. Cross-user computation sharing is off by default: content-addressed dedup across principals is an existence oracle and requires an explicit sharing policy.

4.6 Semantic epoch advance

Epochs are content identities; advancing one never mutates existing artifacts (ADR-0018):

- evidence is epoch-scoped: a receipt remains verifiable under its pinned epoch indefinitely (INV-006, INV-014); an epoch advance never silently revalidates or invalidates a published receipt;
- each advance publishes a typed per-artifact-class compatibility statement — Preserved | Revalidate | Incompatible — and an estimated invalidation blast radius by artifact class before it is applied;
- the daemon may hold at most two epochs concurrently during migration; new work defaults to the newest; continuations resume only under their pinned epoch (§9.6) and are forked, never migrated in place;
- re-derived artifacts receive new identities linked to their predecessors by SUPERSEDES edges; nothing is rewritten in place.

4.7 Engine-defect lifecycle

Engine defects are first-class evidence. Any ReplayDiverged, parity mismatch, engine crash, or explanation-validation failure emits a defect_* artifact: all inputs pinned by content identity, semantic and checker epochs, engine identity, and an automatically minimized reproduction, governed by the same redaction policy as Context Packs (§18.4). continuum doctor assembles a shareable defect bundle. Daemon health and performance export as OpenTelemetry alongside — never inside — semantic evidence.

5. Intent Contract

5.1 Why intent must be explicit

Formal tooling creates unusual reward-hacking opportunities. A patch can make verification green by:

- changing Agreement to a weaker observer;
- adding a fairness assumption that schedules away the bug;
- reducing node count from five to three;
- removing crash-after-submit;
- marking a storage effect opaque;
- mapping two concrete values to one abstract value;
- lowering assurance from exhaustive to sampled;
- excluding the failing state with a constraint.

These are potentially valid design changes, but they are not ordinary repairs.

5.2 Intent structure

An Intent Contract contains:

```
claims
  property AST + stable semantic name
assumptions
  environment, scheduler, timing, storage, network, trust
  domain-pack fidelity profile per effect family
  (ideal | contractual | platform-qualified | adversarial-envelope)
fault model
  crash, recovery, partition, loss/duplication/delay envelopes
fairness
  weak/strong fairness declarations per action family
completion policy
  (stutter-forever | deadlock-violation | finite-trace-only | closed)
nondeterminism classes per choice site
  (demonic | angelic | scheduler | probabilistic† | timed† | epistemic†)
  († declarative until ADR-0016 lanes ship – see B11)
observers
  state/event/knowledge/security projections
abstraction maps
  content identities of the §16 correspondence/abstraction maps the
  claims are stated against; merge/split changes are privileged (§5.3)
scope
  model/program components, abstraction level, and semantic fragment
  declarations (ADR-0025)
trust boundaries
  trusted and opaque components and effects
bounds
  values, nodes, faults, depth
assurance policy
  accepted evidence classes and required checkers
optimization
  hard constraints, soft objectives, non-vacuity
security policy
  data classification, redaction classes, and capability requirements
  (first-class per §0.1; resolves RFC 0037's open question)
change policy
  who/what may modify each field
```

5.3 Semantic intent diff

A semantic diff is not text diff. It classifies:

- strengthened/weakened property;
- strengthened/weakened assumption;
- observer refinement/coarsening;

- bound increase/decrease;
- fault envelope expansion/contraction;
- fairness addition/removal;
- abstraction merge/split;
- proof policy upgrade/downgrade;
- unsupported semantic change;
- unchanged intent.

Where implication is decidable or solver-checkable, Continuum proves the direction. Otherwise it emits a proof obligation, unknown (undecidable or unattempted), or unsupported (outside declared fragments) rather than guessing — the two are distinct per INV-008. All non-affirmative classifications fail closed: an intent change classified unknown, unsupported, or incomparable on a protected field blocks ordinary promotion exactly as a confirmed privileged change does, pending review. The semantic-diff schema enforces this structurally, not only in prose.

Each intent field declares its semantic fragment (ADR-0025: Finite / Symbolic / Temporal / Probabilistic / Theorem / Runtime), so “supported” is a checkable predicate. The diff guarantee is: within declared supported fragments, every property weakening, assumption strengthening, bound decrease, observer coarsening, fault removal, fairness addition or removal, and assurance downgrade is classified as privileged — the seven G3 dimensions; outside them, unsupported.

5.4 Intent locks

CI policy can lock fields:

```
[intent.policy]
properties = "review"
assumptions = "review"
fairness = "review"
trust_boundaries = "no-expansion"
non_vacuity = "no-removal"
completion_policy = "review"
bounds = "no-decrease"
faults = "no-removal"
assurance = "no-downgrade"
observers = "review" # a proof-gated verb requires an RFC 0037 revision
security_policy = "review"
optimization = "review"
scope = "review"
nondeterminism = "review"
abstraction_maps = "review"
```

Verbs are drawn from RFC 0037’s closed set; free-form policy strings are rejected. `scope`, `nondeterminism`, and `abstraction_maps` are protected: RFC 0031 classifies any change to them as a semantic change, so each must be lockable.

Agent repair capabilities exclude intent mutation unless a task explicitly asks for redesign.

`intent.accept` is the only transition from Proposed to protected status; it requires the `revise-intent` capability, produces an audit record, and is never performed implicitly by `init`, `check`, or any repair operation. `intent.lock` edits the §5.4 policy table under the same capability.

6. Context Packs

6.1 Definition

A Context Pack is a bounded, typed, property-directed compilation of the evidence graph for a human or agent task.

It is not a generic summary. It is produced by a query such as:

```
Why did AckImpliesDurable fail?
What code could affect the missing order?
What proof obligations block promotion?
Which assumptions distinguish this from the safe branch?
```

6.2 Contents

A failure Context Pack includes:

- target intent and property;
- exact verdict and assurance envelope;
- causal core and surrounding frontier;
- concrete and abstract state deltas;
- obligation/resource flow;
- missing or reversed order constraints;
- relevant source spans and model actions;
- proof/refinement slice;
- assumptions used and unused;
- counterfactual safe branches;
- candidate repair surfaces, clearly marked heuristic;
- exact replay and debugger handles;
- omitted-node counts and expansion queries;
- schema, semantic epoch, and content hash.

6.3 Context compiler

The compiler combines:

- backward causal slicing;
- property automaton relevance;
- dynamic and static dependence;
- proof dependency slicing;
- observer projection;
- abstraction/refinement correspondence;
- minimal unsatisfied core or correction-set analysis;
- information-gain ranking;
- token/byte budget optimization.

The core slice must be replay-preserving when claimed. Additional explanatory items may be heuristic but are labeled.

6.4 Views

The same pack can render as:

- concise terminal explanation;
- JSON/CBOR agent artifact;
- IDE diagnostics and code lenses;
- causal graph;
- debugger initial state;
- proof-worker task;
- repair transaction seed.

Presentation never mutates the underlying artifact.

7. Verification debugger

7.1 Debugger model

Traditional debuggers step through one execution. Continuum debugs a space of executions.

The debugger state contains:

```
selected configuration
causal past
currently enabled event frontier
property-monitor state
abstract views
concrete program state
```

obligation/region tree
 fault and time state
 unexplored alternatives

State materialization is deterministic re-execution from periodic committed checkpoints; the checkpoint interval is budget-tunable. Reverse-step and branch-fork carry docs/34 latency rows. Concurrently materialized branches per session are capped with LRU spill to the CAS. Branch comparison operates on projected observer state unless full concrete state is explicitly expanded.

7.2 Operations

- step one semantic event;
- step one abstract transition;
- run until property-relevant event;
- reverse to causal predecessor;
- jump to last change of a value/resource;
- branch on a different enabled event;
- inject/remove a fault at a legal frontier;
- compare two branches;
- inspect why an event is enabled or blocked;
- show fairness debt and liveness rank;
- project to selected observer;
- export branch as a crashpack or regression scenario.

7.3 DAP projection

A DAP adapter maps:

- semantic owners/tasks/nodes to threads;
- abstraction/refinement frames to stack frames;
- states, obligations, messages, and enabled events to variables;
- source/model labels to breakpoints;
- causal branches to restart/step targets;
- stable debugger object handles to DAP variable references.

DAP enables broad IDE support, but advanced partial-order operations remain native custom requests.

7.4 Why-enabled explanations

Every action should expose a derivation:

ReceiveVote(n2,n1) enabled because:
 message m17 is pending and deliverable
 n1 is running
 current partition permits $n2 \rightarrow n1$
 cancellation phase is Running
 guard $\text{term}(m17) \geq \text{current_term}(n1)$

Disabled actions expose the smallest blocking conditions where feasible.

8. Repair transactions

8.1 Transaction contents

A Repair Transaction is an immutable proposal plus accumulating evidence:

base snapshot and intent
 failure/crashpack
 repair hypothesis
 patch/model/proof changes
 semantic diff
 intent diff
 exact replay result
 neighboring exploration result

mutation challenge result
proof/refinement impact
performance impact
security review
promotion policy verdict

Each evaluation produces a new transaction version. The original proposal remains addressable.

8.2 Required gates

Default promotion requires:

1. The original failure replays on the base snapshot.
2. The patch applies to the declared snapshot with no hidden edits.
3. Protected intent is unchanged, or the transaction is explicitly reclassified.
4. The exact failure no longer occurs.
5. Neighboring schedules, faults, and values are explored.
6. Property mutations still fail where expected.
7. Known defect mutants remain detected.
8. Refinement coverage is not reduced unexpectedly.
9. Invalidated certificates/proofs are rebuilt.
10. Clean and incremental results agree.
11. Required code tests, static verification, and security gates pass.
12. A promotion receipt is generated.

8.3 Neighboring exploration

Overfitting is combated by constructing a semantic neighborhood:

- alternate enabled events at each causal decision;
- fault insertion/removal around the repaired window;
- cancellation at adjacent checkpoints;
- equivalent value/name permutations;
- changed message duplication/loss/delay;
- schedule perturbations in the same trace class boundary;
- generated variants from the abstraction map;
- hidden corpus-style mutations.

The neighborhood is property-directed and budgeted. Coverage appears in the receipt.

8.4 Counterfactual repairs

Continuum can evaluate proposed interventions on a failure graph:

add Sync → Ack order
move publication after Commit
abort reply obligation on cancellation
make epoch check atomic with write

Counterfactual success does not prove a source patch correct. It prioritizes repair surfaces and produces explicit hypotheses for agents/humans.

8.5 Review UX

A reviewer sees:

Intent: unchanged
Original failure: eliminated
Causal neighborhood: 38,412 classes explored
New failures: none in bounded envelope
Proof impact: 2 receipts rebuilt, 1 Lean theorem reused
Coverage: +4.3%
Cost: p99 verification +1.8%
Unresolved: production storage profile remains assumed

The exact artifacts are one click/handle away.

8.6 Cost governance

Every repair transaction carries a cumulative cost ledger (CPU, wall, solver, memory, token) across all its versions; the promotion receipt includes it. Gate 5–7 evaluations are incremental by default: neighborhood classes and mutants whose causal footprint is disjoint from the patch delta (a Conservative-class §9 dependency query) reuse prior results as Validated edges; anything else re-runs. The §9.5 sampling rate derives from a declared statistical confidence target for mismatch detection per reuse class, reviewed at G5. The daemon enforces per-principal and per-transaction cost ceilings; exceeding one yields BudgetExhausted with a continuation — never a silently smaller campaign (the cost-domain form of INV-007).

9. Incremental semantic database

9.1 Need

Impeccable DX requires subsecond feedback for local edits and resumable deeper searches. Re-running every model, extraction, exploration, proof, and explanation from scratch would make Continuum irrelevant in normal development.

The engine itself is engineering risk, not settled design: whether it is derived from an existing memoization framework or built custom is decided by a Phase B ADR with spike evidence (§21), and the decision is carried as docs/08 risk R21, with the mitigation “edge classes collapse to Conservative” named as the failure mode that destroys interactivity.

9.2 Query model

All derived artifacts are memoized queries over immutable inputs:

```
parse(file)
elaborate(module, imports, semantic_epoch)
extract_rust(crate_snapshot, annotations)
build_model(config, intent)
property_automaton(property)
abstract_state(program_state, map)
explore(model, intent, strategy, budget)
check_certificate(certificate, checker_epoch)
compile_context(evidence_root, query, budget)
```

A query key includes semantic configuration; a source hash alone is insufficient.

9.3 Dependency edge classes

- **Exact:** output is a pure function of named inputs; safe to reuse by content identity.
- **Validated:** translation/reduction output carries a checker witness.
- **Conservative:** invalidation may over-approximate dependencies but cannot miss changes under stated assumptions.
- **Experimental:** reuse may improve speed but result cannot support strong finality until clean validation.

9.4 Semantic dependencies

Dependencies distinguish:

- reads type vs value;
- unfolds definition;
- selects instance/domain profile;
- observes event family;
- relies on assumption/fairness/bound;
- uses abstraction component;
- depends on proof lemma/certificate checker;
- consumes solver encoding epoch;
- consumes source correspondence.

This enables precise invalidation and useful “why did this re-run?” explanations.

9.5 Incremental Parity Audit

(Renamed from “clean-build Tribunal”; **Tribunal** refers exclusively to the TLA+ corpus oracle harness of ADR-0021.)

CI and sampled local runs compare incremental and clean artifacts:

- verdict;
- canonical state graph or digest;
- counterexample class;
- certificate result;
- context slice soundness;
- semantic diff;
- proof axiom manifest.

Queries carry an auditability class, declared on the query definition:

- **Equality-auditable** — deterministic under the docs/19 matrix; compared bit-for-bit. Any disagreement quarantines the reuse class and emits a minimal invalidation counterexample.
- **Certificate-auditable** — solver-backed; the audit compares checked certificates and claim envelopes, never raw solver behavior. A certificate-level disagreement quarantines; a solver-outcome difference with agreeing certificates does not.
- **Budget-sensitive** — anytime results; the audit checks only that the incremental result's evidence labels are no stronger than a clean run's under equal budget (monotone-honesty), and records divergence as drift telemetry without quarantine.

Divergence attributable solely to budget or portfolio nondeterminism never quarantines a reuse class; divergence in an equality- or certificate-auditable query always does. INV-010's exactness claim applies per auditability class. The class is declared on the query definition, not the run, so a mismatch cannot be reclassified away after the fact.

9.6 Long-task continuations

Exploration, proof, and synthesis continuations contain committed frontier/search state and identity of every input. Resume rejects mismatched snapshots or epochs. Forking a continuation with a new budget is explicit.

10. Agent-native protocol

10.1 Design principles

The Agent-Computer Interface follows these rules:

- small orthogonal operations;
- typed arguments and results;
- stable handles instead of cursor positions;
- deterministic ordering;
- concise default payloads;
- explicit expansion;
- error messages with recovery actions;
- idempotency;
- budget and cancellation controls;
- no need to parse terminal prose;
- no implicit authority escalation;
- source text treated as data, not instructions.

10.2 Core operations

```
workspace.create / fork / diff / seal
intent.get / diff / propose_revision / accept / reject / lock
verification.start / result / await
model.check / explore / compare
program.extract / run / replay
refinement.check / explain
proof.goal / attempt / check / slice
correspondence.bind / status / drift
debug.open / state / enabled / step_event / step_abstract / reverse_causal /
  branch / compare / why_enabled / why_blocked / export
context.compile / expand
failure.explain / minimize / branch
```

repair.begin / apply / attach / evaluate / resume / review / promote / reject
observe.ingest / classify / result
forge.create / step / archive / materialize
benchmark.run
task.status / cancel / resume / subscribe
evidence.get / query / verify
query.explain_reuse / explain_invalidation / clean_compare

10.3 Error taxonomy

Errors are actionable and typed:

StaleSnapshot
UnsupportedSemanticFeature
IntentMutationDenied
InsufficientEvidence
BudgetExhausted
ContinuationEpochMismatch
AmbiguousCorrespondence
UntrustedDomainBoundary
CertificateRejected
ReplayDiverged
CapabilityDenied
PolicyGateFailed
ProtocolVersionUnsupported (protocol-level, RFC 0026)
IdempotencyKeyReused (protocol-level, RFC 0026)
MalformedRequest (protocol-level, RFC 0026)

Each error may include safe recovery operations, but never free-form commands with untrusted interpolation.

10.4 MCP adapter

MCP exposes a curated subset of native operations. Explicit handles are threaded through tool calls. The adapter:

- validates auth independently of handle possession;
- bounds result sizes;
- exposes schemas and deterministic tool ordering;
- avoids session-scoped semantic state;
- maps resources to immutable artifacts;
- attaches trace context;
- supports client-independent task handoff.

No MCP prompt or resource may mutate evidence status.

10.5 Agent handoff

A coordinator can give separate agents:

shared: snapshot, intent, evidence graph
isolated: proof search state, debugger branch, Forge candidate pool

This explicit sharing model prevents one giant session boundary from either over-sharing or isolating the wrong state.

11. Multi-agent Evidence Graph

11.1 Why chat is insufficient

Agent conversations are lossy, expensive, difficult to validate, and prone to persuasive but unsupported conclusions. Continuum stores durable work as a graph.

11.2 Node types

IntentClaim
Assumption
Property
ModelVersion
ProgramSnapshot

ProofGoal
InvariantCandidate
RankingCandidate
AbstractionMap
SynthesisCandidate
Counterexample
CausalExplanation
RepairHypothesis
Patch
VerificationRun
Certificate
ProofReceipt
BenchmarkResult
Conflict
ReviewDecision

11.3 Edge types

SUPPORTS
REFUTES
DEPENDS_ON
REFINES
EXPLAINS
REPAIRS
INVALIDATES
GENERALIZES
COUNTEREXAMPLE_TO
CHECKED_BY
DERIVED_FROM
CONFLICTS_WITH
SUPERSEDES

Edges name checker/evidence when applicable.

11.4 Status lattice

Proposed
Observed
Sampled
Bounded
Validated
Proved
Refuted
Inconclusive
Superseded

Only trusted services can promote into Validated or Proved. Agent votes or confidence cannot.

Inconclusive carries a typed reason per INV-008 (Unsupported, ResourceExhausted, EngineError, InsufficientTelemetry, AbstractionAmbiguity, IncompleteProofSearch). Validated records whether solver evidence is CHECKED_CERTIFICATE or TRUSTED_SOLVER; the two never render identically. Parameterized results carry checked($N=k$) / cutoff_checked($N \leq k$) / proved($\forall N$) and are structurally distinct in every surface. Budget exhaustion is never a verdict.

11.5 Whiteboard compiler

For complex invention tasks, agents may use a structured whiteboard:

Goal
Known facts
Candidate invariant
Counterexample
Unresolved obligation
Experiment
Decision

The compiler turns whiteboard entries into typed graph proposals, rejecting references to nonexistent artifacts or unsupported status claims.

11.6 Swarm roles

Recommended role decomposition:

- architect/modeler;
- implementation analyst;
- counterexample diagnostician;
- invariant/ranking synthesizer;
- Lean proof worker;
- repair worker;
- adversarial intent reviewer;
- benchmark/mutation worker;
- evidence integrator.

Roles are capabilities and task views, not separate truth domains.

11.7 Write model

The evidence graph is append-only. Publication is per-artifact atomic (INV-017) and linearized per claim identity; status promotion is a compare-and-set against the claim's current status, so racing promotions cannot regress the lattice. Concurrent contradictory claims materialize a Conflict node rather than resolving by write order. Idempotency keys make agent retries safe. The concurrency section of RFC 0038 is specification debt (§25); the evidence-graph spike validated the authority table, not concurrent enforcement (§25).

12. Explanation engine

12.1 Explanation levels

1. **Outcome:** what property failed and assurance scope.
2. **Causal:** minimal relevant events and order.
3. **Semantic:** abstract/concrete state mismatch and obligations.
4. **Source:** code/model spans and effect boundaries.
5. **Logical:** formula, proof obligation, assumptions, fairness.
6. **Exhaustive:** raw graph/certificate/solver artifacts.

Users can move up or down without rerunning verification.

12.2 Explanation objects

An explanation may include:

- actual cause candidates;
- necessary and sufficient causal sets under a stated model;
- minimal unsatisfied cores;
- minimal correction sets;
- proof slices;
- event/state delta slices;
- contrastive explanation: "why failing branch rather than safe branch?";
- counterfactual interventions;
- uncertainty and incompleteness.

Causal terminology must be precise. "Cause" is not used when only correlation or relevance was computed.

12.3 Contrastive example

Why did this run acknowledge before durability while the sibling run did not?

Only differing relevant choice:

failing: CancelRequested won before SyncCompleted
safe: SyncCompleted won before CancelRequested

The finalizer published the reserved reply in both branches.
The property therefore depends on finalizer behavior, not network order.

12.4 Explanation validation

Explanations are tested with:

- replay preservation;
- removal/addition checks;
- counterfactual execution;
- independent property monitor;
- expert review;
- human task studies;
- agent diagnosis benchmarks.

A concise explanation that omits the actual defect is worse than a long trace.

13. Human developer experience

13.1 Progressive disclosure

Continuum supports four default personas without separate semantics:

- Rust developer: source diagnostics, replay, causal debugger.
- Protocol designer: CML, state graphs, temporal properties.
- Formal-methods expert: formulas, reductions, certificates, Lean.
- Reviewer/operator: intent/assurance diff, production evidence, receipts.

13.2 Diagnostics

Diagnostics should answer:

What happened?
Why does it violate intent?
Where is the smallest relevant code/model region?
How certain is the claim?
What was explored or proved?
What remains unknown?
What is the next useful action?

13.3 IDE

The LSP provides:

- syntax/type/effect/fragment diagnostics;
- semantic hover for actions/properties/assumptions;
- go-to correspondence among model, Rust, proof, and evidence;
- inlay indicators for observer and effect footprints;
- proof/verification code lenses;
- semantic rename and impact preview;
- live bounded checks;
- context-aware completion for model constructs;
- intent-change warnings.

13.4 CLI

Human CLI output is stable, terse by default, and expandable:

```
continuum check --summary
continuum check --json
continuum explain crash_x --level semantic
continuum evidence show receipt_x
```

Exit codes and JSON schemas are documented. Color is never semantically required.

13.5 Learning path

The learning sequence mirrors the TLA+ corpus but uses one environment:

1. finite puzzles and reachability;
2. concurrency/deadlock;
3. invariants and counterexamples;
4. fairness/liveness;
5. symmetry and refinement;
6. real asupersync code;
7. storage/crash/cancellation;
8. production evidence;
9. proof and synthesis.

Every tutorial includes a defect, explanation, repair transaction, and evidence receipt.

13.6 Usability metrics

Measure:

- time to first model and first found bug;
 - diagnosis accuracy;
 - repair correctness;
 - assurance calibration;
 - number of context expansions;
 - abandoned tasks;
 - false confidence;
 - successful transfer to a new protocol;
 - expert vs non-expert divergence.
-

14. Continuum Forge

14.1 Purpose

Forge is the invention engine. It searches for new algorithms and proof artifacts while Continuum enforces intent and evidence.

Forge can synthesize:

- guards and state updates;
- message/acknowledgement rules;
- quorum systems;
- retry and cancellation protocols;
- recovery transitions;
- auxiliary/ghost state;
- invariants and lemmas;
- ranking functions and fairness obligations;
- abstraction maps;
- domain-pack contracts;
- implementation skeletons;
- optimization policies.

14.2 Forge problem

fixed:

Intent Contract
semantic fragment
implementation/effect constraints
hard safety/liveness/refinement obligations
assurance requirement

holes:

typed grammar or sketch locations

objectives:

latency, messages, durable writes, memory, simplicity, proof size

archive:

behaviorally distinct correct candidates

14.3 Search portfolio

- enumerative and constraint-based synthesis;
- CEGIS with exact/generalized counterexamples;
- interpretation reduction;
- SyGuS/SMT/CHC;
- IC3/PDR-guided invariant synthesis;
- game solving and assumption synthesis;
- stochastic/evolutionary search;
- quality-diversity algorithms;
- LLM proposal generation;
- theorem/proof retrieval;
- superoptimization over finite semantic fragments.

No engine is trusted. Candidates flow through the same verifier and proof pipeline.

14.4 Co-synthesis

Protocol synthesis often fails if algorithm, invariant, abstraction, and ranking are searched separately. Forge maintains a coupled candidate:

```
Candidate =  
  algorithm  
  + inductive invariant  
  + refinement map  
  + liveness ranking/fairness argument  
  + implementation correspondence  
  + cost vector
```

Counterexamples are classified against the component that failed:

- unsafe algorithm;
- non-inductive invariant;
- invalid abstraction;
- liveness cycle;
- unrealizable environment;
- implementation mismatch.

14.5 Non-vacuity and anti-gaming

Every synthesis task includes positive behaviors or progress scenarios, such as:

A request must be acknowledged in a failure-free fair run.

At least one write must be admitted when capacity exists.

Leadership must remain possible after recovery.

Property mutation and hidden semantic variants detect overfitting.

14.6 Quality-diversity archive

Candidates are indexed by behavioral descriptors:

- quorum geometry;
- message phase count;
- stable-write count;
- concurrency width;
- recovery mechanism;
- cancellation structure;
- fairness dependence;
- proof complexity.

A diverse archive is useful for discovering qualitatively new algorithms and for agent training.

14.7 Materialization

A promoted candidate can emit:

- CML model;
- executable asupersync Rust skeleton;
- correspondence map;
- generated tests and crash campaigns;
- proof obligations and Lean files;
- benchmark configuration;
- evidence receipt.

Generated code is never presented as production-ready without domain-pack and implementation verification.

15. Proof service and Lean integration

15.1 Proof service

Proof work runs in isolated workers keyed by:

- Lean version;
- Mathlib/dependency closure;
- source snapshot;
- options and resource policy;
- target declaration.

The service supports:

- strict proof checking;
- goal extraction;
- declaration/axiom metadata;
- proof-state stepping;
- proof repair attempts;
- lemma extraction and minimization;
- deterministic source transformations;
- proof receipts.

15.2 Proof Context Pack

A proof worker receives:

- exact goal and local context;
- relevant definitions and theorem statements;
- proof dependency slice;
- failed attempts and diagnostics;
- finite countermodels where meaningful;
- analogous corpus proofs;
- allowed tactics/axioms;
- version identity and budget.

It does not receive a giant repository dump by default.

15.3 Lean theorem program for Revision 3

Formalize:

1. Intent refinement/order for supported property fragments.
2. Soundness of semantic-diff classifications where decidable.
3. Context-slice preservation claims for finite causal graphs.
4. Repair acceptance implies elimination of the named witness under exact replay.
5. Incremental query equality under valid dependency closure.

6. CEGIS candidate acceptance for finite synthesis domains.
7. Composition of evidence/receipts.
8. Lens/conflict laws for proof-oriented correspondence.

Theorems remain small and foundational. High-performance engines emit certificates rather than being verified wholesale first.

15.4 Proof repair

Automated repair may use retrieved lemmas, compiler feedback, proof sketches, tactic search, and agent planning. Acceptance is kernel checking under the pinned environment. Changed axioms or theorem statements are intent/semantic changes, not proof repair.

16. Model/program correspondence

16.1 Correspondence graph

Rather than one abstraction function hidden in code, Continuum stores typed links:

```
Rust type/field/event/effect
  ↳ operational model component
  ↳ abstract model component
  ↳ property observer
  ↳ Lean definition/theorem
```

Links may be generated, inferred, or handwritten, but status and evidence differ.

16.2 Proof-oriented lenses

A correspondence has:

- get: concrete $\rightarrow$ abstract projection;
- optional put: proposed abstract edit $\rightarrow$ concrete candidate edits;
- complement/provenance needed for round trip;
- consistency relation;
- ambiguity conditions;
- generated proof obligations;
- effects and unsupported cases.

When multiple concrete repairs satisfy an abstract change, Continuum returns alternatives or a conflict. It does not choose silently.

16.3 Drift detection

Changes trigger:

- unmapped source effects;
- changed state projection;
- action split/merge;
- observer loss;
- new stuttering class;
- proof invalidation;
- semantic equivalence check where feasible.

The IDE can show “model correspondence stale” before deep verification.

17. Protocol adapters and ecosystem

17.1 LSP

Use standard LSP for editor-neutral authoring. Custom extensions refer only to stable handles and native operations.

17.2 DAP

Use DAP for broad debugger UI compatibility while preserving native partial-order controls.

17.3 SARIF

Export source-located, deduplicated verification diagnostics with artifact URIs, stable rule IDs, severity, code flows, fixes where safe, and evidence handles. SARIF is a report projection, not a proof format.

17.4 MCP

Expose agent operations and immutable resources. Explicit state handles enable multi-agent sharing and resumability. MCP does not define transaction semantics or authorization.

17.5 Cargo and test harness

```
cargo continuum check
cargo continuum explore
cargo continuum replay crash_x
cargo continuum review --base main
```

Rust unit/property tests can call an embedded client, but authoritative artifacts still come from the daemon/core library contract.

17.6 CI

CI produces:

- intent/semantic diff;
 - verification matrix;
 - changed evidence graph;
 - invalidated/rebuilt receipts;
 - benchmark deltas;
 - SARIF;
 - promotion recommendation with explicit unknowns.
-

18. Security architecture

18.1 Threats

- prompt injection through source/comments/logs;
- forged handles or evidence;
- unauthorized intent changes;
- solver/proof worker escape;
- denial of service via state explosion;
- secret exfiltration through context packs;
- production trace privacy leakage;
- dependency/toolchain substitution;
- malicious domain packs;
- replay artifact tampering;
- agent collusion/reward hacking.

18.2 Capabilities

Agents receive explicit capabilities for:

- reading selected snapshots/artifacts;
- proposing patches;
- starting bounded tasks;
- requesting proof work;
- expanding context;
- creating repair transactions.

They do not receive evidence-signing, intent-policy mutation, arbitrary network, or host execution by default.

18.3 Sandboxing

Foreign solvers, Lean workers, generated code, corpus oracles, and agent code run in isolated, resource-bounded environments with pinned images/toolchains and read-only inputs. Output is parsed as untrusted data.

Domain packs are content-addressed, signed, and pinned in the workspace snapshot; the Intent Contract's fidelity profile binds the exact pack identity, so pack substitution is a privileged intent diff (G3), not an environment change. Third-party packs default to adversarial-envelope fidelity until they pass docs/17 conformance qualification, and receipts render pack provenance (first-party | qualified | unqualified) distinctly.

18.4 Context privacy

Context compilation enforces source/trace field policy before slicing. Redaction is represented in the omission manifest. A redacted pack cannot support claims requiring hidden data unless a trusted checker provides a separate receipt.

Capture-time contract for production traces: payloads are recorded as salted commitments by default; raw payload capture is per-field opt-in, tagged with a data classification at ingestion, encrypted per artifact on entry (enabling §4.5 purge), and subject to a declared retention clock. Remote workers receive only post-redaction minimized artifacts; residency constraints are a capability property of the worker pool.

18.5 Audit

Every privileged operation records actor, capability, inputs, policy decision, outputs, and evidence identity. Audit logs are append-only and separate from semantic events.

18.6 Signing identities

Receipts, intent bundles, and domain packs are signed. The signing lifecycle is specified in docs/09: identities are minted through audited daemon operations (the solo-developer default is a local keypair minted on first use and recorded in the audit log); organizational deployments pin an allowed-signers set distributed inside the intent bundle; rotation and revocation are audited operations; a signature that cannot be verified downgrades the artifact to typed unverified provenance rather than failing open — except the §4.2.1 CI acceptance check, which fails closed by policy.

19. ContinuumBench

19.1 Purpose

ContinuumBench measures whether the system actually enables trustworthy agent/human work.

19.2 Task families

- translate prose/TLA+ into CML;
- diagnose finite safety failure;
- diagnose liveness/fairness failure;
- repair concurrent Rust;
- repair model/proof correspondence;
- infer invariant;
- synthesize ranking function;
- prove/refine in Lean;
- port a TLA+ example;
- synthesize protocol holes;
- classify insufficient production evidence;
- review semantic/intent diff;
- resist intent gaming and prompt injection;
- manage stale snapshots/continuations;
- optimize correct candidates.

19.3 Metrics

correctness / proof status
intent integrity
generalization to hidden variants
counterexample quality
repair minimality and robustness
human diagnosis time/accuracy
agent token and tool-call cost
wall/CPU/memory cost
invalid action rate

expensive failure rate
context compression and expansions
assurance calibration
security-policy compliance
behavioral novelty

Grading is ordered per RFC 0034: intent integrity → security → semantic correctness → evidence validity → hidden-variant generalization → cost → explanation. A zero in intent integrity caps the total score at failure. Leaderboards are Pareto fronts; no single ranking hides cost or assurance.

19.4 Dataset construction

Sources include:

- 80 validated TLA+ example families;
- deliberate and generated mutations;
- historical concurrency bugs;
- Continuum domain packs;
- asupersync cancellation/obligation cases;
- proof and refinement tasks;
- synthesized hidden variants;
- real project migrations.

Train/dev/test partitions isolate semantic families and source hashes to reduce leakage.

A named subset of corpus families is held out from all development, tuning, and regression use and graded only by the isolated ContinuumBench grader. G9's "80 families at declared parity" is measured on the development set plus a final, single evaluation of the held-out set.

19.5 Reward-hacking suite

Every benchmark candidate is tested against attempts to:

- weaken property;
- add assumptions;
- lower bounds;
- remove faults;
- hide observer events;
- return unsupported as pass;
- hard-code known trace;
- exploit stale cache;
- forge receipt/status;
- smuggle instructions through source.

Intent integrity is a prerequisite, not a bonus metric.

20. Crate and service architecture

continuum-workspace
continuum-intent
continuum-value
continuum-model-core
continuum-cml-syntax
continuum-cml-elab
continuum-cir
continuum-observer
continuum-refinement
continuum-certificate
continuum-kernel-core
continuum-kernel-sat
continuum-kernel-smt
continuum-kernel-temporal
continuum-evidence

continuum-context
continuum-semantic-diff
continuum-repair
continuum-incremental
continuum-task
continuum-debugger
continuum-forge
continuum-benchmark
continuum-security
continuum-asupersync
continuum-effects-{network,storage,time,process}
continuum-engine-reference
continuum-engine-explicit
continuum-engine-dpor
continuum-engine-symbolic
continuum-engine-liveness
continuum-proof-client
continuum-corpus
continuumd
continuum-cli
continuum-lsp
continuum-dap
continuum-mcp
continuum-sarif

Dependency rules:

- proof/certificate checker does not depend on search engines;
 - the `continuum-kernel-*` crates form the trusted checking base: no shared optimized evaluator code with any engine, no async, no unsafe, no plugins or dynamic loading, a <15,000 non-test-line covenant (docs/03), and a serialization boundary between every engine and the kernel — a certificate is checked from its wire form, never from shared memory;
 - in certified lanes, content identity is exact canonical identity; hash collisions are resolved by canonical comparison; 256-bit-hash identity is permitted only in explicitly labeled non-certified modes (ADR-0013);
 - model core does not depend on asupersync;
 - adapters do not own semantic state;
 - Forge depends on verifier interfaces, never vice versa;
 - UI crates cannot mutate evidence directly;
 - foreign tools remain isolated behind normalized artifacts;
 - kernel crates build reproducibly from pinned sources; every receipt records the checker's build digest and toolchain identity (INV-014), so checker identity is independently re-derivable.
-

21. Implementation program

21.1 Resourcing, licensing, and study posture

Phases are gate-driven, not time-driven. Each phase names an owner and a minimum viable team; a phase without both is **BLOCKED**, not in progress. The owner/team table lives in `notes/START_HERE_IMPLEMENTATION.md`; filling a phase's row is a merge requirement of that phase's opening PR, and an unfilled row records the phase as **BLOCKED**. Phase A additionally resolves: the product license (permissive, compatible with asupersync and solver adapters); a per-family redistribution audit for the corpus before any public benchmark release; and the rule that foreign oracle tooling (TLC, Apache, solvers) never ships in release binaries (ADR-0029). The G8 usability gate is defined against a preregistered minimal study: the three human-executed docs/34 acceptance workflows (new model; existing Rust system; review), two cohorts (Rust newcomers completing the deterministic/causal workflow; distributed-systems experts diagnosing real failures), a raw-trace baseline comparator, and cohort sizes, instruments, and pass thresholds fixed in a preregistration document — an expanded docs/48 — published before the study runs. The fourth docs/34 workflow (agent repair) is covered by the G2 ACI ablation and ContinuumBench, not the human study. Authoring the preregistration is a Phase E deliverable; the study itself runs in Phase F.

Phase A — Trust spine and ACI kernel

Deliver:

- immutable workspace and intent schemas;
- native daemon protocol;
- task/continuation lifecycle;
- evidence graph;
- Context Pack v0;
- semantic diff v0;
- executable finite reference engine and certificates from Revision 2;
- agent protocol spike parity;
- Lean environment pinned and seed modules kernel-checked; T0/T1 theorems (transition-system safety, stuttering simulation, finite-closure certificate soundness) compile with no sorry and empty axiom manifests, per the RFC 0012 theorem ladder and ADR-0035 axiom manifests.

Exit: Die Hard and Dining Philosophers can be checked through native API, CLI, and an agent client with identical artifacts; the ACI ablation shows the typed surface beats disciplined shell use on success and cost, or the protocol is redesigned before freeze (G0-DX-10); the prompt-injection corpus cannot trigger privileged operations (G2); continuation resume validates epochs and inputs (G1).

Phase B — Real-code failure loop

Deliver:

- asupersync semantic adapter (one adapter crate; supported upstream window of versions N and N-1, each covered by the adapter conformance corpus in CI against upstream pre-releases — an upstream breaking release opens a scheduled adapter epoch (§4.6), visible in envelopes; explicit semantic-hook contract; the model core never depends on asupersync);
- CML core-fragment parser and elaborator (Finite fragment; enough for the replicated register and Wave 0 ports; the programmatic model API remains supported);
- storage/network/process packs;
- replicated register;
- causal minimizer;
- debugger v0;
- repair transactions;
- exact and neighboring replay;
- an ADR resolving build-vs-adopt for the §9 incremental engine (salsa-derived vs custom), backed by a spike implementing the four reuse-edge classes and `query.explain_invalidation` over parse/elaborate/explore with a measured invalidation-precision baseline; Phase C is BLOCKED until this ADR exists.

Exit: an agent fixes ack-before-durable without changing intent and produces a promotion receipt under the Phase B gate profile.

Promotion gate profiles are phase-staged. Each §8.2 gate enters the default profile when its producing subsystem ships: gates 1–8 and 11–12 in Phase B; gate 10 (clean/incremental agreement) in Phase C; gate 9 (certificate and proof rebuild) in Phase D. A receipt must name its profile and list not-yet-enforced gates as `NotYetEnforced` — never as passed. A Phase B receipt is therefore structurally distinguishable from a Phase D receipt.

Phase C — Interactive scale

Deliver:

- incremental semantic database;
- LSP/DAP/SARIF;
- Incremental Parity Audit (§9.5);
- proof Context Packs;
- read-only tokio observation lane: journal observable lifecycle, channel, and time events; opaque-effect inventory; envelopes capped at `Observed` status with every claim dimension `Unsupported` — the adoption top-of-funnel for the Phase F instrumentation lane, making no controlled-semantics claim;

- Wave 0/1 corpus interaction tasks (29 families, pinned to `tlaplus/Examples@91c22ea...`) at their required parity per corpus/`tla-examples/PARITY_LEVELS.md`, capped at P2 (bounded semantic parity): 14 Wave 0/1 families are required at P4 and 3 at P3, which need the Phase D proof and liveness machinery; Phase C delivers every family at min(required, P2);
- CML language stability: normalized semantic AST frozen before surface syntax; formatter and migration tool ship before syntax stability (docs/11 §14);
- formative usability sessions (think-aloud, ~5 participants per §13.1 persona) on the edit/check/explain loop, repeated each phase from C onward and feeding docs/34 — distinct from and non-binding on the preregistered summative G8 study (§21.1).

Exit: the edit/check/explain loop meets the docs/34 latency table (p50/p95 per interaction) on the reference workload — the table is gate-normative for G5 and carries an explain-interaction row (failure → rendered causal explanation), both marked in docs/34 itself; incremental results are trustworthy under differential audit; reduction engines show zero reachability mismatch against the unreduced reference on the no-reduction corpus, and certified claims fall back to the unreduced baseline until the reduction’s certificate lane matures (docs/08 R04); semantic artifacts are byte-identical across the docs/19 determinism matrix (worker counts, build modes, platforms, hash seeds).

Phase D — Proof and liveness

Deliver:

- Lean theorem/certificate pipeline;
- fairness/liveness debugger;
- invariant/ranking synthesis;
- proof repair service;
- refinement receipts;
- self-application: a CML model of continuumd’s task/continuation lifecycle and INV-017 publication protocol, bound to the daemon implementation via the §16 correspondence machinery and checked in CI from Phase D onward; a daemon concurrency defect found this way is recorded as G4-class evidence of product value;
- Wave 0/1 families raised from the Phase C P2 cap to their full required parity (P3/P4);
- corpus Waves 2–3 at required parity;
- corpus release checkpoints mapped to phases: 0.2 closes in Phase D; 0.5 (Waves 0–2 at required parity plus ≥5 P5 exemplar families, now designated by name in `validated-examples.csv`) closes in Phase E; 1.0 (all 80) closes in Phase F with G9.

Exit: one nontrivial corpus protocol has safety and liveness evidence plus real-code refinement, produced by a proof service that holds G6: pinned Lean environment, per-request isolation and cancellation, theorem and axiom manifests on every receipt, and agent proof repair accepted only by the kernel.

Phase E — Forge

Deliver:

- typed holes and grammars;
- CEGIS/interpretation reduction;
- quality-diversity archive;
- co-synthesis of algorithm/invariant/ranking/abstraction;
- materialization to asupersync skeleton.

Exit: Forge rediscovers known solutions through typed holes and finite CEGIS, produces at least one behaviorally novel candidate independently verified within the declared envelope, materializes model, Rust skeleton, and proof obligations for a promoted candidate, and reports unrealizability as distinct from unknown.

Phase F — Production and ecosystem

Deliver:

- production partial-order evidence;
- foreign-runtime instrumentation lane (tokio), upgrading the Phase C read-only lane: instrumentation synthesis, fault-window replay where monitorable, and correspondingly bounded assurance envelopes — an adoption bridge, not a claim of controlled semantics;

- instrumentation synthesis;
- remote proof/verification workers;
- all 80 corpus families at declared parity;
- ContinuumBench public release;
- the preregistered G8 usability study (per §21.1) executed and analyzed;
- multi-agent workbench.

Exit: Continuum replaces bespoke DST plus separate TLA+ workflow in at least two materially different real systems. (§22 G10 and docs/52 G10 carry this same two-system criterion; all three statements are reconciled in one commit per §22's reconciliation rule.)

Scope note: 1.0 verifies systems whose controlled participants live in one workspace snapshot. Cross-repository federation — one intent governing independently deployed participants with a version-skew envelope — is an explicit 1.0 non-goal, recorded here so Phase F migrations are recruited within scope. A design note registers the federation-manifest (fed_*) direction for post-1.0.

22. Release gates

The normative gate scheme is docs/52_RELEASE_GATES_REV3.md (G0–G10); this section summarizes it. Legacy gate citations in Revision 2 ADRs (0001–0035), RFCs (0001–0025), and Revision 2-era docs (docs/01–docs/32, including the risk register docs/08 and the objections in docs/20) refer to the Revision 2 scheme in docs/26 and may not be cited without translation; the translation sweep is part of the specification pass in §25 and includes retiring the -Corpus/-Proof gate suffixes (RFCs 0011/0012/0019) and ADR-0022's internal Lean G-ladder, and archiving or Rev-2-bannering docs/04, whose gate graph matches neither docs/26 nor docs/52. No document may introduce a new gate numbering.

docs/52 and this section are reconciled in both directions in one commit: every criterion this section adds is folded into docs/52, no docs/52 criterion is dropped here, and the dossier validator enforces bullet-for-bullet correspondence between the two.

Phase (§21)	Gates it must close
A	G0 (falsification), G1 (workbench identity and lifecycle), G2 (ACI)
B	G3 (intent integrity), G4 (causal debugging and real repair)
C	G5 (incremental trust)
D	G6 (proof service)
E	G7 (Forge)
F	G8 (human usability), G9 (corpus parity), G10 (Continuum 1.0)

G0 — Load-bearing falsification

All items in notes/G0_SPIKE_MATRIX.md have evidence or an explicit redesign decision, recorded in the matrix itself. A failed or unexecuted freeze-blocking item blocks interface freeze (docs/52 G0).

G0 closes in Phase A for every item whose required experiment runs against Phase A machinery: DX-01–05, 07, 08, 10, 12, 13, 14. An unexecuted or failed item in this subset blocks interface freeze. Items whose experiments require later subsystems are re-homed to the gates that own them — DX-06 (neighborhood/mutation campaign) → G4, DX-11 (proof-service isolation) → G6, DX-09 (human diagnosis study) → G8, DX-15 (benchmark leakage) → G9 — and each re-homing is recorded in the matrix as that item's explicit decision. The matrix carries Status, Evidence, and Decision columns; §0.3's counts are derived from it, not asserted beside it.

G1 — Workbench identity and lifecycle

- snapshots, intent contracts, handles, and artifacts are immutable and content-addressed;
- explicit handles across the native API;
- requests are idempotent under idempotency keys;

- continuation resume validates epochs and inputs before any reuse;
- cancellation closes obligations and publishes no partial finality;
- artifact publication is transactional (INV-017);
- authorization is checked independently of handle possession;
- daemon crash recovery leaves no stale index entries or orphan tasks (§4.5).

G2 — Agent-computer interface

- no terminal parsing required;
- explicit handles and resumability;
- stale state rejected;
- generated clients and schemas ship for the native protocol;
- Context Packs are bounded, carry omission manifests and expansion handles, and improve agent benchmark effectiveness;
- native ACI beats the disciplined shell baseline on success and cost, or the protocol is redesigned before freeze (G0-DX-10);
- the prompt-injection corpus cannot trigger privileged operations.

G3 — Intent integrity

- every gaming mutation in the hidden (held-out) suite that falls in a supported fragment is classified as a privileged intent change, across all seven diff dimensions (property/assumption/bound/observer/fault/ fairness/assurance);
- mutations outside supported fragments classify as Unknown and block ordinary promotion rather than passing silently;
- intent policy locks are enforced; evidence is invalidated on intent revision;
- no ordinary repair promotes with a protected-intent change.

G4 — Causal debugging and real repair

- a real asupersync failure — not only injected mutants — is diagnosed and repaired end-to-end, alongside multiple known mutants;
- causal explanation with a replay-preserving core;
- partial-order debugger including alternate-branch exploration;
- exact and neighboring replay;
- mutation challenge;
- promotion receipt;
- human review view over the repair transaction;
- repair transaction closes exact, neighborhood, and mutation gates under the Phase B gate profile (§21).

G5 — Incremental trust

- incremental results continuously match clean builds under the Incremental Parity Audit;
- reuse edges carry their class (Exact/Validated/Conservative/ Experimental) and mismatches are minimized and quarantine the class;
- proof and certificate freshness is tracked;
- interactive latency targets (docs/34) hold on the reference workload;
- evidence queries and context compilation meet the docs/34 targets at $\geq 10^7$ evidence nodes on the reference workload;
- cache and publication are crash-safe.

G6 — Proof service

- Lean foundations kernel-check (Revision 2 and Revision 3 theorems, no placeholders);
- pinned Lean environment with per-request isolation and cancellation;
- every proof receipt carries theorem and axiom manifests;
- agent proof repair is accepted only by the kernel;
- certificate mutations are rejected;
- proof Context Packs improve proof-worker success/cost.

G7 — Forge

- typed holes and finite CEGIS;
- safety and non-vacuity;
- hidden variant generalization;

- independent candidate verification;
- diversity archive has semantic, not merely syntactic, spread;
- explicit unrealizability/unknown distinction;
- unrealizability produces reusable evidence where supported;
- materialized model/Rust/proof obligations.

G8 — Human usability

- the preregistered study (§21.1) covers both cohorts — Rust newcomers completing the deterministic/causal workflow, and distributed-systems experts diagnosing real failures;
- the preregistration (expanded docs/48) is published before the study runs; results are graded only against its fixed thresholds;
- explanation beats the raw-trace baseline on diagnosis accuracy and time;
- assurance confidence is calibrated, not merely no worse than baseline;
- progressive disclosure reaches exact artifacts;
- accessibility: no critical workflow requires color or a rendered graph;
- no critical workflow requires formal-methods folklore.

G9 — Corpus parity

- 80 validated TLA+ families at their declared parity level (corpus/tla-examples/PARITY_LEVELS.md), measured per §19.4's held-out discipline;
- every family carries its interaction artifacts: native model, expected verdict and state facts, meaningful explanation, failure/mutation task, proof/refinement support where applicable, and an agent benchmark artifact (docs/52 G9 — this is *interaction* parity, per B23).

G10 — Continuum 1.0

- two real project migrations;
- two materially different real projects remove bespoke DST infrastructure;
- at least two migrated projects stop requiring a separate TLA+ workflow for normal development;
- production evidence returns valid pass/fail/inconclusive classifications (INV-008);
- public ContinuumBench;
- documented assurance envelopes;
- agent-driven repair used on real changes under review;
- operating cost acceptable;
- zero known paths for unprivileged agent to promote false evidence.

Release blocker doctrine

A missing feature can be documented as unsupported. A misleading assurance result, replay failure, stale receipt, hidden intent change, or unauthorized evidence promotion is a release blocker at every gate. A confirmed false-positive success verdict triggers the soundness incident policy in docs/09: block release, revoke affected claim IDs, publish affected semantic epochs, ship an artifact scanner, add a permanent regression, and reevaluate whether the producing engine remains eligible for certified mode.

23. Success metrics

Product

- median time from defect injection to understood causal explanation;
- median time from explanation to promoted repair;
- percentage of concurrent changes carrying evidence receipts;
- reduction in bespoke DST code;
- number of projects no longer maintaining separate TLA+ models;
- human and agent successful transfer to unseen protocols.

Verification

- counterexample replay rate;
- causal core size and faithfulness;

- state/class reduction with preservation evidence;
- proof/certificate checking time;
- incremental/clean mismatch rate;
- unsupported/inconclusive honesty.

Agents

- task success under fixed model;
- token/tool/wall cost;
- invalid operation rate;
- stale-handle recovery;
- intent-gaming rejection;
- expensive failure rate;
- patch robustness on hidden variants;
- proof success with/without Context Packs.

Forge

- solved synthesis tasks;
 - proof rate;
 - generalization;
 - Pareto improvements;
 - behavioral diversity;
 - rediscovery of known algorithms;
 - independently validated novel candidates.
-

24. Kill criteria

Continuum must narrow or redesign if:

- real code requires pervasive rewrites solely to become observable;
- model/program correspondence remains mostly manual and fragile;
- Context Packs frequently omit defect causes;
- agent-native API does not beat disciplined CLI use;
- intent diff cannot reliably expose gaming in supported fragments;
- incremental trust overhead erases interactivity;
- Lean/certificate integration makes ordinary checks unusably slow;
- Forge mostly discovers vacuous or overfit protocols;
- all practical power comes from a loose collection of external tools rather than shared semantics;
- a second real project requires engine-specific surgery rather than domain packs;
- users systematically misread bounded evidence as proof despite UX controls;
- asupersync integration requires invasive scheduler forks that cannot be stabilized behind the pinned adapter contract (docs/08 R06);
- the foreign-runtime instrumentation lane cannot produce useful envelopes for tokio-based systems, leaving no adoption bridge;
- domain packs cannot demonstrate conformance to their fidelity profiles on real systems, making applied durability/network claims unearned (docs/08 R10);
- a candid comparison shows Quint plus existing DST tooling would be cheaper and equally strong for the target users; a “yes” after G4 closes is a program-level failure signal (docs/08 states this criterion as “after G2” in the Revision 2 scheme; Rev-2 G2 $\approx$ Rev-3 G4).

Failure of a frontier research lane does not kill Continuum. Failure of the single-semantic-contract, intent-integrity, replay, or evidence architecture does.

24.5 Frontier lane register

Every HYPOTHESIS-class capability in this plan is owned by a research lane with a baseline, a quantitative promotion threshold, a kill criterion, and a named fallback. The register is authoritative for lane status; no plan section may claim a lane’s output without its status.

Continuum: Master Implementation Plan & Comprehensive Dossier

A lane is **ratified** when its owner fixes the numeric threshold in the research note and this register quotes it verbatim; the dossier validator checks register↔note quote identity. Until then a row is **draft** and blocks its lane's promotion.

Continuum: Master Implementation Plan & Comprehensive Dossier

Capability	Lane	Threshold / kill	Fallback
General context compilation (§6)	research/25, research/32, research/33	ablation design per research/25 (raw trace vs pack on the agent benchmark); win margin fixed at ratification — draft; kill (research/32): compact packs repeatedly induce incorrect repairs despite preservation checks	plain causal slice + expansion
Exploration reduction (§9, INV-013)	research/01; docs/31	research/01 (stated there as a kill): $\geq 10\times$ reduction in explored classes on a non-artificial corpus subset without a <i>serious</i> regression on dependent workloads; observer-indexed (docs/31): median $\geq 5\times$ on the observer-sensitive class, <i>checker</i> overhead $< 20\%$ (distinct from the certificate-overhead row), zero mutation loss	conservative unreduced exploration
Liveness-preserving reduction (§7.2, Phase D)	research/04, research/13 — lane to be opened	soundness gate per research/04: property-directed reduction is proven fair-cycle-preserving or disabled; speedup target and liveness corpus subset fixed at lane opening — draft	unreduced liveness with explicit cost banner; batch expectations stated in the Phase D exit
Causal minimization (§6, §12)	research/26	replay-preserving core $\leq 10\%$ of trace length on real (non-synthetic) failures — draft, pending ratification; kill if minimization cost dominates verification	1-minimal delta debugging only
Neighborhood adequacy (§8.3)	research/33 — lane to be opened	hidden-variant catch rate of the §8.3 neighborhood on the docs/50 gaming corpus; target fixed at lane opening — draft; kill (research/33): hidden variants are too easy to leak or too hard to grade independently	fixed strategy-list neighborhood with per-receipt coverage disclosure and no adequacy claim
Sub-file incremental trust (§9)	research/27	clean-mismatch rate at the §9.5 sampled audit rate (rate derives from §8.6's confidence target) — draft; kill if dependency capture cannot be made trust-	module-granularity invalidation

research/25, research/32, research/33
 research/01; docs/31
 research/04, research/13 — lane to be opened
 research/26
 research/33 — lane to be opened
 research/27

A register row may not carry an unratified or TBD threshold past Phase A; such a row blocks its lane's promotion. docs/31's remaining quantitative thresholds (cubical reduction $\geq 3\times$ on ≥ 2 real protocol classes with a preservation theorem; semiring within 15% of specialized analyses with three production analyses sharing code; assumption synthesis on ≥ 5 liveness cases; abstraction discovery) are merged into this register during the §25 pass so the program has exactly one lane authority; where docs/31 and a research note disagree, the note is corrected and cited.

25. Immediate execution

The ordered initial PR sequence (PR 0–PR 30, with inserts 4a/15a/25a) is in notes/START_HERE_IMPLEMENTATION.md.

The seven load-bearing Revision 3 RFCs (0026, 0027, 0028, 0030, 0031, 0032, 0037) are expanded to specification form — field types, enums, classification lattices, versioning, RFC-2119 language — and the execution layer is reconciled: START_HERE carries PR 0, the Phase A Lean band (PR 4a; the proof *service* remains PR 28), the four continuum-kernel-* crates at PR 9, the CML parser (15a) and SARIF (25a) PRs, and per-PR gate annotations; the G0 matrix carries Status/Evidence/Decision columns, the DX-09 preregistration requirement, and the §22 re-homing decisions; the dossier validator enforces gate-citation hygiene, bidirectional plan $\leftrightarrow$ docs/52 bullet correspondence, and START_HERE gate annotations. Where plan prose and RFC disagree, the RFC is corrected and becomes normative. The plan is a map, not the spec.

The validator itself is brought up to these claims and its outputs regenerated in the same pass: lettered PR headings (4a/15a/25a) are matched; G0's freeze-blocking subset and the phase $\leftrightarrow$ gate tables are compared structurally, not as empty bullet lists; bare Rev-2 gate citations (including Target gate: metadata in RFCs 0001–0025) require a scheme qualifier or banner; §0.3's G0 counts and §24.5's register rows are derived checks; and VALIDATION_REPORT.md / validation-results.json are regenerated so every advertised check has a recorded result.

The following specification debt remains open and is paid before PR 5 (native protocol kernel) freezes any interface; each item names its target file so closure is checkable:

- RFC 0026: the normative IDL file (currently referenced, not present);
- RFC 0026: absorb the §4.3 requirements it currently omits — the N and N–1 protocol-major window, the evidence/receipt readability decoupling, mid-flight budget updates, and evidence subscriptions;
- RFC 0032: absorb §8.6 — the incremental gate 5–7 reuse rule, cost ceilings, and BudgetExhausted-with-continuation;
- RFC 0037: the intent.accept / intent.lock operations and revise-intent capability (currently only in RFC 0027), and the knowledge/security observer projections dropped from §5.2's four;
- RFC 0038: the evidence-graph write/concurrency model (§11.7);
- schemas: an intent-registry record schema carrying Proposed/accepted status and §4.2.1 acceptance chains; a shared Redacted(reason, commitment) \$defs used by every artifact class (§4.5, §18.4); a promotion-receipt schema carrying the §8.6 cost ledger and gate profile; a B11-complete assurance envelope (all nine dimensions, each naming its producing engine or a typed Unsupported(reason)), with the resource-exhausted verdict removed (§11.4: budget exhaustion is never a verdict); conditional enforcement for INV-008 reasons, validation_basis, and fail-closed diffs; typed Failed reasons on tasks (§4.5); one \$id/versioning convention with a schema-epoch field (§4.3);
- RFC 0037: the intent-bundle distribution and convergence section (§4.2.1);
- schemas/intent-contract.schema.json: a structured property-AST expression form with canonical normalization, replacing the bare expression string;
- docs/35, RFC 0026, ADR-0018, and docs/42: absorb the §4.5/§4.6/§4.7 operational contract (purge and Redacted(reason, commitment), backup and verified restore, the cross-user dedup existence-oracle rule, two-epoch migration, Preserved | Revalidate | Incompatible compatibility statements, engine-defect artifacts);
- docs/41: regenerate around the 12-gate, phase-profile repair design (RFC 0032 is normative in the interim);
- derived-doc contradictions corrected in the same pass (each currently states the opposite of the plan or its normative RFC): docs/42's opt-in parity audit (§9.5 and RFC 0030 make sampling default-on) and its Elaborate query key missing semantic_epoch (§9.2); docs/40's missing fail-closed rule and unbound incomparable (§5.3); docs/41's missing RFC 0032 banner (added immediately, ahead of regeneration); docs/35's cross-user sharing default (§4.5); docs/45's grading order (security is second per §19.3/RFC 0034).

The first demonstration should be brutally concrete:

1. Agent receives a 200-event failing run through a 4-event Context Pack.
2. It identifies acknowledgement-before-durability.

3. It proposes moving publication after sync.
4. Continuum proves intent unchanged.
5. Exact replay passes after the patch.
6. Neighboring cancellation/crash schedules pass.
7. Property mutations and known mutants remain detectable.
8. Refinement and proof receipts rebuild.
9. A human reviews one compact semantic diff and receipt.

Then Forge should solve the corresponding hole from scratch:

```
ack_guard ∈ grammar
Safety: Ack ⇒ Durable
Progress: failure-free synced request eventually acknowledged
Result: ack_guard = synced
```

This is deliberately small. It closes the complete product loop—from intent through invention and proof—before the project scales outward.

The executed spikes behind this demonstration carry their own boundaries, which this plan adopts: the 200→4 event reduction was measured on a synthetic trace whose 196 noise events are semantically inert, so it validates the Context Pack artifact shape, not the general context compiler; the synthesis result was selected from a seven-candidate finite grammar and is not evidence that protocol synthesis will scale; the evidence-graph spike validates the authority policy table, not authenticated enforcement, concurrency, or Byzantine agents.

26. Final doctrine

Continuum will succeed if it makes the correct thing easier than the seductive wrong thing:

- easier to state intent than bury assumptions;
- easier to consume a causal explanation than grep a log;
- easier to repair under a transaction than overfit a test;
- easier to resume by handle than preserve a fragile session;
- easier to inspect an assurance envelope than trust a green badge;
- easier for agents to use typed evidence than hallucinate from prose;
- easier to invent inside proof constraints than bolt verification on afterward.

The long-term ambition is not merely safer concurrency. It is a new mode of systems research:

humans specify values and constraints; agents explore enormous mathematical and implementation spaces;
Continuum turns every proposed leap into replayable evidence, checked refinement, and proof-bearing code.

That is how concurrent and distributed systems stop being artisanal collections of race conditions and become a domain where invention can accelerate without sacrificing truth.

Reference Documents: Specifications & Architecture Documents

Document: docs/00_EXECUTIVE_SPEC.md

Continuum Executive Specification — Revision 2

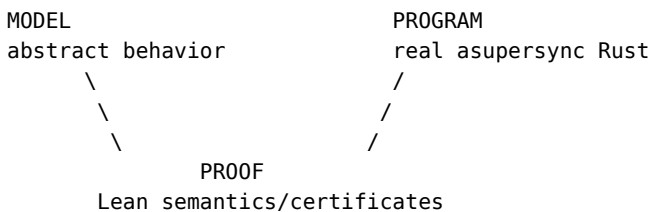
Mission

Continuum shall make formal modeling, deterministic simulation, systematic concurrency exploration, implementation refinement, proof, and production conformance coherent execution modes of one concurrent-Rust engineering environment.

It shall replace, for supported projects, the repeated need for:

- a separate TLA+/Quint model;
- bespoke deterministic-simulation infrastructure;
- Loom/Stateright-like one-off harnesses;
- handwritten model-based tests;
- ad hoc production trace mapping;
- unverifiable “the checker said okay” success claims.

Semantic triptych



- The **Model** is useful before code and chooses the correct abstraction.
- The **Program** is actual production logic under controlled or real effects.
- The **Proof** independently defines and checks strong claims.

CML is the model language. CIR is the causal interchange for concrete execution. Lean is the metatheory/certificate authority. Asupersync is the native concrete execution substrate.

Release contract

Continuum 1.0 requires native semantic equivalents for all **80 CI-validated TLA+ Examples families** at their declared P0–P5 parity level. The corpus is pinned by commit and used as:

- language expressiveness test;
- model-checking and liveness test;
- proof-pattern test;
- PlusCal/procedural-lowering test;
- configuration/symmetry/refinement test;
- mutation benchmark;
- performance corpus.

Semantic parity does not require copying TLA+ syntax or TLAPS proof scripts. It requires preserving the intended behavior and claims under an explicit correspondence.

Constitutional rules

1. **Model, Program, and Proof remain independently meaningful.**
2. **No engine implementation defines semantics accidentally.**
3. **No strongest claim is self-certified.**
4. **Partial orders are primary for concrete concurrency.**
5. **Standalone abstract models are mandatory.**
6. **Fairness and assumptions are explicit and attached to named actions/environment choices.**
7. **No ambient nondeterminism in controlled code.**
8. **Exact equality or collision resolution is mandatory in proof lanes.**

9. **Replay is a compatibility surface.**
10. **Production monitoring may return Inconclusive.**
11. **Foreign TLA+/solver tools are evidence producers, not hidden runtime dependencies.**
12. **Proof receipts expose transformations, epochs, checker identity, Lean theorems, and axioms.**
13. **Property/assumption changes are privileged semantic operations.**
14. **Experimental mathematics must beat a simpler baseline and carry a preservation story.**

Native capabilities

Abstract design

- mathematical states, sets, maps, relations, sequences, graphs;
- nondeterministic relational actions;
- process/procedural surface lowered to actions;
- invariants, temporal properties, fairness, deadlock policy;
- modules, parameters, refinement, auxiliary/history/prophecy variables.

Real-code verification

- asupersync tasks, regions, capabilities, obligations and cancellation;
- virtual network, time, storage, process lifecycle and faults;
- deterministic replay and snapshots;
- schedule and causal exploration;
- exact/minimized crashpacks.

Verification portfolio

- simulation and coverage-guided faults;
- explicit-state search;
- source/optimal/observer-indexed DPOR;
- unfoldings and experimental higher-dimensional reduction;
- bounded symbolic checking and PDR/CHC;
- symmetry, cutoffs, counter abstraction and nominal techniques;
- SCC/lasso/ranking liveness;
- assumption games;
- timed, probabilistic, weak-memory and hyperproperty lanes.

Proof/evidence

- finite closure and invariant certificates;
- refinement simulations;
- reduction witnesses;
- fair-SCC and ranking certificates;
- SAT/PB/solver proof import through verified encodings;
- Lean theorem receipts and axiom manifests;
- optional independent Lean environment checking.

Production

- partial-order trace conformance;
- uncertainty-aware verdicts;
- instrumentation synthesis;
- production-to-Lab reproduction.

Assurance lattice

Continuum never collapses all evidence to “passed.” Representative classes:

example

sampled campaign

bounded schedules

DPOR-complete under declared dependence

finite exact closure

symbolic bounded
inductive/parameterized
liveness proof
refinement proof
Lean theorem receipt
production observation
inconclusive

Every result records bounds, assumptions, observers, semantic epochs, pack profiles, trusted components, and limitations.

Initial wedge

The first end-to-end demonstration is a durable replicated register:

- standalone model;
- real asupersync implementation;
- crash/restart, network, storage, timer and cancellation effects;
- deliberate acknowledgement-before-sync defect;
- observer-indexed causal exploration;
- minimized replayable failure;
- concrete-to-abstract stuttering refinement;
- independently checked finite/Lean receipt.

Scope boundaries

Continuum does not promise push-button proof of arbitrary Rust, automatic discovery of the correct abstraction, universal liveness automation, exhaustive unbounded checking, or conclusive monitoring from insufficient evidence.

Its promise is narrower:

one coherent semantic system, explicit abstraction/refinement, aggressive but checkable analysis, honest assurance, and first-class developer/agent workflows.

Success test

The project has replaced the practical need for TLA+ in its target projects when a user can:

1. model a protocol before implementation;
2. analyze safety and liveness;
3. implement it in asupersync;
4. check the implementation refines the model;
5. reproduce and repair failures causally;
6. obtain independent evidence;
7. validate production traces;
8. port every validated TLA+ example family without semantic special cases.

Document: docs/01_ARCHITECTURE.md

Continuum Architecture — Revision 2

1. Architectural stance

Continuum has six planes joined by typed, versioned contracts:

Authoring → Semantics → Execution → Verification → Evidence
 ↘ Corpus / Tribunal ↗

The three normative artifact families are:

- **CML/core model** — abstract mathematical meaning;
- **CIR/configuration** — concrete causal meaning;
- **proof receipt** — independently checked claim meaning.

No plane owns the others.

2. System diagram

flowchart TD

```

CML[CML standalone models] --> ELAB[Typed elaboration]
PROC[Procedural model surface] --> ELAB
RUST[Rust + asupersync] --> AD[Asupersync semantic adapter]
TLA[TLA+/PlusCal/TLAPS oracle] --> TRIB[Corpus Tribunal]

ELAB --> CORE[Model semantic core]
CORE --> REF[Reference evaluator]
CORE --> EX[Explicit/symbolic/temporal engines]
CORE --> LEAN[Lean semantic mirror]

AD --> CIR[Causal IR]
CIR --> CAUSAL[DPOR/unfolding/symmetry/nominal]
CIR --> CONFORM[Production conformance]
CIR --> RMAP[Program refinement]

REF --> EVID[Evidence and crashpacks]
EX --> EVID
CAUSAL --> EVID
CONFORM --> EVID
RMAP --> EVID
TRIB --> EVID

EVID --> CERT[Independent certificate checker]
CERT --> LEAN
LEAN --> RECEIPT[Proof receipt + axiom manifest]

```

3. Semantic triptych

3.1 Model plane

The model plane contains CML source, typed core, transition/behavior semantics, properties, assumptions, observers and refinement declarations. It exists independently of Rust.

3.2 Program plane

The program plane contains asupersync code and versioned domain packs. It emits CIR using public semantic hooks and reports uncovered effects.

3.3 Proof plane

The proof plane contains Lean definitions, preservation theorems, reflective certificate checkers, corpus theorem libraries and proof receipts.

3.4 Edges

- ModelPortParity
- ModelRefinement
- ProgramRefinement
- TraceConformance
- TransformationPreservation
- CertificateJustification

Each edge records assumptions, observer mapping, semantic epoch and evidence status.

4. Stable boundaries

4.1 Typed model core

Normative concepts:

- values and equality;
- state schemas;
- init/action relations and frames;
- finite/infinite behaviors;

- enabledness and fairness;
- observers;
- refinement;
- fragment requirements.

The reference evaluator is intentionally independent of optimized engines.

4.2 CIR

CIR is append-only within a semantic epoch and contains events, causality, conflict, intervals, effects, resources, obligations, faults, lifecycle and observer projections. It uses canonical binary encoding plus diagnostic JSON.

CIR is the concrete interchange, not the complete abstract model language.

4.3 Proof receipt

The receipt binds claim, model/property/assumption closure, semantic/proof/corpus epochs, transformations, certificate, checker, Lean theorems, imports and axioms.

4.4 Corpus manifest

A port manifest binds a source family/commit to native modules, configurations, state/action correspondence, expected outcomes, mutations, parity and theorem/runtime evidence.

5. Model-language fragments

Finite	exact enumerable semantics
Symbolic	solver-representable constraints
Temporal	infinite behavior and fairness
Probabilistic	distributions/MDPs/games
Theorem	Lean-only propositions/proofs
Runtime	concrete controlled effects
WeakMemory	reads-from/coherence execution graphs
Hyper	relational sets of executions

Elaboration computes the least required fragment and rejects unsupported backend combinations.

6. Asupersync adapter

Asupersync	Continuum
task	task identity and program order
region	ownership/lifecycle resource
Cx	effect authority and provenance
cancellation	phase transitions and reason
obligation	linear resource delta
reserve/commit	effect phase
Lab scheduler choice	replay choice
virtual time	time constraints
chaos	fault event
snapshot	replay checkpoint

The adapter cannot be the sole refinement checker for itself.

7. Verification architecture

7.1 Reference path

CML core → simple exact evaluator → graph/witness. This path defines executable finite meaning and differential truth.

7.2 Optimized paths

- explicit state;
- DPOR/unfolding;

- symbolic/PDR/CHC;
- liveness/games;
- timed/probabilistic;
- weak memory/hyperproperties.

Each optimized path emits evidence checked outside the search implementation.

7.3 Transformation graph

Every lowering/reduction records input/output hashes, preservation class and witness. Strong claims retain the entire chain.

8. Observer lattice

Observers order by factorization. Finer observers preserve more distinctions. Reduction, state slicing, conformance and instrumentation analysis all use this relation.

Independence is indexed by the active observer/property/fairness contract. Static footprints are conservative defaults.

9. Corpus Tribunal

The Tribunal contains foreign tool adapters, oracle containers, graph/trace comparators, mutation generation, proof-intent mapping and a dashboard. It is isolated from product verification.

10. Lean architecture

M0 mathematics
M1 transition/reachability
M2 temporal/fairness
M3 refinement
M4 event structures/observers
M5 effects/cancellation/durability
M6 algorithms/certificates
M7 serialization/encoding bridge
M8 corpus/project theorems

Reflective checkers turn large external evidence into composable Lean theorems. Axiom manifests are mandatory.

11. Domain packs

Packs own operation signatures, fault algebra, phase/lifecycle semantics, footprints, observers, abstraction views, profiles, conformance tests and proof obligations. Storage/network semantics are profiles, not universal mocks.

12. Production architecture

Production events form an uncertain partial order. Conformance solves for legal model alignment. The monitoring API supports valid, invalid, inconclusive, semantic mismatch and resource-limit results. Instrumentation synthesis proposes the minimum additional evidence needed to decide a claim.

13. Dependency constraints

- model core does not depend on asupersync;
- certificate checker does not depend on search;
- Lean does not trust generated facts without a checker theorem;
- foreign TLA tools are Tribunal-only;
- experimental engines cannot enter the kernel dependency closure;
- proof receipts and semantic codecs are versioned independently of UI.

Document: docs/02_SEMANTICS.md

Semantic Contract

1. Why true concurrency is primary

A total trace chooses an arbitrary order for independent events. It is useful for replay but a poor semantic foundation:

```
send A→B ; local write C
local write C ; send A→B
```

may represent one causal behavior, not two. Starting from total traces forces every engine to rediscover commutativity and makes production logs appear more ordered than reality.

Continuum instead uses causal configurations and interval pomsets/event structures. Interleavings are linear extensions.

2. Core mathematical object

A model M defines:

$$M = (S, I, \mathcal{A}, T, O, F)$$

- S : typed states;
- $I \subseteq S$: initial states;
- $\mathcal{A}$: action schemas;
- $T_a \subseteq S \times Params_a \times S$: transition relations;
- O : observations;
- F : fairness assumptions.

A concrete execution is not merely $s_0, s_1, \dots$, but a labeled event structure:

$$\mathcal{E} = (E, \leq, \#, \lambda, \iota)$$

- E : events;
- $\leq$: causality, a partial order;
- $\#$: conflict, symmetric and hereditary;
- λ : semantic labels;
- ι : interval/resource/obligation metadata.

A finite configuration C is conflict-free and downward closed. A cut induces state through a deterministic fold where commuting events are required to commute or be marked dependent.

3. Determinism condition for state projection

For events e and f concurrently enabled in a configuration, if they are declared independent:

$$apply_f(apply_e(s)) = apply_e(apply_f(s))$$

and their generated observations are equivalent under the active observer.

This diamond obligation can be:

- proven statically for a pack schema;
- checked dynamically for a specific transition;
- conservatively rejected, making events dependent.

The default is dependence. Performance never overrides soundness.

4. Conflict and independence

Dependence sources include:

- overlapping semantic writes;
- read/write conflicts;
- same message/timer/obligation identity;
- lifecycle relation;
- durability ordering;
- explicit synchronization;
- observer-sensitive output;
- fairness/progress interaction;
- pack-defined conflict.

Independence is parameterized:

$$Indep(e, f \mid View, PropertyClass)$$

Safety properties often permit more reduction than liveness or hyperproperties. The verifier records which relation was used.

5. Time

CIR supports three distinct notions:

1. **model logical time:** abstract counters or clocks;
2. **virtual execution time:** controlled by Lab;
3. **observed wall intervals:** lower/upper bounds from production instrumentation.

An event may have an interval $[start, end]$. Real-time precedence is:

$$end(e) < start(f) \Rightarrow e \prec f$$

Overlapping intervals remain unordered unless another causal edge exists.

Timed models add clock valuations and constraints. They are not implicitly inferred from wall timestamps.

6. Faults

Faults are events or adversarial choices, never magic mutation. Examples:

- drop/duplicate/delay;
- partition/heal;
- crash/recover;
- volatile-state loss;
- torn or reordered persistence;
- clock jump/skew;
- cancellation;
- budget exhaustion;
- Byzantine corruption, only in packs that explicitly support it.

A fault algebra defines composition and exclusions. For example, a pack may prohibit a disk completion after device destruction unless a recovery model permits it.

7. Cancellation calculus

Lifecycle

Active

```

├ request(cancel_reason) → Cancelling
├ complete(value)       → Completed
└ panic(payload)        → Panicked

```

Cancelling

```

├ drain(effect)*
├ finalize(resource)*
└ obligations == ∅      → Cancelled

```

Effect protocol

Idle → Reserved(token)

Reserved(token) → Committed(result)

Reserved(token) → Aborted(reason)

A cancellation checkpoint may occur in declared interruptible phases. Committed effects cannot be silently rolled back. Reserved effects must either abort or transfer an obligation to a finalizer.

Core invariants

- every obligation has one owner or is discharged;
- ownership transfer is causal;
- region close implies no descendant tasks or obligations;
- visible commit implies all required precommit conditions;
- cancellation does not create a visible half-effect;

- finalization order respects durability/resource dependencies.

8. Fairness

Fairness objects are typed and scoped:

- `WeakFair(action, scope);`
- `StrongFair(action, scope);`
- `EventuallyDelivered(channel predicate);`
- `EventuallyScheduled(task predicate);`
- `EventuallyStable(network partition);`
- `FiniteCrashes(node);`
- `CancellationResponsive(operation, bound?).`

No global “fair scheduler” switch exists. Fairness assumptions can be falsified by a concrete execution or remain outside the modeled environment.

9. Property languages

State safety

$$\forall \text{reachable}(s).P(s)$$

Transition safety

$$\forall \text{step}(s, s').Q(s, s')$$

Trace/temporal

LTL-like operators, past operators where useful, event quantification, and named fairness.

Hyperproperties

Properties over sets or tuples of executions:

- noninterference;
- observational determinism;
- probability-distribution preservation;
- differential privacy-style bounds;
- strong linearizability/refinement.

These require dedicated semantics and cannot be inferred from ordinary trace refinement.

Quantitative

- probability bounds;
- expected cost/time;
- percentile/rare-event targets;
- resource budgets.

10. Refinement modes

Trace inclusion

Every concrete visible trace is accepted by the abstract model.

Stuttering refinement

Concrete internal steps can leave the abstract state unchanged.

Forward simulation

A relation $R(c, a)$ advances with every concrete step.

Progressive forward simulation

Adds a well-founded progress obligation to prevent infinite concrete stuttering, required for hyperliveness-style claims.

Strong observational refinement

Preserves selected hyperproperties/scheduler behavior. This is stronger than ordinary linearizability/trace inclusion.

Approximate refinement

For probabilistic or numeric systems, a metric or error bound relates observations. This is later work and must not share the same verdict as exact refinement.

11. Multi-grained composition

Each component exposes:

- interface events;
- assumed environment behavior;
- guaranteed behavior;
- abstract summary;
- hidden internal events;
- refinement certificate.

A mixed-grain system can instantiate a fine model for changed components and summaries for stable components. Compatibility checks ensure interface assumptions compose.

12. Semantic normalization

Canonical model form includes:

- alpha-normalized IDs;
- explicit action groups;
- normalized expressions;
- deterministic finite-domain enumeration;
- explicit unchanged variables;
- source-map side tables;
- semantic digest independent of formatting.

This digest anchors replay and certificates.

13. Semantic evolution

Changes are classified:

- **representation-only**: same denotation; old artifacts decode;
- **clarification**: previously ambiguous behavior rejected or fixed;
- **semantic extension**: new constructs, old models unchanged;
- **breaking semantic change**: new epoch and migration report.

A model lockfile pins semantic epoch and pack versions.

Document: docs/03_ASSURANCE_AND_TCB.md

Assurance Model and Trusted Computing Base

1. Problem

Verification tools often collapse fundamentally different evidence into “passed.” Continuum forbids that. A result is a typed claim whose meaning is stable across CLI, CI, documentation, and production monitoring.

2. Assurance claim schema

Conceptually:

```
struct AssuranceClaim {
    subject: SubjectDigest,
    semantic_epoch: SemanticEpoch,
    model_scope: Scope,
    property: PropertyRef,

    semantic_coverage: SemanticCoverage,
    exploration: ExplorationClass,
    proof: ProofClass,
    linkage: ImplementationLinkage,
```

```

observation: ObservationCoverage,

assumptions: Vec<Assumption>,
trusted: Vec<TrustedComponent>,
excluded: Vec<ExcludedBehavior>,

witness: Option<ArtifactRef>,
certificate: Option<ArtifactRef>,
reproduction: Reproduction,
}

```

3. Dimensions

Semantic coverage

Level	Meaning
MODEL_ONLY	abstract semantics only
CONTROLLED_CORE	implementation core uses modeled capabilities
CONTROLLED_CLOSURE	relevant dependency closure audited
PRODUCTION_OBSERVED	real execution emitted semantic evidence
HOST_COMPLETE	host boundary coverage justified for declared property

Exploration

Level	Meaning
ONE_RUN	one deterministic execution
SAMPLED	multiple generated executions
BOUNDED_CHOICES	all executions within schedule/fault bounds
BOUNDED_STATES	all states within declared model scope
EXHAUSTIVE_FINITE	complete reachable finite state space
INDUCTIVE	unbounded executions within a symbolic domain
PARAMETERIZED	quantified/cutoff argument for instance sizes
STATISTICAL	confidence/precision statement, not proof of impossibility

Proof evidence

Level	Meaning
ASSERTION_ONLY	engine reports result
REPLAYED_WITNESS	counterexample independently replayed
CROSS_ENGINE	independent engines agree
CHECKED_CERTIFICATE	small kernel checked evidence
MECHANIZED_KERNEL	checker correctness linked to proof assistant artifact

Implementation linkage

Level	Meaning
NO_LINK	abstract model only
TEST_GENERATION	model traces run against code
TRACE_CONFORMANCE	code traces accepted by model
STEP_REFINEMENT	concrete transitions refine abstract transitions
PROGRESSIVE_REFINEMENT	infinite stuttering excluded
STRONG_OBSERVATIONAL	selected hyperproperties preserved

These dimensions form a product lattice; they are not collapsed into a single “level 5.”

4. Human-readable verdict examples**SAFE**

property: Agreement
 exploration: exhaustive finite
 scope: nodes=3, values=2, crashes≤1
 evidence: closure certificate checked by continuum-kernel 0.3
 implementation linkage: none
 assumptions: network may drop/reorder/duplicate; no Byzantine behavior

NO VIOLATION OBSERVED

property: NoLeakedObligation
 exploration: 5,000,000 sampled Lab executions
 evidence: deterministic replay for all retained failures
 implementation linkage: controlled core
 warning: not exhaustive

REFINES

concrete: server crate build 98af...
 abstract: LeaseProtocol digest d31c...
 linkage: step refinement over 44,182 bounded configurations
 evidence: checked refinement certificate
 excluded: FFI metrics exporter and allocator behavior

5. Certificate checker architecture

continuum-kernel should be:

- single-threaded;
- deterministic;
- allocation-bounded where practical;
- `#![forbid(unsafe_code)]`;
- independent of search data structures;
- able to stream large certificates;
- fuzzed against malformed/adversarial input;
- specified separately from the emitter.

Code-size covenant (binding — plan §20 and START_HERE PR 9 adopt it as the kernel-crate rule): fewer than 15,000 non-test lines across decoder, semantics, and certificate checks. The number is a forcing function, not a proof of trustworthiness.

6. Certificate formats**6.1 Closed finite state space**

A certificate includes:

- canonical state table;
- initial-state IDs;
- action schemas;
- for each state/action, exact successor IDs or disabled witness;

- invariant evaluation data or values sufficient to recompute it;
- optional symmetry representative map;
- model and property digests.

Checker:

1. decode and canonicalize;
2. verify all initial states are present;
3. recompute every enabled transition;
4. verify successor closure;
5. evaluate property for every state;
6. verify symmetry map if used.

A smaller certificate can use an independently checkable Merkleized state table, but the checker must still validate closure.

6.2 Inductive invariant

Certificate supplies *Inv*:

- $Init \Rightarrow Inv$;
- $Inv \wedge Step \Rightarrow Inv'$;
- $Inv \Rightarrow Property$.

Backend proofs can be Alethe/LRAT/solver-specific plus checked theory lemmas. Until theory-proof support is mature, the result is TRUSTED_SOLVER, not CHECKED_CERTIFICATE.

6.3 Refinement

Certificate supplies:

- abstraction function or relation;
- initial mapping;
- visible-step simulation;
- stuttering classification;
- progress measure where required;
- fault/fairness mapping.

The checker can enumerate finite concrete domains or validate symbolic obligations via checked solver proofs.

6.4 Partial-order coverage

A certifying POR result needs:

- event schemas;
- explored configurations/prefix;
- dependency relation;
- commutation/diamond witnesses;
- backtracking/source-set coverage evidence;
- property-preservation class.

This is research-grade. Baseline exhaustive claims can fall back to unreduced search until the certificate is trustworthy.

6.5 Liveness

Finite graph:

- SCC decomposition;
- reachable-component witnesses;
- fairness acceptance labels;
- absence of accepting SCCs or explicit fair-cycle witness.

Ranking:

- well-founded domain;
- decrease obligations;

- fairness-to-progress linkage;
- safety side conditions.

6.6 Probabilistic

- rational/interval value function;
- Bellman inequalities;
- scheduler quantifier;
- residual/error bounds;
- exact arithmetic or checked interval rounding.

7. Solver trust classes

SAT:

model replayed successfully	→ witness checked
model not replayed	→ solver-trusted witness

UNSAT:

Alethe/LRAT proof checked	→ certificate checked
proof unavailable	→ solver-trusted
multiple solvers agree	→ cross-engine, still not proof

The CLI makes this distinction prominent.

8. Diversity against common-mode bugs

Independent paths should differ in:

- language/runtime;
- state representation;
- traversal;
- arithmetic;
- parser;
- certificate decoding;
- solver.

Practical progression:

1. simple Rust reference evaluator vs optimized Rust engine;
2. foreign oracle (TLC/Quint/Apalache) for compatible corpus;
3. tiny Rust kernel;
4. mechanized semantics/checker in Lean or Rocq;
5. generated conformance vectors between implementations.

9. Pack axioms and trust

Some host facts cannot be proven by Continuum. Example: “an acknowledged fsync has the documented kernel/filesystem/device semantics.” Packs list:

- modeled guarantees;
- measured assumptions;
- trusted external contracts;
- unsupported hardware behavior;
- observation gaps.

An assurance result includes the exact pack fidelity profile.

10. Claims ledger

Each claim record:

```
id: CLM-EXPLICIT-001
text: "Exact explicit exploration is collision-safe."
status: PROVEN_BY_TEST_AND_REVIEW
scope: "continuum-explicit 0.4, canonical state backend"
fresh_until: 2026-10-01
```

evidence:

- `cargo test -p continuum-explicit collision_corpus`
- `artifacts/CLM-EXPLICIT-001/report.json`

caveats:

- "Does not cover experimental GPU backend."

Documentation CI rejects stronger wording than the ledger allows.

Document: docs/04_IMPLEMENTATION_ROADMAP.md

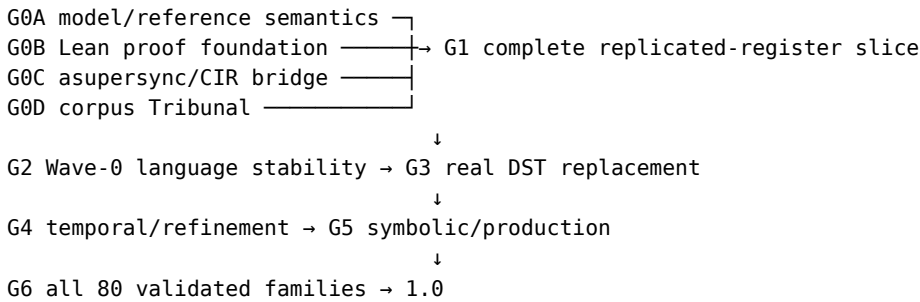
Implementation Roadmap — Revision 2

REVISION 2 — SUPERSEDED. This roadmap and its gate graph (G0A–G0D, G1–G6) predate Revision 3 and match neither docs/26 nor the normative docs/52. For current phases and gates see plan.md §21–§22; for the Rev-2 gate scheme see docs/26. Do not cite gate numbers from this file without translation.

Principle

Sequence by **risk retirement and evidence closure**, not by subsystem prestige. The implementation order is gate-driven; it is not a delivery-time promise.

Gate graph



Workstreams

W1 — CML and typed core

Parser, formatter, diagnostics, value algebra, actions, modules, temporal properties, fragments and refinement declarations.

W2 — Reference semantics

Exact deterministic evaluator, graph exploration, shortest traces, finite certificates and semantic fuzzing.

W3 — Lean metatheory

Reachability, temporal/fairness, refinement, event structures, cancellation, reduction and reflective certificate theorems.

W4 — CIR/asupersync

Semantic event sink, lifecycle/obligation mapping, replay, snapshots and coverage.

W5 — Explicit/causal engines

Parallel exact checking, DPOR, unfoldings, symmetry, nominal canonicalization and minimization.

W6 — Temporal/games

Fair lasso/SCC, ranking, parity/Streett and environment-assumption synthesis.

W7 — Symbolic/parameterized

BMC, k-induction, PDR/CHC, cutoffs, counter abstraction, regular/nominal lanes and solver evidence.

W8 — Corpus Tribunal

80-family inventory, foreign oracles, port manifests, graph/trace parity, mutation suite and dashboard.

W9 — Domain packs

Network, storage, process, time, synchronization, database and weak-memory profiles.

W10 — Production conformance

Partial-order matching, uncertainty, instrumentation synthesis and Lab reproduction.

W11 — DX/agents

CLI, daemon, LSP, visualizer, proof blueprints, semantic diff, generated Rust interfaces and repair loop.

Corpus milestones

- **Wave 0:** finite search, proof basics, elementary concurrency;
- **Wave 1:** mutual exclusion, barriers, readers/writers, auxiliary variables;
- **Wave 2:** fairness, termination, graphs and reachability;
- **Wave 3:** consensus, Byzantine protocols, transactions and refinement;
- **Wave 4:** storage, TCP, cache coherence, lock-free and complex operational models;
- **Wave 5:** language/meta edge cases.

Each wave hardens reusable semantics and libraries. It is not 80 isolated ports.

Proof milestones

1. closure/invariant theorem;
2. stuttering refinement and composition;
3. fair-lasso/SCC certificate;
4. symmetry/observer preservation;
5. cancellation/obligation conservation;
6. solver encoding and reflective import;
7. parameterized/cutoff theorem patterns;
8. corpus proof families.

First vertical slice

The replicated-register slice is complete only when abstract model, real code, faults/cancellation, CIR, exploration, minimization, replay, refinement and proof receipt all exist. A partial implementation cannot be labeled G1.

API freeze rules

Do not freeze:

- higher-order model semantics before corpus pressure;
- observer-indexed DPOR before exhaustive differential evidence;
- proof-receipt binary layout before first Lean import;
- adapter hooks before real asupersync execution;
- production trace schema before uncertainty experiments;
- nominal representation before fresh-name benchmarks.

1.0 gate

All 80 validated TLA+ example families meet required parity; proof-bearing claims have Lean receipts; selected runtime refinements cover major domains; critical claims have current reproducible evidence; no hidden foreign fallback exists.

Document: docs/05_RESEARCH_LANDSCAPE.md

Research Landscape and Gap Analysis

This document summarizes the relevant state of the art as of 2026-07-24. Source IDs refer to 13_BIBLIOGRAPHY.md.

1. Abstract protocol modeling

TLA+ and TLC

TLA+ remains exceptional for unconstrained state-machine abstraction, safety/liveness, fairness, and mathematical state. TLC provides explicit-state exploration but carries installation and implementation friction and maintains a hard

model/code boundary. Continuum should preserve TLA+’s freedom to choose abstraction, not merely its syntax. [S01][S64]

Quint and Apalache

Quint modernizes authoring with types, a REPL, simulation, and model-based testing, while Apalache lowers TLA+-style models to SMT and supports bounded checking and inductiveness. They validate the value of a typed frontend and a layered IR. Continuum should interoperate rather than fork users unnecessarily. [S02][S03][S04]

P

P’s event-driven machines, monitors, systematic testing, inductive verifier, and runtime observation come closest to lifecycle integration. Its architectural commitment to machines/events is useful but narrower than arbitrary mathematical abstraction. [S05]

Gap: none of these systems make a production Rust runtime’s cancellation, durability, task ownership, and causal events native to the same semantics as the abstract model.

2. Deterministic simulation and systematic concurrency

FoundationDB and TigerBeetle

Both demonstrate the decisive engineering pattern: run production code in a deterministic environment with virtualized nondeterminism, accelerate time, inject faults, and preserve exact seeds. This is the strongest empirical evidence for replacing bespoke DSTs with a common substrate. [S06][S07]

Loom, Shuttle, and GenMC

Loom instruments Rust synchronization but does not model the full C11 memory model and is intentionally local. Shuttle offers randomized/replay/PCT-style schedule exploration. GenMC performs stateless model checking over LLVM memory models. [S08][S09][S43]

MODIST and related transparent checkers

MODIST showed that interposing on an unmodified distributed system can expose actions to a centralized deterministic explorer and found many protocol bugs. The cost is semantic opacity and platform-specific interposition. [S12]

Gap: local concurrency, distributed faults, cancellation, and production semantics are typically controlled by different tools.

3. Model/code conformance

Quint Connect

Quint Connect generates model traces and runs them against Rust implementations. This validates model-based testing as a practical bridge. [S04]

OmniLink

OmniLink maps black-box concurrent events with timeboxes into TLA+ meanings and solves for a legal total order. It demonstrates that useful validation need not require a fabricated observed total order and can find previously unknown bugs. [S13]

Multi-grained ZooKeeper verification

Recent work on ZooKeeper uses fine and coarse TLA+ component models together, addressing the tradeoff between state explosion and model/code gaps and finding severe bugs. [S14]

Gap: conformance remains a post hoc workflow. Continuum makes multi-grain views and partial-order trace completion native.

4. Partial-order reduction and true concurrency

DPOR

Classic DPOR, optimal DPOR, observer variants, and parsimonious optimal DPOR provide increasingly precise exploration of Mazurkiewicz trace classes. They are the practical baseline. [S15][S16][S17]

Petri-net unfoldings

Complete finite prefixes represent concurrent executions without enumerating all interleavings. Recent work extends symbolic prefixes to high-level nets and infinite-marking classes. Modular unfoldings summarize component behavior through interfaces. [S21][S22A]

Higher-dimensional automata

Recent Kleene, Myhill–Nerode, and logical characterization results give a serious formal-language theory for interval pomsets and non-interleaving concurrency. This is not yet a production model-checking recipe, but it offers tools for canonicalization and compositional languages beyond traces. [S22][S23][S24A]

Directed topology

Spaces of directed executions and homotopy classes formalize families of schedules. They may support reduction and visualization, but practical value must be benchmarked. [S24B]

Gap: practical verification rarely treats partial-order objects as the stable interchange artifact.

5. High-performance state exploration

LTSmin/PINS

LTSmin separates modeling languages from verification algorithms using the Partitioned Next-State Interface. It combines explicit, symbolic, parallel, LTL, and POR algorithms. This strongly supports Continuum’s engine-independent semantic interface. [S18]

Sylvan and saturation

Parallel decision diagrams and saturation can outperform flat exploration on asynchronous systems by exploiting transition locality. They must be portfolio engines, because representation performance is highly model-dependent. [S19][S20]

Gap: a modern Rust-native system could combine partial-order events, PINS-style action groups, exact state encodings, and proof evidence, but should reuse mature algorithmic ideas rather than assume a concurrent hash set is enough.

6. Inductive and parameterized verification

PDR/IC3 and CHCs

IC3/PDR learns inductive invariants through property-directed blocking. Spacer generalizes this style to infinite-state systems and CHCs; global-guidance work addresses local-generalization instability. [S26][S27]

IC3PO and Ivy

IC3PO connects symmetry with quantification and cutoff discovery, automatically deriving quantified invariants for protocols including Paxos. Ivy restricts models into decidable fragments to automate parameterized proofs. [S28][S29]

Well-structured transition systems

WSTS theory gives decidable coverability for monotone infinite-state systems using well-quasi-orders and ideal completions. It is powerful for counters, multisets, lossy channels, and broadcast-like systems, but not a general distributed verifier. [S61]

Gap: users manually choose and reformulate systems for each proof technology. Continuum can detect structure and lower one semantic model into restricted lanes with explicit applicability conditions.

7. Liveness

Finite-state liveness can use SCC/automata methods, but parameterized distributed liveness remains difficult. LVR reduces many protocol liveness proofs to safety using ranking functions; newer relational-ranking work aims at scalable progress proofs. [S30][S31]

Gap: deterministic simulators usually test finite progress, while model checkers separate fairness from concrete runtime cancellation and fault assumptions. Continuum can connect fairness and rankings to asupersync’s structural lifecycle.

8. Refinement and verified distributed systems

IronFleet and Verdi established end-to-end or framework-based verified distributed implementations. Aneris, Grove, Perennial, and Trillium provide expressive separation logics and refinement frameworks for concurrency, distributed systems, crashes, and TLA+-style models. [S32]–[S37]

These systems demonstrate that layered refinement is the right theoretical shape. Their proof-engineering cost and language/tool separation remain barriers for routine systems development.

Gap: automated, executable, multi-grained refinement tied to ordinary Rust workflows.

9. Rust verification

Verus, Creusot, Kani, Aeneas, hax, Crux, RustBelt, and GenMC cover complementary regions:

- deductive functional correctness;
- bit-precise bounded code checking;
- translation to proof assistants;
- unsafe-code semantics;
- weak-memory exploration;
- machine-checked foundations. [S38]–[S45]

Interior mutability and concurrency remain difficult; library-defined capabilities and hybrid safe/unsafe verification are active research.

Gap: code verifiers do not replace an abstract distributed protocol model. Continuum should export local obligations and use these tools below the protocol-refinement boundary.

10. Certificates

Alethe provides an SMT proof format; Carcara is a Rust checker. LRAT has efficient verified checkers. Model-checking certificates exist for finite state, probabilistic systems, and interactive certification. [S47]–[S49]

Gap: mainstream distributed-system model checking rarely makes certifying success a first-class developer artifact.

11. Timed and probabilistic systems

UPPAAL, IMITATOR, Storm, and PRISM provide mature specialized engines for timed, parametric timed, probabilistic, and stochastic systems. Rare-event simulation and statistical model checking address failures too rare for naive sampling. [S50]–[S52][S56]

Gap: these semantics are typically separate models. Continuum should add them as typed domain/property extensions without weakening the deterministic core.

12. Hyperproperties and strong refinement

Ordinary refinement may not preserve distributions or security hyperproperties. Strong observational refinement and progressive forward simulations characterize stronger preservation conditions. HyperLTL tools automate selected hyperproperties. [S53][S54][S55]

Gap: system verifiers routinely claim refinement without stating which property classes are preserved.

13. Composition, effects, and choreographies

Algebraic/higher-order effects provide modular interpretations of operations. Choreographic programming and multi-party session types can project global protocols to local endpoints with safety/liveness properties, including refined and timed protocols. [S58]–[S60]

Opportunity: Continuum’s effect packs and views can learn from these systems, while avoiding the claim that every distributed implementation should be generated from one global choreography.

14. Synthesis and agents

SyGuS/CEGIS, reactive synthesis, and AI-assisted Verus work show that invariants, functions, and proofs can be generated effectively in constrained domains. Every useful result still depends on a mechanical verifier. [S57][S69][S70]

Gap: agents receive poor semantic feedback. Continuum’s causal counterexamples and explicit proof obligations can make agentic repair much more reliable.

15. 2025–2026 frontier delta

Several recent results sharpen the architecture:

- QSM-Cutoff systematically derives quantified cutoff formulas from symmetry-aware finite reachability, strengthening the case for an orbit-lifted parameterized lane rather than naïve finite extrapolation. [S77]
- Await-aware optimal DPOR removes semantically pure waiting executions and weakens conflict where justified; Continuum’s runtime semantics should expose await/quiescence structure rather than treating every poll as an event. [S78]
- Asynchronous fault-tolerant runtime-verification results establish decisive monitorability limits for real-time-order-sensitive properties, requiring Inconclusive as a first-class production verdict. [S79]
- Algorithm-selection results in hardware verification support a measured portfolio scheduler, but only as a performance layer over independently sound engines. [S80]
- RustMC demonstrates stateless checking of compiled Rust/FFI code through GenMC, reinforcing the value of an adapter lane below Continuum’s protocol semantics. [S81]
- Partial-order conformance through Petri-net unfoldings provides a concrete baseline for Continuum’s production alignment engine. [S82]
- LRAT-Catcher shows that large external solver certificates can be imported compositionally into Lean through reflection, strengthening the long-term path from Continuum certificates to theorem-prover artifacts. [S84]
- Thrust and Gillian-Rust show rapid progress in CHC/refinement-type automation and hybrid safe/unsafe Rust verification; Continuum should generate local proof obligations rather than attempting to subsume these tools. [S71][S86]

16. Overall conclusion

No existing system combines all of the following as one semantic product:

- arbitrary standalone abstract models;
- real Rust/asupersync execution;
- cancel-correct effect semantics;
- deterministic distributed simulation;
- partial-order-first exploration;
- zoomable refinement;
- inductive/liveness lanes;
- production partial-order conformance;
- typed assurance;
- independently checked certificates.

The project is plausible because nearly every ingredient has independent evidence. The risk lies in semantic integration and scope, not in the absence of algorithms.

Document: docs/06_RESEARCH_AGENDA.md

Frontier Research Agenda

The research agenda separates foundational work from speculative work. Nothing in the speculative lane is required for the first useful product.

Track A — Foundational semantics

A1. Causal Intermediate Representation

Question: what is the smallest event/configuration structure that faithfully represents:

- asupersync lifecycle;
- state transitions;
- interval time;
- conflict and independence;
- storage durability;
- faults;
- observations;
- refinement views?

Deliverable: formal semantics, executable reference evaluator, mechanized core definitions, and conformance corpus.

A2. Cancellation and obligation calculus

Develop a labeled transition system for:

- request/drain/finalize;
- reserve/commit/abort;
- linear obligation ownership;
- region quiescence;
- bounded responsiveness assumptions.

Prove compositional rules such as:

```
operation cancel-correct
+ finalizer cancel-correct
+ region obligation closure
⇒ no visible half-effect after region close
```

This is a genuinely underdeveloped area compared with safety of message protocols.

A3. Effect-pack refinement

For each pack, distinguish:

- ideal semantics;
- operational Lab semantics;
- host/production semantics;
- observation relation.

Use refinement transformers inspired by verified distributed-system frameworks: application proofs should survive when a verified pack implementation replaces an ideal pack.

Track B — Partial-order engines

B1. Observer-sensitive parsimonious DPOR

Extend dependence with active view/property observers. Internal effects may commute even when their concrete footprints overlap if the abstraction proves commutativity.

Research questions:

- can abstraction-proved commutation safely reduce implementation exploration?
- how are liveness/fairness dependencies retained?
- can independence witnesses be certified compactly?

B2. Complete finite causal prefixes

Translate bounded CIR systems into occurrence structures and compute adequate-order cutoffs. Explore symbolic high-level events to avoid grounding every data value.

Benchmarks against:

- source DPOR;
- parsimonious optimal DPOR;
- explicit BFS;
- TLC/Stateright where comparable.

B3. Causal Cubical Reduction

Construct a cubical complex:

- vertices: configurations;
- edges: events;
- n -cubes: n mutually commuting events.

Potential uses:

- canonical representatives of schedule families;

- homotopy-class exploration;
- detection of “holes” corresponding to forbidden synchronization patterns;
- compact visualization;
- search guidance.

Required discipline:

- topology is heuristic unless connected to a proven quotient;
- Betti numbers are not correctness evidence;
- retain only if practical reduction or diagnosis improves.

B4. Event-language minimization

Recent higher-dimensional Myhill–Nerode/Kleene results suggest residual languages over interval pomsets. Explore whether repeated protocol subbehaviors can be minimized directly in a non-interleaving automaton.

This could create reusable causal summaries for components.

Track C — Compositionality and abstraction

C1. Multi-grained verification

Automatically compute impact cones from:

- changed code;
- effect footprints;
- view dependencies;
- learned invariants.

Keep affected modules concrete/fine; replace unaffected modules with certified summaries. Validate composition using assume-guarantee obligations.

C2. Sheaf gluing

Model:

- local component behavior as sections;
- interfaces as restrictions;
- global execution as a compatible gluing.

Research hypotheses:

1. incompatible local abstractions produce a computable obstruction;
2. the obstruction identifies missing interface state/assumptions;
3. local proof certificates can be assembled without constructing the whole state graph.

Comparison baseline: interface automata, assume-guarantee learning, modular unfoldings.

C3. Abstract-domain synthesis

Candidate methods:

- predicate abstraction;
- CEGAR;
- anti-unification of counterexample states;
- IC3-learned clauses;
- symmetry orbits;
- e-graph equality saturation for expression candidates;
- SyGuS grammar for abstraction functions;
- abstract interpretation with Galois insertions.

All synthesized maps are verified independently.

C4. Coalgebraic behavioral interfaces

Coalgebra offers a uniform language for state-based systems and bisimulation. Explore a restricted coalgebraic interface for domain packs and views so that behavioral equivalences and compositional operators are not reinvented per engine.

Kill if it adds abstraction vocabulary without simplifying implementations or proofs.

Track D — Inductive and parameterized proof

D1. Symmetry-to-quantification

Integrate orbit analysis and IC3PO-style clause generalization:

- infer quantified invariants from finite instances;
- discover candidate cutoffs;
- validate with first-order induction;
- explain which symmetries justify generalization.

D2. WSTS lane

Detect monotone fragments over:

- counters;
- multisets;
- lossy channels;
- process populations;
- broadcast protocols.

Generate coverability proofs using ideals/backward reachability. Report precisely that only upward-closed safety was proved.

D3. CHC/PDR portfolio

Lower model/refinement/liveness obligations to CHCs. Run multiple engines and preserve learned invariants. Develop model-specific global guidance using causal/action partitions.

D4. Proof repair

When an inductive invariant fails:

- classify initiation/consecution/property failure;
- produce minimal counterexample-to-induction;
- mine relevant predicates from CIR footprints and views;
- propose repairs;
- mechanically recheck.

Track E — Liveness

E1. Ranking synthesis

Represent eventualities as obligations. Synthesize lexicographic, multiset, ordinal, or relational rankings under fairness.

Use the runtime obligation graph as a candidate ranking vocabulary.

E2. Fairness diagnostics

Given a liveness counterexample, compute:

- unfair actions;
- permanently enabled actions;
- failure assumptions needed to sustain the cycle;
- weakest fairness strengthening that excludes it;
- whether that strengthening matches production reality.

The tool should not simply print an SCC.

E3. Cancellation progress

Properties:

- every cancellation request reaches region quiescence under declared responsiveness;
- no obligation is transferred infinitely often without progress;
- race losers drain;

- finalizers terminate or expose a bounded-failure certificate.

E4. Ordinal progress

Some nested retries/recovery protocols need rankings beyond natural numbers. Explore ordinal templates and well-founded transition invariants, but keep generated proofs explicit and checkable.

Track F — Production conformance

F1. Partial-order trace completion

Input:

- event labels;
- causal IDs/vector clocks;
- timeboxes;
- missing-event model;
- abstract semantics.

Output:

- satisfying completion/linearization;
- minimal inconsistency core;
- or explicit insufficiency.

Use SAT/SMT/CP backends, but replay the resulting abstract execution.

F2. Observability analysis

Compute which properties are monitorable from an instrumentation schema. Suggest the cheapest additional event needed to distinguish valid from invalid executions.

F3. Trace-to-simulation reconstruction

Infer a constrained Lab scenario from a production trace:

- network envelopes;
- timer order;
- crash epochs;
- storage completions;
- cancellation points.

Then search neighboring schedules to find a smaller or more deterministic reproduction.

F4. Hyperconformance

For security/randomized systems, ordinary trace inclusion is insufficient. Implement strong observational/progressive refinement for selected finite-state models, then evaluate scalability.

Track G — Timed and probabilistic verification

G1. Unified time constraints

Combine virtual time and production intervals through a common difference-constraint layer. Avoid conflating simulated exact times with observed uncertain times.

G2. Timed partial orders

Use zones over event intervals and causality, reducing permutations that differ only by commuting timed events.

G3. Rare-event verification

Combine:

- importance splitting;
- cross-entropy tuning;
- semantic distance-to-violation;
- exact replay;
- confidence sequences.

Statistical evidence remains statistical.

G4. Probabilistic refinement

Define scheduler-sensitive probability-preserving refinement, using strong observational refinement where appropriate. This is a high-risk research area.

Track H – Agentic verification

H1. Semantic agent protocol

Expose tools for:

- inspect model state/action;
- request successor slice;
- inspect counterexample core;
- propose invariant;
- check induction;
- propose abstraction;
- run mutant;
- replay exact trace.

Agents never edit evidence artifacts directly.

H2. Proof portfolio planning

Use agents to choose between explicit, PDR, refinement, and liveness tactics. The policy may be learned; each result remains mechanically gated.

H3. Spec/code co-repair

Given a conformance failure, classify:

- implementation bug;
- model omission;
- abstraction mismatch;
- instrumentation gap;
- pack fidelity issue.

Require explicit human or policy approval before changing the specification to “fix” a failing implementation.

Research scorecard

Every experiment records:

Field	Meaning
hypothesis	falsifiable statement
baseline	strongest reasonable comparator
corpus	public and internal benchmarks
metric	soundness, reduction, time, memory, diagnosis
threshold	minimum practical win
failure mode	what would invalidate result
disposition	promote, continue research, kill

Novelty without a scorecard does not enter the core.

Revision-2 expansion

Track G — Corpus-complete semantic design

G1. Behavioral parity methodology

Develop relational graph/trace comparison that can establish parity without requiring identical state encodings. Produce reusable correspondence proofs for common TLA+ idioms: stuttering closure, module instantiation, UNCHANGED, fairness, symmetry, views and auxiliary variables.

G2. Corpus-derived language minimization

Determine the smallest typed CML core that can express semantic equivalents of all 80 validated families. Prefer reusable desugarings over one-off operators. Record which source features require theorem-only or symbolic fragments.

G3. Mutation adequacy

Design mutants that expose false parity: missing frame conditions, fairness drift, wrong quorum arithmetic, invalid symmetry, altered deadlock policy and weakened properties. Measure whether source and port fail equivalently.

Track H — Lean and proof-producing verification

H1. Reflective closure/refinement checkers

Implement executable Lean checkers with soundness theorems. Compare explicit proof terms, native reflection and external Rust checking plus imported witness.

H2. Verified encodings

Prove correspondence from CML finite/symbolic semantics to SAT/PB/SMT instances. This is more important than verifying an already detached solver certificate.

H3. Temporal proof library

Bridge Continuum temporal definitions to LeanLTL where compatible; formalize fair lassos, SCC exclusion, rankings and refinement composition.

H4. Proof receipts

Formalize enough receipt identity to prevent epoch/model/property substitution. Evaluate independent environment checking with Lean4Lean-class tools.

Track I — Nominal and parameterized systems

I1. First-occurrence canonicalization

Prove alpha-renaming invariance for fresh IDs and quantify reduction on realistic request/transaction workloads.

I2. Orbit-finite automata

Evaluate nominal automata and finite-support representations for dynamic membership and allocation. Compare with finite symmetry and bounded-ID abstraction.

I3. Cutoff/counterexample lifting

Combine finite corpus models, symmetry-to-quantification, IC3PO-style inference and Lean proof to lift bounded evidence.

Track J — Games and assumptions

J1. Maximal-permissive safety environments

Synthesize environment moves that must be prohibited for a safety property. Minimize and translate them into domain-pack policies.

J2. Fairness/progress games

Synthesize scheduler/network/recovery assumptions under which liveness is realizable. Generate monitors that distinguish system failure from violated environment assumptions.

J3. Byzantine liveness

Investigate strategy/assumption proof patterns for partial synchrony and adversarial scheduling, informed by current Byzantine-liveness verification work.

Track K — Weak memory and hierarchical verification

Verify local concurrent components under execution-graph memory semantics and export atomic contracts to distributed models. Avoid the full Cartesian product. Compare against Loom, RustMC and dedicated litmus suites.

Track L — Checked choreography and generation

Project global CML protocols to role-local automata, Rust endpoint traits and monitors. Prove or translation-validate projection, branch knowledge and communication compatibility.

Track M — Automatic abstraction and proof-guided agents

Use abstract interpretation, CEGAR, invariant inference, interpretation reduction and corpus templates to propose abstractions. Require held-out schedules, mutants and independent refinement evidence before promotion.

Document: docs/07_BENCHMARKS_AND_EVALUATION.md

Benchmarks and Evaluation

1. Evaluation principles

1. Compare semantics, not filenames. A faster engine that checks a weaker model is not a win.
2. Publish bounds and assumptions.
3. Separate startup, exploration, checking, and certificate time.
4. Include hostile cases where each algorithm loses.
5. Reproduce every performance number from a manifest.
6. Measure diagnostic quality, migration cost, and proof burden—not only states/second.

2. Benchmark suites

Suite A — Local concurrency

- bounded MPMC queue;
- lock-free stack;
- once initialization;
- cancellation-safe channel;
- semaphore obligation transfer;
- executor race/loser drain.

Comparators: Loom, Shuttle, GenMC where semantics match.

Metrics:

- schedules/classes explored;
- bug discovery;
- replay;
- memory-model coverage caveats;
- DPOR overhead.

Suite B — Distributed safety

- two-phase commit;
- replicated register;
- primary/backup;
- Raft election/log safety;
- Paxos/Synod;
- lease/fencing service;
- membership reconfiguration;
- deduplicated request processing.

Comparators: TLC, Quint simulator/Apache, Stateright, P when encodings are faithful.

Suite C — Crash consistency

- write-ahead log;
- double-write buffer;
- manifest swap;
- checkpoint/recovery;
- object-store multipart commit;
- queue acknowledgement and persistence.

Metrics include durability-state cardinality and mutation score.

Suite D — Cancellation

- cancellation during reserve;
- nested region cancellation;
- finalizer failure;
- task race;
- cancellation with storage sync;
- cancellation across request/reply obligation.

This suite is a Continuum differentiator; comparators may not have equivalent semantics.

Suite E — Parameterized protocols

- mutual exclusion ring;
- cache coherence abstraction;
- broadcast protocols;
- Paxos variants;
- quorum systems;
- token protocols.

Comparators: Ivy, IC3PO, P verifier, CHC solvers.

Suite F — Liveness

- leader election;
- retry under eventual delivery;
- fair lock;
- two-phase termination;
- cancellation quiescence;
- starvation defect;
- unfair scheduler counterexample.

Suite G — Timed/probabilistic

- lease expiry with skew;
- timeout/retry race;
- randomized consensus toy models;
- failure probability under replication;
- rare split-brain scenario.

Comparators: UPPAAL/IMITATOR, Storm/PRISM.

Suite H — Production trace conformance

- concurrent queue traces;
- storage transaction traces;
- staging cluster traces;
- deliberately incomplete traces;
- timestamp uncertainty;
- instrumentation loss.

Comparator: OmniLink-style total-order solving and conventional linearizability checkers where appropriate.

3. Core metrics

Correctness

- known mutants killed;
- false positives;
- differential mismatches;
- certificate rejection/acceptance;
- replay mismatch;
- property-preservation tests.

Search efficiency

- generated events;
- explored configurations;
- explored interleavings;
- equivalence classes;
- state count;
- reduction ratio;
- time to first violation;
- exhaustive wall time;
- peak RSS;
- bytes per state/configuration;
- parallel scaling.

Proof efficiency

- invariant size;
- certificate size;
- checker time;
- solver time;
- human annotations;
- failed proof attempts;
- parameterized scope.

Conformance

- instrumentation overhead;
- observed-event volume;
- valid completion time;
- minimal core size;
- false “inconclusive” rate;
- production-to-Lab reproduction rate.

Usability

- lines deleted from bespoke DST;
- model LOC;
- abstraction/refinement LOC;
- time to first useful invariant;
- number of semantic concepts users must understand;
- diagnostic task completion;
- CI flake rate.

4. Benchmark manifest

```
id: raft-election-3
semantic_epoch: 1
model: models/raft_election.ctm
scope:
  nodes: 3
  terms: 3
```

```

faults:
  crashes: 1
  partitions: 1
properties:
  - ElectionSafety
engines:
  - reference-explicit
  - dpor
  - unfold
  - smt-bmc
comparators:
  - tlc
  - apalache
expected:
  mutants_killed: [stale-vote, double-leader]

```

5. Fair comparator rules

For each comparator:

- document translation;
- verify initial/successor equivalence on small scopes;
- use equivalent fairness and fault assumptions;
- disclose unsupported constructs;
- run recommended configuration;
- report both raw and normalized results.

No “Rust vs Java” marketing benchmark without data-layout and semantic analysis.

6. Performance gates

Replay

- 100% stable on retained cases;
- deterministic semantic digest across worker counts;
- replay startup under an agreed interactive threshold for small cases.

DPOR

- zero reachability mismatch versus exhaustive baseline on corpus;
- overhead below 20% on serial/no-concurrency cases or auto-disable;
- meaningful reduction on concurrency-heavy suite.

Explicit engine

- exact collision handling;
- deterministic trace;
- memory use competitive with TLC/Stateright on at least half the matched corpus;
- no global performance claim.

Certificate kernel

- checker substantially simpler and faster than search;
- malformed certificates cannot panic or allocate unboundedly;
- independent mutation suite.

Domain packs

- all declared fault behaviors reached;
- forbidden behaviors rejected;
- fidelity tests on supported host configurations.

7. Research kill thresholds

Research lane	Promote threshold
cubical/HDA reduction	$\geq 2\times$ memory or representative reduction on 3/5 target benchmarks without regressions >25% elsewhere
sheaf gluing	solves a modular case conventional assume-guarantee cannot, or cuts proof/exploration cost materially with better diagnostics
topological coverage	statistically useful prediction of new bug classes beyond edge/state/trace coverage
observer DPOR	soundness corpus clean and net win on view-heavy implementations
learned abstraction	checked abstractions reduce total human+compute cost on multiple projects
portfolio ML	improves solved instances/time, never changes claim soundness
rare-event guidance	orders-of-magnitude improvement for at least one realistic failure with calibrated bounds

8. Artifact reproducibility

Every result bundle contains:

- source commit;
- dirty-tree patch;
- toolchain lock;
- CPU/OS metadata;
- model and pack digests;
- command;
- seed/choices;
- raw results;
- processed tables;
- certificate/check log;
- comparator versions.

9. Continuous benchmark tiers

- **PR smoke:** seconds/minutes, mutant subset.
- **nightly:** full safety/replay corpus.
- **weekly:** comparator matrix, scalability, symbolic.
- **release:** fresh evidence for all public claims.
- **research:** expensive campaigns, isolated from release claims.

10. Evaluation report structure

1. semantic equivalence;
2. correctness/mutation results;
3. bug-finding latency;
4. exhaustive performance;
5. proof/certificate burden;
6. model/implementation linkage;
7. migration cost;
8. failure analysis;
9. threats to validity;
10. exact claims supported.

Document: docs/08_RISK_REGISTER.md

Risk Register and Kill Criteria

Note: This file's internal gate citations (e.g. "before G2") use the Revision 2 gate scheme of docs/26_RELEASE_GATES_REV2.md and may not be cited without translation per plan §22 (Rev-2 G2 $\approx$ Rev-3 G4).

Risk scale

- Probability: low / medium / high.
- Impact: moderate / severe / existential.
- Evidence state: observed / plausible / speculative.

R01 — Scope collapse under ambition

Probability: high

Impact: existential

Failure mode: the project simultaneously attempts a language, runtime, TLC replacement, theorem prover, production monitor, distributed checker, and IDE; nothing becomes indispensable.

Controls:

- gate-driven delivery;
- first vertical slice;
- no frontier engine before G2;
- explicit non-goals;
- separate core/product/research lanes.

Kill signal: after G0 effort, no replayable real-code demo or no project willing to migrate its DST.

R02 — Excellent DST, failed TLA+ replacement

Probability: high

Impact: severe

Failure mode: implementation exploration works, but no standalone abstract modeling, arbitrary abstraction, liveness, or refinement.

Controls:

- abstract model language starts before second project;
- formal-methods gate owner;
- G2 explicitly requires model-before-code;
- liveness roadmap.

R03 — Circular trust

Probability: high

Impact: existential for assurance

Failure mode: asupersync execution, CIR generation, and checker share the same bug and agree.

Controls:

- reference evaluator;
- foreign differential oracles;
- independent kernel;
- certificate evidence;
- semantic mutation tests.

R04 — Unsound independence

Probability: medium

Impact: existential

Failure mode: DPOR/unfolding skips a violating execution because a pack or observer analysis declares dependence incorrectly.

Controls:

- conservative default;
- no-reduction differential corpus;
- pack-level commutation tests;
- checked witnesses;
- certified claims fall back to baseline until POR certificate matures.

R05 — Hidden nondeterminism

Probability: high

Impact: severe

Failure mode: dependencies read ambient time/RNG, spawn foreign threads, or perform untracked I/O; replay and exploration are incomplete.

Controls:

- capability lints;
- MIR audit;
- dependency closure report;
- Lab traps;
- production coverage report;
- assurance downgrade.

R06 — Asupersync coupling or instability

Probability: medium

Impact: severe

Failure mode: Continuum depends on private runtime internals or semantic behavior changes frequently.

Controls:

- one adapter crate;
- exact pin;
- semantic hook contract;
- adapter conformance corpus;
- compatibility branches;
- own CIR and replay format.

Kill signal: required hooks force invasive scheduler forks that cannot be stabilized.

R07 — State explosion remains dominant

Probability: certain

Impact: moderate/severe

Controls:

- partial-order primary;
- multi-grain views;
- symmetry;
- symbolic/PDR;
- compositionality;
- explicit budget/inconclusive results.

The project never claims to solve state explosion universally.

R08 — Model language becomes a bad Rust clone

Probability: medium

Impact: severe

Controls:

- mathematical value semantics;
- relational actions;
- no operational async constructs in abstract layer;
- user studies against Quint/TLA+;
- normalized IR independent of syntax.

R09 — Model/implementation mapping burden

Probability: high

Impact: severe

Controls:

- semantic event generation from capabilities;
- derived views;
- multi-grain mapping;
- mapping diagnostics;
- optional synthesis with independent checking.

Kill signal: mapping code approaches implementation size on multiple projects.

R10 — Domain packs lie

Probability: high

Impact: existential for applied claims

Failure mode: “disk” or “network” abstraction excludes relevant behavior.

Controls:

- fidelity profiles;
- ideal/Lab/host refinement;
- mutation/fault coverage;
- empirical conformance;
- pack version in every claim;
- no generic universal disk semantics.

R11 — Liveness becomes intractable

Probability: high

Impact: severe for TLA+ replacement

Controls:

- finite SCC lane first;
- explicit fairness;
- ranking reduction;
- targeted fragments;
- progress diagnostics;
- accepted inconclusive results.

R12 — Certificate system too large

Probability: medium

Impact: severe

Controls:

- start with finite closure;
- streaming formats;
- code-size covenant;
- proof format reuse;
- certificate benchmarks;

- no certificate for every heuristic until justified.

R13 — Semantic IR overgeneralization

Probability: medium

Impact: severe

Failure mode: CIR tries to encode every model class and becomes unusable.

Controls:

- small mandatory core;
- typed extensions;
- domain packs;
- semantic epochs;
- concrete vertical slices.

R14 — “Alien math” theater

Probability: medium

Impact: reputational/severe

Failure mode: sheaves, homology, HDA, category theory appear in marketing without improving the system.

Controls:

- research labels;
- baseline and threshold;
- no correctness claim from heuristic topology;
- kill criteria;
- publish negative results.

R15 — Solver proof gap

Probability: high

Impact: moderate

Failure mode: SMT solver says UNSAT but proof production/checking lacks theory support.

Controls:

- precise TRUSTED_SOLVER label;
- multiple solvers;
- bounded witness replay;
- gradually add Alethe/LRAT/theory checkers;
- never mislabel.

R16 — Rust compiler churn

Probability: high

Impact: moderate/severe

Controls:

- pinned toolchain;
- narrow compiler-internal usage;
- extraction adapters isolated;
- compatibility CI;
- prefer stable metadata over rustc internals where possible.

R17 — Production tracing overhead/privacy

Probability: medium

Impact: severe

Controls:

- semantic sampling;

- payload abstraction/commitments;
- local aggregation;
- configurable event classes;
- overhead budgets;
- explicit monitorability analysis.

R18 — Agent changes spec to fit bug

Probability: high

Impact: existential for trust

Controls:

- spec changes require separate approval;
- agent provenance;
- mutation tests;
- compare property strength;
- no automatic evidence elevation.

R19 — Performance obsession destroys determinism

Probability: medium

Impact: severe

Controls:

- canonical outputs independent of thread schedule;
- deterministic merge;
- thread-count matrix;
- reference mode;
- performance gates do not waive replay.

R20 — Adoption friction

Probability: high

Impact: existential as product

Controls:

- incremental DST-first migration;
- ordinary Rust;
- useful before proof;
- superb counterexamples;
- stable CLI;
- compatibility with Quint/TLA+;
- avoid requiring users to understand every engine.

R21 — Incremental engine substrate cannot support precise invalidation

Probability: medium

Impact: severe

Failure mode: the chosen memoization substrate (salsa-derived or custom; the build-vs-adopt decision is a Phase B ADR per plan §9.1) cannot support the four reuse-edge classes (Exact/Validated/Conservative/Experimental) with precise invalidation; edge classes collapse to Conservative, destroying interactivity (G5).

Controls:

- Phase B build-vs-adopt ADR backed by a spike implementing the four reuse-edge classes and `query.explain_invalidation`, with a measured invalidation-precision baseline;
- Incremental Parity Audit (plan §9.5).

Kill signal: Conservative-collapse on the reference workload.

Project-level kill questions

At each gate ask:

1. Is Continuum deleting more bespoke infrastructure than it creates?
2. Does the same semantic artifact drive at least three modes?
3. Are claims stronger or merely more convenient?
4. Is replay actually perfect?
5. Can a user model before implementation?
6. Can the independent path catch shared bugs?
7. Is the next frontier feature demanded by evidence?
8. Would using Quint plus existing DST be cheaper and equally strong?

A “yes” to question 8 after G2 is a serious failure signal.

Document: docs/09_THREAT_MODEL.md

Threat Model

1. Security objective

Continuum must not create false confidence. Its primary security property is **sound communication of evidence**:

An adversary, malformed artifact, engine bug, pack bug, incomplete trace, or resource limit must not cause a stronger assurance claim than the available evidence justifies.

Availability and confidentiality matter, but an unsound “proved” verdict is the catastrophic failure.

2. Assets

- semantic definitions;
- model/property digests;
- replay integrity;
- certificate validity;
- domain-pack fidelity profiles;
- production traces;
- implementation build identity;
- claim ledger;
- solver/proof artifacts;
- private payloads contained in traces;
- user trust.

3. Adversaries and failure sources

Accidental

- verifier bugs;
- unsound optimization;
- ambiguous semantics;
- hidden nondeterminism;
- incorrect model;
- stale pack;
- incomplete instrumentation;
- integer overflow;
- noncanonical serialization;
- compiler/runtime behavior change.

Malicious

- crafted model/certificate causing panic or resource exhaustion;
- forged production trace;
- malicious third-party pack lying about independence;

- solver returning malformed evidence;
- supply-chain dependency compromise;
- agent modifying properties to make verification pass;
- artifact substitution;
- hash collision attack;
- path traversal in crashpack extraction.

4. Trust boundaries

untrusted:

- models
- imported TLA+/Quint artifacts
- third-party packs
- production trace streams
- solver output
- certificates before checking
- agent output
- crashpacks from others

semi-trusted:

- first-party optimized engines
- asupersync adapter
- first-party packs
- instrumentation runtime

trusted for certified claim:

- semantic epoch definition
- canonical decoder
- property evaluator
- continuum-kernel
- explicitly listed external axioms

5. Threats and controls

T01 — Parser/decoder memory exhaustion

Controls:

- length/depth limits;
- streaming decoding;
- checked arithmetic;
- allocation budgets;
- no recursive descent on unbounded attacker structures;
- fuzzing and corpus tests.

T02 — Hash collision changes reachability

Controls:

- fingerprints are indexes, not identity;
- exact canonical comparison on collision;
- cryptographic digests for artifact identity;
- adversarial collision corpus.

T03 — Unsound POR

Controls:

- conservative dependence;
- no-reduction reference;
- pack commutation tests;
- observer/property class in result;
- certificate/fallback for strong claims.

T04 — Forged trace or artifact substitution

Controls:

- content-addressed manifests;
- hash chain/Merkle root over events;
- build identity;
- optional signing/attestation;
- reject digest mismatch;
- preserve redaction commitments.

T05 — Incomplete production observation presented as validity

Controls:

- instrumentation schema declares observability;
- sequence gaps and dropped buffers are events;
- checker has `INCONCLUSIVE_OBSERVATION`;
- property monitorability analysis;
- no default closed-world assumption.

T06 — Pack claims host semantics it does not provide

Controls:

- versioned fidelity profile;
- host conformance tests;
- platform matrix;
- explicit axioms in assurance result;
- pack can only raise assurance for declared configurations.

T07 — Malicious pack independence rule

Controls:

- sandbox untrusted packs;
- dynamic footprint validation;
- baseline cross-check;
- first-party review for certified mode;
- pack signature/provenance.

T08 — Solver lies or proof checker disagrees

Controls:

- SAT witness replay;
- proof checking;
- solver-trusted label;
- multiple solver diversity;
- preserve raw proof and command.

T09 — Agent weakens property

Controls:

- semantic property diff;
- review gates;
- immutable baseline property digest in CI;
- mutation score;
- agent cannot modify claim ledger or certificate.

T10 — Replay executes hostile code

Controls:

- explicit user action;

- container/sandbox mode;
- network disabled by default;
- filesystem capability restrictions;
- resource/time limits;
- no automatic replay of downloaded artifacts.

T11 — Secret leakage through traces

Controls:

- typed redaction at event schema;
- salted commitments;
- local abstraction;
- field-level retention;
- access control and encryption;
- crashpack scrubber;
- property declared against redacted observations where possible.

T12 — Dependency/unsafe compromise

Controls:

- pinned lockfile;
- dependency allowlist;
- cargo-vet/advisory scanning;
- unsafe boundary audit;
- reproducible build metadata;
- minimal kernel dependencies.

T13 — Semantic downgrade

A newer tool silently interprets an old artifact with weaker semantics.

Controls:

- explicit semantic epoch;
- exact pack digest;
- migration report;
- reject unknown breaking epoch;
- no “best effort” decode for evidence.

T14 — Resource exhaustion mistaken for proof

Controls:

- INCONCLUSIVE_RESOURCE_LIMIT;
- partial exploration artifacts clearly marked;
- no success from frontier exhaustion unless closure is verified;
- certificate checker separately confirms closure.

T15 — Common-mode compiler bug

Controls:

- foreign oracle corpus;
- independent kernel implementation path;
- potentially compile kernel with diverse toolchain;
- mechanized reference;
- serialization-level conformance vectors.

6. Production journal design

Requirements:

- append-only;
- monotonic local sequence number;

- causal predecessor IDs;
- clock uncertainty;
- explicit loss marker;
- schema digest;
- implementation build digest;
- field redaction map;
- integrity chain.

A central collector is not trusted to invent causal edges. It aggregates signed/local evidence.

7. Certificate decoder constraints

- bounded integer widths or arbitrary precision with allocation limits;
- canonical representation rejection;
- duplicate ID rejection;
- cycle/conflict validation;
- no path/URL dereference;
- no executable payload;
- stable error codes;
- checker never panics on input.

8. Security testing

- cargo-fuzz targets for every decoder;
- property tests for canonical encodings;
- malicious graph/certificate generator;
- pack honesty mutants;
- trace loss/reordering/tampering;
- solver proof corruption;
- replay sandbox escape tests;
- supply-chain bill of materials.

9. Incident policy

Any confirmed false-positive success verdict:

1. blocks release;
2. revokes affected claim IDs;
3. publishes affected semantic epochs/packs/engines;
4. supplies artifact scanner;
5. adds permanent mutation/regression;
6. reevaluates whether the optimizing engine remains eligible for certified mode.

Soundness incidents are treated more seriously than crashes or performance regressions.

10. Signing identities

Receipts, intent bundles, and domain packs are signed; plan §18.6 delegates the signing-identity lifecycle here.

- **Minting.** Identities are minted through audited daemon operations. The solo-developer default is a local keypair minted on first use and recorded in the audit log.
- **Trust-root distribution.** Organizational deployments pin an allowed-signers set distributed inside the intent bundle (plan §4.2.1).
- **Rotation and revocation.** Both are audited daemon operations.
- **Loss recovery.** A lost key is not recovered; recovery re-mints under a new identity with an audit-linked supersession record.
- **Verification failure.** A signature that cannot be verified downgrades the artifact to typed unverified provenance rather than failing open — except the plan §4.2.1 CI acceptance check, which fails closed by policy.

Document: docs/10_INTEROP_AND_MIGRATION.md

Interoperability and Migration Strategy

1. Principle

Continuum should win by being useful with existing formal ecosystems before asking users to abandon them. Interoperability is an oracle, migration path, and risk-control mechanism.

2. TLA+ interoperability

Import

Initial path:

TLA+ source

↓ SANY or Apalache frontend

resolved semantic representation

↓ adapter

Continuum typed model/CIR

Do not reimplement SANY before the engine has value.

Compatibility levels:

- parse/import;
- initial-state equivalence;
- successor-set equivalence;
- invariant result equivalence;
- finite liveness equivalence;
- trace round-trip;
- unsupported-feature diagnostics.

Export

Useful exports:

- TLA+ behavior from CIR;
- model skeleton for external review;
- counterexample trace;
- finite state relation;
- proof obligation comments.

Export is not claimed to preserve every source-level construct.

Corpus Differential Tribunal

For compatible corpus:

- enumerate initial states;
- compare normalized successor sets;
- compare state counts;
- compare invariants;
- compare counterexample existence;
- compare fairness/liveness on restricted cases.

3. Quint and Apalache

Priority interoperability:

- ITF trace read/write;
- Quint Connect-compatible state mappings;
- typed IR adapter if stable;
- Apalache JSON-RPC invocation;
- bounded SMT result import;
- model witness replay.

Potential strategy: use Quint as an optional frontend while Continuum's .ctm surface matures.

4. Stateright

Use cases:

- compare executable Rust models;
- import small protocol corpus;
- benchmark trace quality and state storage;
- validate linearizability-style examples.

No attempt to make Stateright models magically production code.

5. P

Potential interchange:

- event/machine models to CIR;
- runtime monitor traces;
- compare inductive verifier on machine-oriented protocols.

P's event queues and monitors can inform domain-pack semantics.

6. Rust verifier integration

Verus/Creusot

Continuum exports local proof obligations:

- pack implementation refines operation contract;
- abstraction function purity;
- canonical encoder injectivity for a type;
- data structure invariants;
- unsafe boundary contracts.

A verified local component can become an atomic/certified summary in higher-level exploration.

Kani/Crux

Use bounded bit-precise checking for:

- serialization;
- arithmetic;
- unsafe/FFI adapters;
- compact state encoding;
- certificate parser;
- pack micro-semantics.

GenMC/Loom/Shuttle

Use for local scheduler/memory-model differential tests. Continuum does not claim to subsume weak-memory verification immediately.

hax/Aeneas

Potentially translate restricted pack/kernel code to Lean/Rocq/F* for independent proofs. Avoid making these translations mandatory for baseline use.

7. Solver interoperability

Protocols:

- SMT-LIB 2.x;
- CHC;
- Alethe;
- LRAT/FRAT where bit-blasted;
- solver model format normalized by replay;
- proof artifacts retained.

External process isolation is default. Embedded solver libraries are optional performance features.

8. Trace interoperability

CIR adapters:

- OpenTelemetry spans/events;
- asupersync trace;
- ITF;
- TLA+ behavior;
- JSON event logs;
- vector-clock logs;
- linearizability histories.

Adapters must state information loss. OpenTelemetry, for example, does not automatically provide complete semantic causality or effect phases.

9. Migrating a bespoke DST

Inventory

Classify existing code:

- scheduler;
- time;
- RNG;
- network;
- storage;
- process/faults;
- scenarios;
- generators;
- invariants;
- replay;
- trace visualization;
- domain fakes.

Mapping

scheduler/time/RNG/replay	→ asupersync Lab
network/storage/process	→ standard Continuum packs
scenarios/generators	→ Continuum scenario API
invariants	→ properties
domain fakes	→ project/domain packs
abstract model	→ .ctm

Differential migration

Run old and new frameworks with common scenarios:

- compare externally visible outcomes;
- compare known bug seeds;
- compare fault reachability;
- retain old system until parity.

Deletion gate

Remove old infrastructure only when:

- known scenarios pass;
- known mutants are killed;
- replay parity is established;
- semantic exclusions are documented;
- CI workflow improves.

10. Compatibility policy

Every adapter has:

- supported version range;
- semantic fidelity level;
- known loss;
- conformance corpus;
- owner;
- deprecation policy.

No adapter is labeled “compatible” as a single percentage.

Document: docs/11_LANGUAGE_AND_DX.md

Model Language and Developer Experience

1. Why a standalone language is necessary

A Rust-only model API would improve implementation linkage but weaken abstraction:

- Rust evaluation order becomes visible;
- mathematical sets/relations become library encodings;
- finiteness is unclear;
- nondeterministic relations become awkward iterators;
- compiler restrictions leak into the model;
- modeling before code becomes less approachable.

Continuum therefore has two surfaces over one semantics:

1. .ctm, optimized for abstract models;
2. Rust annotations/traits, optimized for implementation and views.

2. Language design principles

- relational, not command-sequence-first;
- pure and total by default;
- typed;
- deterministic semantics;
- explicit nondeterminism;
- explicit finite scopes for exhaustive checking;
- arbitrary mathematical abstraction;
- first-class actions, views, fairness, and properties;
- no hidden I/O/time/RNG;
- source maps preserved through normalization;
- simple enough for agents and humans to transform safely.

3. Type universe

Initial:

- Bool;
- bounded/unbounded mathematical Int and Nat;
- finite model atoms;
- tuples/records/variants;
- Option;
- sequences;
- finite sets;
- finite maps;
- relations;
- functions over finite domains.

Later:

- algebraic data types;
- rational/real symbolic values;
- clocks;
- probabilities;
- opaque user sorts;
- quantified parameterized sorts.

Implementation types are not automatically model types. Conversion is explicit.

4. Actions

Actions are relations over pre/post state. Syntactic sugar may look imperative, but lowering is relational.

```
action Commit(n: Node, e: Entry) {
  require role[n] == Leader;
  logs' = logs.put(n, logs[n].push(e));
  unchanged(term, role);
}
```

The checker rejects unspecified state changes unless the action explicitly opts into relational postconditions.

5. Nondeterminism

```
let n = choose Node where alive[n];
let subset = choose Set<Node> where quorum(subset);
```

Choice is semantic, not randomized. Simulation chooses by seed; exhaustive engines enumerate or symbolize.

6. Modules and views

```
view Service from Protocol {
  state {
    committed: Map<Key, Value>
  }

  map {
    committed = derive_committed(Protocol.logs)
  }

  visible action ClientCommit maps Protocol.ApplyCommitted;
  invisible action Protocol.Replicate;
}
```

View checker emits exact obligations.

7. Property syntax

```
invariant Agreement { ... }

transition invariant DurableAck {
  event.kind == ReplyCommitted
  => exists w in history:
    w.kind == StorageSynced
    && w.write_id == event.write_id
    && w <= event
}

eventually RequestCompletes {
  assuming EventuallyStableNetwork, WeakFair(HandleRequest);
  forall r: Request:
    accepted(r) => eventually completed(r);
}

hyperproperty ObservationalDeterminism {
  forall traces a, b:
    same_public_inputs(a, b) => same_public_outputs(a, b);
}
```

Syntax remains provisional until semantics are validated.

8. Static analyses

- type/effect checking;
- finite-domain analysis;
- totality/termination for model functions;
- action read/write sets;
- cone of influence;
- symmetry candidate detection;
- hidden state update;
- unused fairness;
- vacuity;
- inconsistent assumptions;
- state-space cardinality estimate;
- unsupported symbolic fragment.

9. CLI

```
continuum check model.ctm
continuum simulate model.ctm --seed 42
continuum explore model.ctm --engine dpor
continuum verify model.ctm --property Agreement
continuum refine model.ctm --crate ./server
continuum replay crashpack/
continuum explain crashpack/
continuum observe trace.cir --against model.ctm
continuum prove model.ctm --property Progress
continuum claims
```

cargo continuum resolves crate/build context.

10. REPL

Capabilities:

- evaluate expressions;
- inspect initial states;
- list enabled actions;
- step/choose;
- inspect view state;
- ask why action disabled;
- evaluate property;
- fork execution;
- export crashpack.

11. LSP/IDE

- semantic highlighting;
- types and cardinality;
- action dependency graph;
- view mapping;
- state explorer;
- counterexample timeline and causal DAG;
- jump from semantic event to Rust source;
- fairness and assumption display;
- one-click replay;
- proof-obligation panels.

12. Diagnostics

Bad:

Invariant failed.

Required:

CTV2104 Agreement violated

abstract state: chosen[epoch=7] = {X, Y}

minimal causal explanation:

e31 A accepted X in epoch 7

e44 B accepted Y in epoch 7

missing causal fact: B observed epoch 8 before e44, but action guard ignored it

model: models/consensus.ctm:118

implementation: src/replica.rs:442

replay: continuum replay crashpacks/CTV2104

13. Agent-facing protocol

Structured operations:

- model.inspect;
- model.successors;
- trace.slice;
- trace.minimize;
- property.check;
- invariant.check_inductive;
- refinement.check_step;
- pack.explain_effect;
- replay.run;
- mutant.run.

Responses use stable schemas, not terminal prose.

14. Avoiding syntax lock-in

The normalized semantic AST is stable before the surface language. Format and language revisions can lower to the same AST. A formatter and migration tool are mandatory before declaring syntax stable.

Document: docs/12_GOVERNANCE_AND_ENGINEERING.md

Governance and Engineering Discipline

1. Repository constitution

Code policy

- Rust edition/toolchain pinned.
- unsafe forbidden by default.
- deterministic collections in semantic paths.
- no ambient time/RNG in core.
- no platform-dependent hashing in canonical formats.
- no network access in certificate checking.
- dependency additions require rationale and TCB classification.

Semantic policy

- semantic changes require ADR;
- every breaking change increments semantic epoch;
- every pack operation has a normative contract;
- reference semantics precedes optimization;
- ambiguous behavior is an error, not implementation freedom.

2. Decision process

ADRs have statuses:

- proposed;
- accepted;
- superseded;
- rejected;
- experimental.

An ADR must include:

- context;
- decision;
- formal consequences;
- alternatives;
- compatibility;
- security;
- performance hypothesis;
- validation plan;
- rollback.

3. Claim governance

Public docs use controlled verbs:

- observed;
- tested;
- bounded;
- exhaustively checked;
- proved under;
- certificate checked;
- hypothesized.

CI cross-checks claim IDs.

4. Review requirements

Semantic changes

Require:

- reference tests;
- metamorphic tests;
- differential tests;
- updated schemas;
- migration note;
- security review;
- claim impact.

POR/reduction changes

Require:

- no-reduction differential campaign;
- mutation of dependence relation;
- liveness/property-class analysis;
- performance report.

Pack changes

Require:

- fidelity profile;
- host/Lab conformance;
- fault coverage;
- independence review;

- version bump.

Kernel changes

Require:

- two reviewers;
- fuzz corpus;
- mutation tests;
- code-size report;
- no unchecked optimization.

5. Reproducibility

Release assets include:

- source;
- locked dependencies;
- toolchain;
- benchmark manifests;
- claim ledger;
- known unsupported features;
- certificate schemas;
- conformance corpus.

6. Experimental feature policy

Research features are behind explicit flags and cannot emit stable certified claims unless promoted.

Example:

```
--engine cubical-experimental  
assurance.proof = CROSS_ENGINE at most  
warning = "experimental reduction; certified coverage unavailable"
```

7. Compatibility epochs

Separate versions:

- CLI/API semver;
- model-language version;
- semantic epoch;
- CIR format version;
- certificate version;
- pack semantic version;
- crashpack version.

These must not be conflated.

8. Community strategy

Early collaborators should be chosen for:

- real distributed Rust systems;
- willingness to expose known bugs/seed corpus;
- formal-methods expertise;
- storage/network semantics;
- independent skepticism.

Avoid optimizing for broad beginner adoption before the semantic product works.

9. Publication strategy

Potential research outputs:

- causal IR and cancellation calculus;
- observer-sensitive certifying DPOR;

- proof-carrying partial-order prefixes;
- multi-grained refinement for production Rust;
- production partial-order conformance;
- sheaf/cubical experimental results, including negative findings;
- verified certificate kernel.

Each publication artifact should strengthen the production repository rather than become a disconnected prototype.

10. Licensing and ecosystem

Choose a permissive license compatible with asupersync and solver adapters. Keep certificate formats and CIR openly specified. Domain packs may have separate host-specific dependencies, but the core remains lightweight.

11. Release blocker classes

- soundness regression;
- replay instability;
- semantic digest nondeterminism;
- malformed artifact panic in kernel;
- claim-ledger inconsistency;
- undocumented pack behavior change;
- unresolved known false assurance;
- benchmark artifact unreproducible.

Performance regressions are serious but secondary to these.

Document: docs/13_BIBLIOGRAPHY.md

Primary-Source Bibliography

Cutoff: sources reviewed through 2026-07-24.

This is curated for architectural relevance, not exhaustive. Preprints are labeled.

A. TLA+, Quint, P, and protocol-modeling ecosystems

[S01] TLA+

Leslie Lamport, TLA+ home and materials.

<https://lamport.azurewebsites.net/tla/tla.html>

[S02] Apalache

Apalache symbolic model checker, official site and documentation.

<https://apalache-mc.org/>

<https://github.com/apalache-mc/apalache>

[S03] Quint

Quint specification language, official documentation and repository.

<https://quint-lang.org/>

<https://github.com/informalsystems/quint>

[S04] Quint Connect

“Quint Connect: Model-Based Testing for Real Implementations,” 2025; Emerald case study, 2026.

https://quint-lang.org/posts/quint_connect

https://quint-lang.org/posts/quint_connect_emerald

<https://github.com/informalsystems/quint-connect>

[S05] P

P programming language and verification system, official documentation.

<https://p-org.github.io/P/>

[S64] TLC architecture

TLA+ tools codebase architecture.

<https://docs.tlapl.us/codebase%3Aarchitecture>

<https://github.com/tlaplus/tlaplus>

[S65] Quint transpiler architecture

Quint ADR: transpiler architecture and layered IR.

<https://quint-lang.org/docs/development-docs/architecture-decision-records/adr001-transpiler-architecture>

[S67] PObserve

P runtime monitoring/observation materials.

<https://p-org.github.io/P/>

B. Deterministic simulation and implementation checking

[S06] FoundationDB simulation

FoundationDB testing and engineering documentation.

<https://apple.github.io/foundationdb/testing.html>

<https://apple.github.io/foundationdb/engineering.html>

[S07] TigerBeetle VOPR and architecture

TigerBeetle architecture and deterministic simulation.

<https://github.com/tigerbeetle/tigerbeetle/blob/main/docs/ARCHITECTURE.md>

[S08] Loom

Tokio Loom repository and documentation.

<https://github.com/tokio-rs/loom>

<https://docs.rs/loom/latest/loom/>

[S09] Shuttle

Shuttle deterministic concurrency testing.

<https://github.com/aws-labs/shuttle>

<https://docs.rs/shuttle/latest/shuttle/>

[S10] Asupersync

Asupersync repository and README.

<https://github.com/Dicklesworthstone/asupersync>

[S11] Asupersync Lab/trace/DPOR notes

Asupersync reference document for Lab, traces, and DPOR.

<https://github.com/Dicklesworthstone/asupersync/blob/main/skills/asupersync-mega-skill/references/LAB-TRACE-DPOR.md>

[S12] MODIST

Junfeng Yang et al., “MODIST: Transparent Model Checking of Unmodified Distributed Systems,” NSDI 2009.

<https://www.microsoft.com/en-us/research/publication/modist-transparent-model-checking-of-unmodified-distributed-systems/>

[S13] OmniLink

Finn Hackett et al., “Trace Validation of Unmodified Concurrent Systems with OmniLink,” preprint, 2026.

<https://arxiv.org/abs/2601.11836>

[S14] Multi-grained ZooKeeper specifications

Lingzhi Ouyang et al., “Multi-Grained Specifications for Distributed System Model Checking and Verification,” EuroSys 2025.

<https://doi.org/10.1145/3689031.3696069>

<https://arxiv.org/abs/2409.14301>

[S68] TLA+ trace validation

Horatiu Cirstea et al., “Validating Traces of Distributed Programs Against TLA+ Specifications,” SEFM 2024.
https://doi.org/10.1007/978-3-031-77382-2_8
<https://arxiv.org/abs/2404.16075>

C. Partial-order reduction, unfoldings, and true concurrency

[S15] Dynamic Partial-Order Reduction

Cormac Flanagan and Patrice Godefroid, “Dynamic Partial-Order Reduction for Model Checking Software,” POPL 2005.
<https://doi.org/10.1145/1040305.1040315>

[S16] Optimal DPOR

Parosh Aziz Abdulla et al., “Optimal Dynamic Partial Order Reduction,” POPL 2014.
<https://doi.org/10.1145/2535838.2535845>

[S17] Parsimonious Optimal DPOR

Parosh Aziz Abdulla et al., “Parsimonious Optimal Dynamic Partial Order Reduction,” CAV 2024.
https://doi.org/10.1007/978-3-031-65630-9_2
<https://arxiv.org/abs/2405.11128>

[S21] Symbolic complete finite prefixes

Nick Würdemann et al., “Taking Complete Finite Prefixes To High Level, Symbolically,” Fundamenta Informaticae, 2024.
<https://doi.org/10.3233/FI-242196>

[S22A] Modular finite complete prefixes

Agnes Madalinski and Eric Fabre, “Modular Construction of Finite and Complete Prefixes of Petri Net Unfoldings,” 2009.
<https://doi.org/10.3233/FI-2009-148>

[S22] Kleene theorem for higher-dimensional automata

Uli Fahrenberg et al., “Kleene Theorem for Higher-Dimensional Automata,” LMCS 2024.
[https://doi.org/10.46298/lmcs-20\(4:22\)2024](https://doi.org/10.46298/lmcs-20(4:22)2024)
<https://arxiv.org/abs/2202.03791>

[S23] Myhill–Nerode theorem for HDA

Uli Fahrenberg and Krzysztof Ziemiański, “Myhill–Nerode Theorem for Higher-Dimensional Automata,” 2024.
<https://doi.org/10.3233/FI-242194>

[S24A] Logic and languages of HDA

Amazigh Amrane et al., “Logic and Languages of Higher-Dimensional Automata,” preprint, 2024.
<https://arxiv.org/abs/2403.19526>

[S24B] Directed paths in HDA

Martin Raussen, “Strictifying and Taming Directed Paths in Higher Dimensional Automata,” 2021.
<https://doi.org/10.1017/S0960129521000280>

[S24] Sheaf-theoretic distributed tasks

Stephan Felber, Bernardo Hummes Flores, Hugo Rincon Galeana, “A Sheaf-Theoretic Characterization of Tasks in Distributed Systems,” preprint, 2025.
<https://arxiv.org/abs/2503.02556>

D. High-performance model checking

[S18] LTSmin

Gijs Kant et al., “LTSmin: High-Performance Language-Independent Model Checking,” TACAS 2015.
https://doi.org/10.1007/978-3-662-46681-0_61

[S19] Sylvan

Tom van Dijk and Jaco van de Pol, “Sylvan: Multi-Core Decision Diagrams,” TACAS 2015; extended STTT paper.
https://doi.org/10.1007/978-3-662-46681-0_60
<https://doi.org/10.1007/s10009-016-0433-2>

[S20] Saturation

Gianfranco Ciardo, Gerald Lüttgen, Radu Siminiceanu, “Saturation: An Efficient Iteration Strategy for Symbolic State-Space Generation,” 2001.
<https://ntrs.nasa.gov/citations/20010022506>

[S62] PINS architecture

See LTSmin paper and associated PINS materials.
<https://www.tvandijk.nl/publication/kantlmpbd15/>

[S71A] Recomposition

Ian Dardik, April Porter, Eunsuk Kang, “Recomposition: A New Technique for Efficient Compositional Verification,” preprint, 2024.
<https://arxiv.org/abs/2408.03488>

E. Abstract interpretation, induction, parameterization, and infinite state

[S25] Abstract interpretation

Patrick Cousot and Radhia Cousot, “Abstract Interpretation: A Unified Lattice Model for Static Analysis,” POPL 1977.
<https://www.di.ens.fr/~cousot/COUSOTpapers/POPL77.shtml>

[S26] IC3/PDR

Aaron R. Bradley, “SAT-Based Model Checking without Unrolling,” VMCAI 2011.
https://doi.org/10.1007/978-3-642-18275-4_7

[S27] Spacer/global guidance

Hari Govind Vadiramana Krishnan et al., “Global Guidance for Local Generalization in Model Checking,” Formal Methods in System Design, 2024.
<https://doi.org/10.1007/s10703-023-00412-3>

[S28] IC3PO

Aman Goel and Karem Sakallah, “On Symmetry and Quantification: A New Approach to Verify Distributed Protocols,” 2021.
<https://arxiv.org/abs/2103.14831>

[S28A] Automatic Paxos invariant

Aman Goel and Karem Sakallah, “Towards an Automatic Proof of Lamport’s Paxos,” 2021.
<https://arxiv.org/abs/2108.08796>

[S29] Ivy

Microsoft Research Ivy project and CAV tool paper.
<https://www.microsoft.com/en-us/research/project/ivy/>

[S61] Well-structured transition systems

Alain Finkel and Philippe Schnoebelen, WSTS foundations and later ideal-theory work.
[https://doi.org/10.1016/0890-5401\(90\)90009-7](https://doi.org/10.1016/0890-5401(90)90009-7)
<https://doi.org/10.1016/j.ic.2020.104582>

[S71] Thrust

Hiromi Ogawa, Taro Sekiyama, Hiroshi Unno, “Thrust: A Prophecy-Based Refinement Type System for Rust,” PLDI 2025.
<https://doi.org/10.1145/3729333>

F. Liveness and temporal verification

[S30] LVR

Jianan Yao et al., “Mostly Automated Verification of Liveness Properties for Distributed Protocols with Ranking Functions,” POPL 2024.

<https://doi.org/10.1145/3632877>

[S31] Toward Liveness Proofs at Scale

Kenneth L. McMillan, CAV 2024.

https://doi.org/10.1007/978-3-031-65627-9_13

[S31A] Incremental progress model checking

Aaron R. Bradley et al., “An Incremental Approach to Model Checking Progress Properties,” FMCAD 2011.

<https://plv.colorado.edu/papers/fair-fmcad11.html>

[S55] AutoHyper

Raven Beutner and Bernd Finkbeiner, AutoHyper for full HyperLTL.

<https://autohyper.github.io/>

https://doi.org/10.1007/978-3-031-30823-9_8

<https://doi.org/10.1007/s10009-025-00801-5>

G. Refinement and verified distributed systems

[S32] IronFleet

Chris Hawblitzel et al., “IronFleet: Proving Practical Distributed Systems Correct,” SOSP 2015.

<https://www.microsoft.com/en-us/research/project/ironclad/publications/>

[S33] Verdi

James R. Wilcox et al., “Verdi: A Framework for Implementing and Formally Verifying Distributed Systems,” PLDI 2015.

<https://doi.org/10.1145/2737924.2737958>

[S34] Aneris

Aneris project: higher-order separation logic for distributed systems.

<https://iris-project.org/aneris/>

[S35] Grove

Grove: distributed separation logic for realistic systems.

<https://github.com/mit-pdos/gokv>

<https://doi.org/10.1145/3571202>

[S36] Perennial

Perennial: verifying concurrent crash-safe systems.

<https://github.com/mit-pdos/perennial>

[S37] Trillium

Trillium: higher-order concurrent and distributed separation logic/refinement.

<https://gitlab.mpi-sws.org/iris/trillium>

[S53] Strong observational refinement

Hagit Attiya and Constantin Enea, “Putting Strong Linearizability in Context,” DISC 2019; extended 2025 work.

<https://doi.org/10.4230/LIPIcs.DISC.2019.2>

<https://doi.org/10.1007/s00236-025-00500-3>

[S54] Progressive forward simulation

Brijesh Dongol, Gerhard Schellhorn, Heike Wehrheim, “Weak Progressive Forward Simulation Is Necessary and Sufficient for Strong Observational Refinement,” CONCUR 2022.

<https://doi.org/10.4230/LIPIcs.CONCUR.2022.31>

H. Rust verification and semantics

[S38] Verus

Verus official repository and guide.

<https://github.com/verus-lang/verus>

<https://verus-lang.github.io/verus/guide/>

[S39] RustBelt

Ralf Jung et al., “RustBelt: Securing the Foundations of the Rust Programming Language,” POPL 2018.

<https://plv.mpi-sws.org/rustbelt/popl18/>

[S40] Creusot

Creusot official site.

<https://creusot.rs/>

[S41] Aeneas

Aeneas Rust-to-functional translation and verification project.

<https://github.com/AeneasVerif/aeneas>

[S42] Kani

Kani Rust verifier.

<https://model-checking.github.io/kani/>

<https://github.com/model-checking/kani>

[S43] GenMC

GenMC stateless model checker.

<https://github.com/MPI-SWS/genmc>

[S44] Crux

Stuart Pernsteiner et al., “Crux, a Precise Verifier for Rust and Other Languages,” 2024.

<https://arxiv.org/abs/2410.18280>

<https://crux.galois.com/>

[S45] hax/hacspect

hax high-assurance Rust translation.

<https://hax.cryspen.com/>

<https://github.com/cryspen/hax>

<https://hacspect.org/>

[S72] Library-defined capabilities for interior mutability

Federico Poli et al., “Reasoning about Interior Mutability in Rust using Library-Defined Capabilities,” 2024.

<https://arxiv.org/abs/2405.08372>

[S73] Gillian-Rust hybrid verification

Sacha-Élie Ayoun et al., “A Hybrid Approach to Semi-Automated Rust Verification,” 2024.

<https://arxiv.org/abs/2403.15122>

I. Proof certificates and trustworthy checking

[S47] Alethe

Alethe proof format for SMT.

<https://verit.loria.fr/documentation/alethe-spec.pdf>

<https://github.com/SMT-COMP/alethe>

[S48] Carcara

Carcara, Rust Alethe proof checker.

<https://github.com/ufmg-smite/carcara>

[S49] LRAT and verified checking

Nathan Wetzler, Marijn Heule, Warren Hunt, “DRAT-trim/LRAT” line of work and recent verified LRAT checkers.
<https://www.cs.utexas.edu/~marijn/publications/LRAT.pdf>

[S49A] CreuSAT

A SAT solver written in Rust and verified with Creusot.
<https://github.com/sarsko/CreuSAT>

J. Timed and probabilistic verification

[S50] Storm

Storm probabilistic model checker.
<https://www.stormchecker.org/>
<https://github.com/moves-rwth/storm>

[S51] UPPAAL

UPPAAL official site.
<https://uppaal.org/>

[S52] IMITATOR

IMITATOR parametric timed-automata tool.
<https://www.imitator.fr/>

[S56] PRISM

PRISM probabilistic model checker.
<https://www.prismmodelchecker.org/>

[S76] Timed multiparty sessions in Rust

Ping Hou, Nicolas Lagaillarde, Nobuko Yoshida, “Fearless Asynchronous Communications with Timed Multiparty Session Protocols,” ECOOP 2024.
<https://doi.org/10.4230/LIPIcs.ECOOP.2024.19>

K. Effects, sessions, choreographies, and synthesis

[S57] Syntax-Guided Synthesis

Rajeev Alur et al., SyGuS.
<https://www.microsoft.com/en-us/research/publication/syntax-guided-synthesis/>
<https://sygus.org/>

[S58] Formal theory of choreographic programming

Luís Cruz-Filipe, Fabrizio Montesi, Marco Peressotti, 2023.
<https://doi.org/10.1007/s10817-023-09665-3>

[S58A] Real-world choreographic programming

Lovro Lugović and Fabrizio Montesi, 2024.
<https://doi.org/10.22152/programming-journal.org/2024/8/8>

[S59] Refined multiparty protocols

Fangyi Zhou et al., “Statically Verified Refinements for Multiparty Protocols,” OOPSLA 2020.
<https://doi.org/10.1145/3428216>

[S59A] Hybrid multiparty session types

Lorenzo Gheri and Nobuko Yoshida, compositional protocol specification.
<https://arxiv.org/abs/2302.01979>

[S60] Algebraic effects and handlers

Andrej Bauer and Matija Pretnar, “An Effect System for Algebraic Effects and Handlers,” LMCS 2014.
[https://doi.org/10.2168/LMCS-10\(4:9\)2014](https://doi.org/10.2168/LMCS-10(4:9)2014)

[S60A] Higher-order effects

Birthe van den Berg and Tom Schrijvers, “A Framework for Higher-Order Effects & Handlers,” 2023.
<https://arxiv.org/abs/2302.01415>

[S60B] Relational separation logic for effect handlers

Paulo Emílio de Vilhena et al., POPL 2026.
<https://doi.org/10.1145/3776676>

L. Agent-assisted formal verification

[S69] AutoVerus

Chenyuan Yang et al., “AutoVerus: Automated Proof Generation for Rust Code,” 2024.
<https://arxiv.org/abs/2409.13082>

[S70] VeriStruct

Chuyue Sun et al., “VeriStruct: AI-assisted Automated Verification of Data-Structure Modules in Verus,” 2025/2026.
<https://arxiv.org/abs/2510.25015>

[S70A] VeruSAGE

Chenyuan Yang et al., “VeruSAGE: A Study of Agent-Based Verification for Rust Systems,” preprint, 2025.
<https://arxiv.org/abs/2512.18436>

M. Architectural inspiration

[S63] FrankenLean comprehensive plan

Jeff Emmanuel, `COMPREHENSIVE_PLAN_FOR_THE_DESIGN_OF_FRANKEN_LEAN.md`.
https://github.com/Dicklesworthstone/franken_lean/blob/main/COMPREHENSIVE_PLAN_FOR_THE_DESIGN_OF_FRANKEN_LEAN.md

Notes on evidence quality

- Official documentation and peer-reviewed papers are preferred.
- ArXiv entries from 2025–2026 are research signals, not settled facts.
- Tool capability claims must be rechecked against pinned versions before implementation decisions.
- Continuum’s speculative research tracks do not become soundness claims merely because related mathematics exists.

N. Additional 2025–2026 frontier work

[S77] QSM-Cutoff

Yun-Rong Luo, Aman Goel, Karem Sakallah, “QSM-Cutoff: Systematic Derivation of Quantified Cutoff Formulas for Distributed Protocols,” CAV 2025.
https://doi.org/10.1007/978-3-031-98682-6_14

[S78] Await-aware optimal DPOR

Bengt Jonsson, Magnus Lång, Konstantinos Sagonas, “Awaiting for Godot: Stateless Model Checking that Avoids Executions Where Nothing Happens,” Formal Methods in System Design, 2025.
<https://doi.org/10.1007/s10703-025-00479-0>

[S79] Asynchronous runtime-verification decidability

Armando Castañeda, Gilde Valeria Rodríguez, “Asynchronous Fault-Tolerant Language Decidability for Runtime Verification of Distributed Systems,” preprint, 2025.
<https://arxiv.org/abs/2502.00191>

[S80] Verification algorithm selection

Roderick Bloem et al., “Btor2-Select: Machine Learning Based Algorithm Selection for Hardware Model Checking,” CAV 2025.
https://doi.org/10.1007/978-3-031-98668-0_15

[S81] RustMC

Oliver Pearce, Julien Lange, Dan O’Keeffe, “RustMC: Extending the GenMC Stateless Model Checker to Rust,” preprint/tool paper, 2025.

<https://arxiv.org/abs/2502.06293>

[S82] Partial-order conformance via unfoldings

Ariba Siddiqui, Wil M. P. van der Aalst, Daniel Schuster, “Computing Alignments for Partially-ordered Traces Through Petri Net Unfoldings,” preprint, 2025.

<https://arxiv.org/abs/2504.00550>

Xixi Lu, Douwe Geurtjens, “FoldA: Computing Partial-Order Alignments Using Directed Net Unfoldings,” preprint, 2025.

<https://arxiv.org/abs/2506.08627>

[S83] Runtime verification over Mazurkiewicz traces

Martin Leucker, “A Note on Runtime Verification of Concurrent Systems,” 2025.

<https://www.isp.uni-luebeck.de/research/publications/note-runtime-verification-concurrent-systems>

[S84] LRAT-Catcher

Stefan Szeider, “LRAT-Catcher: Importing SAT Solver Certificates into Lean4 by Reflection,” preprint, 2026.

<https://arxiv.org/abs/2607.00815>

[S85] Three-dimensional refinement algebra

Yu Zhang, Jérémie Koenig, Yuting Wang, Zhong Shao, “Unifying Compositional Verification and Certified Compilation with a Three-Dimensional Refinement Algebra,” POPL 2025 artifact.

<https://doi.org/10.5281/zenodo.14065332>

<https://github.com/CertiKOS/rbgs/tree/popl25-artifact>

[S86] Gillian-Rust, PLDI 2025

Sacha-Élie Ayoun et al., “A Hybrid Approach to Semi-Automated Rust Verification,” PLDI 2025.

<https://doi.org/10.1145/3729289>

<https://gillianplatform.github.io/publications/rust.html>

[S87] Partial-order streaming conformance

“Back to the Order: Partial Orders in Streaming Conformance Checking,” Information Systems, 2025.

<https://doi.org/10.1016/j.is.2025.102566>

[S88] HyperLTL counterexamples and explanations

Sarah Winter, Martin Zimmermann, “Tracy, Traces, and Transducers: Computable Counterexamples and Explanations for HyperLTL Model-Checking,” Acta Informatica, 2025.

<https://doi.org/10.1007/s00236-025-00499-7>

[S89] Modular distributed hybrid analysis

Eduard Kamburjan, “Modular Analysis of Distributed Hybrid Systems Using Post-Regions,” Formal Methods in System Design, 2026.

<https://doi.org/10.1007/s10703-026-00491-y>

O. Combinatorial geometry of concurrency

[S90] Median graphs, cube complexes, and event structures

Laurine Bénêteau, Jérémie Chalopin, Victor Chepoi, Yann Vaxès, “Medians in Median Graphs and Their Cube Complexes in Linear Time,” Journal of Computer and System Sciences, 2022.

<https://doi.org/10.1016/j.jcss.2022.01.001>

<https://arxiv.org/abs/1907.10398>

[S91] Domains and event structures

Paolo Baldan, Andrea Corradini, Fabio Gadducci, “Domains and Event Structures for Fusions,” 2017.
<https://arxiv.org/abs/1701.02394>

[S92] Antimatroids in discrete-event systems

Paul Glasserman and David Yao, “Generalized Semi-Markov Processes: Antimatroid Structure and Second-Order Properties,” *Mathematics of Operations Research* 17(2), 1992.
<https://doi.org/10.1287/moor.17.2.444>

[S93] Trace-monoid combinatorics

Cartier–Foata trace-monoid theory and later work on dependence-graph growth and clique automata. One recent example:
<https://www.mdpi.com/2504-3900/123/1/8>

P. Revision-2 corpus, Lean, projection, and verification-frontier sources

[S94] TLA+ Examples corpus

TLA+ Foundation, “TLA+ Examples,” pinned by Continuum revision 2 at commit 91c22ea537853196ed1e03e9ad91693ec37642de. The repository serves as an example library, language-tool corpus, and case-study collection.
<https://github.com/tlaplus/Examples>

[S95] TLA+ Examples corpus tooling

Manifest generation, feature census, model-state recording, PlusCal translation, proof checking, and CI scripts.
<https://github.com/tlaplus/Examples/tree/master/.github/scripts>

[S96] TLAPS

TLA+ Proof Manager implementation and documentation.
<https://github.com/tlaplus/tlapm>

[S97] LeanLTL

Eric Vin, Kyle A. Miller, Daniel J. Fremont, “LeanLTL: A Unifying Framework for Linear Temporal Logics in Lean,” preprint, 2025.
<https://arxiv.org/abs/2507.01780>

[S98] PBLean

Stefan Szeider, “PBLean: Pseudo-Boolean Proof Certificates for Lean 4,” preprint, 2026.
<https://arxiv.org/abs/2602.08692>

[S99] Lean4Lean

Mario Carneiro, “Lean4Lean: Towards a Verified Typechecker for Lean, in Lean,” 2024.
<https://arxiv.org/abs/2403.14064>

[S100] Lean Kernel Arena

Independent Lean kernel/checker interoperability and benchmarking project.
<https://arena.lean-lang.org/>

[S101] Rust-to-Lean verification pipeline

Natalia Klaus, Palina Tolmach, Juan Conejero, “A Rust-to-Lean Verification Pipeline with AI Provers: An Experience Report,” preprint, 2026.
<https://arxiv.org/abs/2605.30106>

[S102] VeruSAGE

Chenyuan Yang et al., “VeruSAGE: A Study of Agent-Based Verification for Rust Systems,” preprint, 2025.
<https://arxiv.org/abs/2512.18436>

[S103] KVerus

Yuwei Liu et al., “KVerus: Scalable and Resilient Formal Verification Proof Generation for Rust Code,” preprint, 2026.
<https://arxiv.org/abs/2605.03822>

[S104] Lean-guided TLA+ proof automation

Yuhao Zhou and Stavros Tripakis, “Towards Language Model Guided TLA+ Proof Automation,” preprint, 2025.
<https://arxiv.org/abs/2512.09758>

[S105] Stateful partial-order reduction

Berk Cirisci et al., “A Pragmatic Approach to Stateful Partial Order Reduction,” 2022.
<https://arxiv.org/abs/2211.11942>

[S106] Data-centric DPOR

Marek Chalupa et al., “Data-Centric Dynamic Partial Order Reduction,” 2016.
<https://arxiv.org/abs/1610.01188>

[S107] DPOR for transaction isolation

Ahmed Bouajjani, Constantin Enea, Enrique Román-Calvo, “Dynamic Partial Order Reduction for Checking Correctness against Transaction Isolation Levels,” 2023.
<https://arxiv.org/abs/2303.12606>

[S108] Structural temporal logic

Eleftherios Ioannidis et al., “Structural Temporal Logic for Mechanized Program Verification,” 2024.
<https://arxiv.org/abs/2410.14906>

[S109] Sound and complete projection for global types

Dawit Tiore, Jesper Bengtson, Marco Carbone, “A Sound and Complete Projection for Global Types,” Journal of Automated Reasoning, 2025.
<https://doi.org/10.1007/s10817-025-09726-9>

[S110] Generalized asynchronous projection

Rupak Majumdar, Madhavan Mukund, Felix Stutz, Damien Zufferey, “Generalising Projection in Asynchronous Multiparty Session Types,” 2021.
<https://arxiv.org/abs/2107.03984>

[S111] Formal choreographic programming

Luís Cruz-Filipe, Fabrizio Montesi, Marco Peressotti, “A Formal Theory of Choreographic Programming,” 2022; mechanized in Coq.
<https://arxiv.org/abs/2209.01886>

[S112] Pirouette

Andrew K. Hirsch and Deepak Garg, “Pirouette: Higher-Order Typed Functional Choreographies,” POPL 2022; mechanized in Coq.
<https://doi.org/10.1145/3498684>

[S113] Multiparty asynchronous session types

Kohei Honda, Nobuko Yoshida, Marco Carbone, “Multiparty Asynchronous Session Types,” JACM 2016.
<https://doi.org/10.1145/2827695>

[S114] Event-structure semantics for multiparty sessions

“Event Structure Semantics for Multiparty Sessions,” Journal of Logical and Algebraic Methods in Programming, 2023.
<https://doi.org/10.1016/j.jlamp.2022.100844>

[S115] Weak-memory formalism survey

Roger C. Su and Robert J. Colvin, “Weak Memory Model Formalisms: Introduction and Survey,” 2026.
<https://doi.org/10.1002/cpe.70484>

[S116] Endive

William Schultz, Ian Dardik, Stavros Tripakis, “Plain and Simple Inductive Invariant Inference for Distributed Protocols in TLA+,” FMCAD 2022.
<https://arxiv.org/abs/2205.06360>

[S117] Scythe

Derek Egolf, William Schultz, Stavros Tripakis, “Efficient Synthesis of Symbolic Distributed Protocols by Sketching,” 2024.
<https://arxiv.org/abs/2405.07807>

[S118] Interpretation reduction

Derek Egolf and Stavros Tripakis, “Accelerating Protocol Synthesis and Detecting Unrealizability with Interpretation Reduction,” 2025.
<https://arxiv.org/abs/2501.14585>

[S119] IC3PO

Aman Goel and Karem A. Sakallah, “On Symmetry and Quantification: A New Approach to Verify Distributed Protocols,” 2021.
<https://arxiv.org/abs/2103.14831>

[S120] Inductive proof slicing

William Schultz, Edward Ashton, Heidi Howard, Stavros Tripakis, “Scalable, Interpretable Distributed Protocol Verification by Inductive Proof Slicing,” 2024.
<https://arxiv.org/abs/2404.18048>

[S121] Shipwright

Derek Leung, Nickolai Zeldovich, Frans Kaashoek, “Shipwright: Proving Liveness of Distributed Systems with Byzantine Participants,” preprint, 2025.
<https://arxiv.org/abs/2507.14080>

[S122] Regular abstractions for array systems

Chih-Duo Hong and Anthony W. Lin, “Regular Abstractions for Array Systems,” POPL 2024.
<https://arxiv.org/abs/2401.02618>

[S123] Automated cutoff-based verification

Shreesha G. Bhat and Kartik Nagar, “Automating and Mechanizing Cutoff-based Verification of Distributed Protocols,” 2022/2023.
<https://arxiv.org/abs/2211.15175>

[S124] Ranking-function liveness verification

Jianan Yao, Runzhou Tao, Ronghui Gu, Jason Nieh, “Mostly Automated Verification of Liveness Properties for Distributed Protocols with Ranking Functions,” POPL 2024.
<https://doi.org/10.1145/3632877>

[S125] IC3Syn

Weining Cao et al., “Synthesizing Inductive Invariants for Distributed Protocols via IC3 and Large Language Models,” preprint, 2026. Candidate invariants are independently checked; the preprint reports TLAPS proofs for unbounded instances.
<https://arxiv.org/abs/2605.24619>

M. Agent-computer interfaces, proof agents, and developer experience

[S126] SWE-agent and Agent-Computer Interfaces

John Yang et al., “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” NeurIPS 2024. The work isolates interface design as a major determinant of coding-agent effectiveness.
<https://arxiv.org/abs/2405.15793>

[S127] Pantograph

Leni Aniva et al., “Pantograph: A Machine-to-Machine Interaction Interface for Advanced Theorem Proving, High Level Reasoning, and Data Extraction in Lean 4,” TACAS 2025.
<https://arxiv.org/abs/2410.16429>
https://doi.org/10.1007/978-3-031-90643-5_6

[S128] AXLE

Jimmy Xin et al., “AXLE: A Cloud Infrastructure for Lean 4 Theorem Proving Utilities,” preprint, 2026.
<https://arxiv.org/abs/2606.26442>

[S129] OProver

David Ma et al., “OProver: A Unified Framework for Agentic Formal Theorem Proving,” preprint, 2026.
<https://arxiv.org/abs/2605.17283>

[S130] LAMP

Santhana Srinivasan R and Maithilee Patawar, “LAMP: Lean-based Agentic framework with MCP and Proof Repair,” preprint, 2026.
<https://arxiv.org/abs/2606.28841>

[S131] LeanDojo

Kaiyu Yang et al., “LeanDojo: Theorem Proving with Retrieval-Augmented Language Models,” NeurIPS 2023.
<https://arxiv.org/abs/2306.15626>
<https://github.com/lean-dojo/LeanDojo>

[S132] SWE-Effi

Zhiyu Fan et al., “SWE-Effi: Re-Evaluating Software AI Agent System Effectiveness Under Resource Constraints,” preprint, 2025.
<https://arxiv.org/abs/2509.09853>

N. Interaction protocols and report formats

[S133] Language Server Protocol 3.18

Microsoft, official Language Server Protocol specification.
<https://microsoft.github.io/language-server-protocol/specifications/lsp/3.18/specification/>

[S134] Debug Adapter Protocol 1.71

Microsoft, official Debug Adapter Protocol specification.
<https://microsoft.github.io/debug-adapter-protocol/specification>

[S135] SARIF 2.1.0

OASIS Static Analysis Results Interchange Format specification.
<https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html>

[S136] MCP explicit state handles

Model Context Protocol SEP-2567, “Sessionless MCP via Explicit State Handles,” Final Standards Track, 2026.
<https://modelcontextprotocol.io/seps/2567-sessionless-mcp>

O. Counterexample explanation and human factors

[S137] Counterexample explanation systematic review

Arut Prakash Kaleeswaran et al., “A systematic literature review on counterexample explanation,” 2022.
<https://arxiv.org/abs/2201.03061>

[S138] Bosch formal-verification explanation study

Arut Prakash Kaleeswaran et al., “A User Study for Evaluation of Formal Verification Results and their Explanation at Bosch,” Empirical Software Engineering, 2023.
<https://arxiv.org/abs/2304.08950>
<https://doi.org/10.1007/s10664-023-10353-4>

[S139] Halpern–Pearl actual causality

Joseph Y. Halpern and Judea Pearl, foundational structural-model definitions of actual causality and subsequent refinements.
<https://www.cs.cornell.edu/home/halpern/papers/causalitybook-ch1-3.html>

P. Incrementality and bidirectional correspondence

[S140] Salsa

Salsa, incremental computation framework for Rust and design documentation.
<https://salsa-rs.github.io/salsa/>
<https://github.com/salsa-rs/salsa>

[S141] Certifying incremental SAT solving

Katalin Fazekas, Florian Pollitt, Mathias Fleury, Armin Biere, “Certifying Incremental SAT Solving,” LPAR 2024.
<https://doi.org/10.29007/pdcc>
<https://easychair.org/publications/paper/TbPs>

[S142] Unrealizability proof synthesis and reusable summaries

Shaan Nagy et al., “Automating Unrealizability Logic: Hoare-Style Proof Synthesis for Infinite Sets of Programs,” 2024.
<https://arxiv.org/abs/2401.13244>

[S143] KBX

Jianhong Zhao et al., “KBX: Verified Model Synchronization via Formal Bidirectional Transformation,” 2024.
<https://arxiv.org/abs/2404.18771>

[S144] Effectful lenses

Ruifeng Xie, Tom Schrijvers, Zhenjiang Hu, “Effectful Lenses: There and Back with Different Monads,” ICFP 2025; formalized in Agda.
<https://doi.org/10.1145/3747523>

[S145] BiGUL

Hsiang-Shang Ko, Tao Zan, Zhenjiang Hu, “BiGUL: a formally verified core language for putback-based bidirectional programming,” PEPM 2016.
<https://doi.org/10.1145/2847538.2847544>

Q. Synthesis, diversity, and verified invention

[S146] SyGuS

Syntax-Guided Synthesis community, standard and benchmark resources.
<https://sygus-org.github.io/>

[S147] Quality-diversity algorithms

Jean-Baptiste Mouret and Jeff Clune, “Illuminating search spaces by mapping elites,” 2015, and subsequent MAP-Elites/quality-diversity literature.
<https://arxiv.org/abs/1504.04909>

[S148] Diversifying to Verify

“Diversifying to Verify: When Task-Equivalent Programs Differ in Verifiability,” preprint, 2026. The paper studies how semantically equivalent programs can differ sharply in downstream formal-verification success, motivating proof-aware quality-diversity search.

<https://arxiv.org/abs/2607.09366>

[S149] Vericoding benchmark

“Vericoding: Benchmarking Code Generation with Formal Verification,” preprint, 2025.

<https://arxiv.org/abs/2509.22908>

[S150] SEVerA

“SEVerA: Specification-Evolving Verification Agent,” preprint, 2026. A contemporary baseline for agents that co-evolve implementation, specification, and proof obligations.

<https://arxiv.org/abs/2603.25111>

[S151] MOUR-QD

Research on uncertainty-aware quality-diversity optimization for robust exploration, used as a comparative baseline for Forge’s repertoire search.

<https://doi.org/10.1145/3712256.3726394>

Document: docs/14_GLOSSARY.md

Glossary

Abstract interpretation — sound approximation framework relating concrete and abstract domains, often through Galois connections.

Action — a relation between pre-state, parameters, and post-state.

Adequate order — ordering used to choose cutoffs in unfolding/complete-prefix construction.

Assurance claim — typed machine-readable statement of what was established and under which assumptions.

Causal Intermediate Representation (CIR) — Continuum’s versioned representation of semantic events, causality, conflict, footprints, observations, and evidence.

Certificate — independently checkable evidence for a verification result.

Configuration — a conflict-free, causally closed set of events.

Conflict — relation indicating events cannot coexist in one execution.

Crashpack — deterministic reproduction artifact for a violation or notable execution.

Domain pack — semantics and handlers for a family of effects such as storage or networking.

DPOR — dynamic partial-order reduction.

Effect phase — lifecycle stage such as proposed, reserved, committed, or aborted.

Event structure — true-concurrency model with events, causality, and conflict.

Fairness — assumption restricting infinite executions, such as weak fairness of an action.

Fidelity profile — explicit statement relating a domain pack to host behavior and listing exclusions.

Footprint — typed semantic resources read, written, transferred, or observed by an event.

Higher-dimensional automaton (HDA) — automaton where higher-dimensional cells represent concurrent execution.

Hyperproperty — property of sets/tuples of traces, not individual traces.

Incremental Parity Audit — differential audit that continuously checks incremental query results against clean recomputation, minimizing and quarantining mismatches (plan §9.5); formerly called the “clean-build Tribunal”.

Inductive invariant — predicate true initially, preserved by every step, and implying a safety property.

Interval pomset — partially ordered multiset of events with interval-order structure.

Linearization — total order consistent with a partial order.

Model scope — finite domains and bounds used for a verification run.

Observation alphabet — events/state visible at a refinement boundary.

Obligation — linear semantic responsibility that must be owned, transferred, or discharged.

Pack axiom — external guarantee assumed rather than proved by Continuum.

PDR/IC3 — property-directed reachability algorithms that infer inductive invariants.

PINS — partitioned next-state interface separating language semantics from analysis algorithms.

Prime event structure — event structure with causality and hereditary conflict.

Progressive refinement — refinement with a well-founded progress condition preventing infinite invisible stuttering.

Refinement view — abstraction from a concrete configuration/system to an abstract model.

Semantic epoch — identifier for normative semantics used by models, traces, and certificates.

Stuttering — concrete step with no abstract-state change.

Strong observational refinement — refinement strong enough to preserve selected hyperproperties/scheduler-sensitive behavior.

Tribunal — the TLA+ corpus oracle harness (ADR-0021) that differentially validates Continuum verdicts against the validated examples corpus. This term refers exclusively to the corpus-oracle harness; the incremental-vs-clean audit formerly called the “clean-build Tribunal” is the **Incremental Parity Audit**.

True concurrency — semantics representing concurrent events without choosing arbitrary interleavings.

WSTS — well-structured transition system, an infinite-state system ordered by a well-quasi-order enabling decidability of some coverability questions.

Zoomable refinement — multiple related abstraction grains used selectively by component or verification goal.

Document: docs/15_COMPETITIVE_MATRIX.md

Competitive Matrix

The matrix is directional, not a marketing score. Capabilities vary by version and model.

Continuum: Master Implementation Plan & Comprehensive Dossier

Capa- bility	TLA+/TLC	Quint/ Apache	P	Stateright	Loom/ Shuttle	Verus/ Creusot/ Kani	Founda- tionDB- style DST	Contin- uum target
arbi- trary ab- stract state	excellent	strong	con- strained	Rust- shaped	poor	code- shaped	poor	excellent
model before code	yes	yes	yes	yes	no	no	no	yes
explicit safety	yes	yes/sym- bolic	yes	yes	schedules	local/code	sampled	yes
live- ness/ fair- ness	strong	partial/ varies	monitors/ proof	limited	limited	limited	sampled	strong tar- get
para- meter- ized proof	TLAPS/ manual	inductive support	verifier	no	no	local proofs	no	restricted automated lanes
pro- duction code execu- tion	no	Connect bridge	generated/ observed	model code	near-real local	actual code	yes	yes
cancel- lation seman- tics	ad hoc model	ad hoc model	machine events	ad hoc	drop/ schedule	code-level	project- specific	structural via asuper- sync
crash/ dura- bility seman- tics	modeled manually	modeled manually	modeled	modeled	poor	local	project- specific	standard packs
deter- minis- tic re- play	TLC traces	traces	systematic tests	yes	yes	counterex- amples	yes	compati- bility sur- face
model/ code refine- ment	manual/ TLAPS	Connect/ testing	integrated modes	manual	no	contracts	no	first-class
pro- duction partial- order valida- tion	external	external	PObserve	no	no	no	logs	first-class target
proof	TLAPS	solver-de-	verifier-	no	no/solver	proof sys-	no	core target

certifi-
cates
mon-
itors
chain

no proofs
mon
schema

no dependent
comment
schema

no dependent
mon
schema

Rust mod-
els

primitives
wrappers

yes

no

proof sys-
tem
specific

no

yes explicit
semantics
packs

Where Continuum should not compete

- replacing interactive theorem provers;
- proving arbitrary mathematical theorems;
- full weak-memory semantics in the first releases;
- being the easiest first formal-methods tool before the core is stable;
- matching every TLA+ language feature;
- outperforming specialized timed/probabilistic tools on every workload.

Defensible wedge

1. asupersync production and Lab integration;
2. cancellation/obligation semantics;
3. reusable network/storage/process packs;
4. causal counterexamples and replay;
5. standalone abstract models;
6. direct refinement into real Rust;
7. typed, certificate-backed assurance.

Document: docs/16_PROOF_OBLIGATIONS.md

Proof Obligation Catalog

This catalog defines stable obligation classes. Diagnostics and certificates refer to these IDs.

Model well-formedness

PO-MOD-001 — Initial satisfiability

There exists at least one state satisfying `Init`, unless emptiness is explicitly expected.

PO-MOD-002 — Action total evaluation

For all well-typed inputs and states, action guard/postcondition evaluation terminates without semantic error.

PO-MOD-003 — State closure

Every enabled action produces a state in the declared state domain.

PO-MOD-004 — Deterministic operator semantics

Pure operator evaluation is independent of implementation iteration order.

Event/CIR

PO-CIR-001 — Acyclic causality

$\leq$ is acyclic.

PO-CIR-002 — Conflict heredity

If $e \# f$ and $f \leq g$, then $e \# g$.

PO-CIR-003 — Configuration validity

Accepted configurations are downward closed and conflict-free.

PO-CIR-004 — Independent diamond

Declared independent enabled events commute in state and relevant observation.

PO-CIR-005 — Canonical encoding

Semantically equal artifacts encode identically for a semantic epoch.

Effects and cancellation

PO-EFF-001 — Phase legality

Effect lifecycle transitions follow the pack state machine.

PO-EFF-002 — Commit uniqueness

A reservation commits at most once.

PO-EFF-003 — Abort invisibility

An aborted effect does not emit a committed observation.

PO-CAN-001 — Obligation ownership

Every live obligation has exactly one owner.

PO-CAN-002 — Transfer causality

Obligation transfer is causally ordered and preserves identity.

PO-CAN-003 — Region close

Closed region implies no live child tasks, finalizers, or obligations.

PO-CAN-004 — Cancellation progress

Under declared responsiveness/fairness, cancellation reaches terminal lifecycle state.

Domain packs

PO-PACK-001 — Choice completeness

Pack choices includes every behavior claimed by its fidelity profile.

PO-PACK-002 — Independence conservatism

Events declared independent satisfy the diamond obligation for the applicable property class.

PO-PACK-003 — Handler refinement

Production and Lab handlers refine the pack semantics under declared host assumptions.

PO-PACK-004 — Fault closure

Fault events preserve pack state well-formedness.

Safety

PO-SAF-001 — Initiation

$Init \Rightarrow Inv.$

PO-SAF-002 — Consecution

$Inv \wedge Step \Rightarrow Inv'.$

PO-SAF-003 — Strength

$Inv \Rightarrow Property.$

Refinement

PO-REF-001 — Initial mapping

Concrete initial states map to abstract initial states.

PO-REF-002 — Visible-step simulation

Every visible concrete step maps to one or more allowed abstract steps.

PO-REF-003 — Stuttering classification

Invisible concrete steps preserve abstract state/observation.

PO-REF-004 — Fault mapping

Concrete faults map to allowed abstract environment steps.

PO-REF-005 — Progress

Infinite concrete invisible behavior is excluded or matched according to the refinement class.

PO-REF-006 — Hyperproperty preservation

The chosen simulation/refinement rule preserves the declared hyperproperty class.

POR/unfolding

PO-POR-001 — Backtracking coverage

Every unexplored dependent reorder has a covered backtracking/source-set representative.

PO-POR-002 — Sleep-set soundness

Sleeping an event does not remove the sole representative of a relevant trace class.

PO-UNF-001 — Prefix completeness

Every reachable marking/configuration has a represented equivalent in the finite prefix.

PO-UNF-002 — Cutoff adequacy

Cutoff order satisfies completeness conditions.

Liveness

PO-LIV-001 — Fairness encoding

Acceptance condition exactly represents named fairness assumptions.

PO-LIV-002 — Ranking well-foundedness

Ranking domain has no infinite descending chain.

PO-LIV-003 — Ranking decrease

Relevant fair progress steps decrease ranking or discharge eventuality.

PO-LIV-004 — No accepting SCC

Finite graph contains no reachable SCC satisfying liveness violation acceptance.

Timed/probabilistic

PO-TIME-001 — Zone soundness

Zone abstraction contains all concrete clock valuations.

PO-TIME-002 — Time-elapse closure

Successor computation accounts for allowed delay.

PO-PROB-001 — Distribution normalization

Probability distributions are nonnegative and sum to one.

PO-PROB-002 — Scheduler quantification

Min/max probability claim uses the declared adversary/scheduler class.

PO-PROB-003 — Fixed-point certificate

Provided value bounds satisfy Bellman inequalities and error bounds.

Production trace

PO-OBS-001 — Trace integrity

Local sequence/integrity constraints hold.

PO-OBS-002 — Completion consistency

Inserted/unordered events respect causality and time intervals.

PO-OBS-003 — Observation sufficiency

The property is monitorable under the instrumentation schema or result is inconclusive.

PO-OBS-004 — Build/model identity

Trace, implementation, packs, and model digests match the claim.

Certificate kernel**PO-KER-001 — Decoder totality**

All inputs yield a verdict/error without panic.

PO-KER-002 — Resource bounds

Declared limits are enforced before allocation/recursion.

PO-KER-003 — Evidence binding

Certificate is cryptographically/structurally bound to model, property, and assumptions.

Document: docs/17_DOMAIN_PACK_CONTRACT.md**Domain Pack Contract****1. Purpose**

A domain pack is a semantic component, not a mock library. It must be precise enough for exploration, refinement, replay, and production conformance.

2. Required contents

```
pack/
├─ PACK.md
├─ schema/
├─ model/
├─ lab/
├─ production/
├─ fidelity/
├─ tests/
├─ mutants/
└─ examples/
```

3. PACK.md requirements

- operation list;
- state model;
- event lifecycle;
- fault algebra;
- ordering guarantees;
- cancellation behavior;
- durability/visibility points;
- independence rules;
- fairness assumptions;
- unsupported behaviors;
- host configurations;
- semantic versioning.

4. Operation schema

Each operation defines:

```
struct OperationContract {
  name: Symbol,
  input_type: Type,
  output_type: Type,
  phases: PhaseMachine,
  resources: FootprintTemplate,
  observations: Vec<ObservationSchema>,
  faults: Vec<FaultSchema>,
  cancel_points: Vec<CancelPoint>,
}
```

5. Three implementations

Ideal

Small mathematical relation used in high-level models.

Lab

Executable deterministic handler exposing every nondeterministic choice.

Production

Host implementation plus semantic instrumentation.

Required relations:

Lab refines Pack

Production observations refine Pack under FidelityProfile

Ideal is abstracted/refined by Pack according to declared direction

6. Fidelity profile example: durable storage

`profile: linux-ext4-local-ssd-v1`

`claims:`

- write becomes visible to process after successful syscall
- sync completion is the modeled durability boundary

`modeled_faults:`

- process crash
- loss of unsynced writes
- write reordering before sync

`excluded:`

- device firmware lies about flush
- latent sector corruption
- kernel/filesystem bugs
- power-loss torn sector below declared atomic unit

`assumptions:`

- mount options: [...]
- device cache configuration: [...]

There is no universal Disk.

7. Network pack baseline

State:

- endpoints;
- message envelopes;
- in-flight multiset;
- partitions;
- connection epochs;
- MTU/frame state;
- delivery history.

Choices:

- deliver;
- delay;
- drop;
- duplicate;
- reorder through choice of deliverable envelope;
- partition/heal;
- connection reset;
- endpoint crash.

Optional profiles:

- reliable FIFO stream;

- QUIC-like stream abstraction;
- datagram;
- adversarial Byzantine.

8. Cancellation contract

Every async pack operation declares:

- reservation point;
- commit point;
- abort behavior;
- finalizer obligations;
- whether cancellation can return before host completion;
- late-completion handling;
- idempotence key;
- replay choice.

9. Independence contract

A pack supplies a conservative predicate and witness:

```
fn independent(a: &Event, b: &Event, observer: &Observer) -> Independence {  
    Independence::Proved(witness)  
    // or Independence::Unknown  
}
```

Unknown means dependent.

Typical storage dependence:

- same object/generation: dependent;
- distinct objects may still depend through global flush/barrier;
- crash event depends on all volatile operations;
- sync depends on writes in its durability domain.

10. Fault algebra

Faults compose only where semantics says they do. A pack defines constraints such as:

- crash clears volatile state;
- recovery increments epoch;
- delayed completion from old epoch is rejected or explicitly modeled;
- partition affects matching routes;
- clock jump changes wall mapping, not monotonic logical time;
- cancellation is not process crash.

11. Pack conformance suite

- model vs Lab successor equality on finite scopes;
- production handler trace validation in controlled integration tests;
- every fault reachable;
- forbidden transitions rejected;
- phase mutants killed;
- independence mutants detected by baseline comparison;
- serialization/replay stable;
- host profile tests.

12. Versioning

- patch: implementation/performance, same semantics;
- minor: additive operation/fault, old behavior unchanged;
- major: changed semantics/fidelity;
- semantic epoch bump if core interpretation changes.

Every crashpack pins exact pack digest.

13. Third-party packs

Untrusted packs can be used for sampled testing. Certified claims require:

- conformance suite;
- review/signature policy;
- independence audit;
- fidelity statement;
- kernel-recognized schema or checked extension proof.

Document: docs/18_CLAIMS_MATRIX.md

Claims Matrix

This document is the seed for a machine-readable ledger. Public claims must never outrun the evidence state.

Evidence states:

HYPOTHESIS

TARGET

OBSERVED

ESTABLISHED - BOUNDED

ESTABLISHED

BLOCKED

REFUTED

Continuum: Master Implementation Plan & Comprehensive Dossier

ID	Claim	Required evidence	Initial state
C001	Same semantic inputs produce the same Lab CIR	cross-thread-count/cross-run corpus	TARGET
C002	Crashpacks replay exactly within a semantic epoch	retained replay corpus, clean processes	TARGET
C003	CIR preserves relevant partial-order behavior	reference semantics and projection theorems/tests	HYPOTHESIS
C004	Asupersync adapter captures all supported non-terminism	API inventory, lint/MIR audits, mutation corpus	TARGET
C005	Baseline DPOR preserves finite safety verdicts	exhaustive differential corpus	TARGET
C006	Exact finite explorer has no fingerprint unsoundness	exact equality and collision injection	TARGET
C007	Abstract model Agreement holds in configured scope	checked finite certificate	TARGET
C008	Runtime implementation refines abstract register in configured scope	checked refinement evidence	TARGET
C009	Continuum replaces one project's bespoke DST	migration metrics and bug corpus	TARGET
C010	Continuum replaces practical TLA+ safety use in two projects	standalone models, refinement, certificates	TARGET
C011	Inductive lane proves an unbounded protocol property	independently checked invariant proof	HYPOTHESIS
C012	Liveness lane handles explicit fairness correctly	differential corpus and ranking/SCC certificates	HYPOTHESIS
C013	Production conformance avoids false total orders	partial-order trace experiments	HYPOTHESIS
C014	Observer-sensitive DPOR improves reduction soundly	property-aware differential benchmarks	HYPOTHESIS
C015	Cubical reduction outperforms optimal DPOR on high-width systems	preregistered benchmark	HYPOTHESIS
C016	Sheaf gluing yields useful compositional diagnostics	direct-SAT comparison and theorem	HYPOTHESIS
C017	Topological coverage improves bug yield	held-out mutant campaign	HYPOTHESIS
C018	Proof-carrying results materially shrink the TCB	independent checker audit and mutation	TARGET
C019	Production instrumentation overhead is acceptable	per-tier real workload benchmarks	TARGET
C020	Agent repairs do not weaken properties/assumptions silently	semantic diff and hidden-mutant evaluation	TARGET

Documentation rule

A generated table maps claim states to allowed wording. Examples:

State	Allowed
HYPOTHESIS	“we hypothesize,” “research target”
TARGET	“designed to,” “planned”
OBSERVED	“observed on [scope]”
ESTABLISHED-BOUNDED	“established within [exact bounds]”
ESTABLISHED	wording matching formal theorem/contract
BLOCKED	no positive public claim
REFUTED	explicitly document the negative result

CI should reject bare “verified,” “sound,” “complete,” “deterministic,” or “production-equivalent” unless linked to an adequate claim row.

Revision-2 claim additions

ID	Claim	Required evidence	Initial state
C021	All 80 validated TLA+ example families have native equivalents	corpus dashboard, port manifests, oracle/proof evidence	TARGET
C022	CML is expressive enough to replace ordinary TLA+ usage in target projects	corpus completion plus real-project modeling studies	TARGET
C023	The semantic triptych avoids circular self-validation	independent path audit and mutation tests	TARGET
C024	Lean receipts faithfully justify Continuum claims	verified encoding/checker, axiom manifest, independent replay	TARGET
C025	Cyclic symmetry preserves the dining model	exhaustive spike over 573 states	OBSERVED-BOUNDED
C026	Observer-indexed independence is monotone under observer coarsening	Lean theorem plus engine differential tests	TARGET
C027	Nominal canonicalization removes only identifier renaming	theorem and fresh-name corpus	HYPOTHESIS
C028	Safety games synthesize actionable environment contracts	benchmark against hand-written minimal assumptions	HYPOTHESIS
C029	Checked projection reduces model/code drift	real protocol study and projection theorem	HYPOTHESIS
C030	Weak-memory local receipts compose safely with distributed models	local refinement theorem and integrated example	TARGET
C031	Proof-producing transformations prevent optimizer-induced false proofs	transformation mutation campaign	TARGET
C032	Inconclusive monitoring is calibrated under missing evidence	synthetic and production trace campaigns	TARGET
C033	Agents can repair concurrent Rust without semantic weakening	locked-spec hidden-mutant benchmark	TARGET
C034	Semiring-valued traversal provides reusable analysis without claim confusion	specialized-baseline equivalence and performance	HYPOTHESIS
C035	Lean seed files are verified	successful pinned lake build, zero sorry, axiom reports	BLOCKED

Document: docs/19_TEST_STRATEGY.md

Test and Validation Strategy

1. Testing layers

unit properties
→ generated semantic litmus tests
→ differential tiny exhaustive tests
→ metamorphic tests
→ mutation campaigns
→ retained crashpack replay
→ cross-engine differential harness
→ performance/evidence gates

The key principle is to test **semantic equivalence**, not just API outputs.

2. Generated transition systems

A small generator creates finite systems with:

- typed state variables;
- guarded transitions;
- explicit read/write footprints;
- independent/dependent pairs;
- conflicts;
- obligations;
- cancellation phases;
- fairness annotations.

For small sizes, enumerate all interleavings and configurations. Compare every optimized engine and reduction against this oracle.

3. Metamorphic relations

Expected preservation:

- alpha-renaming;
- stable reordering of declarations;
- set/map insertion order;
- splitting a deterministic action into stuttering substeps with a valid view;
- joining adjacent internal stutter steps;
- symmetry renaming;
- independent-event swap;
- equivalent guard normalization;
- serialization round trip;
- snapshot restore versus root replay.

Expected non-preservation tests deliberately alter observations, fairness, or effect phases.

4. Mutation testing

Semantic engine

- drop causal edge;
- declare conflicting events independent;
- skip obligation discharge;
- use hash equality only;
- ignore epoch;
- map submitted to stable;
- accept unknown field as old meaning;
- omit fairness edge;
- use timestamp total order.

Protocol corpus

- all first-demo mutants;
- stale term/epoch;
- double counting;
- non-idempotent retry;
- ack-before-durability;
- forgotten loser drain;
- timeout via silent drop;
- restart timer leak;
- recovery livelock.

A mature suite has kill expectations per engine. Not every engine must kill every mutant, but the assurance result cannot overclaim.

5. Differential oracles

Use independently implemented systems where semantics overlap:

- reference model evaluator versus optimized evaluator;
- exhaustive enumerator versus DPOR;
- TLC/Quint/Alpache for model subsets;
- asupersync Lab reports versus Continuum obligation model;
- Kani/Verus/other Rust tools for local components;
- SAT/SMT solvers and proof checkers;
- external temporal/probabilistic tools for extension lanes.

Disagreement halts the relevant claim and creates a minimized fixture.

6. Fuzzing

- parser and canonical encodings;
- CIR validator;
- certificate formats;
- domain-pack commands/faults;
- trace importers;
- solver proof parsers;
- replay state machine;
- schema migrations.

Fuzzing is resource-limited and includes malicious cyclic/oversized artifacts.

7. Determinism matrix

Test each retained scenario over:

- worker counts 1, 2, 8, 32;
- debug/release;
- supported OS/architectures;
- different hash seeds where internal structures allow;
- snapshot intervals;
- root versus snapshot replay;
- process restarts.

Semantic artifacts must be identical or differences must be explicitly non-semantic and normalized away.

8. Performance regression

Benchmarks have confidence intervals, warm/cold distinctions, and hardware manifests. Performance claims require:

- comparison at equal semantics/assurance;
- time to first bug and full closure;
- memory/certificate overhead;
- variance across runs;

- raw data retention.

9. Security validation

- dependency and unsafe audits;
- malformed untrusted trace/certificate inputs;
- denial-of-service limits;
- path traversal in crashpacks;
- solver sandboxing;
- secret-redaction tests;
- signature/provenance verification.

10. Release gates

No release promotes a capability from experimental unless:

- schemas stable for that semantic epoch;
- claims matrix evidence updated;
- all known semantic disagreements resolved/documented;
- replay corpus green;
- mutation thresholds met;
- unsupported cases explicit;
- migration path tested.

Document: docs/20_ADVERSARIAL_DESIGN_REVIEW.md

Adversarial Design Review

This document argues against Continuum as currently conceived. Every objection needs evidence, mitigation, or acceptance.

Objection 1: This is five PhDs and ten products disguised as one repository

Correct. The full vision spans language design, async runtime semantics, model checking, theorem proving, runtime verification, storage modeling, and developer tooling.

Mitigation: the product wedge is much smaller:

asupersync Lab → CIR → systematic exploration → causal crashpack

Standalone models and refinement follow only after one real DST migration. Frontier engines are separate research programs. The project succeeds commercially/operationally before completing the whole vision if it replaces bespoke DSTs with superior diagnostics.

Kill signal: G1 cannot remove substantial bespoke infrastructure from a real project.

Objection 2: “One semantics” is a category error

Production hardware, Rust abstract machine behavior, asupersync runtime, model language, and SMT encodings cannot literally be one operational semantics.

Resolution: one **normative semantic contract** with multiple abstractions and refinements. Implementations remain distinct. The project must never use shared code agreement as proof of correspondence.

Objection 3: Building on asupersync makes adoption tiny

Asupersync is young and nonstandard. Requiring it may make Continuum irrelevant to Tokio-dominated Rust.

Response: asupersync is the high-fidelity native substrate because cancellation/lifecycle semantics are structurally superior. Continuum also defines adapter boundaries and standalone models. Tokio/foreign adapters can exist at lower assurance. Do not weaken the core to chase early ubiquity.

Kill signal: required asupersync hooks become invasive or the runtime cannot sustain production adoption.

Objection 4: A model extracted from code is not abstraction

Correct. Automatic extraction tends to preserve implementation complexity and prove the wrong thing.

Resolution: standalone relational models are mandatory. Rust views connect code to abstraction; they do not replace abstraction.

Objection 5: Partial-order-first semantics complicates everything

Yes. State-based model checking is mature, simple, and often enough.

Response: the project needs partial orders for DPOR, production traces, and causal explanation anyway. CIR can project to a state graph. The proof burden is justified only if reduction/diagnostics outperform a state-first baseline.

Kill signal: CIR overhead dominates and no meaningful partial-order benefits appear by G2.

Objection 6: Domain packs become a new universe of fake models

This is likely. Storage/network fidelity is context-dependent and easy to oversell.

Resolution: profile classes, qualification evidence, adversarial envelopes, explicit unsupported behavior, and claims tied to exact profile hashes. The best default is conservative nondeterminism, not realism theater.

Objection 7: Proof certificates are premature architecture astronautics

Counterexample checking and closed-state witnesses can be built early with modest effort. Full SMT/liveness certificates are not.

Resolution: start with small witness checking. Expand certificate families only when an engine produces strong positive claims. The typed TCB disclosure exists from day one.

Objection 8: Rust is not the right model language

Agreed. Rust is the implementation language and one authoring surface. The standalone model language is relational and mathematical.

Objection 9: State explosion wins

It always can. Continuum does not promise universal verification. It promises a coherent escalation path:

sample → partial-order search → exact finite → symbolic → induction → proof

Every lane reports its scope. Some systems remain beyond reach.

Objection 10: The exotic mathematics is cosplay

It will be unless each idea beats a simpler baseline or produces a theorem/certificate/diagnostic that ordinary techniques cannot.

Resolution: experimental governance, preregistration, negative benchmarks, and deletion. No topology/sheaf/category code in the critical path before evidence.

Objection 11: Runtime trace validation can never confirm correctness

Often true under incomplete asynchronous evidence. Some properties are only refutable, not confirmable.

Resolution: monitorability analysis and Inconclusive. Production evidence complements, never replaces, design-time proof.

Objection 12: The project could verify its own mistaken semantics

This is the deepest risk.

Mitigation:

- independent reference evaluator;
- differential foreign oracles;
- small kernel;
- proof/counterexample certificates;
- mechanized core semantics eventually;
- semantic mutation testing;
- avoid shared optimized code between producer/checker;
- public adversarial corpus.

Objection 13: Developer friction will kill adoption

Explicit capabilities, models, views, packs, and properties are work.

Response: Continuum must delete more bespoke framework code than it adds. Macros/generation can remove ceremony, but semantic declarations remain. Measure integration LOC and time-to-first-bug.

Objection 14: The project will optimize benchmarks and miss reality

Use real migrations, retained historical bugs, hidden mutants, negative workloads, and production traces. Freeze benchmark governance before performance work.

Objection 15: Agentic features create security and epistemic risk

Agents can weaken properties, assumptions, or instrumentation.

Response: agents have no authority over claims. Semantic diffs, hidden mutants, replay, certificates, and human review gate changes.

Decision

Proceed only as a sequence of falsifiable vertical slices. The vision is worth pursuing because the semantic center is coherent; it is not licensed to skip gates because the end state is exciting.

Revision-2 objections

Objection 12: The TLA+ Examples corpus will turn Continuum into a compatibility clone

It could. The remedy is semantic parity rather than syntax parity. The corpus controls completeness, not UX or architecture. Features are generalized into reusable semantics and libraries; port-specific operators are rejected.

Kill signal: corpus work produces a growing pile of named exceptions instead of shrinking shared infrastructure.

Objection 13: Lean will consume the project before users get value

This is a serious risk. The formalization boundary is semantic load-bearing walls and certificate checkers, not the optimized runtime or every domain pack implementation.

Kill signal: proof work repeatedly blocks executable counterexamples without protecting a real claim. Narrow the theorem surface, use per-instance translation validation, and preserve the proof receipt contract.

Objection 14: “Independent paths” will duplicate everything

Some duplication is intentional. A tiny reference evaluator and certificate checker are cheaper than discovering that all engines share one semantic bug. Independence is concentrated at trust boundaries, not every utility function.

Objection 15: 80 examples is a vanity metric

It becomes vanity if ports only parse or reproduce expected answers. Required parity includes behavior, failures, temporal/refinement semantics, mutations and theorem intent. The corpus is necessary but not sufficient; real asupersync refinements remain mandatory.

Objection 16: Observer-indexed independence is too subtle to trust

Correct. The default remains conservative dependence. Observer-specific reduction is opt-in until preservation theorems, witness checking and differential campaigns mature.

Objection 17: Nominal sets, cubical complexes and sheaves are academic distraction

They are quarantined research lanes with baselines and kill criteria. First-occurrence renaming, ordinary DPOR and SAT unsat cores are the baselines. No exotic abstraction enters the critical path on aesthetic grounds.

Objection 18: Generated protocol interfaces will force users into an actor DSL

Projection generates traits/types/monitors, not an entire architecture. If idiomatic Rust cannot implement the generated seam cleanly, narrow or remove generation.

Objection 19: A proof receipt can create false confidence through provenance theater

A receipt is useful only if its checker recomputes closure hashes, validates transformation evidence and links a Lean theorem to the original semantics. JSON fields and signatures alone prove nothing.

Objection 20: Weak-memory support explodes the scope again

Weak memory is a separate hierarchical lane. SC is acceptable for abstract corpus ports. Real lock-free refinement requires a local receipt; distributed verification consumes the atomic summary.

Objection 21: Agents will “fix” bugs by changing the spec

Property, assumption, observer, bound and fault-model changes are privileged. Repair evaluation locks them and emits semantic diffs. Any such change requires explicit review.

Document: docs/21_NOVEL_CONTRIBUTIONS.md

Candidate Novel Contributions

This distinguishes actual research contributions from competent integration.

Contribution A: Partial-order semantic substrate spanning model, runtime, and production

Existing systems usually begin with one of:

- abstract transition systems;
- controlled runtime schedules;
- program traces;
- process-mining event logs.

Continuum proposes one typed causal representation that can project into all of them, with checkable projection obligations.

Novelty risk: event structures/partial orders are old; the contribution must be the cross-layer contract and evidence architecture, not the data structure alone.

Contribution B: Cancellation and obligation calculus tied to a production runtime

Structured cancellation, reserve/commit effects, region ownership, and quiescence become formal state and progress obligations.

Novelty risk: linear/session/resource logics already exist. The publishable result needs a precise calculus, sound mapping to asupersync, useful automation, and bugs/results beyond existing tools.

Contribution C: Observer-sensitive partial-order reduction

Dependence is parameterized by property observers, fairness, obligations, and fault semantics rather than only concrete memory/resource conflict.

Novelty risk: property-driven POR exists. The advance must be the rich distributed/runtime observer model and certifying implementation.

Contribution D: Proof-carrying partial-order closure

An unfolding/DPOR exploration emits compact evidence that all relevant causal classes are covered, checked independently.

Novelty risk: model-checking certificates and unfolding cutoffs exist. The contribution must improve certificate practicality for runtime-derived systems.

Contribution E: Multi-grain refinement with executable counterexample transport

Counterexamples move between service, protocol, operational, runtime, and production views; infeasible witnesses drive localized grain refinement.

Novelty risk: CEGAR and multi-grain specs exist. The novel part is unified authoring/runtime replay and causal diagnostics.

Contribution F: Partial-order production conformance plus instrumentation synthesis

Validate incomplete interval/causal traces without fake total ordering; analyze monitorability; synthesize minimal probes needed to decide a property.

Novelty risk: trace validation and process conformance are active fields. Instrumentation synthesis and direct Lab reproduction may be differentiators.

Contribution G: Causal Cubical Reduction

Use higher-dimensional cells to collapse families of commuting distributed effects and preserve observer-relevant directed behavior.

Novelty risk: highest. It may fail entirely. It needs a clear preservation theorem and benchmark wins over optimal DPOR/unfoldings.

Contribution H: Sheaf-based refinement gluing and blame

Local refinement witnesses glue to a global witness; obstruction/minimal ungluable covers diagnose inconsistent composition.

Novelty risk: strong mathematics, unclear engineering payoff. Direct SAT is the baseline.

Contribution I: Evidence-domain algebra

Partial verification artifacts form a composable information order with safe joins, resumable/distributed checking, and no scalar confidence collapse.

Novelty risk: related to proof lattices/domain theory and build systems. Value depends on clear semantics and real artifact reuse.

Contribution J: Proof-gated autonomous repair

Agents consume causal failures and emit proof-carrying repairs subject to neighborhood closure and semantic-diff gates.

Novelty risk: integration novelty unless evaluated on difficult real distributed bugs with low false-fix rates.

Publication sequence

1. **Cancellation + CIR + asupersync mapping.**
2. **Partial-order runtime/model refinement and causal crashpacks.**
3. **Production partial-order conformance and instrumentation synthesis.**
4. **Certifying observer-sensitive DPOR/unfoldings.**
5. **Multi-grain refinement.**
6. **Cubical/sheaf methods only after positive evidence.**

A failed hypothesis is still valuable if the benchmark and negative result are rigorous.

Revision-2 candidate contributions

Contribution K: Corpus-governed semantic replacement rather than source compatibility

Use the TLA+ Examples corpus as a behavioral/theorem contract for a different, typed, implementation-linked environment. The contribution is a measurable definition of “replaces TLA+ for this domain.”

Contribution L: Semantic triptych with independent evidence paths

Model, real program and proof remain first-class, with typed edges and proof receipts. This avoids both drift and circular self-validation.

Contribution M: Observer lattice as a shared optimization/monitoring object

The same observer refinement structure governs DPOR independence, state slicing, refinement granularity, production observability and cache reuse.

Contribution N: Assumption synthesis connected to domain packs and production monitors

Game-derived environment contracts become executable fault policies and monitored deployment assumptions rather than isolated synthesis output.

Contribution O: Nominal causal verification for fresh identifiers

Combine alpha-equivalent fresh-name quotienting with causal event structures, preserving order-sensitive observers and runtime replay.

Contribution P: Checked choreographic projection to cancel-correct Rust

Project global protocol models into asupersync role interfaces, lifecycle-aware monitors and refinement targets.

Contribution Q: Proof-receipt supply chain for model checking

Bind semantic closure, reductions, solver certificates, Lean theorem/axiom metadata and independent checker identity into a reusable artifact.

Contribution R: Property-directed abstraction repair loop for agents

Agents propose views/invariants/ghost state; CEGAR, mutants and proof obligations prevent trace-fitting and semantic weakening.

Document: docs/22_TLA_EXAMPLES_COMPATIBILITY_PROGRAM.md

TLA+ Examples Compatibility Program

Mandate

Continuum 1.0 must provide native semantic equivalents for every CI-validated family in the pinned `tlaplus/Examples` corpus. This is the strongest practical defense against building a beautiful system that handles only Continuum-shaped problems.

The upstream repository states that it is simultaneously:

- an example library;
- a diverse development/testing corpus for TLA+ tools;
- a collection of formal-specification case studies.

Continuum adopts all three roles.

Snapshot

Pinned commit: 91c22ea537853196ed1e03e9ad91693ec37642de

Validated families: 80

Additional tracked examples: 39

README-indicated proof-bearing families: 34

PlusCal or PlusCal-variant families: 22

TLC-model families: 77

Apalache-indicated families: 28

Counts are generated from `corpus/tla-examples/validated-examples.csv`, transcribed from the pinned README. The Tribunal's first bootstrap task is to regenerate these facts directly from upstream manifests and fail on drift.

Why this changes the architecture

The corpus proves that “replace TLA+” is not equivalent to “build an explicit-state protocol checker.” It forces:

1. arbitrary mathematical abstraction;
2. procedural and relational authoring;
3. finite and symbolic domains;
4. stuttering and temporal logic;
5. fairness and liveness;
6. model configuration, symmetry, views, aliases, constraints, and deadlock policy;
7. refinement with auxiliary variables;
8. theorem proving;
9. distributed, shared-memory, storage, network, probabilistic, and hardware domains;
10. deliberately failing models.

This motivates the semantic-fragment architecture and the Lean proof plane.

Port artifact

Every family gets a directory with five independent layers:

```
upstream source lock
native Continuum model
behavioral correspondence
```

oracle/evidence artifacts
Lean theorem package (when required)

Selected distributed/concurrent cases add an asupersync implementation and P5 refinement evidence.

Equivalence criteria

Values

A correspondence maps TLA+ values to canonical Continuum values. It must preserve equality, set/function membership, record/sequence structure, and model-atom identity required by the model.

States

State correspondence may be:

- bijective;
- projected through auxiliary-variable erasure;
- relational where prophecy/history variables are involved;
- quotient-based under symmetry.

Actions

Action labels may map one-to-one, many-to-one, or to stuttering. A hidden concrete action is not ignored informally; it is included in the refinement contract.

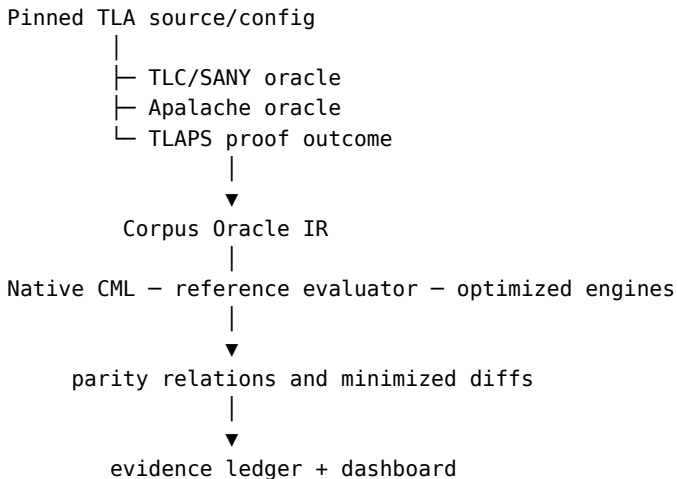
Behaviors

For P3, correspondence covers infinite behaviors, stuttering, fairness, and temporal acceptance. Finite trace agreement alone is insufficient.

Proofs

For P4, theorem intent and dependencies are mapped to Lean. The proof can be structurally different. What matters is the theorem over the corresponding semantics and a checked bridge to the executable model.

Tribunal architecture



The oracle IR is not CIR. Corpus Oracle IR describes state/action/value/temporal facts from foreign tools. CIR remains Continuum's native causal execution representation.

Feature-driven scheduling

Ports are scheduled to retire unknowns:

- Wave 0 freezes values, actions, stuttering, BFS and basic proofs.
- Wave 1 freezes procedural lowering, deadlock, fairness and shared-memory symmetry.
- Wave 2 freezes asynchronous messaging, topology and termination.
- Wave 3 freezes consensus, Byzantine adversaries, transactions and parameterized induction.
- Wave 4 freezes storage, probability and larger composition.
- Wave 5 proves integrated scalability on TCP, German protocol, and TLC's own checker model.

How ports are validated

Each port must include:

- positive reference run;
- property mutation that must fail;
- transition mutation that changes the expected graph/verdict;
- semantic metamorphisms;
- at least one minimized disagreement fixture created during development;
- deterministic reproduction;
- performance budget and state-space facts.

What we do not promise

- automatic translation of all TLA+ source in 1.0;
- identical error messages;
- identical search order;
- equal internal state count when auxiliary encodings differ;
- a Lean translation of every TLAPS proof script;
- support for arbitrary Java operator overrides.

We promise equivalent formal work and explicit differences.

1.0 acceptance dashboard

The release dashboard must show all 80 rows with:

P0 inventory	80/80
P1 native port	80/80
P2 finite parity	all rows requiring P2+
P3 temporal/refinement	all rows requiring P3+
P4 theorem parity	all proof-bearing required rows
P5 runtime exemplars	selected minimum set across domains
unclassified diffs	0
vacuous properties	0
replay failures	0

Document: docs/23_LEAN4_FORMALIZATION_PROGRAM.md

Lean 4 Formalization Program

Role

Lean 4 is Continuum's mathematical court of final appeal. It validates semantics, reductions, encodings, and certificates; it does not replace the Rust product or become the normal execution engine.

The dossier pins v4.32.1, observed as the current stable patch on 2026-07-24. The pin advances only through a proof epoch that rebuilds all theorem packages and records any changed axioms or performance.

The metatheory stack

- M0 Logic-neutral mathematics
 - sets, relations, orders, finite maps, group actions
- M1 Transition semantics
 - init, step, reachability, behaviors, observations
- M2 Temporal semantics
 - stuttering, LTL-X, fairness, omega acceptance
- M3 Refinement
 - simulations, relations, composition, hyperproperties
- M4 Causal semantics
 - event structures, configurations, conflict, intervals

M5 Effect semantics
obligations, cancellation, durability, network/storage faults

M6 Algorithms and certificates
closure, DPOR, symmetry, SCC, ranking, solver encodings

M7 Concrete bridge
CIR wire format and asupersync event adapter contracts

Dependencies point downward. Corpus proofs sit above M1–M7.

Trusted computing base

The high-assurance claim ultimately trusts:

- Lean kernel and compiler/runtime to the degree stated by Lean’s own trust model;
- Continuum’s Lean definitions;
- explicit axioms reported by theorem manifests;
- certificate bytes and verified parser/checker/encoding path.

The optimized Rust explorer, SMT/SAT solver, agents, and foreign TLA+ tools are not trusted.

For especially sensitive results, Lean4Lean or another independent checker can validate generated .olean environments, adding diversity to the proof-checking path.

Proof families

Safety

Core theorem:

$$\begin{array}{l} \text{Init} \subseteq I \\ \text{Post}(I) \subseteq I \\ I \subseteq \text{Safe} \end{array}$$

Reachable $\subseteq$ Safe

Certificate instantiations:

- explicit closed set;
- symbolic inductive formula;
- quantified invariant;
- compositional invariant graph.

Refinement

Initial core:

$$\begin{array}{l} \text{InitC}(c) \Rightarrow \text{InitA}(\alpha c) \\ \text{StepC}(c, c') \Rightarrow \alpha c = \alpha c' \vee \text{StepA}(\alpha c, \alpha c') \end{array}$$

Extensions cover relational refinement, event projection, prophecy/history variables, fair refinement, and strong observational refinement for hyperproperties.

Liveness

Finite certificates:

- absence of fair accepting SCC;
- ranking on SCC condensation or helpful transitions;
- Streett/justice progress obligations.

Symbolic certificates:

- well-founded rank relation;
- fairness premises;
- action-local decrease/nonincrease lemmas;
- proof graph linking progress obligations.

Reduction

- symmetry quotient preserves init, transitions and property;
- observer-indexed commutation preserves observation language;
- DPOR source sets cover all relevant Mazurkiewicz classes;
- unfolding prefix is complete for target property;
- abstract interpretation is sound via Galois connection/simulation.

Cancellation

- phase transitions are well-formed;
- every reservation is committed or aborted;
- obligation ownership is linear;
- drain/finalize preserves safety;
- under responsiveness and finite budgets, rank decreases to quiescence;
- cancellation-aware refinement maps partial effects correctly.

Reflective certificate pipeline

Recent Lean work demonstrates the practicality of verified reflective checkers for LRAT and pseudo-Boolean certificates. Continuum follows this pattern:

1. define a Boolean checker in Lean;
2. prove `check = true` $\rightarrow$ proposition;
3. compile checker to native code;
4. verify large certificate efficiently;
5. obtain a composable Lean theorem.

The harder and more important task is the verified encoding from Continuum semantics to the solver problem.

Proof engineering conventions

- no sorry in release packages;
- theorem names include semantic epoch where meaning can change;
- `#print` axioms captured in machine-readable manifests;
- proofs are sliced by action/property dependency;
- executable examples accompany abstract definitions;
- definitions are kept reducible enough for reflection but opaque where abstraction stability matters;
- proof performance is benchmarked and regression-gated;
- model-derived finite facts are imported as certificates, not enormous generated source terms.

Corpus theorem program

The proof-bearing TLA+ examples are divided into patterns:

1. elementary arithmetic and finite combinatorics;
2. loop invariants/program correctness;
3. inductive distributed safety;
4. refinement and auxiliary variables;
5. termination/liveness;
6. graph/topology properties;
7. protocol families with reusable quorum/broadcast libraries.

The goal is not 34 unrelated proof ports. It is a reusable library where later corpus theorems collapse to instantiations of established patterns.

Initial theorem milestones

1. finite closure safety for DieHard TypeOK;
2. shortest path witness validity;
3. deadlock witness validity for Dining Philosophers;
4. reserve/commit register stuttering refinement;
5. mutual exclusion invariant schema;

6. finite symmetry quotient theorem;
7. fair-lasso certificate theorem;
8. quantified quorum intersection library;
9. cancellation obligation conservation;
10. proof-producing DPOR safety theorem.

Formalization risk controls

Lean can consume the project if every implementation detail is formalized too early. The rule is:

Formalize semantic load-bearing walls and certificate checkers; validate changing optimizations per instance.

Each proof workstream has a product gate, executable test, theorem statement, and maximum tolerated proof-maintenance cost.

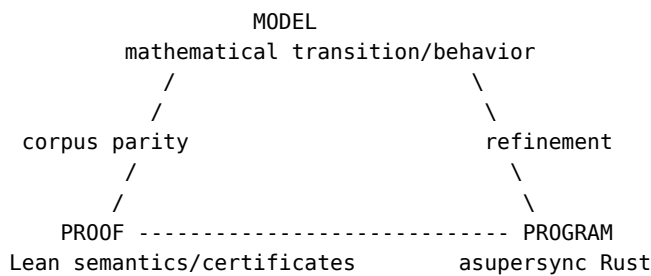
Document: docs/24_REVISION_2_ARCHITECTURE.md

Revision-2 Architecture: The Semantic Triptych

Core shift

Revision 1 centered CIR. Revision 2 keeps CIR but recognizes that a causal interchange format alone cannot replace TLA+ or justify its own optimizations.

The new center is a triangle of independently meaningful artifacts:



CIR is the language spoken on the Program edge and by causal engines. CML typed core is the language spoken on the Model edge. Lean definitions govern theorem meaning on the Proof edge.

Six planes

1. Authoring plane

- CML relational model language;
- procedural algorithm surface;
- Rust model/refinement attributes;
- TLA+/Quint importers;
- generated agent APIs.

2. Semantic plane

- typed values and expressions;
- transition systems and infinite behaviors;
- event structures/configurations;
- observers/views;
- temporal/fairness semantics;
- faults, obligations, cancellation and durability.

3. Execution plane

- exact reference evaluator;
- asupersync production and Lab adapters;
- domain packs;
- replay and snapshots;

- semantic event journal.

4. Verification plane

- simulation/fuzzing;
- source/optimal DPOR;
- unfoldings/HDA experiments;
- explicit and symbolic state engines;
- PDR/CHC/parameterized lanes;
- liveness and games;
- probabilistic/timed lanes.

5. Evidence plane

- crashpacks;
- closure/invariant/refinement/reduction certificates;
- proof-producing compiler transformations;
- Lean reflection/import;
- evidence ledger and assurance lattice.

6. Corpus/Tribunal plane

- TLA+ Examples parity;
- differential foreign oracles;
- operator-level semantic fuzzing;
- mutation and metamorphic suites;
- performance and proof regressions.

Semantic fragments

CML does not lie about executability. Every expression/action/property carries fragment requirements:

Finite	exact bounded evaluation
Symbolic	solver-representable mathematics
Temporal	infinite behavior/fairness
Probabilistic	distributions/MDP/game semantics
Theorem	Lean-only propositions and proof obligations
Runtime	controlled concrete effects

Elaboration computes the least required fragment and reports why a backend applies or does not.

One model, multiple grains

A project can define:

```
ServiceSpec
  refines <- ProtocolSpec
  refines <- OperationalSpec
  refines <- AsupersyncProgram
  observed by <- ProductionTrace
```

Each edge carries:

- state relation/view;
- event map;
- hidden/stuttering actions;
- assumptions and fairness;
- proof strategy;
- evidence status.

This graph is a first-class build artifact. “The implementation matches the model” is never an unqualified boolean.

Independence and observer lattice

Views/properties induce an observer lattice. A fine observer preserves more distinctions; a coarse observer permits more commutations and abstraction.

This lattice drives:

- property-specific DPOR;
- trace canonicalization;
- state slicing;
- refinement granularity;
- production observability requirements;
- cache reuse.

The relationship is formalized so performance optimization remains subordinate to semantics.

Proof-producing pipeline

Every semantic pass declares its preservation mode:

```
source
→ elaborated core      equivalence
→ finite instance      instantiation theorem
→ property slice       property preservation
→ symmetry quotient    bisimulation
→ DPOR/unfolding       trace/property preservation
→ solver formula       equisatisfiability
→ proof certificate    checked theorem
```

The user sees the chain attached to the result.

Why this can replace bespoke DST and TLA+

DST replacement comes from the Program/Execution/CIR side:

- controlled time, scheduling, faults, network, storage, cancellation, replay.

TLA+ replacement comes from the Model/Temporal/Proof side:

- abstract authoring before code, arbitrary mathematical views, exhaustive/symbolic checking, fairness/liveness, refinement, proofs.

The triangle connects them without requiring duplicated hand-maintained models.

Document: docs/25_SEMANTIC_FEATURE_MATRIX.md

Semantic Feature and Backend Matrix

Legend: N native baseline, S supported by specialized lane, P proof-only initially, R research, – rejected/meaningless.

Continuum: Master Implementation Plan & Comprehensive Dossier

Feature	Reference	Explicit	DPOR	SMT/PDR	Lean	Runtime
finite sets/ maps/se- quences	N	N	N	S	N	N
un- bounded integers	symbolic	bounded	bounded	N	N	concrete
uninter- preted sorts	symbolic	model-bound	model-bound	N	N	mapped
higher-or- der opera- tors	N	elaborated	elaborated	restricted	N	—
recursive definitions	guarded	finite	finite	restricted	N	—
relational nondeter- minism	N	N	N	N	N	controlled
stuttering	N	N	N	N	N	event hiding
weak/ strong fairness	N	N	S	S	N	assumption
liveness	N	SCC/lasso	fair DPOR	ranking/PDR	N	monitor/in- conclusive
symmetry	N	N	N	N	proof	IDs/atoms
fresh atoms/ names	nominal	R	R	S	N	IDs
refine- ment	N	bounded	causal	symbolic	N	trace/runtime
hyper- properties	product	S	restricted	S	N	partial evi- dence
real time	zones	S	interval R	SMT	N	virtual/real
probabil- ity	distributions	MDP	probabilistic POR R	S	N	sampled
adversar- ial choices	games	game	game POR R	game solving	N	fault model
crash/ durability	domain pack	N	N	bounded	contracts	N
cancellat- ion/oblig- ations	calculus	N	N	bounded	N	asupersync
source- level Rust	—	—	concrete SMC	CHC/BMC adapters	contracts	N

Capability inference

The elaborator produces a diagnostic such as:

Model requires:

Finite(Set[Node]) because action Deliver enumerates messages
 Temporal(StrongFairness[Deliver]) because property EventualCommit depends on SF
 Runtime(Storage@v2) because concrete view references sync completion

Applicable engines:

reference, explicit, fair-DPOR, liveness-SCC, Lean

Not applicable:

pure IC3: sequence operator unsupported by selected encoding
 probability: model has no probabilistic choices

Definedness

CML separates type correctness from semantic definedness. Division by zero, sequence indexing, recursive nontermination, non-enumerable choice, and opaque foreign operations produce proof obligations or rejection—not arbitrary host behavior.

TLA+ correspondence

CML equivalents cover TLA+ concepts through semantics rather than glyphs:

TLA+ idea	CML core
VARIABLES, priming	state schema and relational next state
UNCHANGED	frame inference or explicit unchanged set
[A]_v	action-or-stutter combinator
ENABLED A	enabledness predicate over relation
WF_v(A), SF_v(A)	named fairness monitors
functions as values	finite/symbolic map/function values
INSTANCE	parameterized module instantiation
CHOOSE	theorem witness or explicit executable selection contract
PlusCal	procedural surface lowering to labeled actions
TLC config	model configuration artifact
TLAPS theorem	Lean theorem/verified certificate

Document: docs/26_RELEASE_GATES_REV2.md

Revision-2 Program Gates

A gate is passed by artifacts and adversarial tests, not a meeting.

G0A — semantic kernel

- exact reference evaluator supports Wave-0 value/action core;
- deterministic finite exploration and path witnesses;
- independent closure checker;
- semantic epoch and canonical serialization;
- operator-level differential fixture generator.

Exit: DieHard, N-Queens, CoffeeCan, Majority, TransitiveClosure pass P2.

G0B — Lean foundation

- transition/reachability/invariant/refinement theorems compile without sorry;
- theorem axiom manifest generated;
- closure certificate imported from Rust/Python fixture;
- malformed certificates rejected;
- CI installs pinned Lean toolchain reproducibly.

G0C — corpus Tribunal

- pinned upstream epoch and manifest scraper;

- TLC output/trace oracle adapter;
- parity.toml schema and dashboard;
- minimized discrepancy fixture;
- mutation and metamorphic harness.

G1 — procedural/concurrency semantics

- labeled procedural lowering;
- deadlock and terminal policies;
- finite process symmetry;
- weak/strong fairness core;
- baseline source-DPOR;
- Dining Philosophers and Barriers P3;
- one shared-memory P5 implementation.

G2 — distributed DST replacement

- asupersync adapter with coverage report;
- network/time/storage/process domain packs;
- crash/restart and cancellation;
- exact replay and causal minimization;
- observer-indexed conservative independence;
- one existing bespoke DST removed without lost scenarios.

G3 — refinement and evidence

- standalone views and stuttering simulations;
- proof-producing finite closure and refinement certificates;
- property cone slicing;
- EWD/graph Wave-2 corpus parity;
- second materially different project integrates without engine changes.

G4 — temporal and parameterized assurance

- fair SCC/lasso checking and certificates;
- ranking-function interface;
- PDR/CHC backend;
- quantified invariant/cutoff lane;
- assumption objects and strength tracking;
- selected Paxos/termination examples P3/P4.

G5 — production conformance

- semantic events in production mode;
- partial-order/timebox trace ingestion;
- Valid/Invalid/Inconclusive/SemanticsMismatch verdicts;
- instrumentation sufficiency analysis;
- trace-to-Lab replay for supported cases;
- known observability impossibilities surfaced.

G6 — corpus completion and frontier promotion

- all 80 validated corpus families at required parity;
- proof-bearing corpus theorem suite;
- TCP/German/TLC stress cases;
- at least ten P5 examples across concurrency, consensus, storage, network and cancellation;
- experimental engines promoted only where they beat certified baselines and retain soundness evidence.

1.0 blocker classes

- unknown semantic discrepancy;
- replay instability;
- unproved certifying reduction;

- vacuous corpus property;
- hidden ambient nondeterminism in verified boundary;
- fairness assumption without production interpretation;
- proof package containing sorry;
- strong result dependent on hash collision absence;
- production Valid verdict from insufficient evidence.

Document: docs/27_SPIKE_FINDINGS_REV2.md

Revision-2 Spike Findings

Executable artifacts live under `spikes/`. They use a deliberately small Python reference kernel because Rust and Lean toolchains were not present in the assembly environment.

Spike 1 — exact finite reference exploration

The kernel implements:

- deterministic BFS;
- exact hashable state identity;
- labeled transitions;
- shortest witness reconstruction;
- finite closure certificate generation;
- independent certificate checking.

It is intentionally too simple to hide optimization bugs.

Spike 2 — Die Hard corpus seed

The model directly ports the six actions from the upstream TLA+ example.

Observed:

- 16 reachable states;
- 96 enumerated action edges including self/stuttering-equivalent updates;
- shortest big = 4 witness at depth 6;
- independently accepted type/closure certificate.

The six action labels and resulting states are stored in `spikes/results/spike-results.json`.

Architectural consequence

Corpus parity can be built incrementally around exact facts. A P2 port should compare normalized state graph and shortest violation semantics, not just “both tools eventually find a solution.”

Spike 3 — Dining Philosophers

A five-philosopher left-then-right model produced:

- 573 reachable states;
- 2,365 action edges;
- canonical deadlock with every philosopher holding its left fork;
- shortest deadlock depth 10;
- accepted finite type/closure certificate.

Architectural consequence

Deadlock is a semantic property distinct from invariant failure. The result also supplies the first procedural/concurrency fixture for DPOR and symmetry spikes.

Spike 4 — stuttering refinement

A concrete register splits an abstract atomic write into:

```
Reserve(v) → Commit
           ↘ Abort
```

The abstraction forgets pending reservations. Exhaustive checking confirmed every concrete step either stutters or performs an abstract write.

Architectural consequence

Reserve/commit/abort and cancellation can be connected to abstract atomic specifications with ordinary stuttering simulation before introducing more exotic true-concurrency refinement.

Spike 5 — observer-indexed independence

`write_x` and `write_y` commute for observers of final (x,y) state and for an x -only observer, but not for an order-sensitive audit observer.

Architectural consequence

Independence must be indexed by the observer/property contract. A universal conflict table would either erase audit bugs or miss reductions available to state-only properties.

Not established

The spikes do not prove:

- CML semantics;
- DPOR soundness;
- TLA+ source parity beyond the hand port;
- Lean code compilation;
- Rust performance;
- fairness/liveness preservation;
- correctness of asupersync adapters.

Each missing fact is attached to a gate rather than buried in prose.

Document: docs/28_THEORY_OF_SUCCESS.md

Theory of Success

The product wedge

Continuum succeeds first by replacing bespoke deterministic simulation in real Rust projects:

one dependency
one semantic event model
one replay artifact
one fault/scenario language
one verifier command

The initial buyer/user does not need to care about category theory or theorem provers. They need a concurrency bug reduced from 80,000 log lines to nine causal events with a one-command replay.

The credibility wedge

The TLA+ Examples corpus prevents the product wedge from becoming a glorified test runtime. Lean-checked certificates prevent performance optimizations from becoming a trust exercise.

Together:

immediate utility		long-term assurance
DST replacement	+	corpus/Lean parity

Neither is sufficient alone.

The agent wedge

Agents amplify both sides:

- they can write ports, models, invariants, proof sketches and repairs;
- Continuum turns their output into checked artifacts;
- minimized causal counterexamples reduce context and hallucination surface;
- proof goals and evidence graphs give agents objective progress signals.

The likely immediate-future workflow is not a human hand-writing every invariant. It is an agent generating candidates against a verifier that refuses bullshit.

Adoption path

Level 0 — replayable DST

No model language required. Existing Rust tests move onto asupersync/Continuum domain packs.

Level 1 — semantic events and assertions

Concrete invariants, fault campaigns, causal minimization.

Level 2 — abstract view

One abstraction map and model-generated scenarios.

Level 3 — standalone model and refinement

Design-before-code plus bounded proof that implementation events refine it.

Level 4 — temporal/parameterized proof

Fairness, liveness, unbounded node counts, Lean certificates.

Teams receive value at every level.

Why comparable systems have not unified this

The pieces historically optimize different workflows:

- theorem provers privilege proof expressiveness;
- model checkers privilege abstract models;
- DST frameworks privilege implementation fidelity;
- concurrency checkers privilege schedule control;
- runtime monitors privilege low-overhead observation.

Asupersync’s explicit capabilities, cancellation, obligations, and Lab semantics create an unusually coherent concrete substrate. CIR and CML connect it upward; Lean checks the bridges.

Key existential risks

1. **Asupersync immaturity.** Mitigate with adapter isolation and semantic conformance.
2. **Language scope explosion.** Let corpus waves drive features; reject unneeded generality.
3. **Proof-program sinkhole.** Formalize kernels and transformations, not every optimizer.
4. **State explosion.** Portfolio plus abstraction; never promise universality.
5. **Rust ecosystem friction.** Offer incremental adoption and Tokio/foreign boundaries outside verified core.
6. **False “same code” claim.** Explicitly track modeled/contracted/unverified dependencies.
7. **Agent-generated noise.** Only evidence changes claim status.

Metrics that matter

- known bugs reproduced from migrated DSTs;
- median minimized causal-core size;
- exact replay rate;
- integration code removed per project;
- corpus parity coverage;
- mutation kill rate;
- time to first counterexample;
- certificate check/search ratio;
- proof maintenance per semantic change;
- agent repair success under proof gates;
- production traces classified without false validity.

The north-star demonstration

A production Rust service has a rare cancellation/durability race. Continuum:

1. observes an ambiguous production trace;
2. identifies missing instrumentation rather than guessing;
3. reproduces the permitted causal envelope under Lab;
4. finds the violating schedule;
5. minimizes it;
6. maps it to an abstract model violation;
7. an agent proposes a fix and invariant;
8. exact replay and neighboring exploration pass;
9. a refinement/safety certificate checks in Lean;
10. the same semantic instrumentation guards production.

That is a micro-revolution, not a faster model checker.

Document: docs/29_PROOF_ENGINEERING_STRATEGY.md

Proof Engineering Strategy

Principle: prove interfaces, validate optimizations

A full formal verification of a high-performance parallel Rust model checker is a poor first objective. The better decomposition is:

- prove semantic kernels and certificate checkers once;
- require optimized engines to produce per-instance evidence;
- use translation validation for complex passes;
- verify small concurrency-critical implementation components with code-level tools where useful.

Proof graph

Every theorem/certificate is a node in a dependency graph:

Safety

```
├─ TypeOK
├─ QuorumIntersection
├─ LogPrefix
└─ ActionPreservation
    ├── ReceiveVote
    ├── Commit
    └─ Recover
```

This supports inductive proof slicing, incremental rebuild, localized agent goals, and exact blame when a model action changes.

Four proof modes

1. Native Lean proof

Best for foundational theorems, reusable mathematics, and small corpus proofs.

2. Reflection

Best for large finite certificates and decision procedures. A proved checker computes.

3. External solver certificate

Best for SAT/PB/SMT/PDR obligations. Solver is untrusted; certificate and verified encoding are checked.

4. Translation validation

Best for changing compiler/reduction optimizations. Each transformed artifact carries a witness checked against its input.

Proof object sizing

Evidence must be streamable and content-addressed. Large state closures use:

- sorted chunked state tables;
- Merkle roots;

- partition closure witnesses;
- edge batches;
- local checker summaries;
- a final composition theorem.

Lean reflection verifies chunks and root composition without constructing a term per transition.

Incrementality

Semantic identities are declaration/content hashes. A model change invalidates only:

- dependent operators/actions;
- proof graph slices;
- affected certificate partitions;
- refinement edges observing changed fields/events.

This is a long-term requirement, not a v0 optimization, because proof rebuild latency determines whether humans and agents keep verification enabled.

Proof automation

Automation is layered:

1. simplification and decision procedures;
2. action-local VC generation;
3. finite counterexample-to-induction generation;
4. grammar-based lemma/invariant synthesis;
5. proof slicing;
6. solver-backed arithmetic/set reasoning;
7. agent proposal/search;
8. human theorem design for genuinely new abstractions.

No tactic's success is trusted beyond the kernel-checked term it produces.

Code-level verification

Continuum exports local obligations to complementary tools:

- Kani for bounded bit-precise Rust behaviors;
- Loom/asupersync Lab for concrete schedules;
- Verus/Thrust-style tools for functional/refinement properties;
- Miri/sanitizers/fuzzers for implementation safety;
- Lean for semantic/certificate theorems.

The project does not pretend protocol-level model checking proves unsafe FFI or memory-model correctness.

Document: docs/30_AGENT_NATIVE_MICRO_REVOLUTION.md

Agent-Native Concurrent Systems Engineering

Premise

Agents will soon generate more concurrent Rust code than humans can manually audit. Current feedback loops are inadequate:

agent writes code → tests happen to pass → merge

Continuum changes the loop to:

agent proposes model/code/proof
→ semantics and effects checked
→ schedules/faults explored
→ counterexample minimized
→ repair proposed
→ certificate checked
→ evidence attached to change

Agent-facing primitives

continuum model

Create or update a standalone model from a design, with corpus analogues and unresolved assumptions listed.

continuum challenge

Generate adversarial mutations, missing fairness cases, crash windows, cancellation points, and abstraction counterexamples.

continuum explain

Return causal core, abstract delta, violated lemma, proof slice, assumption involvement, and source locations.

continuum repair

Produce candidate code/model changes but no claim; automatically launches replay and neighboring exploration.

continuum prove

Emit Lean goals, candidate invariant grammar, finite counterexamples to induction, and solver certificate tasks.

continuum review

Compare evidence before/after a PR: weaker properties, stronger assumptions, reduced coverage, new opaque effects, invalidated proof edges.

Stable machine contracts

Agents consume JSON/CBOR schemas, not human terminal output. Every command supports:

- deterministic IDs;
- bounded resource budget;
- resumable search state;
- evidence references;
- exact semantic epoch;
- structured diagnostics;
- no implicit prompt interpretation in the verifier core.

Autonomous swarm decomposition

A complex proof/repair can be decomposed into agents for:

- corpus analogy search;
- invariant synthesis;
- liveness ranking;
- abstraction/refinement mapping;
- Rust repair;
- Lean proof;
- adversarial mutation;
- evidence review.

They communicate through the proof graph and claim ledger. A coordinator cannot declare success until machine gates close.

Anti-reward-hacking design

Agents may be tempted to:

- weaken the property;
- strengthen fairness;
- reduce model bounds;
- hide events;
- change abstraction maps;
- mark external effects opaque;
- overfit the exact counterexample.

Continuum computes semantic diffs on all of these and treats them as first-class review items. Mutation tests and neighboring exploration detect trivial properties and narrow patches.

Training/evaluation corpus

The 80 TLA+ examples plus generated mutations form a high-quality agent benchmark:

- translate formal models;
- repair semantic defects;
- infer invariants;
- prove theorems;
- classify liveness assumptions;
- connect model to Rust.

Unlike text-only benchmarks, every output has executable or kernel-checked grading.

Document: docs/31_FALSIFICATION_AND_KILL_CRITERIA.md

Falsification and Kill Criteria

Boundary-pushing ideas must be easy to kill. Otherwise the project becomes a museum of impressive nouns.

Observer-indexed independence

Hypothesis: property/view-specific observers materially reduce exploration beyond conservative resource conflicts.

Promote if: median $\geq 5\times$ schedule reduction on observer-sensitive corpus class with checker overhead $< 20\%$ and zero mutation loss.

Kill/default-off if: witness cost dominates or property changes invalidate caches too often.

Higher-dimensional/cubical reduction

Hypothesis: representing k-way concurrency as cells beats optimal DPOR/unfoldings on high-width workloads.

Promote if: $\geq 3\times$ memory/time improvement on at least two real protocol classes with a preservation theorem.

Kill if: only synthetic grid benchmarks win or counterexample explanations worsen materially.

Sheaf gluing

Hypothesis: local refinement/telemetry evidence can be glued compositionally and cohomological obstruction localizes inconsistency.

Promote if: it proves or diagnoses a multi-component case that pairwise checks cannot, with understandable output.

Kill if: it restates constraint solving less efficiently or explanations require specialists.

Nominal/orbit-finite lane

Hypothesis: fresh identifiers and dynamic names can be verified without arbitrary finite bounds.

Promote if: a real session/request protocol scales unboundedly with tractable orbit growth and Lean-proved equivariance.

Kill if: models routinely violate symmetry or orbit explosion matches concrete bounding.

Semiring-valued analysis

Hypothesis: one algebraic traversal framework simplifies reachability, shortest witnesses, counting and provenance.

Promote if: three production analyses share code and match specialized performance within 15%.

Kill if: it obscures numerical/game semantics or introduces hot-path abstraction cost.

Assumption synthesis games

Hypothesis: counterstrategies and synthesized fairness contracts improve protocol design.

Promote if: produces weaker, deployable assumptions or repairs on at least five liveness cases.

Kill if: grammars require hand-encoding the answer or synthesized assumptions are operationally meaningless.

Automatic abstraction discovery

Hypothesis: agents/CEGAR can propose useful views and refinement maps.

Promote if: maps survive independent checking and reduce human work on multiple projects.

Kill as an assurance feature if: results require manual semantic repair; retain as suggestion-only.

Lean proof import on every strong result

Hypothesis: reflective checking keeps proof overhead practical.

Promote if: certificate checking is $\leq 10\%$ of search time for large finite proofs or provides acceptable asynchronous CI latency.

Adjust if: Lean import is too expensive—native verified checker remains routine path, Lean validates checker and sampled/full release artifacts.

Full TLA+ source importer

Hypothesis: source compatibility materially accelerates adoption beyond native ports.

Promote if: users bring significant existing specs and importer maintenance remains bounded.

Defer indefinitely if: corpus ports and SANY oracle export satisfy migration needs more cheaply.

Document: docs/32_CORPUS_PORTING_PLAYBOOK.md

Corpus Porting Playbook

1. Pin and inventory

Record source commit, modules, configurations, imports, proof files, expected tool modes/results, and notable features.

2. State the semantic intent

Before translating syntax, write:

- state variables and mathematical domains;
- initial predicate;
- action relation;
- stuttering policy;
- safety/liveness properties;
- fairness assumptions;
- expected deliberate failures;
- abstraction/refinement structure.

3. Choose the CML fragment

Classify each operator/action/property as finite, symbolic, temporal, probabilistic, theorem, or runtime. Any unsupported combination becomes a tracked language issue.

4. Create value/state correspondence

Define canonical translation. Identify auxiliary variables, model atoms, function values, sequence indexing conventions, and symmetry.

5. Obtain upstream oracle facts

At minimum:

- verdict;
- generated/distinct state counts where meaningful;
- depth;
- counterexample trace;
- liveness loop;
- deadlock behavior.

For P2+, export graph/transition facts or enough traces for bisimulation/language comparison.

6. Build native model

Prefer semantic clarity over literal structure. Preserve named action boundaries when they matter for fairness, traces or proof correspondence.

7. Differential check

Run the native reference evaluator and upstream oracle. Minimize differences before optimizing anything.

8. Add metamorphisms and mutations

Prove the port is semantic and the properties are non-vacuous.

9. Add Lean theorem parity

For P4:

- state theorem correspondence;
- formalize bridge to executable model;
- prove or import certificate;
- record axioms.

10. Add runtime exemplar when selected

Implement using asupersync and domain packs, instrument semantic events, define view/refinement, explore schedules/faults, and replay failures.

Review questions

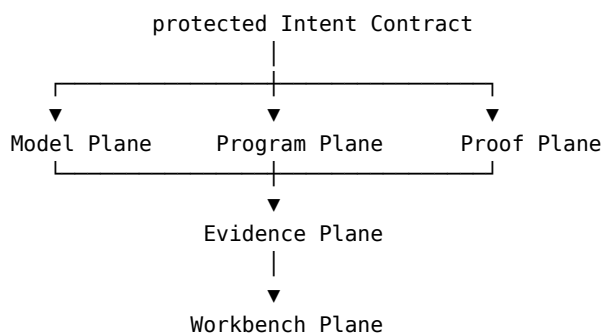
- Did the port accidentally make an action atomic that was not?
- Did typing remove behaviors the TLA+ model allowed?
- Did finite bounds become source semantics?
- Are fairness clauses attached to equivalent actions?
- Does CH00SE/selection have the same determinism/underspecification?
- Are symmetry and model values preserved?
- Is a shorter/different counterexample merely search order or semantic divergence?
- Did the Lean theorem prove the intended property or an encoding artifact?
- Can a mutation make the property fail?

Document: docs/33_REVISION_3_ARCHITECTURE.md

Revision 3 Architecture: Intent, Evidence, and Workbench

Decision

Revision 3 changes Continuum from a verifier with agent features into an **intent-preserving verification workbench**. The semantic core remains independent of the interaction layer, but the interaction layer becomes a load-bearing correctness component.



Why the semantic triptych was insufficient

Model, program, and proof can agree for the wrong reason. An automated repair can:

- replace a strong property with a weak one;
- add a favorable fairness assumption;
- exclude the failing state through bounds or constraints;
- coarsen an observer until distinct bad states look equal;
- remove a fault from the environment;
- downgrade exhaustive evidence to simulation.

All three planes may then become “green.” The missing object is the user’s intended claim and acceptable assurance.

Protected intent

Intent is a typed immutable artifact. It names:

- claims and property formulas;
- observer scope;
- environment assumptions;
- fault and recovery envelope;
- timing/fairness conditions;
- bounds/cutoffs;
- trust boundaries;
- assurance requirements;
- hard/soft synthesis objectives;
- non-vacuity conditions;
- change policy.

Intent revisions are legitimate, but they are never hidden inside repairs. A semantic diff identifies the direction and impact of the change. Dependent evidence is invalidated.

Evidence plane

The evidence plane normalizes results from heterogeneous engines into typed artifacts. It distinguishes:

Observed	one or more concrete executions
Sampled	probabilistic/randomized campaign
Bounded	exhaustive/symbolic within a stated envelope
Validated	independent certificate/translation checker passed
Proved	Lean kernel accepted the theorem under named axioms
Refuted	valid counterexample or proof of inconsistency
Inconclusive	available evidence cannot decide the claim

These states are not totally ordered. A production observation and a bounded proof answer different questions. The assurance envelope records every relevant dimension.

Workbench plane

The workbench is continuumd plus adapters. It provides:

- immutable snapshots and content addressing;
- incremental semantic queries;
- task and budget lifecycle;
- Context Packs;
- causal debugger;
- semantic/intent diff;
- repair transactions;
- synthesis/Forge;
- proof isolation;
- evidence graph;
- access control and audit.

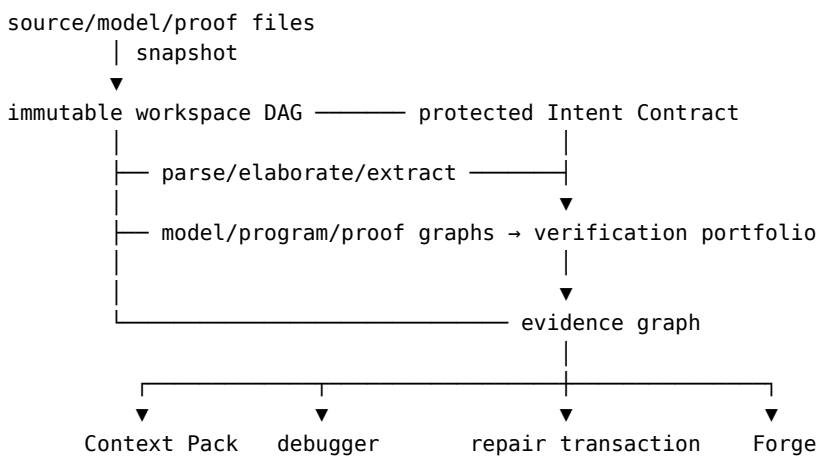
Architectural separation

The following separations are mandatory:

Producer	Independent consumer/checker
optimized explicit explorer	closure/certificate checker
DPOR reducer	unreduced differential oracle + reduction witness checker
Rust extractor	translation validation/refinement checker
Forge/LLM candidate generator	verifier and proof pipeline
proof agent	Lean kernel
incremental query engine	Incremental Parity Audit
Context compiler	replay/property preservation checks
semantic synchronizer	correspondence-law checker

Shared schemas are allowed. Shared decision logic that causes both sides to accept the same bug is not.

Dataflow



Failure model

Continuum itself is a concurrent system. Workbench tasks use asupersync regions and obligations. Publication is two-phase:

```

reserve artifact identity
  → compute/stream provisional evidence
  → validate references
  → commit artifact and ledger edge
  
```

Cancellation before commit leaves no authoritative artifact. Suspended search state is committed as a continuation with explicit partial-evidence status.

Scaling model

The local daemon is the first architecture. Remote workers later consume immutable inputs and return candidate artifacts. The authority service validates and commits them. The design avoids shared mutable solver/session state and permits:

- local single-process development;
- isolated Lean/solver processes;
- parallel workers;
- remote proof farms;
- content-addressed caching;
- reproducible task handoff.

Trust boundary

Trusted or independently checked components should converge toward:

- canonical value decoder;

- core transition/certificate semantics;
- proof-receipt verifier;
- Intent Contract parser/policy;
- content identity and authorization;
- Lean kernel/toolchain closure.

The daemon, search engines, UIs, agents, solvers, and optimizers remain outside the smallest trust base.

Consequence

Revision 3 raises initial implementation cost. It also prevents the most dangerous failure mode: an autonomous system producing convincing formal evidence for a subtly altered question.

Document: docs/34_DEVELOPER_EXPERIENCE_PRODUCT_CONTRACT.md

Developer Experience Product Contract

Product promise

A user should move from a concurrent-systems question to trustworthy evidence without becoming an expert in every engine Continuum employs.

The product contract is:

Every operation is discoverable, reproducible, interruptible, explainable, and honest about its assurance envelope.

Human contract

Immediate comprehension

The first screen or terminal result must communicate:

1. verdict;
2. property and intent version;
3. one-sentence semantic explanation;
4. causal-core size;
5. assurance class;
6. unresolved unknowns;
7. next useful operations.

It must not lead with engine banners, JVM flags, solver statistics, or serialized states.

Exactness on demand

Every simplified item links to exact evidence. Progressive disclosure is lossless navigation, not information destruction.

Stable vocabulary

Use consistent terms:

- **failure** for a checked counterexample;
- **engine error** for tool failure;
- **unsupported** for unavailable semantics;
- **inconclusive** for insufficient evidence;
- **assumption** for environmental restriction;
- **bound** for finite scope;
- **proof** only for independently checked theorem/certificate classes defined by policy.

Reproduction

Every failure has a single stable replay handle. Every success names the command/API and artifact needed for independent checking.

No surprise cost

Before expensive work, Continuum reports the planned portfolio and budget. It can return useful partial evidence at budget boundaries.

Agent contract

No screen scraping

All semantic operations have stable versioned schemas. Human output is not an API.

Bounded observations

Results fit declared byte/token budgets. Larger graphs are paginated by stable handle and semantically meaningful expansion operations.

Actionable failures

An error includes:

- machine code;
- affected handle/input;
- whether retry is safe;
- valid recovery operations;
- whether any evidence was committed.

Explicit state

Agents can save, share, fork, and resume tasks by handles. No operation depends on hidden chat history or connection affinity.

Capability clarity

Tool descriptions distinguish read, propose, execute, and promote authority. Possession of an artifact handle does not grant mutation or promotion rights.

Latency targets

This table is gate-normative for G5 (plan §21 Phase C exit): at G5 the targets are enforced on the reference workload, measured with a saturating background swarm present (plan §4.1). Before Phase C they are design targets.

Interaction	p50 target	p95 target
parse/type/intent diagnostics	50 ms	200 ms
semantic hover/correspondence	50 ms	150 ms
cached bounded check	100 ms	500 ms
local incremental exploration feedback	250 ms	2 s
explain: failure → rendered causal explanation	200 ms	1 s
Context Pack compilation from existing evidence (at $\geq 10^7$ evidence nodes)	100 ms	1 s
evidence query over the evidence graph at $\geq 10^7$ nodes	100 ms	500 ms
replay/minimized branch load	250 ms	1 s
debugger reverse-step (check-point re-execution)	100 ms	500 ms
debugger branch fork at a frontier	250 ms	1 s
task cancellation acknowledgment	50 ms	200 ms

The $\geq 10^7$ -node scale qualifier on the evidence-query and Context Pack rows is gate-normative for G5's evidence-query bullet (plan §22 G5).

Long tasks stream monotonic progress and return continuations.

Output principles

Terminal

- one result per line/group;
- stable rule IDs;
- no color-only meaning;
- paths relative to workspace where possible;
- exact --json equivalent;
- no progress noise when noninteractive.

IDE

- source-local diagnostics when mapping exists;
- evidence handle in diagnostic metadata;
- code lens for check/explain/debug;
- semantic diff preview before protected changes;
- no automatic model/code rewrite without a preview transaction.

Agent

- typed enums rather than prose classifications;
- canonical ordering;
- stable IDs;
- content hashes and epochs;
- explicit omissions;
- no duplicate payload when a reference suffices.

Workflow acceptance tests

These four workflows are the referent of the preregistered G8 study (plan §21.1): the three human-executed workflows (new model; existing Rust system; review) are covered by the two-cohort human study; the agent-repair workflow is covered by the G2 ACI ablation and ContinuumBench, not the human study.

New model

A developer can create, check, inspect a shortest witness, and add an invariant in under ten minutes without reading implementation docs.

Existing Rust system

A developer can identify uncontrolled effects, run one deterministic campaign, and replay a failure without replacing production logic.

Agent repair

An agent can diagnose and propose a repair using only Context Pack and expansion operations. It cannot promote a patch that changes intent.

Review

A reviewer can answer within one screen:

- what changed semantically;
- whether intent changed;
- what evidence was invalidated/rebuilt;
- what remains unknown.

Anti-goals

- reproducing every engine's native flags in the primary UX;
- pretending bounded checking is proof;
- forcing users to maintain duplicate configurations;
- exposing raw internal IDs without semantic labels;
- generating unreviewable model/code changes;
- optimizing agent token count by hiding uncertainty;
- treating a successful tool invocation as a successful verification result.

Measurement

DX is evaluated continuously through human and agent task suites. "It felt clear to the implementer" is not evidence.

Document: docs/35_CONTINUUMD_WORKBENCH_DAEMON.md

continuumd: Authoritative Workbench Daemon

Status: This document predates plan §4.5–§4.7 (the daemon's operational contract: purge and Redacted stubs, backup and verified restore, index fsck, two-epoch migration, compatibility statements, engine-defect artifacts, priority classes and memory budget) and has not yet absorbed those sections; that absorption is open specification debt per plan §25. Plan §4.5–§4.7 governs in the interim.

Responsibility

continuumd is the sole authority for mutable workbench coordination. Semantic artifacts themselves are immutable.

It owns:

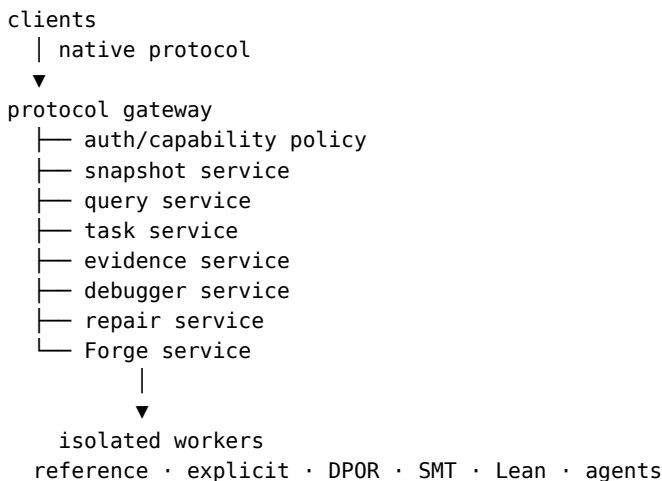
- snapshot creation and overlay resolution;
- Intent Contract registry and policy;
- query dependency graph and cache;
- tasks, budgets, cancellation, and continuations;
- artifact CAS and publication transactions;
- evidence graph/status transitions;
- proof/solver worker dispatch;

- repair and Forge transactions;
- authorization and audit.

It does not own:

- truth of solver results;
- proof acceptance;
- source control;
- agent planning;
- UI presentation semantics.

Service topology



Snapshot transaction

1. Client submits root paths, overlay buffers, dependency/config references.
2. Daemon normalizes paths and content.
3. Parsers may produce diagnostics but cannot alter content identity.
4. Snapshot manifest is canonicalized and hashed.
5. Intent is attached by identity, not copied implicitly.
6. Snapshot is sealed and immutable.

An editor may create many cheap overlay snapshots. Garbage collection follows reachability from named roots, tasks, receipts, and retention policy.

Task identity

A semantic task identity includes:

```

operation
snapshot
intent
semantic/proof/engine epochs
normalized parameters
strategy class
  
```

Budget may be excluded from semantic identity when continuation semantics are monotonic; it remains in execution identity. Two clients of the same principal requesting an identical task share computation freely. Cross-user computation sharing is off by default: content-addressed dedup across principals is an existence oracle and requires an explicit sharing policy (plan §4.5).

Idempotency

Clients supply an idempotency key. The daemon records the canonical request digest. Reusing a key with different content is an error. Reusing it with identical content returns the existing task/result.

Cancellation

Task cancellation is a request–drain–finalize protocol:

- child workers receive cancellation;
- provisional streams close;
- committed partial artifacts are finalized;
- continuation is emitted if supported;
- task transitions exactly once to terminal/suspended state;
- obligations and resource leases are resolved.

Hard-killed foreign workers produce an explicit worker-failure result, never a logical verdict.

Continuations

A continuation contains:

- exact semantic task identity;
- frontier/search state;
- committed evidence roots;
- engine version;
- random/choice state where relevant;
- resource accounting;
- integrity checksum.

Resume validates all referenced inputs. Forking with a changed strategy or intent creates a new task, not a resume.

Evidence publication

Workers return untrusted candidate artifacts. The daemon:

1. validates schema and size;
2. verifies referenced inputs;
3. invokes required independent checker;
4. computes content identity;
5. commits artifact;
6. commits evidence-graph edges/status;
7. emits event/subscription update.

The worker cannot select its own final status.

Incremental query database

The query engine resembles a self-adjusting computation graph but adds proof-oriented edge classes and evidence provenance. Every cached result records:

- inputs and dependency reasons;
- implementation/query version;
- validation receipt;
- reuse class;
- last clean-comparison outcome.

Deployment modes

Embedded/local

A process-local or Unix-domain-socket daemon for cargo continuum; no external service required.

Shared workstation

Multiple IDEs/agents run under per-user auth; each principal's sessions share immutable artifacts and computations freely among themselves. Artifacts and computations are shared across principals only under an explicit sharing policy (plan §4.5) — cross-principal sharing is not a default of this mode.

Remote organization

CAS and workers scale independently. Intent/evidence policy remains centralized. Sensitive source can use local extraction with remote proof over minimized artifacts.

Reliability

The daemon is itself verified through:

- asupersync Lab campaigns;
- model of task/publication state machine;
- crash recovery tests;
- linearizability checks for handle publication;
- Loom/weak-memory checks for local concurrent structures where appropriate;
- independent audit-log consistency checks.

Observability

Operational telemetry is separate from semantic evidence. It includes task latency, cache behavior, worker health, queueing, resource use, and cancellation. A fast worker is not a correct worker; dashboards must not conflate them.

Document: docs/36_AGENT_PROTOCOL_AND_TOOL_CONTRACTS.md

Agent Protocol and Tool Contracts

Non-normative projection. This document is a projection of RFC 0026 (continuumd native protocol), RFC 0027 (agent tool protocol), and plan §10.2, and is regenerated from them. On any divergence, the RFCs and the plan win.

Objective

Give agents the equivalent of a purpose-built verification IDE: compact operations, semantic handles, explicit state, exact feedback, and safe authority boundaries.

Native request envelope

```
{
  "protocol_version": "3.0",
  "request_id": "req_...",
  "idempotency_key": "...",
  "operation": "verification.start",
  "actor": "agent:repairer-1",
  "capability": "cap_...",
  "snapshot": "ws_...",
  "intent": "in_...",
  "arguments": {},
  "budget": {},
  "output_policy": {}
}
```

Native result envelope

```
{
  "request_id": "req_...",
  "status": "completed",
  "verdict": "refuted",
  "assurance": {},
  "artifacts": [{"kind": "context_pack", "handle": "ctx_..."}],
  "task": "task_...",
  "continuation": null,
  "omissions": [],
  "warnings": [],
  "cost": {"states": 48211, "wall_ms": 9120},
  "epochs": {},
  "next_operations": ["context.expand", "failure.explain", "repair.begin"]
}
```

Tool design rules

1. Prefer semantic verbs over filesystem mechanics.
2. Return handles for reusable state.

3. Keep default results small.
4. Allow precise expansion by relation or budget.
5. Include valid next operations.
6. Make destructive/privileged operations visually and structurally distinct.
7. Avoid overloaded “run” tools.
8. Never place untrusted source text inside an instruction field.
9. Separate hypothesis from evidence.
10. Make Unknown and Inconclusive easy to represent.

Example failure workflow

```
workspace.create → ws_1
verification.start(ws_1, in_1, property) → task_1
verification.await(task_1) → crash_1 + ctx_1
context.expand(ctx_1, relation="source") → ctx_2
repair.begin(crash_1) → rt_1
repair.apply(rt_1, patch, hypothesis) → rt_2
repair.evaluate(rt_2) → rt_3
repair.promote(rt_3) → receipt_1 or PolicyGateFailed
```

Error taxonomy

Errors use the 15 stable typed codes of plan §10.3: twelve semantic codes (StaleSnapshot ... PolicyGateFailed) plus three protocol-level codes from RFC 0026 (ProtocolVersionUnsupported, IdempotencyKeyReused, MalformedRequest). Each error may carry recovery as a list of allowed operations with pre-filled arguments — never free-form commands. BudgetExhausted is never a semantic verdict; it carries the continuation when one exists.

Context expansion

Expansion is graph-based:

```
{
  "context": "ctx_1",
  "anchor": "event:e_ack",
  "relation": "causal_predecessors",
  "depth": 2,
  "budget": {"max_nodes": 50}
}
```

Useful relations include:

- causal predecessors/successors;
- conflicting alternatives;
- source correspondence;
- abstract projection;
- proof dependencies;
- assumptions used;
- state-change provenance;
- minimal correction candidates;
- sibling safe branches.

Proof operations

Proof state handles are explicit. Agents operate on goals rather than cursor positions:

```
proof.goal(obligation) → exact goal + proof Context Pack + proof_state
proof.attempt(state, tactic/term) → child states + diagnostics
proof.check(candidate) → kernel receipt
proof.slice(goal) → proof dependency slice (curated declarations)
```

Repair authority

A repair agent may:

- read context;

- propose patches/models/proofs;
- start bounded evaluation;
- request expansion.

It may not:

- alter locked intent;
- declare evidence validated;
- sign promotion receipts;
- access unrelated secrets;
- modify benchmark graders.

MCP mapping

MCP tool calls mirror native operations and carry handles as ordinary typed string fields. Immutable artifacts may be exposed as resources. The adapter remains stateless with respect to semantic work; the client threads handles.

Protocol conformance

Ship:

- JSON Schema/OpenAPI or equivalent IDL;
- generated Rust/TypeScript/Python clients;
- golden protocol traces;
- fuzzing for malformed requests/results;
- compatibility matrix;
- deterministic ordering tests;
- authorization tests;
- replay/idempotency tests.

Token economics

Measure protocol effectiveness rather than optimizing payload size blindly:

useful semantic facts per token
successful task per token/tool call
time to correct diagnosis
expansion precision
invalid action frequency

A 500-token pack that induces the wrong repair is worse than a 2,000-token faithful pack.

Document: docs/37_HUMAN_WORKFLOWS.md

Human Workflows

Workflow 1: design a protocol before code

1. continuum model new creates CML module and Intent Contract.
2. IDE shows state, action, invariant, fairness, and fragment types.
3. Bounded exploration runs incrementally.
4. Counterexample appears as state delta and causal/action trace.
5. User branches in debugger or edits model.
6. Stronger proof lanes produce obligations/receipts.
7. Later Rust implementation binds to model via correspondence graph.

The user never has to invent a separate test harness for model execution.

Workflow 2: bring an existing Rust project under control

1. cargo continuum init audits ambient effects.
2. Report classifies time, entropy, tasks, synchronization, network, storage, FFI, and opaque dependencies.
3. User chooses verification boundary and domain profiles.
4. Asupersync adapter runs deterministic smoke scenarios.
5. Context Packs explain unsupported/uncontrolled effects.

6. Model can be authored manually or bootstrapped as a draft from observed behavior.

Continuum must not claim a generated operational model is the intended abstraction.

Workflow 3: debug a failure

1. Open causal explanation.
2. Inspect abstract state mismatch.
3. Step backward to last relevant write/obligation transition.
4. View enabled frontier before divergence.
5. Branch to alternative schedule/fault.
6. Compare branches.
7. Save hypothesis into repair transaction.

Workflow 4: review a pull request

The review view groups changes by:

- intent;
- model semantics;
- concrete effects;
- abstraction/refinement;
- proof/certificate impact;
- verification/benchmark envelope;
- performance.

Textual source diff remains available but is not the only unit of review.

Workflow 5: reason about liveness

The user sees:

- candidate fair cycle;
- fairness obligations currently owed;
- continuously/intermittently enabled actions;
- ranking/progress measures;
- scheduler/environment assumptions;
- whether the cycle is unfair, genuine, or inconclusive.

This avoids the common experience of staring at a long lasso without knowing which fairness clause matters.

Workflow 6: production incident

1. Import partial-order telemetry under a named instrumentation profile.
2. Continuum reports matched model execution, violation, or insufficient evidence.
3. Missing telemetry is listed as concrete instrumentation needs.
4. A compatible trace is replayed/simulated if possible.
5. Result links to model/intent version deployed at incident time.

Workflow 7: learn Continuum

Tutorials progress from puzzles to real code. Every lesson asks the learner to predict, run, explain, and repair. Tooltips explain formal vocabulary in situ. Advanced formulae remain accessible.

Design standards

- no modal dialog blocks long work;
- cancellation is always available;
- every progress view has semantic milestones, not just percent;
- path from summary to raw evidence is at most three interactions;
- keyboard/CLI parity for core workflows;
- artifacts are linkable/shareable by handle;
- failures retain user annotations without mutating evidence;
- uncertainty is visually prominent.

User research plan

Recruit:

- Rust engineers without formal-methods background;
- experienced distributed-systems engineers;
- TLA+/model-checking users;
- formal verification experts.

Tasks measure diagnosis, repair, assurance comprehension, and transfer. Compare raw traces, conventional model-checker output, and Continuum explanations. Record incorrect confidence, not just speed.

Document: docs/38_COUNTEREXAMPLE_EXPERIENCE.md

Counterexample Experience

Product principle

A counterexample is the beginning of diagnosis, not the end of verification.

Canonical failure artifact

A crashpack contains:

- snapshot and Intent Contract;
- violated property and monitor state;
- canonical execution/configuration;
- exact choice/fault/replay data;
- concrete and abstract states;
- source/model/proof correspondence;
- causal/conflict graph;
- minimization record;
- assurance envelope;
- engine and semantic epochs.

Explanation pipeline

raw witness

- validate/replay
- normalize causal graph
- property-directed slice
- minimize while preserving failure
- compute state/obligation deltas
- contrast with nearest safe branch
- map to source/model/proof
- compile Context Pack

Causal core

A causal core is closed under the dependencies needed to reproduce the violation. It may include:

- events directly observed by the property;
- causal predecessors;
- conflicts that selected the branch;
- obligation and resource transfers;
- fault preconditions;
- abstraction-relevant hidden events.

A smaller set based only on textual relevance is not accepted as replay-preserving.

Minimality classes

- **1-minimal:** removing any one selected item destroys the witness.
- **cardinality-minimal:** no smaller selected set witnesses the failure.
- **causally minimal:** minimal configuration under causal closure.
- **value-minimal:** domains/names reduced.

- **owner-minimal:** tasks/nodes reduced.
- **fault-minimal:** no unnecessary injected fault.
- **explanation-minimal:** human/agent study objective, not purely set size.

The artifact records which class was achieved.

State delta

Instead of dumping whole states:

Concrete:

disk.pending[e7]	+ value X
replies[e7]	reserved → published
disk.stable[e7]	unchanged

Abstract:

acknowledged[e7]	false → true
durable[e7]	false

Violation:

acknowledged[e7] $\wedge$ $\neg$ durable[e7]

Missing order

For concurrency bugs, show the order constraint required by intent:

required: SyncCompleted(e7) → ReplyPublished(e7)
observed: ReplyPublished(e7) || SyncCompleted(e7 absent)

Use →, conflict, concurrency, and absence precisely.

Contrastive branch

The engine searches for a nearby safe execution and reports the minimal relevant difference. Distance may combine:

- changed scheduler choices;
- fault edit distance;
- causal graph edit distance;
- abstract state distance;
- owner switches.

The distance definition is part of the evidence.

Source mapping

Each semantic event links to:

- source span;
- macro/generated origin;
- runtime effect primitive;
- model action;
- abstraction edge;
- proof obligation.

If mapping is incomplete, the UI says so and suggests instrumentation/annotation.

Suggested repairs

Suggestions are hypotheses only. They are ranked by:

- intervention success on the witness;
- locality;
- semantic intent preservation;
- neighboring branch robustness;
- proof impact;
- cost.

They enter a Repair Transaction before any claim.

Failure of explanation

Continuum must return `ExplanationIncomplete` when it cannot produce a faithful bounded explanation. It may still provide the raw witness. It must not fabricate a tidy narrative.

Document: docs/39_VERIFICATION_DEBUGGER.md

Verification Debugger

Problem

A normal debugger answers “what did this execution do?” A verification debugger must answer:

- which semantic events could occur next;
- why each event is enabled or disabled;
- which alternatives were pruned and why;
- where the chosen execution diverged from a safe one;
- how concrete steps map to abstract transitions or stuttering;
- what fairness/progress obligations accumulate;
- which fault or cancellation choices matter.

Debug state

```
struct DebugState {
    snapshot: SnapshotHandle,
    intent: IntentHandle,
    execution: ExecutionHandle,
    configuration: ConfigurationId,
    causal_past: EventSet,
    enabled: Vec<EnabledEvent>,
    observers: Vec<ObserverState>,
    abstractions: Vec<AbstractState>,
    obligations: ObligationTree,
    fairness: FairnessLedger,
    frontier: FrontierSummary,
}
```

Handles are stable within an immutable execution artifact. Large values use lazy child handles.

Stepping modes

Semantic event

Execute one chosen event from the enabled frontier.

Abstract step

Run concrete internal/stuttering events until the selected abstraction changes or a property event occurs.

Owner turn

Run until task/node/region ownership changes.

Property step

Run until the monitor changes state.

Causal reverse

Move to a causally closed predecessor configuration. If multiple maximal events can be removed, return choices.

Counterfactual branch

Select a different legal event/fault at a prior configuration and create a new branch handle.

Breakpoints

Breakpoints can target:

- source line/function;
- model action;

- event kind or effect phase;
- property monitor state;
- abstract predicate;
- obligation creation/resolution;
- cancellation phase;
- durability boundary;
- fault injection;
- fairness debt threshold;
- correspondence mismatch.

A breakpoint is compiled into an observer/query and versioned with the debug task.

Why enabled

The engine returns a proof-like derivation tree:

```
Deliver(m17, n2)
├─ pending(m17)
├─ destination(m17) = n2
├─ running(n2)
├─ connected(n1,n2)
├─ now ≥ earliest_delivery(m17)
└─ budget allows delivery
```

For disabled actions, return blocking leaves and whether they can become true.

Branch comparison

Comparing branches produces:

- common causal prefix;
- first conflicting choice;
- event additions/removals;
- state/observer deltas;
- property outcome difference;
- assumption/fairness differences;
- cost difference.

This is a semantic diff between executions, not just a textual trace diff.

Partial-order visualization

The primary visualization is a layered causal graph:

- horizontal grouping by node/task/region;
- causality arrows;
- conflict/alternative edges;
- intervals for operations;
- resource/obligation flow;
- abstract transition bands;
- property-monitor timeline.

A total-order timeline remains available for replay but is explicitly marked as one linearization.

DAP adaptation

DAP object references are ephemeral adapter handles backed by stable Continuum handles. Suggested mapping:

Continuum: Master Implementation Plan & Comprehensive Dossier

DAP	Continuum
thread	node/task/abstract process
stack frame	abstraction/source/effect frame
scope	concrete state, abstract state, obligations, frontier
variable reference	lazy artifact query
breakpoint	semantic predicate/event query
step in	semantic event or abstraction descent
step over	abstract step
step back	causal reverse
restart frame	branch from configuration

Custom requests expose causal frontier and branch comparison.

Correctness

The debugger does not mutate original evidence. Every branch is a new execution artifact. Replay validates selected transitions. The UI cannot manufacture an event not accepted by the semantic engine.

Agent use

Agents can ask bounded questions:

show the three events enabled before the violation
branch by forcing SyncCompleted first
compare property and abstract state
show the last event that changed reply obligation

This is substantially more reliable than asking an agent to infer concurrency from a linear log.

Document: docs/40_SEMANTIC_DIFF_AND_IMPACT.md

Semantic Diff and Impact Analysis

Objective

Determine what a change means, what evidence it invalidates, and whether it changes protected intent.

Diff layers

Text diff
Syntax/AST diff
Type/effect diff
Intent diff
Transition-system diff
Observer/property diff
Correspondence/refinement diff
Proof dependency diff
Behavior/evidence diff
Cost/performance diff

A change may be textually large and semantically null, or one token may weaken a critical property.

Intent diff categories

- property strengthened/weakened/incomparable;
- assumption strengthened/weakened/incomparable;
- observer refined/coarsened/incomparable;
- bounds expanded/contracted;
- fault envelope expanded/contracted;
- fairness added/removed/changed;
- trust boundary expanded/contracted;
- assurance upgraded/downgraded;

- optimization/non-vacuity changed;
- no protected change;
- unknown, proof required.

incomparable on a protected field has the same blocking status as a confirmed privileged change (see Soundness policy).

Model diff

Compare elaborated relations, not source text alone:

- states/variables/types;
- initial relation;
- action guards/updates/frames;
- stuttering policy;
- temporal formulae;
- symmetry declarations;
- constraints/views;
- refinement edges.

For finite configurations, exact relational inclusion/equivalence may be checked. For symbolic/infinite fragments, generate solver or Lean obligations.

Program semantic diff

Use extraction and semantic journal metadata to identify:

- new/removed effects;
- changed atomicity boundaries;
- changed resource footprints;
- cancellation checkpoints/finalizers;
- durability phase changes;
- task/region ownership changes;
- observer publication changes;
- opaque/FFI boundary changes.

Behavioral diff

Within an assurance envelope, compute:

- newly reachable states/configurations;
- removed behavior;
- changed counterexample classes;
- changed liveness SCCs;
- refinement coverage changes;
- trace-language inclusion samples/proofs;
- performance/cost distribution changes.

Impact graph

Evidence edges classify how changes propagate:

```
source value read
source type read
definition unfolded
model action used
observer event consumed
assumption/fairness consumed
proof lemma used
certificate checker epoch used
engine encoding used
```

Impact output distinguishes definitely invalidated from conservatively rebuilt.

Review presentation

Protected intent

- ✓ properties unchanged
- ✓ assumptions unchanged
- ! bound N: 5 → 3 (contraction; blocked)

Program semantics

- reply publication moved after storage sync
- cancellation finalizer no longer publishes reserved reply

Behavior

- failing causal class removed
- no new bounded violations
- availability traces preserved under declared fair profile

Evidence

- 2 certificates invalidated/rebuilt
- 1 Lean theorem reused
- 4 benchmark tasks affected

Soundness policy

A claim such as “property strengthened” requires implication evidence in the supported fragment. Otherwise output unknown (undecidable or unattempted), unsupported (outside declared fragments), or incomparable — unknown and unsupported are distinct classifications per INV-008.

All non-affirmative classifications fail closed: any intent change classified unknown, unsupported, or incomparable on a protected field blocks ordinary promotion exactly as a confirmed privileged change does, pending review (plan §5.3, RFC 0031). Text heuristics may prioritize review but cannot authorize promotion.

Use in Forge

Forge candidates are compared semantically to maintain a behaviorally diverse archive and detect duplicate implementations. Syntactic novelty alone is not counted.

Document: docs/41_REPAIR_TRANSACTIONS.md

Repair Transactions

Status: This document predates the 12-gate, phase-profile repair design and is scheduled for regeneration (plan §25). **RFC 0032 and schemas/repair-transaction.schema.json are normative**; where this document’s 9-step pipeline disagrees with RFC 0032’s twelve gates (including incremental_parity as a gate, gate_profile, and not_yet_enforced statuses), the RFC governs.

Purpose

Make autonomous repair safe, reproducible, and reviewable by treating repair as an evidence-bearing transaction rather than an edit followed by tests.

State machine

Draft

- Applied
- Evaluating
- Ready | Blocked | Inconclusive
- Promoted | Rejected | Superseded

Each transition creates a new immutable transaction version.

Required fields

```
struct RepairTransaction {
    base_snapshot: SnapshotHandle,
    base_intent: IntentHandle,
    failure: EvidenceHandle,
```

```
hypothesis: Hypothesis,  
proposed_changes: ChangeSet,  
actor: ActorId,  
evaluation_policy: RepairPolicy,  
evidence: Vec<EvidenceHandle>,  
status: RepairStatus,  
}
```

The hypothesis is separate from evidence. Agents can be wrong without corrupting the record.

Change sets

A transaction may contain:

- Rust patch;
- CML change;
- correspondence change;
- proof change;
- domain-pack/config change;
- explicit Intent Contract revision.

The last category automatically changes transaction class and review policy.

Evaluation pipeline

1. Base validation

Reproduce failure on the exact base snapshot and intent. If replay diverges, block with `ReplayDiverged`.

2. Apply and seal

Apply changes in a forked snapshot. Normalize and hash. Reject hidden filesystem edits.

3. Semantic/intent diff

Classify changes. Protected intent change blocks ordinary repair policy.

4. Exact regression

Replay original execution choices where still meaningful. If the patch eliminates an event, use correspondence to report where replay diverges and continue under a declared policy.

5. Causal neighborhood

Explore alternate choices around the failure slice.

6. Mutation challenge

Run property/model/code mutants and hidden variants to detect overfitting or accidental verifier disablement.

7. Proof/refinement rebuild

Invalidate and rebuild affected evidence. Reject stale receipts.

8. Nonfunctional checks

Measure performance/resource/security changes relevant to intent.

9. Policy decision

A trusted policy engine evaluates evidence and unknowns.

Promotion receipt

A receipt contains:

- base and candidate snapshot IDs;
- intent IDs and diff;
- failure identity;
- hypothesis and patch hash;
- exact replay outcome;
- neighborhood definition and coverage;

- mutation results;
- proof/certificate receipts;
- clean/incremental parity;
- performance/security findings;
- unresolved unknowns;
- policy and signer/checker identities.

Agent swarm

Different agents may attach:

- diagnosis;
- candidate patch;
- invariant/proof repair;
- adversarial review;
- benchmark result.

They do not edit one mutable transaction document. They append typed proposals/evidence nodes. The transaction manager derives status.

Minimality

Continuum may compute patch subsets and semantic changes to identify unnecessary edits. Minimal textual patch is not always best; the review shows both textual and semantic footprint.

Failure modes

- **Exact overfit:** original trace fixed, sibling trace fails.
- **Intent gaming:** property/assumption/bound changed.
- **Verifier gaming:** instrumentation or property monitor disabled.
- **Stale proof:** receipt from old snapshot reused.
- **Availability collapse:** safety fixed by suppressing progress.
- **Abstraction gaming:** concrete bad states merged.
- **Opaque escape:** affected effect marked unsupported/opaque.

Each has an explicit gate and benchmark mutation.

Document: docs/42_INCREMENTAL_VERIFICATION.md

Incremental Verification

Thesis

Continuum should feel interactive without making cached correctness an article of faith.

Query graph

Derived computations are pure or explicitly effectful queries over immutable artifacts. Examples:

```
Parse(file)
Elaborate(module, imports, semantic_epoch)
InferFragment(model)
ExtractRust(crate, annotations)
BuildCorrespondence(model, program)
CompileProperty(property, observer)
Explore(model, config, strategy)
CheckRefinement(concrete, abstract, map)
CheckCertificate(certificate)
CompileContext(evidence, question, budget)
SemanticDiff(before, after, intent)
```

A query key includes semantic configuration; a source hash alone is insufficient (plan §9.2).

Granularity

Use declaration/action/property/effect granularity rather than file granularity where provenance is sound. Do not chase maximal granularity before semantic dependencies are trustworthy.

Reuse classes

Exact

Content-addressed pure result; no validation beyond decoder/integrity.

Validated

Result includes a witness checked independently, such as translation validation or certificate.

Conservative

Dependency tracking may over-invalidate but is proven/argued not to under-invalidate for the supported fragment.

Experimental

Useful for provisional UI/search. Cannot support final promotion until clean comparison.

Incremental exploration

Potential reuse:

- unchanged canonical states and successors;
- frontier partitions;
- property monitor results;
- symbolic clauses/lemmas;
- DPOR independence and backtracking metadata;
- proof summaries;
- context slices.

Danger: a small transition change can invalidate reachability globally. Reuse must be guarded by semantic change classification, not source locality.

Incremental SAT/SMT/PDR

Track assumption frames, formula identities, proof artifacts, and solver epoch. Incremental solving that cannot emit/check suitable evidence remains provisional for strong claims.

Proof reuse

Lean declaration hashes and dependency closure allow exact theorem reuse. Changed elaboration environment invalidates even text-identical proof terms where semantics differ.

Dirty/clean dual lane and the Incremental Parity Audit

The clean/dirty comparison mechanism is the **Incremental Parity Audit** (plan §9.5). It is on by default, not opt-in.

Interactive lane:

- maximizes reuse;
- returns provisional/exact classifications quickly;
- a configurable fraction of interactive queries — default 1 in 64, selected uniformly by query key hash, per RFC 0030 — is recomputed clean and compared.

Promotion lane:

- every promotion-relevant query is recomputed clean or independently validated at promotion time;
- compares canonical outputs;
- emits receipt.

Deployments may adjust the sampling rate by policy; an opt-out is recorded in assurance envelopes (INV-010).

Auditability classes

Queries carry an auditability class, declared on the query definition (plan §9.5):

- **Equality-auditable** — deterministic under the docs/19 matrix; compared bit-for-bit. Any disagreement quarantines the reuse class and emits a minimal invalidation counterexample.

- **Certificate-auditable** — solver-backed; the audit compares checked certificates and claim envelopes, never raw solver behavior. A certificate-level disagreement quarantines; a solver-outcome difference with agreeing certificates does not.
- **Budget-sensitive** — anytime results; the audit checks only that the incremental result's evidence labels are no stronger than a clean run's under equal budget (monotone-honesty), and records divergence as drift telemetry without quarantine.

Invalidation debugging

Developers can query:

```
why was this result reused?  
why was this query invalidated?  
which edge caused the rebuild?  
what clean comparisons support this reuse class?
```

The query graph itself is inspectable evidence.

Verification of the incremental engine

- property-based edit sequences;
- differential clean builds;
- intentionally corrupted edges;
- crash/recovery/cancellation tests;
- content-hash collision injection;
- cross-thread deterministic scheduling;
- Lean theorem for a simplified dependency-closure model;
- proof-producing incremental solver formats where available.

Success metric

Not cache hit rate alone. Optimize:

latency × correctness × evidence freshness × recomputation cost

A stale green result is infinitely worse than a slow one.

Document: docs/43_CONTINUUM_FORGE.md

Continuum Forge: Verified Algorithm Invention

Vision

Forge lets humans define what a concurrent/distributed system must accomplish while agents and synthesis engines search the space of algorithms, invariants, abstractions, and implementations. Continuum checks every candidate.

Forge is not code generation

Code generation maps a known design into syntax. Forge searches behavioral space under formal constraints.

Problem definition

```
struct ForgeProblem {  
    snapshot: SnapshotHandle,  
    intent: IntentHandle,  
    sketch: TypedSketch,  
    holes: Vec<TypedHole>,  
    hard_constraints: Vec<Claim>,  
    positive_scenarios: Vec<Scenario>,  
    objectives: Vec<Objective>,  
    diversity: Vec<BehaviorDescriptor>,  
    assurance: AssuranceRequirement,  
    budget: ForgeBudget,  
}
```

Typed holes

Hole categories:

- predicate/guard;
- deterministic or nondeterministic update;
- message rule;
- quorum family;
- retry/finalizer policy;
- state summary;
- invariant/lemma;
- ranking function;
- abstraction map;
- auxiliary state;
- environment assumption;
- implementation primitive.

Assumption holes are dangerous and require explicit synthesis policy. Forge should prefer algorithm repair over environment restriction and report unrealizability separately.

Candidate lifecycle

Proposed

- Type/fragment checked
- Fast finite screen
- Counterexample-guided refinement
- Portfolio verification
- Proof/refinement obligations
- Cost evaluation
- Diversity archive
- Materialization proposal

No candidate becomes “correct” because an LLM says so.

CEGIS loop

1. Generate candidate from grammar/sketch and learned priors.
2. Check positive scenarios/non-vacuity.
3. Check safety/liveness/refinement under current examples.
4. Obtain counterexample or proof evidence.
5. Generalize counterexample to eliminate an interpretation class when sound.
6. Refine candidate/invariant/ranking/abstraction together.
7. Repeat or prove unrealizable within the finite grammar.

Interpretation reduction

Candidates that are syntactically different but semantically identical over relevant interpretations should share one representative. This can reduce enormous redundant search spaces. The reduction must be scoped to the current synthesis domain and checked/validated.

Co-synthesis graph

algorithm candidate

- ├ requires invariant candidate
- ├ requires abstraction candidate
- ├ requires liveness ranking/fairness
- ├ induces implementation correspondence
- └ has objective vector

A counterexample is routed to the appropriate component rather than forcing blind algorithm mutation.

Search engines

- grammar enumeration;
- SAT/SMT/SyGuS;
- CHC solving;
- IC3/PDR invariant discovery;
- game solving;

- program synthesis agents;
- evolutionary strategies;
- novelty/quality-diversity search;
- MCTS over typed transformations;
- proof-guided search;
- corpus retrieval and analogy.

The orchestrator uses empirical portfolio selection but preserves reproducibility.

Quality-diversity

Archive cells are semantic descriptors, for example:

(message rounds, quorum shape, durable writes, state bytes,
cancellation points, recovery style, fairness strength, proof size)

Candidates in one cell compete by objective. Distinct cells preserve alternative ideas and enable human insight.

Unrealizability

Forge must distinguish:

- no candidate found under budget;
- finite grammar exhausted;
- verified unrealizability under a formal synthesis model;
- intent internally inconsistent;
- implementation/effect constraints too restrictive;
- proof engine inconclusive.

Where possible, return reusable unrealizability cores or environment assumptions required for realizability.

Materialization to Rust

Generation uses verified/translation-validated templates around asupersync effects. Output contains explicit TODO/proof obligations for unsupported components. Generated code enters a Repair/Design Transaction and is tested against the same model.

Agent roles

- sketch architect;
- candidate proposer;
- invariant synthesizer;
- liveness analyst;
- proof worker;
- adversarial falsifier;
- performance optimizer;
- novelty curator.

Evidence Graph prevents persuasive coordination from replacing checks.

Research target

After rediscovering known protocols, test genuinely open design spaces:

- cancellation-safe distributed commit;
- lower-write durable queues;
- dynamic quorum/replication tradeoffs;
- recovery protocols under explicit storage profiles;
- lock-free/async structures with proof-carrying linearization;
- resource schedulers optimizing tail latency under safety constraints.

Novel results must be published with model, code, intent, evidence, and proof artifacts.

Document: docs/44_MULTI_AGENT_EVIDENCE_GRAPH.md**Multi-Agent Evidence Graph****Principle**

Agents may converse, but Continuum trusts only typed artifacts and checked edges.

Graph model

```

struct EvidenceNode {
  id: ContentId,
  kind: NodeKind,
  payload: ArtifactHandle,
  status: EvidenceStatus,
  provenance: Provenance,
}

struct EvidenceEdge {
  from: ContentId,
  to: ContentId,
  relation: Relation,
  justification: Option<ArtifactHandle>,
}

```

Why a graph

Concurrent-systems work is naturally non-linear:

- one counterexample refutes several candidates;
- one invariant supports many properties;
- one patch repairs one failure but invalidates a proof;
- two agents may propose conflicting abstraction maps;
- one theorem generalizes many finite observations.

A chat transcript obscures this structure.

Status authority

Status transition	Authority
(creation) → proposed	any authorized human/agent – there is no draft status (plan §11.4)
proposed → observed	execution service
proposed → sampled	execution/verification service
proposed → bounded	verification service
proposed/bounded → validated	independent checker
proposed → proved	Lean proof service
any → refuted	valid counterexample/checker
any → inconclusive	producing service, with a typed INV-008 reason
any → superseded	policy/owner with explicit edge

Conflict handling

Conflicts are first-class nodes:

Abstraction A maps reply publication to Ack

Abstraction B maps storage stability to Ack

Conflict: both cannot satisfy observed correspondence

Required experiment/proof: distinguish observer contract

A coordinator cannot resolve conflict by selecting the most confident agent answer.

Task generation

The graph generates bounded tasks:

- close missing proof edge;
- find counterexample to candidate;
- explain conflict;
- synthesize invariant for failed induction;
- repair source span attached to causal core;
- review intent diff;
- compare candidate behavior/cost.

Each task contains only relevant graph slice and handles.

Whiteboard

A human-friendly whiteboard view permits provisional notes. Compilation rules:

- every claim becomes a proposed node;
- references must resolve;
- status words in prose do not promote status;
- experiments become task proposals;
- conclusions require supporting edges;
- unresolved contradictions remain visible.

Merging agent work

Agents do not merge mutable branches of “reasoning.” They publish immutable candidates. The integrator computes:

- duplicate semantic candidates;
- supporting/refuting evidence;
- conflicts;
- missing obligations;
- Pareto/dominance relations;
- current frontier of justified options.

Credit and provenance

Every node records actor/tool/model version, prompts/tool inputs as policy permits, source retrievals, and derivation. This supports scientific credit, debugging, benchmark analysis, and reproduction without granting authority based on identity.

Garbage collection

Superseded proposals may be compacted but remain reachable from receipts/audits. Large derivations use content-addressed shared subgraphs.

Security

- source text cannot instantiate privileged graph edges;
- actors may append only allowed node types;
- promotion endpoints require checker capabilities;
- graph queries enforce trace/source privacy;
- untrusted attachments are sandboxed;
- provenance is signed/hashed where organization policy requires.

Document: docs/45_CONTINUUMBENCH.md

ContinuumBench

Mission

ContinuumBench evaluates whether humans and agents can use Continuum to produce trustworthy concurrent/distributed systems—not merely whether a model emits code that passes visible tests.

Benchmark object

Each task bundle contains:

public workspace snapshot
protected Intent Contract
allowed capabilities
budget
public task description
hidden semantic variants
hidden mutations
independent grader
expected evidence class
security policy

The grader is isolated from the agent and workbench clients.

Task tracks

Modeling

- formalize a prose protocol;
- port a TLA+ example;
- identify missing assumptions;
- select correct observers/fairness;
- construct abstraction layers.

Diagnosis

- safety failure;
- deadlock/futurelock;
- liveness/fairness cycle;
- cancellation/obligation leak;
- durability/recovery;
- weak-memory ordering;
- refinement mismatch;
- insufficient production telemetry.

Repair

- Rust source;
- model;
- correspondence;
- invariant/proof;
- instrumentation;
- domain-pack profile.

Proof

- inductive invariant;
- refinement theorem;
- ranking function;
- certificate checker extension;
- Lean proof repair.

Synthesis

- finite guard/update holes;
- quorum/message rules;
- invariant/ranking co-synthesis;
- unrealizability classification;
- cost optimization with behavioral diversity.

Governance/security

- detect property weakening;
- reject assumption/bound/fault gaming;
- avoid stale snapshot/receipt;
- resist prompt injection in source/logs;
- respect capabilities and privacy;
- distinguish inconclusive from pass.

Sources

1. TLA+ Examples corpus, with license/provenance preserved.
2. Generated semantics-preserving and defect mutations.
3. Historical bugs from concurrent Rust/runtime/storage systems.
4. Asupersync Lab/cancellation cases.
5. Continuum's own engine bugs.
6. Real project migrations with sanitized artifacts.
7. Forge-generated hidden variants.

Split discipline

Avoid benchmark leakage through:

- family-level train/test separation;
- source-hash and semantic-clone detection;
- renamed/restructured variants;
- hidden observers and fault profiles;
- generated values/topologies;
- proof-library version shifts;
- held-out real projects.

A named subset of corpus families is held out from all development, tuning, and regression use, and is graded only by the isolated grader (plan §19.4). G9's "80 families at declared parity" is measured on the development set plus a single final evaluation of the held-out set.

Grading

A task score is a vector:

intent_integrity
security compliance
semantic_correctness
required_evidence
proof/certificate validity
hidden_variant_generalization
repair robustness
cost and latency
invalid actions
explanation quality

RFC 0034 is the normative grading order: intent integrity → security → semantic correctness → evidence validity → hidden-variant generalization → cost → explanation.

A zero in intent integrity caps the total at failure.

Agent effectiveness

Measure both accuracy and resources:

- wall time;
- verifier CPU/memory;
- model tokens;
- tool calls;
- context bytes and expansion count;
- retries/stale operations;

- expensive unsuccessful trajectories.

This prevents a scaffold from appearing superior solely by consuming extreme resources.

Explanation evaluation

For agents:

- correct defect classification;
- relevant source localization;
- causal mechanism statement;
- appropriate next operation;
- no invented evidence.

For humans:

- diagnosis accuracy/time;
- repair choice;
- assurance comprehension;
- confidence calibration;
- transfer to a new example.

Mutation challenges

Per task, generate:

- trivial true property;
- property with removed conjunct;
- stronger assumption;
- smaller bound;
- removed fault;
- hidden observer event;
- return unsupported as pass;
- stale receipt;
- exploit stale cache / incremental invalidation;
- exact-trace hard-code;
- disabled instrumentation;
- semantically equivalent distractor patch;
- source comment containing malicious agent instructions.

Intent integrity is a prerequisite, not a bonus metric.

Leaderboards

Report Pareto fronts, not one opaque score:

- strongest evidence under fixed cost;
- lowest cost at fixed success;
- highest intent integrity/generalization;
- human-agent team performance;
- Forge novelty under proof constraints.

Scientific artifacts

Every benchmark run stores:

- tool/model versions;
- task snapshot and intent hashes;
- operation trace;
- evidence graph;
- final patch/model/proof;
- grader receipt;
- resource metrics.

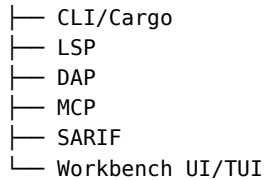
This makes agent-system research reproducible and exposes where interface design, model capability, and verifier quality contribute.

Document: docs/46_IDE_CLI_DAP_MCP_AND_SARIF.md

IDE, CLI, DAP, MCP, and SARIF Integration

One authority, multiple projections

continuum native protocol



Adapters may cache presentation data but not semantic state or evidence authority.

CLI/Cargo

Primary commands:

```
continuum init
continuum snapshot
continuum check/explore/prove
continuum explain/debug/replay
continuum review
continuum repair ...
continuum forge ...
continuum evidence ...
continuum task ...
```

cargo continuum discovers Rust workspace configuration and forwards explicit snapshot operations.

Machine use:

- --json and JSONL streaming;
- stable exit codes;
- no ANSI/progress on non-TTY;
- artifact handles in every result;
- no need to parse explanation prose.

LSP

Use standard LSP for:

- document synchronization;
- diagnostics;
- completion;
- hover;
- symbols/references/rename;
- code actions/lenses;
- semantic tokens;
- workspace impact.

Custom data carries snapshot/evidence handles. Unsaved documents become explicit overlays. The server never assumes editor content equals disk content.

DAP

Use standard DAP for common debugger controls. Continuum custom requests include:

```
continuum/enabledFrontier
continuum/branch
continuum/compareBranches
continuum/stepAbstract
```

```
continuum/reverseCausal
continuum/whyEnabled
continuum/whyBlocked
continuum/fairnessLedger
continuum/exportCrashpack
```

MCP

Expose bounded agent tools:

```
create_workspace
start_verification
get_context_pack
expand_context
open_debug_branch
begin_repair
apply_repair
verify_repair
start_forge
get_evidence
```

Tool results contain explicit handles. Immutable artifacts are resources. Adapter descriptions emphasize capability and result class.

SARIF

Map verification failures to:

- stable rule/property ID;
- primary source location;
- related model/proof/effect locations;
- code flow representing one linearization of causal core;
- properties with evidence/crashpack handles;
- assurance and intent fingerprints;
- safe fix only when a Repair Transaction has produced one.

SARIF consumers may deduplicate and annotate, but the proof artifact lives in Continuum.

Workbench UI

The optional UI integrates:

- intent/assurance dashboard;
- causal graph and branch comparison;
- state delta explorer;
- evidence graph;
- repair transaction review;
- Forge archive and Pareto map;
- proof goals/receipts;
- corpus parity.

It must remain usable without cloud service.

Versioning

Track separately:

- native protocol version;
- adapter protocol version;
- semantic epoch;
- schema version;
- proof/checker epoch.

An adapter can be wire-compatible while unable to render a new semantic artifact; it must expose a generic artifact path rather than silently drop fields.

Document: docs/47_ONBOARDING_AND_PROGRESSIVE_DISCLOSURE.md

Onboarding and Progressive Disclosure

Goal

A Rust engineer should benefit from Continuum before learning temporal logic. An expert should never be trapped behind simplified views.

Concept ladder

Level 0 — Deterministic replay

Learn:

- controlled time/entropy/effects;
- one seed/choice log reproduces a failure;
- cancellation and crash windows.

Level 1 — Invariants and causal failures

Learn:

- state property;
- shortest/minimized counterexample;
- causal vs chronological order;
- Context Pack and debugger.

Level 2 — Abstract model and refinement

Learn:

- model simpler than code;
- abstraction map;
- stuttering;
- model/program drift.

Level 3 — Liveness/fairness

Learn:

- infinite behavior;
- fair cycle;
- environment/scheduler assumptions;
- ranking/progress.

Level 4 — Proof and certificates

Learn:

- inductive invariants;
- independently checked evidence;
- Lean theorem receipts;
- assurance envelope.

Level 5 — Synthesis and invention

Learn:

- typed holes;
- CEGIS;
- non-vacuity;
- quality-diversity;
- unrealizability.

continuum init

Initialization performs an audit and creates a draft, not fake certainty:

```
✓ asupersync runtime detected
✓ virtual time path available
! 3 ambient entropy calls
! filesystem adapter has no crash profile
? abstract model not defined
```

Next: continuum boundary propose

The tool explains why each issue matters and links to automatic or manual actions.

Tutorials

Each tutorial follows:

predict → execute → observe → explain → repair → verify → generalize

Examples:

- Die Hard: reachability and shortest witness;
- Dining Philosophers: deadlock and symmetry;
- barrier: fairness and liveness;
- replicated register: durability/cancellation/refinement;
- Paxos family: abstraction and invariant proof;
- storage service: production evidence and insufficient telemetry;
- Forge guard: synthesis/non-vacuity.

Error messages as teaching

Bad:

Invariant violation at state 3920

Good:

AckImpliesDurable failed.

A reply became externally visible while its write was only submitted, not stable.

This is possible because cancellation finalization publishes the reserved reply.

Formal details remain expandable.

Documentation architecture

- task-first guides;
- concept pages;
- exact semantic reference;
- engine/assurance reference;
- examples/corpus gallery;
- troubleshooting by typed error;
- architecture and proof docs;
- agent API cookbook.

Docs are versioned with semantic epochs and generated schema references.

Agent onboarding

Agents receive a capability/resource manifest and a small “workflow grammar” describing valid operation sequences.

They should not need a giant prompt explaining terminal conventions.

Friction budget

Every mandatory annotation/configuration must justify:

- what semantic ambiguity it resolves;
- what evidence it enables;
- whether it can be inferred and checked;
- whether it remains stable across refactors.

Boilerplate that exists solely for the tool is a design defect.

Document: docs/48_EXPLANATION_SCIENCE.md

Explanation Science Program

Motivation

Formal verification adoption is often limited not by the absence of counterexamples but by the difficulty of understanding them. Continuum treats explanation quality as an empirical and formal research problem.

Taxonomy

Trace explanations

One or more ordered behaviors. Easy to produce, often hard to interpret.

Slices

Remove events/state/formulae irrelevant to a target property.

Minimal counterexamples

Minimize length, events, faults, owners, values, or causal configuration.

Causal explanations

Identify events/interventions satisfying a declared causality definition.

Contrastive explanations

Explain why failure occurred rather than a nearby safe outcome.

Logical explanations

Unsat cores, proof slices, failed induction obligations, assumptions, fairness.

Repair explanations

Minimal correction sets or interventions likely to restore intent.

Formal claims

Every explanation object declares its guarantee:

replay-preserving
property-preserving
1-minimal
cardinality-minimal
causally closed
counterfactual under model M
heuristic relevance only

Do not call a heuristic attention score a cause.

Multi-objective explanation

Optimal explanation is not simply shortest. Objectives may include:

- faithfulness;
- size;
- semantic abstraction level;
- source locality;
- number of owners/faults;
- cognitive chunk count;
- repair utility;
- uncertainty.

Continuum should expose a small Pareto set where objectives conflict.

Active diagnosis

When several defect hypotheses explain the current evidence, Continuum can choose the next experiment maximizing expected information gain:

- branch a schedule;
- inject a fault;
- request an event field;
- enable instrumentation;
- check a derived property;
- run a model bound.

The proposed experiment and its assumptions are explicit.

Human study

This is the preregistered G8 study (plan §21.1): it covers the three human-executed docs/34 acceptance workflows (new model; existing Rust system; review) and two cohorts — Rust newcomers completing the deterministic/causal workflow, and distributed-systems experts diagnosing real failures — against a raw-trace baseline comparator. The fourth docs/34 workflow (agent repair) is covered by the G2 ACI ablation and ContinuumBench, not the human study. Cohort sizes, instruments, and pass thresholds are fixed in a preregistration expansion of this document, authored in Phase E and published before the study runs in Phase F (docs/52 G8).

Questions:

- Does causal/state-delta presentation improve diagnosis over raw trace?
- Which explanation level best serves experts vs newcomers?
- Do users correctly understand bounds and assumptions?
- Does contrastive branching improve repair quality?
- Does simplification create false confidence?

Metrics:

- accuracy;
- time;
- confidence calibration;
- retained understanding;
- repair correctness;
- ability to explain to another engineer.

Agent study

Ablate:

- raw logs vs Context Packs;
- source-only vs model/refinement context;
- causal vs chronological traces;
- explicit omissions vs silent truncation;
- native expansion operations vs repository search;
- counterfactual branch availability.

Measure task success, token cost, invalid edits, overfitting, and invented claims.

Research outputs

- benchmark corpus of explanations and diagnoses;
- causal-core algorithms;
- contrastive branch search;
- explanation certificates;
- adaptive context compiler;
- instrumentation recommendations;
- human/agent design guidelines.

Document: docs/49_SECURITY_FOR_AUTONOMOUS_AGENTS.md**Security for Autonomous Agents****Security posture**

Agents are useful untrusted principals operating on valuable source, production traces, proof infrastructure, and compute resources.

Capability matrix

Capability	Modeler	Repairer	Prover	Reviewer	Trusted promoter
read granted snapshots	yes	yes	scoped	yes	yes
propose model/patch/proof	yes	yes	yes	no	yes
start bounded task	yes	yes	yes	yes	yes
revise intent	proposal only	no	no	proposal only	policy-dependent
promote evidence	no	no	no	no	yes/checker
access production secrets	no/default	no	no	scoped	scoped
arbitrary network/host exec	no	no	no	no	no/default

Prompt injection boundary

Source code, comments, documentation, logs, model labels, counterexample payloads, and production messages are untrusted content. The agent adapter:

- labels data fields;
- never concatenates them into system/tool instructions;
- escapes rendering;
- enforces capabilities independently of model output;
- logs attempted privileged operations;
- can redact or summarize sensitive fields through trusted code.

Artifact integrity

- content hashes cover canonical bytes and schema version;
- receipts cover referenced artifact identities;
- signed organizational attestations optional;
- CAS publication is transactional;
- handles are opaque and unguessable where confidentiality matters;
- authorization is checked on every dereference.

Worker isolation

Lean, solvers, corpus oracles, generated Rust, and third-party analyzers run with:

- pinned image/toolchain;
- read-only input mount;
- no ambient credentials;
- resource limits;

- network disabled unless required and allowlisted;
- output size/schema limits;
- cancellation and hard-kill fallback;
- audit trace.

Denial of service

Verification naturally permits explosive workloads. Controls:

- preflight complexity estimates;
- per-actor/project budgets;
- state/solver/proof/token quotas;
- resumable tasks;
- fair scheduling;
- duplicate task coalescing;
- artifact/result size limits;
- Forge grammar restrictions;
- cancellation.

Budget exhaustion is not a negative/positive verdict.

Evidence forgery

Only checker services can create validated/proved edges. Clients cannot submit arbitrary status. Every promotion re-fetches and verifies referenced artifacts.

Intent attacks

Policy blocks:

- bound contraction;
- property weakening;
- assumption strengthening;
- fault removal;
- observer coarsening;
- assurance downgrade;
- opaque-boundary expansion.

Allowed redesign creates a new intent version and explicit review.

Benchmark integrity

Hidden tasks/graders reside outside agent-readable snapshots. Tool outputs cannot reveal expected patches. Semantic clone detection reduces training leakage.

Supply chain

Pin Rust/Lean/solver/domain-pack dependencies. Receipts name closure hashes. A dependency epoch change invalidates evidence according to policy.

Incident response

Security events produce separate audit incidents, not model counterexamples. Relevant semantic tasks may be quarantined. Reproducible evidence helps determine whether a malicious patch exploited verifier, adapter, policy, or proof service.

Document: docs/50_AGENT_EVALUATION_AND_REWARD_HACKING.md

Agent Evaluation and Reward Hacking

Core risk

An agent optimized for “make verification pass” may discover shortcuts that are formally valid for a modified problem and disastrous for the real one.

Attack classes

Intent attacks

- weaken property;
- strengthen assumptions/fairness;
- shrink bounds;
- remove faults;
- coarsen observer;
- lower assurance.

Instrumentation attacks

- suppress semantic events;
- mark effects opaque;
- alter correspondence;
- bypass controlled runtime;
- fabricate production telemetry.

Evidence attacks

- reuse stale receipts;
- exploit incremental invalidation bug;
- forge status/handle;
- present sampled result as exhaustive;
- hide unknowns.

Overfitting attacks

- special-case exact seed/trace/value;
- fix one linearization;
- disable progress;
- exploit visible benchmark cases;
- hard-code corpus names.

Resource attacks

- consume unlimited search until lucky;
- spawn redundant workers;
- avoid terminal state;
- return huge contexts to obscure failure.

Defenses

- protected Intent Contract;
- semantic diff;
- hidden variants and mutations;
- neighborhood exploration;
- non-vacuity/positive scenarios;
- Incremental Parity Audit;
- capability security;
- resource-normalized evaluation;
- independent graders/checkers;
- explanation/evidence requirements.

Evaluation methodology

Compare agent systems under:

- identical model/backend where possible;
- fixed wall/token/tool budgets;
- same native operations;
- public and hidden task variants;

- repeated seeds;
- cost and failure trajectory reporting.

Separate contributions from:

- base model;
- ACI design;
- context compiler;
- orchestration;
- verification engine;
- proof retrieval;
- search budget.

Expensive failures

Track trajectories that consume high resources without progress. Continuum can terminate or suspend when:

- evidence graph has no novel nodes;
- repeated invalid actions;
- candidate semantics duplicates archive;
- proof state cycles;
- task is proven unrealizable within grammar;
- budget threshold reached.

The final result includes why work stopped and a continuation if useful.

Agent confidence

Agent confidence is metadata only. Calibration is measured against checked outcomes, but it never changes evidence status.

Training signals

High-quality trajectories include:

- intent-preserving diagnosis;
- valid tool sequencing;
- targeted context expansion;
- counterexample-guided repair;
- proof feedback and correction;
- explicit inconclusiveness;
- successful promotion receipts.

Negative trajectories include gaming attempts and stale/unsupported claims. This can train agents to interact with verification honestly.

Benchmark success

A system succeeds only if it produces the required artifact and evidence. Natural-language answers alone cannot solve code/proof/synthesis tasks.

Document: docs/51_PRODUCT_WALKTHROUGHS.md

Product Walkthroughs

Walkthrough A: cancellation corrupts durable acknowledgement

Intent

Property: every published acknowledgement refers to a durable value.
Faults: cancellation at every checkpoint; crash before/after submit/sync.
Progress: failure-free synced request is eventually acknowledged.
Assurance: bounded exhaustive for 2 replicas, 1 crash, 1 cancellation.

Failure

cargo continuum check returns a four-event causal core:

```
WriteSubmitted(e7)
ReplyReserved(e7)
CancelRequested(task3)
ReplyPublished(e7)
```

The abstract delta is acknowledged += e7 while durable is unchanged. The missing required event is SyncCompleted(e7).

Debug

The developer opens the branch before cancellation. Frontier:

```
SyncCompleted(e7)
CancelRequested(task3)
Deliver(other-message)
```

Selecting SyncCompleted yields a safe branch. Selecting cancellation reproduces failure. Reverse causal step shows finalizer publication.

Repair

An agent proposes to resolve the reply obligation only after sync. Intent diff is unchanged. Neighborhood exploration moves cancellation through adjacent phases. Known lost-abort and stale-epoch mutants remain detected. Promotion receipt is generated.

Walkthrough B: liveness bug hidden by fairness

A model has Retry continuously enabled but the scheduler may forever choose Tick. The original intent declares weak fairness for Retry only under network connectivity.

An agent attempts to add unconditional strong fairness. Semantic diff classifies a stronger environment/scheduler assumption and blocks ordinary repair.

The debugger shows fairness debt and the fair lasso. The valid repair changes the protocol to persist retry intent and couples timer progress to an enabled retry transition.

Walkthrough C: model/program drift

Rust refactor splits Commit into PrepareCommit and PublishCommit. Correspondence maps both as stuttering, so the abstract model never changes. Continuum reports an uncovered observer publication and an ambiguous lens conflict.

The developer selects a correspondence where PublishCommit implements the abstract Commit; a refinement obligation checks the intermediate state cannot leak to observers.

Walkthrough D: production evidence is insufficient

Telemetry records request and acknowledgement but not disk sync completion. Continuum cannot decide AckImpliesDurable and returns Inconclusive with an instrumentation proposal:

```
Record StorageStable(entry_id, epoch) before ReplyPublished.
Required correlation: entry_id + node_epoch.
Estimated added event volume: 0.8%.
```

It does not infer durability from timestamps.

Walkthrough E: Forge invents a protocol variant

The user supplies a broadcast model with holes for acknowledgement and retransmission. Hard constraints require agreement and eventual delivery under eventual connectivity. Objectives minimize messages and stable writes. Forge:

1. retrieves analogous corpus protocols;
2. proposes candidates;
3. uses generalized counterexamples;
4. co-synthesizes an invariant and ranking;
5. archives behaviorally distinct candidates;
6. emits two Pareto candidates with bounded proof receipts;
7. materializes asupersync skeletons.

A human compares quorum geometry, recovery behavior, proof complexity, and benchmark cost before selecting one.

Walkthrough F: multi-agent proof repair

A Rust change invalidates a Lean refinement theorem. Evidence Graph creates:

- proof goal;
- changed correspondence edge;
- finite countermodel to old induction hypothesis;
- relevant lemma slice.

Planner proposes two lemma decompositions. Proof workers attempt both. Adversarial reviewer checks theorem statement and axioms unchanged. Lean accepts one proof; receipt closes transaction.

Document: docs/52_RELEASE_GATES_REV3.md

Revision 3 Release Gates

Note: Reconciled bullet-for-bullet with plan.md §22; the two are updated together.

Phase↔Gate mapping

Phase (plan §21)	Gates it must close
A	G0 (falsification), G1 (workbench identity and lifecycle), G2 (ACI)
B	G3 (intent integrity), G4 (causal debugging and real repair)
C	G5 (incremental trust)
D	G6 (proof service)
E	G7 (Forge)
F	G8 (human usability), G9 (corpus parity), G10 (Continuum 1.0)

G0 – Falsification

All load-bearing experiments in ../notes/G0_SPIKE_MATRIX.md have evidence or an explicit redesign decision, recorded in the matrix itself. A failed or unexecuted freeze-blocking item blocks interface freeze.

Staging rule: G0 closes in Phase A for every item whose required experiment runs against Phase A machinery – the freeze-blocking subset DX-01–05, 07, 08, 10, 12, 13, 14. An unexecuted or failed item in this subset blocks interface freeze. Items whose experiments require later subsystems are re-homed to the gates that own them – DX-06 (neighborhood/mutation campaign) → G4, DX-11 (proof-service isolation) → G6, DX-09 (human diagnosis study) → G8, DX-15 (benchmark leakage) → G9 – and each re-homing is recorded in the matrix as that item’s explicit decision. The matrix carries Status, Evidence, and Decision columns; plan §0.3’s counts are derived from it, not asserted beside it.

G1 – Workbench identity and lifecycle

- snapshots, intent contracts, handles, and artifacts are immutable and content-addressed;
- explicit handles across native API;
- requests are idempotent under idempotency keys;
- continuation resume validates epochs and inputs before any reuse;
- cancellation closes obligations and publishes no partial finality;
- artifact publication is transactional (INV-017);
- authorization is checked independently of handle possession;
- daemon crash recovery leaves no stale index entries or orphan tasks (plan §4.5).

G2 – Agent-computer interface

- generated clients and schemas ship for the native protocol;
- no terminal parsing required;

- explicit handles and resumability;
- stale state rejected;
- Context Packs are bounded, carry omission manifests and expansion handles, and improve agent benchmark effectiveness;
- native ACI beats the disciplined shell baseline on success and cost, or the protocol is redesigned before freeze (G0-DX-10);
- prompt injection corpus cannot trigger privileged operations.

G3 — Intent integrity

- every gaming mutation in the hidden (held-out) suite that falls in a supported fragment is classified as a privileged intent change, across all seven diff dimensions (property/assumption/bound/observer/fault/fairness/assurance);
- mutations outside supported fragments classify as Unknown and block ordinary promotion rather than passing silently;
- intent policy locks are enforced; evidence is invalidated on intent revision;
- no ordinary repair promotes with a protected-intent change.

G4 — Causal debugging and repair

- a real asupersync failure — not only injected mutants — is diagnosed and repaired end-to-end, alongside multiple known mutants;
- causal explanation with a replay-preserving core;
- partial-order debugger including alternate-branch exploration;
- exact and neighboring replay;
- mutation challenge;
- repair transaction closes exact, neighborhood, and mutation gates under the Phase B gate profile (plan §21);
- promotion receipt;
- human review view over the repair transaction.

G5 — Incremental trust

- incremental results continuously match clean builds under the Incremental Parity Audit (plan §9.5);
- reuse edges carry their class (Exact/Validated/Conservative/Experimental) and mismatches are minimized and quarantine the class;
- proof and certificate freshness is tracked;
- crash-safe cache/publication;
- evidence queries and context compilation meet the docs/34 targets at $\geq 10^7$ evidence nodes on the reference workload;
- interactive latency targets (docs/34) hold on reference workloads.

G6 — Proof service

- Lean foundations kernel-check (Revision 2 and Revision 3 theorems, no placeholders);
- pinned Lean environment with per-request isolation and cancellation;
- every proof receipt carries theorem and axiom manifests;
- agent proof repair accepted only by kernel;
- certificate mutations are rejected;
- proof Context Packs improve proof-worker success/cost.

G7 — Forge

- typed holes and finite CEGIS;
- safety and positive/non-vacuity scenarios;
- independent candidate verification;
- diversity archive has semantic, not merely syntactic, spread;
- hidden variant generalization;
- explicit unrealizability/unknown distinction;
- unrealizability produces reusable evidence where supported;
- materialized model/Rust/proof obligations.

G8 — Human usability

- the preregistered study (plan §21.1) covers both cohorts — Rust newcomers completing the deterministic/causal workflow, and distributed-systems experts correctly diagnosing real failures;

- the preregistration (expanded docs/48) is published before the study runs; results are graded only against its fixed thresholds;
- explanation beats raw trace baseline on diagnosis accuracy and time;
- assurance confidence is calibrated, not merely no worse than baseline;
- progressive disclosure reaches exact artifacts;
- accessibility and non-color CLI/UI semantics: no critical workflow requires color or a rendered graph;
- no critical workflow requires formal-methods folklore.

G9 — Corpus interaction parity

- 80 validated TLA+ families at their declared parity level (corpus/tla-examples/PARITY_LEVELS.md), measured per plan §19.4's held-out discipline;
- every family carries its interaction artifacts: native model, expected verdict and state facts, meaningful explanation, failure/mutation task, proof/refinement support where applicable, and an agent benchmark artifact — this is *interaction* parity, per B23, not a family count.

G10 — Continuum 1.0 (real adoption)

- two real project migrations;
- two materially different real projects remove bespoke DST infrastructure;
- at least two migrated projects stop requiring a separate TLA+ workflow for normal development;
- agent-driven repair is used on real changes under review;
- production evidence returns valid pass/fail/inconclusive classifications (INV-008);
- public ContinuumBench;
- documented assurance envelopes;
- operating cost is acceptable;
- zero known paths for an unprivileged agent to promote false evidence.

Release blocker doctrine

A missing feature can be documented as unsupported. A misleading assurance result, replay failure, stale receipt, hidden intent change, or unauthorized promotion is a release blocker at every gate.

A confirmed false-positive success verdict triggers the soundness incident policy in docs/09: block release, revoke affected claim IDs, publish affected semantic epochs, ship an artifact scanner, add a permanent regression, and reevaluate whether the producing engine remains eligible for certified mode.

Document: docs/53_SPIKE_FINDINGS_REV3.md

Revision 3 Executable Spike Report

Executed with: Python 3.13.5

Result artifact: results/r3-spike-results.json

Runner: run_r3_spikes.py

All eight Revision 3 spike groups passed their internal assertions.

1. Context Pack causal slicing

The synthetic durability trace contained **200 events**: four causal events and 196 observer-independent noise events.

Results:

- replay-preserving core: **4 events**;
- omitted events: **196**;
- raw JSON: **17,645 bytes**;
- Context Pack: **747 bytes**;
- compression ratio: **23.62×**;
- core was 1-minimal under single-event deletion;
- core replay still produced `acked = true ∧ durable = false`.

Core:

WriteSubmitted
 → ReplyReserved
 → CancelRequested
 → ReplyPublished

This validates the basic artifact shape, not the general context compiler. Real traces need conflict, abstraction, proof, and domain semantics.

2. Semantic intent diff

The spike correctly distinguished an ordinary source guard repair from six formal reward-hacking mutations:

Mutation	Classification
source guard repair	protected intent unchanged
remove property term	property weakened
add favorable assumption	assumptions strengthened
nodes 5 → 3	bound contracted
hide SyncCompleted	observer coarsened
remove crash fault	fault envelope contracted
validated → sampled	assurance downgraded

All expected protected changes were detected.

3. Explicit agent protocol state

The in-memory workbench demonstrated:

- canonical workspace snapshot identities;
- idempotent identical task creation;
- rejection of idempotency-key reuse with different request;
- resumable continuation;
- rejection of continuation under a different workspace snapshot.

This supports the explicit-handle architecture. It does not yet test concurrency, authorization, persistence, or crash recovery.

4. Forge finite CEGIS

Candidate grammar:

true
 false
 reserved
 ~reserved
 ~synced
 reserved $\wedge$ synced
 synced

Constraints:

- safety: acknowledgement implies synced;
- non-vacuity/progress: a reserved and synced request can acknowledge.

CEGIS iterations:

1. false rejected by progress counterexample;
2. reserved rejected by safety counterexample;
3. synced accepted.

The only valid grammar candidates were synced and reserved $\wedge$ synced; AST-size objective selected synced. Invalid candidates were exhausted in the finite grammar.

This validates coupling safety with positive behavior. It is not evidence that broad protocol synthesis will scale.

5. Incremental query invalidation

A ten-query synthetic graph was tested under property, source, model, proof-only, and domain-profile edits.

Findings:

- all incremental outputs matched clean recomputation;
- property edit reused source parsing/extraction and model elaboration;
- proof-only edit invalidated only the proof receipt;
- source edit invalidated extraction, correspondence, refinement, context, and proof receipt;
- domain-profile edit invalidated both model and program semantic paths.

The spike validates the edge taxonomy idea but not production dependency capture.

6. Causal debugger

At a configuration with submitted and reserved work, the enabled frontier was:

SyncCompleted
CancelRequested

Branches:

- SyncCompleted → NormalPublishes produced $\text{acked} \wedge \text{durable}$;
- CancelRequested → FinalizerPublishes produced $\text{acked} \wedge \neg \text{durable}$.

The spike emitted why-enabled conditions, first conflicting choices, abstract branch difference, and causal reverse choice.

7. Proof-oriented lens conflict

The abstract field durable could legitimately map to either:

- local storage stability;
- quorum acknowledgement under a stronger domain contract.

The reverse abstract edit therefore had two concrete candidates. The spike returned AmbiguousCorrespondence and a distinguishing obligation instead of selecting silently. Direct acknowledgement/publication mapping remained unambiguous.

8. Multi-agent Evidence Graph

The immutable graph spike coordinated two independent patch proposals and verifier/kernel evidence. It demonstrated:

- byte-identical proposals deduplicate by content identity;
- an agent cannot publish validated or proved status;
- stale clean-parity evidence cannot complete a repair envelope;
- the safe acknowledgement patch promotes only after fresh exact replay, neighborhood, mutation-challenge, and clean-parity evidence exist;
- a fault-model contraction is blocked as a protected intent change;
- contradictory claims about the same subject and predicate become an explicit conflict rather than last-writer-wins state.

The run produced ten immutable nodes, seven typed edges, one surfaced claim conflict, and one accepted repair envelope. This validates authority separation and coordination semantics, not distributed storage or Byzantine agent resistance.

What the spikes changed

The experiments support five architecture decisions:

1. bounded Context Packs can be dramatically smaller while preserving a witness;
2. intent integrity needs a dedicated semantic diff, not code review convention;
3. explicit handles provide deterministic agent handoff and stale-state rejection;
4. Forge requires positive/non-vacuity constraints and independent verification.
5. Multi-agent work needs immutable evidence, authority-separated promotion, stale-evidence rejection, and explicit conflicts.

What remains unproven

- context slicing on real causal/conflict graphs;
- human/agent performance improvements;
- sound semantic implication for rich temporal properties;
- persistent/concurrent daemon behavior;
- incremental correctness under real edits;
- scalable CEGIS/co-synthesis;
- verified lens laws and effectful correspondence;
- Lean kernel checking of Revision 3 seed modules.

Document: docs/54_THEORY_OF_AGENTIC_ACCELERATION.md

Theory of Agentic Acceleration

Claim

Advanced coding agents can expand the design space humans explore, but only when the environment supplies dense truthful feedback and prevents objective gaming.

Current bottleneck

Concurrent/distributed algorithm work has a hostile search landscape:

- failures are sparse and schedule-dependent;
- tests provide weak gradients;
- logs are noisy and linearized;
- specifications drift;
- proofs require specialized context;
- performance objectives conflict with correctness;
- a local fix can create a remote liveness failure.

An agent can generate candidates faster than humans can validate them. Without Continuum, that increases risk rather than progress.

Acceleration loop

```
human intent
  ↓
agent proposes model/algorithm/invariant/proof
  ↓
Continuum finds exact counterexample or evidence
  ↓
Context compiler produces dense semantic feedback
  ↓
agent revises the right artifact
  ↓
repair/synthesis transaction resists gaming
  ↓
independent checker/Lean accepts evidence
  ↓
quality-diversity archive preserves discoveries
```

The critical quantity is not raw candidate throughput. It is **validated information gained per unit cost**.

Feedback density

A raw failing test gives roughly one bit: pass/fail. A Continuum failure can provide:

- causal core;
- violated abstract relation;
- missing order;
- proof obligation;
- safe contrastive branch;
- intent constraints;

- source correspondence;
- counterexample generalization.

This turns search from blind mutation toward structured inference.

Search decomposition

Agents are strongest when tasks are bounded and contexts are relevant. Evidence Graph decomposes work by missing edge rather than arbitrary role prompts. This supports parallelism without losing coherence.

Trust amplification

Verification allows the system to use more aggressive candidate generators because correctness authority remains external. An untrusted creative agent may search unusual ideas; a small checker decides promotion.

Invention modes

Rediscovery

Reconstruct known algorithms from intent/holes. Validates Forge and provides training trajectories.

Optimization

Find lower-cost implementations/refinements of a fixed protocol.

Variant discovery

Explore alternative quorum/recovery/cancellation structures while retaining intent.

Assumption frontier

Map tradeoffs between achievable guarantees and environment assumptions.

New protocol discovery

Search broad typed spaces, preserve semantic diversity, and produce proof-bearing artifacts suitable for human mathematical analysis.

Scientific safeguards

A claimed novel algorithm requires:

- explicit prior-art search;
- precise intent/assumptions;
- reproducible model and implementation;
- evidence/proof receipt;
- comparison baselines;
- adversarial review and hidden variants;
- performance evaluation;
- clear limits.

Continuum accelerates discovery; it does not confer novelty by itself.

Long-term research opportunity

The combination of:

- true-concurrency semantics;
- property-directed causal abstraction;
- proof-producing reduction;
- program/model/proof co-synthesis;
- agent-native interaction;
- quality-diversity search;
- Lean-checked metatheory;

could support a new experimental mathematics of distributed algorithms. Candidate families become data; counterexamples become lemmas about impossible regions; proofs and implementations co-evolve; humans inspect the resulting structural patterns.

That is the micro-revolution: not agents writing more race-prone code, but agents and humans exploring systems design with a verification substrate strong enough to make radical experimentation responsible.

Document: docs/55_AGENT_API_REFERENCE_SKETCH.md

Agent API Reference Sketch

Non-normative projection. This document is a projection of RFC 0026 (continuumd native protocol), RFC 0027 (agent tool protocol), and plan §10.2, and is regenerated from them. On any divergence, the RFCs and the plan win.

This is a shape contract, not the final wire IDL.

Handles

Full prefix set per plan §4.4:

WorkspaceHandle	ws_...
IntentHandle	in_...
ModelHandle	model_...
CausalGraphHandle	cir_...
CrashpackHandle	crash_...
ContextHandle	ctx_...
ProofArtifactHandle	proof_...
ProofStateHandle	ps_...
TaskHandle	task_...
EvidenceHandle	ev_...
DebugHandle	dbg_...
ReceiptHandle	receipt_...
RepairHandle	rt_...
ForgeHandle	forge_...
ContinuationHandle	cont_...
CapabilityHandle	cap_...
DiffHandle	diff_...

Workspace

workspace.create

Inputs: root/overlay/dependency/config references.

Output: immutable workspace handle plus diagnostics.

workspace.fork

Inputs: base handle and patch/overlays.

Output: new handle and textual/semantic pre-diff availability.

workspace.diff

Inputs: two handles and requested layers.

Output: diff artifact.

workspace.seal

Seals a snapshot as immutable; sealed snapshots are the only valid semantic inputs.

Intent

intent.get

Returns canonical Intent Contract.

intent.diff

Returns classified changes, proof obligations, and policy impact.

intent.propose_revision

Creates a proposal only; cannot mutate existing intent.

intent.accept

Privileged. Accepts a proposed intent into the registry; drafts gain INV-001 protection only on explicit acceptance.

intent.reject

Privileged. Rejects a proposed intent revision with a typed reason.

intent.lock

Privileged. Applies an intent lock (plan §5.4); subsequent mutation attempts fail with `IntentMutationDenied`.

Verification

verification.start

Inputs:

```
{
  "snapshot": "ws_...",
  "intent": "in_...",
  "target": {"kind": "property", "id": "AckImpliesDurable"},
  "portfolio": "interactive|promotion|custom",
  "budget": {},
  "context_policy": {}
}
```

Outputs task handle or cached result.

verification.result

Returns typed verdict, assurance, evidence roots, continuation, omissions.

verification.await

Blocks (within budget) on a started verification task and returns its result.

Model

model.check

Checks a model against a property under intent and budget; returns typed verdict and evidence.

model.explore

Explores model behavior under a strategy and budget; returns explored-region evidence.

model.compare

Compares two models/configurations; returns a semantic diff artifact.

Program

program.extract

Extracts the semantic model from a Rust crate snapshot and annotations.

program.run

Runs the program under controlled semantics; returns execution artifacts.

program.replay

Replays a recorded execution deterministically; divergence fails with `ReplayDiverged`.

Refinement

refinement.check

Checks a refinement correspondence between program and model; returns typed verdict.

refinement.explain

Explains a refinement result or failure with a Context Pack.

Context

context.compile

Inputs: evidence/question/audience/budget.

Output: Context Pack.

context.expand

Inputs: context, anchor, relation, budget.

Output: new immutable Context Pack referencing parent.

Debug

debug.open

Inputs: crashpack/execution/configuration.

Output: debugger handle and current frontier.

debug.state

Inputs: handle and optional observer projection.

Output: current concrete/abstract state view.

debug.enabled

Inputs: handle.

Output: enabled events at the current frontier.

debug.step_event

Inputs: handle, event.

Output: new handle/state delta (steps one semantic event).

debug.step_abstract

Inputs: handle.

Output: new handle/state delta (steps one abstract transition).

debug.reverse_causal

Inputs: handle.

Output: handle at the causal predecessor.

debug.branch

Inputs: handle and alternate enabled event/fault.

Output: sibling handle.

debug.compare

Inputs: two handles and observer.

Output: semantic branch diff.

debug.why_enabled

Inputs: handle, event.

Output: explanation of why the event is enabled.

debug.why_blocked

Inputs: handle, event.

Output: explanation of why the event is blocked.

debug.export

Inputs: handle.

Output: branch exported as a crashpack or regression scenario.

Failure

failure.explain

Returns a causal explanation of a failure at the requested level.

failure.minimize

Returns a minimized counterexample preserving the failure.

failure.branch

Explores an alternate branch from a failure's causal frontier.

Repair

repair.begin

Inputs: failure, base snapshot/intent, policy.

Output: transaction handle.

repair.apply

Inputs: transaction, patch/model/proof changes, hypothesis.

Output: new transaction and candidate snapshot.

repair.attach

Attaches additional evidence/artifacts (e.g. from another agent) to a transaction; produces a new transaction version.

repair.evaluate

Inputs: transaction and optional budget.

Output: evidence progress/status/continuation.

repair.resume

Resumes a suspended evaluation from its continuation; inputs and epochs are validated.

repair.review

Returns the review view of a transaction: semantic/intent diff, gate status, evidence.

repair.promote

Privileged. Returns receipt or typed policy failure.

repair.reject

Privileged. Rejects a transaction with a typed reason.

Forge

forge.create

Inputs: snapshot, intent, sketch/holes/objectives/diversity/assurance/budget.

forge.step

Runs one or bounded portfolio iteration; returns archive deltas and counterexamples.

forge.archive

Queries the Pareto/quality-diversity archive by objective/descriptor/evidence status.

forge.materialize

Creates a design transaction with model/Rust/proof artifacts.

Proof

proof.goal

Returns exact goal, proof Context Pack, and proof-state handle.

proof.attempt

Applies tactic/term to a proof-state handle and returns children/diagnostics.

proof.check

Strict isolated kernel check and receipt.

proof.slice

Returns the proof dependency slice (curated declarations) for a goal.

Benchmark

benchmark.run

Runs a benchmark task under declared budget and graders; returns evidence, not self-scored results.

Tasks

task.status

Includes semantic milestones, resource use, committed evidence, and terminal reason.

task.cancel

Requests cancel-correct shutdown.

task.resume

Requires continuation plus matching inputs; optional larger budget.

task.subscribe

Streams progress events over the connection; events are hints — committed artifacts and `task.status` are authoritative.

Evidence

evidence.get

Fetch typed artifact metadata or bounded content.

evidence.query

Graph query with node/edge/status/property filters.

evidence.verify

Runs relevant independent checker; does not accept client-declared status.

Query

query.explain_reuse

Explains which memoized results were reused for a derivation and why.

query.explain_invalidations

Explains which results a change invalidated and through which dependency edges.

query.clean_compare

Compares incremental results against a clean recompute (Incremental Parity Audit, plan §9.5).

Error taxonomy

Stable codes per plan §10.3:

StaleSnapshot
UnsupportedSemanticFeature
IntentMutationDenied
InsufficientEvidence
BudgetExhausted
ContinuationEpochMismatch
AmbiguousCorrespondence
UntrustedDomainBoundary
CertificateRejected
ReplayDiverged
CapabilityDenied
PolicyGateFailed

Continuum: Master Implementation Plan & Comprehensive Dossier

ProtocolVersionUnsupported (protocol-level, RFC 0026)
IdempotencyKeyReused (protocol-level, RFC 0026)
MalformedRequest (protocol-level, RFC 0026)

Errors may carry recovery as a list of allowed operations with pre-filled arguments — never free-form commands.
BudgetExhausted is never a semantic verdict; it carries the continuation when one exists.

Common result rules

Every result includes:

- request/task identity;
- snapshot and intent;
- semantic/proof/engine epochs;
- verdict/status;
- artifact handles;
- omissions/unknowns;
- cost/budget;
- valid next operations;
- audit correlation.

Reference Documents: Architecture Decision Records (ADRs)

Document: adr/0001-asupersync-execution-substrate.md

ADR-0001: Use asupersync as the execution substrate

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

Continuum needs production execution, deterministic Lab execution, structured task ownership, virtual time, explicit capabilities, cancellation semantics, and replay. Implementing another runtime would duplicate asupersync and create two incompatible notions of task, cancellation, and quiescence.

Decision

Continuum uses asupersync for concrete production and Lab execution. Continuum owns neither a competing executor nor replacement task/region lifecycle. Integration is isolated in continuum-asupersync, which translates a stable public semantic-event contract into CIR.

Continuum remains capable of model-only execution without asupersync. The normative abstract semantics are owned by Continuum, so the runtime cannot silently redefine model behavior.

Consequences

Benefits:

- cancel-correctness and obligations become structural;
- real code can run in deterministic verification mode;
- bespoke DST scheduler/time/replay layers can be deleted;
- the product has an immediate practical wedge.

Costs/risks:

- dependency on a young runtime;
- possible pressure to use private Lab internals;
- shared-runtime/common-mode bugs;
- narrower initial audience than Tokio-based systems.

Alternatives considered

1. Build on Tokio: larger ecosystem, but cancellation/task ownership remain conventional and Continuum would need to invent the missing semantics.
2. Runtime-neutral World trait only: attractive abstraction, but risks lowest-common-denominator semantics and repeated adapters.
3. Build a custom executor: rejected as wasteful and semantically divergent.

Validation and rollback

G0 requires stable event hooks, replay sufficiency, and no private scheduler dependency. If these cannot be achieved, preserve the abstract/CIR design and replace the concrete adapter rather than forking the runtime indefinitely.

Document: adr/0002-causal-intermediate-representation.md

ADR-0002: Adopt a causal intermediate representation as the semantic interchange

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

State vectors and total traces are insufficient as a common format for DPOR, runtime traces, cancellation, refinement, and production conformance. Total orders contain arbitrary scheduling choices; flat states lose causality and lifecycle evidence.

Decision

Define CIR as a versioned event/configuration representation with causality, conflict, typed footprints, phases, obligations, durability, time constraints, faults, observations, abstract deltas, and provenance.

State graphs and total traces are derived projections. CIR is a semantic format, not merely a logging schema.

Consequences

Enables one artifact to drive replay, reduction, refinement, conformance, and diagnosis. It also introduces complexity: event-structure validation, canonical encoding, and state projection become foundational.

Alternatives considered

1. Transition-system IR only: simpler but weak for production partial orders and true-concurrency reduction.
2. OpenTelemetry/JSON traces: interoperable but semantically incomplete.
3. Rust AST/MIR as IR: too implementation-specific and unstable.
4. TLA+ AST as IR: too language-specific.

Validation and rollback

The CIR reference semantics must reconstruct equivalent states under independent reorders, round-trip canonical encodings, and support the first vertical slice. If true-concurrency fields prove unnecessary, they may remain optional extensions; causality and typed footprints remain mandatory.

Document: adr/0003-no-ambient-nondeterminism.md

ADR-0003: Prohibit ambient nondeterminism in verified cores

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

Deterministic testing and exhaustive exploration are unsound when code can bypass controlled time, entropy, scheduling, I/O, or process lifecycle. Discipline alone is insufficient across a dependency closure.

Decision

Verified cores must obtain nondeterministic or externally observable effects through Cx or Continuum capabilities. Continuum adds lint, MIR-audit, runtime-trap, dependency-report, and production-coverage layers. Unsupported effects downgrade assurance or stop analysis.

Consequences

Signatures become more explicit and migration may be intrusive. The payoff is replay, effect provenance, and a meaningful verification boundary.

Alternatives considered

1. Best-effort mocking: rejected because hidden effects become silent omissions.
2. LD_PRELOAD/syscall interposition: useful for compatibility experiments, not a primary semantic foundation.
3. Whole-VM determinism: broad but expensive and opaque.

Validation and rollback

G1 migration reports quantify remaining ambient effects. The rule may be scoped to declared verified cores; application shells can remain unverified with explicit boundaries.

Document: adr/0004-standalone-models-and-zoomable-refinement.md

ADR-0004: Support standalone models and zoomable refinement views

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

Executing real code under faults is not a replacement for TLA+ if users cannot model before implementation or abstract away operational detail. Conversely, a separate model without refinement drifts.

Decision

Continuum supports standalone typed relational models and multiple views from service semantics through protocol, operational model, implementation, and production observation. Adjacent views carry explicit refinement obligations and can be mixed by component.

Consequences

The language and refinement subsystem become major workstreams. This complexity is necessary to replace abstract modeling rather than only DST infrastructure.

Alternatives considered

1. Rust-only executable models: too concrete.
2. One abstract model plus code tests: retains the model/code gap.
3. Automatic extraction of the model from arbitrary Rust: unrealistic and tends to reproduce implementation detail.

Validation and rollback

G2 requires model-before-code plus refinement of a real implementation. If mapping cost remains excessive, prioritize explicit views and multi-grain summaries over speculative automatic extraction.

Document: adr/0005-partial-order-primary-semantics.md

ADR-0005: Make partial-order executions primary

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

Interleaving semantics replicate equivalent schedules and distort production observations. Modern DPOR, unfoldings, interval-pomset theory, and partial-order trace validation all operate on causal structure.

Decision

CIR executions are partial orders/configurations. Deterministic total linearizations exist for replay and legacy engines. Search engines may use DPOR, unfolding, state graphs, or symbolic encodings while binding results to the same causal semantics.

Consequences

This creates richer semantics and stronger reduction opportunities. It also raises soundness burden around independence, observers, fairness, and certificate checking.

Alternatives considered

1. Total traces primary with vector-clock annotations: easier, but reduction and conformance remain add-ons.
2. State graphs only: loses causal explanations.
3. Petri nets as the only core: too restrictive for arbitrary state and effects.

Validation and rollback

The no-reduction reference must agree with partial-order engines. If an advanced true-concurrency backend loses on benchmarks, it can be killed without changing the CIR contract.

Document: adr/0006-typed-assurance-lattice.md

ADR-0006: Represent assurance as a multidimensional typed claim

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

“Passed” conflates sampling, bounded search, exhaustive finite checking, inductive proof, trace observation, and refinement. A single maturity number hides assumptions and TCB differences.

Decision

Every result records semantic coverage, exploration class, property class, evidence class, implementation linkage, assumptions, exclusions, and trusted components. Human output is derived from this object.

Consequences

Claims become honest and composable, but UI and CI must manage more metadata. Marketing simplicity is intentionally sacrificed.

Alternatives considered

1. Linear assurance levels: too lossy.
2. Engine-specific output: impossible to compare or automate safely.
3. Prose caveats: not machine-checkable.

Validation and rollback

Schemas and golden verdict examples are part of G0. Claim rendering is tested against the ledger; unsupported combinations are rejected.

Document: adr/0007-verification-engine-portfolio.md

ADR-0007: Use a portfolio of verification engines over one semantics

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

Explicit search, DPOR, decision diagrams, BMC, PDR, liveness rankings, timed zones, and probabilistic methods have complementary performance envelopes. Selecting one algorithm would cap the system.

Decision

Define stable semantic/partitioned-next-state interfaces and let several engines compete or cooperate. A portfolio scheduler can allocate resources, but each engine reports typed evidence and cannot elevate assurance without proof.

Consequences

The system is more capable and can exploit model structure. Integration and maintenance cost rise; therefore engines are promoted individually behind gates.

Alternatives considered

1. Build only a fast explicit checker: insufficient for unbounded/liveness work.
2. Shell out to unrelated tools without shared semantics: workflow collage and inconsistent evidence.
3. One universal symbolic IR: risks forcing all models into solver-friendly forms.

Validation and rollback

Baseline engines are reference explicit, simulation, and DPOR. Later engines must demonstrate unique solved cases or practical wins before becoming core dependencies.

Document: adr/0008-proof-carrying-results.md

ADR-0008: Require checkable certificates for strong success claims

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

A high-performance verifier will be large, parallel, and heuristic. Trusting its “safe” verdict makes the verifier part of the critical system. Counterexamples are easier to validate than successful exhaustion or induction.

Decision

Continuum defines certificate families and a small independent continuum-kernel. Certified assurance requires the kernel to bind evidence to model, property, scope, assumptions, and semantic epoch.

Consequences

The TCB shrinks and results become portable. Certificate engineering can be difficult, especially for POR, SMT theories, liveness, and probabilistic claims.

Alternatives considered

1. Trust the Rust engine: rejected for high assurance.
2. N-version agreement only: useful but not proof.
3. Require a proof assistant for every run: too expensive initially.

Validation and rollback

Start with counterexample replay and finite-state closure certificates. Solver/liveness/POR certificates remain lower assurance until their checker paths mature.

Document: adr/0009-production-partial-order-conformance.md

ADR-0009: Validate production traces as partial orders

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

Distributed traces are incomplete and only partially ordered. Forcing collector order into a model can create false violations or false validity. Recent trace-validation work shows the value of solving for legal orders.

Decision

Production semantic events carry causal predecessors, local sequence, and time intervals. Offline checking solves for an execution/completion consistent with the abstract model and reports valid, invalid, insufficient observation, semantic mismatch, or resource inconclusive.

Consequences

Continuum can close the production feedback loop. Instrumentation, privacy, integrity, and solver scaling become significant work.

Alternatives considered

1. Total-order logging: expensive and still not necessarily truthful.
2. Linearizability histories only: too narrow for non-atomic protocols.
3. Runtime monitors for local properties only: insufficient for global conformance.

Validation and rollback

G5 requires real staging/production traces, injected violations, explicit incompleteness, and trace-to-Lab reproduction. Until then the feature is experimental.

Document: adr/0010-cancellation-calculus.md

ADR-0010: Make cancellation and obligations part of formal semantics

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

Cancellation is ubiquitous in async systems yet often modeled as task disappearance. Asupersync supplies structural lifecycle and obligations that can support stronger reasoning.

Decision

Define first-class cancellation events, phases, responsiveness assumptions, reserve/commit/abort semantics, obligation ownership/transfer, finalization, and region quiescence. Add safety and liveness property primitives over them.

Consequences

Continuum gains a unique correctness dimension and can verify shutdown/race behavior. The calculus must remain compositional across adapters and host boundaries.

Alternatives considered

1. Treat cancellation as crash: semantically wrong.
2. Ignore it in models: leaves a major implementation gap.
3. Model manually per protocol: duplicates subtle semantics.

Validation and rollback

The calculus is accepted provisionally. It graduates when standard channel/storage/process packs satisfy compositional rules and real bugs are expressed more clearly than with ad hoc state.

Document: adr/0011-semantic-domain-packs.md

ADR-0011: Represent external systems as versioned semantic domain packs

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

Bespoke DSTs repeat network, storage, clock, process, and service fakes. Generic mocks hide fidelity and fault assumptions.

Decision

A domain pack includes ideal, Lab, and production handlers; operation schemas; event phases; fault algebra; independence; abstraction; fidelity profile; tests; and mutants.

Consequences

Substantial reuse and explicit assurance become possible. Pack development is expensive and domain expertise is required.

Alternatives considered

1. Simple traits with mocks: too weak.
2. Monolithic world simulator: poor modularity.
3. Assume host libraries are correct: no fault semantics.

Validation and rollback

G1 proves the contract by replacing one real DST. A second project must use packs without engine modifications.

(G1 here refers to the Revision 2 gate scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22.)

Document: adr/0012-dual-authoring-surfaces.md

ADR-0012: Provide a standalone model language and a Rust integration surface

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

A single syntax cannot simultaneously optimize arbitrary mathematics and ordinary implementation integration. Rust-only modeling harms abstraction; a separate language alone harms linkage.

Decision

.ctm is the abstract model surface. Rust macros/traits expose semantic actions and views in implementation code. Both lower to the same typed semantic AST/CIR.

Consequences

Two frontends add work but avoid a false compromise. Normalized semantics rather than syntax is the compatibility contract.

Alternatives considered

1. Quint only: viable bootstrap but makes Continuum dependent on an external evolving language and limits custom runtime semantics.
2. Rust DSL only: rejected for abstraction.
3. TLA+ frontend first: too much compatibility burden.

Validation and rollback

Prototype syntax is explicitly unstable. User tests compare .ctm expressiveness and friction with Quint/TLA+ before stabilization.

Document: adr/0013-exact-state-identity.md

ADR-0013: Use exact canonical state identity in exhaustive and certified modes

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

Hash compaction and fingerprints are fast but collisions can suppress states. A system promising stronger assurance should not depend on collision improbability.

Decision

Canonical structural encodings define identity. Hashes index and partition; collisions resolve by exact comparison. Artifact digests use cryptographic hashes, but proof validity still derives from decoded structure.

Consequences

Memory/time may increase relative to fingerprint-only systems. Delta encoding, interning, compression, and batch processing recover performance.

Alternatives considered

1. 64-bit fingerprints: rejected for certified modes.
2. 256-bit hashes as identity: extremely safe but still an assumption; acceptable only for non-certified modes if labeled.
3. Full object comparison without canonicalization: nondeterministic and slow.

Validation and rollback

Benchmark exact overhead. If too high, support clearly labeled probabilistic-identity mode while retaining exact certified mode.

Document: adr/0014-explicit-fairness-and-ranking-liveness.md

ADR-0014: Use explicit fairness plus automata and ranking-function liveness lanes

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

TLA+-class replacement requires liveness. Finite SCC checking and symbolic/parameterized liveness need different techniques, and hidden global fairness is dangerous.

Decision

Fairness is typed and scoped. Finite models use temporal automata/SCC/Emerson-Lei conditions. Symbolic and parameterized models use ranking/progress reductions where applicable. Certificates expose fairness and progress evidence.

Consequences

Liveness becomes explainable and connected to runtime responsiveness. It remains one of the largest research risks.

Alternatives considered

1. Safety only: cannot replace TLA+.
2. Finite-run timeouts: testing, not liveness.
3. One global fair scheduler: obscures unrealistic assumptions.

Validation and rollback

G4 requires known liveness bugs, fair-cycle diagnostics, ranking proofs, and progressive refinement. Unsupported liveness fragments remain inconclusive.

Document: adr/0015-strong-refinement-for-hyperproperties.md

ADR-0015: Distinguish ordinary refinement from hyperproperty-preserving refinement

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

Trace inclusion/linearizability does not preserve all probability distributions or security hyperproperties. Calling every mapping “refinement” is misleading.

Decision

Continuum defines trace, stuttering, forward, progressive, and strong observational refinement classes. A property declares which class is sufficient. Hyperproperty claims use dedicated engines and proof obligations.

Consequences

Results become semantically precise. Stronger refinement can be much harder or impossible for some implementations.

Alternatives considered

1. Ignore hyperproperties: acceptable only for systems without those claims.
2. Assume linearizability preserves everything: unsound.
3. Always demand strongest refinement: impractical and unnecessarily restrictive.

Validation and rollback

Initial implementation supports finite trace/forward simulation. Hyperproperty modes remain experimental until validated against AutoHyper-style tools and known examples.

Document: adr/0016-typed-timed-and-probabilistic-extensions.md

ADR-0016: Add timed and probabilistic semantics as typed extensions

Status: Accepted in principle; deferred

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

Time and randomness matter for leases, retries, randomized protocols, and reliability. Mixing them implicitly into the deterministic core risks semantic confusion and unsound POR.

Decision

Keep deterministic nondeterministic semantics foundational. Add explicit clock/probability types, schedulers, zones, MDPs, statistical claims, and specialized certificates behind separate property/engine classes.

Consequences

The core remains comprehensible. Specialized tools can be integrated without weakening deterministic evidence.

Alternatives considered

1. Encode probability as nondeterministic choice: loses quantitative meaning.
2. Put real-valued time everywhere: harms finite exploration.
3. Defer forever: misses important systems.

Validation and rollback

No implementation before G4 unless a concrete project demands it. Compare against UPPAAL/IMITATOR/Storm/PRISM.

Document: adr/0017-solver-and-tcb-policy.md

ADR-0017: Treat solvers as accelerators unless their evidence is checked

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

SMT/SAT/CHC solvers are essential but large and fallible. Solver diversity is useful but does not shrink the formal TCB.

Decision

SAT models are replayed. UNSAT results are CHECKED_CERTIFICATE only when proof artifacts are independently checked; otherwise they are TRUSTED_SOLVER. Solver invocations are pinned and preserved.

Consequences

Honest claims may appear weaker than other tools, but users can see the actual trust boundary.

Alternatives considered

1. Trust solver unconditionally: common but inconsistent with proof-carrying goal.
2. Implement all solvers: infeasible.
3. Avoid solvers: cripples symbolic verification.

Validation and rollback

Alethe/Carcara and LRAT-style paths are first targets. Unsupported theory proofs remain explicitly solver-trusted.

Document: adr/0018-semantic-versioning-and-replay.md

ADR-0018: Version semantics independently from APIs and artifact formats

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

Semver of crates does not identify the meaning of an old model or trace. Replay can silently change under new pack/runtime semantics.

Decision

Pin distinct model-language, semantic epoch, CIR, certificate, crashpack, and pack versions. Every evidence artifact binds all digests. Breaking semantics cause refusal or explicit migration.

Consequences

Long-lived replay and evidence become possible. Version management and migration tooling are mandatory.

Alternatives considered

1. Use crate semver only: insufficient.
2. Best-effort backward compatibility: dangerous for evidence.
3. Freeze semantics permanently: unrealistic before maturity.

Validation and rollback

G0 schemas include all version fields. Release tests replay a corpus from prior supported epochs.

Document: adr/0019-adapter-isolation.md

ADR-0019: Isolate runtime/compiler/foreign-tool adapters from semantic core

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

Asupersync, rustc internals, TLA+/Quint frontends, and solvers evolve. Direct dependencies throughout the workspace would make semantic code unstable and expand the TCB.

Decision

Each foreign system has one adapter crate/process boundary. Adapters emit normalized, versioned objects and maintain conformance fixtures. The kernel does not depend on them.

Consequences

Churn and licensing/dependency complexity are contained. Some zero-copy opportunities may be sacrificed.

Alternatives considered

1. Shared foreign types across crates: convenient initially, expensive later.
2. Dynamic plugins everywhere: unstable and insecure.

Validation and rollback

Dependency graph CI enforces the boundary. Adapter replacement must not change semantic digests for the conformance corpus.

Document: adr/0020-experimental-mathematics-governance.md

ADR-0020: Gate speculative mathematics behind falsifiable research programs

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture group

Context

Higher-dimensional automata, directed topology, sheaves, cohomology, and persistent homology may unlock real advances—or become decorative complexity.

Decision

Experimental engines are separately named, benchmarked, and assurance-capped. Each has a hypothesis, baseline, threshold, and kill criterion. No public soundness claim relies on them before independent validation.

Consequences

Continuum can pursue frontier research without contaminating the product or credibility. Negative results are acceptable outputs.

Alternatives considered

1. Avoid speculative work: leaves boundary-pushing potential unexplored.
2. Make it core architecture immediately: reckless.
3. Use mathematical vocabulary only for marketing: explicitly rejected.

Validation and rollback

Research scorecards in docs/06_RESEARCH_AGENDA.md and docs/07_BENCHMARKS_AND_EVALUATION.md govern promotion.

Document: adr/0021-tla-examples-corpus-contract.md

ADR-0021: Make the TLA+ Examples corpus a release contract

Status: Accepted

Date: 2026-07-24

Decision owners: Continuum architecture and semantics groups

Context

A project claiming to replace the practical need for TLA+ can drift into verifying only the examples its own language makes convenient. Hand-selected demonstrations do not expose semantic blind spots around stuttering, fairness, symmetry, model configuration, refinement, recursive operators, procedural lowering, or deliberately failing models.

The `tlaplus/Examples` repository explicitly serves both as an example library and as a corpus for testing TLA+ tools. At pinned commit `91c22ea537853196ed1e03e9ad91693ec37642de`, its README lists 80 CI-validated specification families and 39 additional in-tree or external examples.

Decision

The 80 validated families are mandatory compatibility cases for Continuum 1.0. “Compatibility” means semantic equivalence at a declared parity level, not necessarily TLA+ source compatibility.

Every port receives:

- a pinned upstream source closure and toolchain;
- a native Continuum model;
- a state/action/temporal correspondence;
- expected model configurations and verdicts;
- differential and metamorphic tests;
- mutation tests demonstrating non-vacuous properties;
- Lean theorems or certificate imports when theorem parity is required;
- optional asupersync implementation refinement for selected cases.

Parity levels P0–P5 are normative in `corpus/tla-examples/PARITY_LEVELS.md`.

Consequences

The corpus becomes an external force on language and engine design. Features are not accepted because they work on one flagship protocol; they are accepted when they survive a heterogeneous semantic workload.

This adds substantial scope. It also creates a clear stopping condition and prevents the project from becoming a permanent research prototype.

Rejected alternatives

1. **Five showcase protocols.** Too easy to overfit.
2. **Only import TLA+ and run TLC.** Does not create native sovereignty or bridge to Rust.
3. **Line-by-line source translation.** Preserves syntax accidents rather than semantic intent.
4. **Raw state-count equality as the sole oracle.** Invalid under legitimate auxiliary-state and encoding differences.

Evidence and gates

- Corpus inventory is machine-readable and pinned.
- Each passing case publishes its parity artifact and reproduction command.
- Continuum CI runs a fast subset on every commit and the complete Tribunal nightly.
- Upstream disagreements are minimized and classified.
- 1.0 cannot ship while any required row is unsupported, unknown, or manual-only.

Document: adr/0022-lean4-metattheory-and-certificate-authority.md**ADR-0022: Use Lean 4 as metatheory and certificate authority****Status:** Accepted**Date:** 2026-07-24**Context**

Continuum will contain complex, parallel, optimized search engines. Proving the entire Rust implementation correct before it becomes useful is unrealistic; trusting every optimization defeats the assurance goal. Several planned claims—DPOR preservation, symmetry quotienting, stuttering refinement, liveness certificates, cancellation progress, solver encodings—are mathematical theorems rather than testing problems.

Lean 4 provides a small kernel, executable reflection, a strong mathematical ecosystem, and emerging evidence that very large SAT and pseudo-Boolean certificates can be imported through verified checkers without constructing enormous explicit proof terms.

Decision

Lean 4 is the normative home for Continuum metatheory and high-assurance certificate import.

Lean is used to formalize:

- transition, behavior, observation, and refinement semantics;
- soundness of certificate families;
- preservation theorems for reductions and abstractions;
- cancellation and obligation calculus;
- verified encodings from CIR/model fragments to SAT, PB, SMT, or graph obligations;
- key theorems corresponding to proof-bearing corpus examples.

Lean is **not** placed on the per-transition hot path. Rust engines search aggressively and emit evidence. A small native checker validates evidence for routine use; Lean reflection can validate the same artifact and produce a theorem for high-assurance workflows.

The core metatheory starts on Lean core/Std. Mathlib is a separately versioned dependency for research developments requiring advanced algebra, topology, probability, order theory, or category theory.

Consequences

Continuum gains a principled trust story without requiring a verified optimizer. It also incurs a serious proof-engineering program and version-management cost.

The Rust and Lean semantics must not be maintained as informal twins. Serialization schemas, executable test vectors, generated lemmas, and differential interpretation are required.

Rejected alternatives

1. **Trust Rust unit tests.** Insufficient for semantic reduction soundness.
2. **Verify everything in Rust with Verus only.** Valuable for code-level obligations but weaker as a general metatheory and theorem interchange.
3. **Make Lean the implementation language.** Conflicts with the Rust-native runtime/performance/product goal.
4. **Defer formalization until after 1.0.** Allows architecture to ossify around unprovable interfaces.

Milestones

(Renamed from an internal G0–G4 ladder: gate numbering is reserved for the docs/52 release gates, per plan §22. The T0/T1 theorem ladder is defined in RFC 0012; axiom manifests in ADR-0035.)

- L0: core transition/refinement/certificate theorems build with no sorry.
- L1: one Rust closure certificate is imported and checked in Lean.
- L2: observer-indexed reduction theorem covers the baseline DPOR fragment.
- L3: fairness/lasso certificate theorem supports corpus liveness cases.
- L4: asupersync cancellation primitives have formal contracts linked to emitted events.

Document: adr/0023-semantic-triptych-and-independent-paths.md**ADR-0023: Maintain a semantic triptych with independent paths****Status:** Accepted**Date:** 2026-07-24**Context**

“One semantics” is often implemented as one shared library used by model, runtime, and checker. This improves consistency but creates circular assurance: a bug in shared code makes every layer agree.

Conversely, fully independent implementations drift and become unaffordable.

Decision

Continuum distinguishes three authoritative artifacts:

1. **Model semantics:** mathematical transition/behavior definitions and native CML interpretation.
2. **Program semantics:** asupersync concrete execution and semantic event production.
3. **Proof semantics:** Lean definitions and certificate theorems.

They share versioned data contracts and generated fixtures, but not all evaluation code. Strong claims require at least two independent paths and, for proof-level assurance, a checked certificate under the Lean semantics.

The reference evaluator is simple, deterministic, exact, and optimization-hostile. The production Rust engine is optimized. Solver encodings are a third path. Corpus fixtures and generated bounded models compare all available paths.

Consequences

The project must budget for differential testing and semantic change management. It cannot “fix” a disagreement by changing all three sides simultaneously without an explicit semantic decision record.

This architecture makes latent ambiguity visible early, which is exactly the point.

Required independence matrix

Claim	Minimum independent evidence
Counterexample	replay under reference semantics
Finite safety	native certificate checker + reference evaluator
Proof safety	Lean certificate theorem
Concrete refinement	program event trace + independent model/refinement checker
Solver proof	proof certificate + verified encoding/checker
Reduction soundness	Lean theorem + mutation/differential corpus

Rejected alternatives

- one shared evaluator everywhere;
- fully duplicated full-scale engines;
- “N-version agreement” without a theorem or evidence model;
- treating the upstream TLA+ oracle as the permanent semantic authority.

Document: adr/0024-observer-indexed-independence.md**ADR-0024: Index independence by observation and property contracts****Status:** Accepted for baseline design; theorem work required**Date:** 2026-07-24**Context**

Two events can commute in concrete state while differing in audit order, fairness obligations, timing, durability, information flow, or a refinement observer. A global event-kind conflict table is either unsound or needlessly conservative.

Context-sensitive independence research shows that observers can enable exponential reductions. Continuum additionally has explicit views, obligations, and lifecycle events that must participate in the definition.

Decision

Independence is parameterized by an `ObservationContract` containing:

- state/view abstraction;
- visible event alphabet;
- active safety/temporal/hyperproperties;
- fairness monitors;
- obligation and cancellation observations;
- timing and durability sensitivity;
- fault semantics.

An independence witness for events `e` and `f` at configuration `C` must establish:

1. both orders are enabled or both disabled as required;
2. both orders reach states equivalent under the active view;
3. visible observations and obligation flow agree;
4. neither order changes fairness eligibility incorrectly;
5. future enabledness is preserved for the supported property fragment.

Observer refinement induces a monotonicity law: a finer observer permits no more independence than a coarser observer. This law is formalized in Lean and exploited for cache reuse across property sets.

Consequences

Reduction becomes property-aware and potentially far stronger. Cache keys must include the observer contract hash. Adding a property can invalidate prior independence witnesses.

The initial implementation remains conservative: static resource conflicts plus checked dynamic witnesses. Research engines may infer stronger relations but cannot raise assurance without certificates.

Kill criterion

If observer-indexed analysis costs more than the schedules it removes across the corpus, retain only the formal contract and use coarse conservative observers by default.

Document: `adr/0025-stratified-model-language-fragments.md`

ADR-0025: Stratify the model language into declared semantic fragments

Status: Accepted

Date: 2026-07-24

Context

TLA+ permits highly expressive mathematical specifications, while executable model checking requires finiteness or decidable symbolic fragments. A typed Rust-like language can accidentally hide this boundary until an engine fails or silently bounds an unbounded value.

The TLA+ examples corpus spans finite enumeration, symbolic Apache models, TLAPS proofs, temporal properties, probability, refinement, and procedural PlusCal.

Decision

CML has one surface syntax and typed semantic core, but every definition is assigned capabilities/effects from explicit fragments:

- **Finite:** exactly enumerable values and transitions;
- **Symbolic:** solver-representable values and relations;
- **Temporal:** infinite-behavior properties and fairness;
- **Probabilistic:** probability distributions, MDPs, games, rewards;
- **Theorem:** propositions delegated to Lean;
- **Runtime:** concrete effects mapped to `asupersync`/domain packs.

Cross-fragment use is checked. Examples:

- a `Finite` action cannot call an opaque runtime function;
- a symbolic set cannot be enumerated without a finite model assignment;
- probabilistic and adversarial nondeterminism remain distinct;
- theorem-only choice is not given an arbitrary executable witness;
- liveness claims identify the temporal/fairness fragment used.

Consequences

Users see why a model can be simulated but not exhaustively checked, or proved but not executed. Engines can select only sound encodings. Corpus coverage maps directly to fragment completeness.

The language is slightly more explicit than TLA+, intentionally trading convenience for truthful assurance.

Rejected alternatives

- unrestricted language with runtime failures;
- silently finite integers/sets;
- separate unrelated languages per backend;
- require every model to be executable Rust.

Document: `adr/0026-proof-producing-reductions.md`

ADR-0026: Require proof-producing reductions for strong assurance

Status: Accepted as target architecture

Date: 2026-07-24

Context

DPOR, symmetry, partial-order unfoldings, abstraction, slicing, and compositional summaries deliberately omit executions or state detail. Their bugs are dangerous because they can turn an unsafe system into a false pass.

Decision

Every reduction has two modes:

1. **exploratory:** optimized, assurance-capped, may rely on tested algorithms;
2. **certifying:** emits a reduction witness checked independently.

Witness families include:

- commutation diamonds and source/backtrack-set obligations for DPOR;
- orbit representatives plus permutation witnesses for symmetry;
- covering/event-extension obligations for unfolding prefixes;
- Galois connection or simulation obligations for abstraction;
- rely/guarantee interface closure for composition;
- proof-graph dependencies for inductive slicing.

The certificate need not replay the entire search. It must be sufficient to establish the preservation theorem assumed by the result.

Consequences

Certifying engines will initially be slower and support fewer optimizations. This is acceptable. Fast exploratory lanes find bugs; certifying lanes justify absence claims.

The project must resist “certificate” formats that merely serialize internal data without a small semantic checker.

Promotion rule

A reduction may become the default for proof-level results only after:

- a Lean preservation theorem exists;
- the certificate checker is independently tested;
- adversarial malformed certificates are rejected;
- the TLA+ corpus and mutation suite show no semantic loss;

- performance beats unreduced or previously certified baselines on a defined benchmark class.

Document: adr/0027-nominal-orbit-finite-symmetry.md

ADR-0027: Add a nominal/orbit-finite lane for names and symmetry

Status: Accepted as research lane; not a baseline dependency

Date: 2026-07-24

Context

Distributed models contain process IDs, request IDs, session IDs, keys, fresh nonces, and dynamically allocated resources. Bounding each name domain creates artificial state explosion and weakens parameterized claims. Ordinary symmetry reduction handles fixed finite populations but not fresh-name generation elegantly.

Nominal-set and orbit-finite automata theory provides finite representations modulo permutations of atoms, including models with name allocation. It is a mature but underused body of semantics relevant to modern distributed systems.

Decision

CML introduces an optional atom kind with explicit symmetry theory:

- equality atoms;
- ordered atoms where supported;
- fresh allocation and support tracking;
- orbit canonicalization by equality/order pattern;
- alpha-equivalent state identity;
- nominal automata or symbolic transitions for supported fragments.

The lane is used only when the model's operations are equivariant under the declared group action. Non-equivariant observations—hashing raw IDs, stable numeric ordering, external identity—break or refine the symmetry and must be declared.

Consequences

Continuum may verify classes of dynamic-name systems without arbitrary ID bounds and may synthesize stronger parameterized invariants. The implementation and proof burden is significant.

Falsification gates

- demonstrate an orbit-finite model with fresh request/session IDs that finite symmetry cannot scale to;
- prove canonicalization/equivariance soundness in Lean;
- compare against standard symmetry plus small-model cutoffs;
- delete or quarantine the lane if it provides no decisive corpus/real-system win.

Document: adr/0028-assumption-synthesis-as-games.md

ADR-0028: Treat environment and fairness assumption synthesis as games

Status: Accepted as strategic research direction

Date: 2026-07-24

Context

Liveness failures are often dismissed by adding fairness assumptions until the model passes. This is dangerous: the assumptions may be stronger than production can provide, and the model checker gives little help identifying the weakest adequate contract.

Reactive synthesis and game theory provide a more honest formulation. The implementation and environment/fault scheduler are players. A property is realizable only under some environment strategy constraints.

Decision

Continuum models distinguish controlled system choices, admissible implementation nondeterminism, stochastic choices, and adversarial environment/fault choices.

For supported finite omega-regular fragments, Continuum can:

- solve the corresponding safety/parity/Streett game;
- emit an environment counterstrategy when the property is unrealizable;
- synthesize candidate safety restrictions and fairness assumptions;
- order assumptions by a declared strength relation;
- validate generated assumptions against production/domain-pack capabilities;
- expose assumptions as versioned contracts, never hidden solver artifacts.

“Weakest” is used only relative to a specified assumption grammar/order; global weakest assumptions may not exist or be tractable.

Consequences

The tool can answer a more useful question than “liveness failed”: what must the network, scheduler, storage, operator, or clients guarantee, and can that guarantee be implemented?

This also creates a foundation for agent-assisted protocol repair using counterstrategies rather than random patches.

Document: [adr/0029-semantic-equivalence-not-tla-source-cloning.md](#)

ADR-0029: Target semantic equivalence, not a TLA+ source clone

Status: Accepted

Date: 2026-07-24

Context

The TLA+ corpus is a critical benchmark, but reproducing every parser, module-system, proof-language, TLC override, and historical edge case would consume the project and constrain better language design.

Users need the ability to express and verify the same systems and properties. They do not necessarily need every file to run unchanged.

Decision

Continuum 1.0 requires native semantic equivalents for the pinned corpus. Source compatibility is an independent interoperability lane.

The optional TLA frontend proceeds in stages:

1. SANY/TLC oracle export in the Tribunal;
2. import of a normalized semantic IR;
3. native parser/elaborator for the executable subset;
4. diagnostics and source mapping;
5. only then, broader compatibility.

Java or upstream tools are never silently invoked by release binaries.

Consequences

The model language can be typed, fragment-aware, and designed for refinement into Rust. Corpus parity still prevents semantic retreat.

Users with existing TLA+ assets receive a migration path, but the product’s core value does not wait on full language emulation.

Document: [adr/0030-semiring-valued-analysis.md](#)

ADR-0030: Generalize exploration results with typed semiring analyses

Status: Experimental

Date: 2026-07-24

Context

The same transition structure supports many analyses:

- Boolean reachability;
- shortest counterexample length;

- number of causal classes;
- accumulated cost or latency;
- probability bounds;
- provenance explaining which faults/assumptions contribute;
- reliability or risk scores.

Implementing each as unrelated traversal logic duplicates work and loses algebraic structure. Weighted-automata and provenance-semiring theory suggest a unifying formulation.

Decision

The semantic graph APIs permit analyses parameterized by a typed algebra when its laws and interpretation are declared:

- Boolean semiring for reachability;
- tropical semiring for shortest/least-cost witnesses;
- natural-number or generating-function semirings for counting;
- probability/expectation structures for probabilistic fragments;
- provenance polynomials over event/fault labels;
- product semirings for simultaneous metrics.

The assurance result states the algebra and required conditions. Quantitative probability is not treated as an ordinary commutative semiring when nondeterminism or scheduler adversaries require MDP/game semantics.

Consequences

This can unify algorithms and produce richer explanations, but algebraic elegance must not erase semantic distinctions. The feature remains experimental until it produces measurable implementation simplification or novel analysis wins.

Kill criteria

Reject the abstraction if it introduces dynamic dispatch in hot loops, obscures numerical error, conflates probability with nondeterminism, or fails to simplify at least three concrete analyses.

Document: adr/0031-corpus-derived-language-governance.md

ADR 0031: Corpus-Derived Language Governance

Status

Accepted for revision 2.

Context

A new formal language can accumulate elegant features without proving practical completeness. Continuum claims it can replace ordinary TLA+ usage in concurrent and distributed Rust projects.

Decision

The pinned 80-family TLA+ Examples validated corpus governs CML and engine priorities. Every language feature proposal identifies the corpus families and real-system use cases it unlocks. Continuum 1.0 requires all validated families at their declared parity level.

Corpus pressure does not prohibit features absent from TLA+. It prevents the project from declaring victory while missing known modeling patterns.

Consequences

- corpus manifests and dashboards are release artifacts;
- semantic epochs include corpus compatibility changes;
- port-specific hacks are rejected;
- language ergonomics remain free to improve on TLA+ syntax;
- the extended 39-family set remains tracked but is not silently counted as validated.

Document: adr/0032-weak-memory-execution-graph-lane.md

ADR 0032: Weak Memory Is a Separate Execution-Graph Lane

Status

Accepted as architecture; non-SC semantics remain gated.

Context

Loom-style schedule exploration and distributed-system simulation do not fully model Rust/C11 weak memory. Combining every atomic choice with every packet/crash schedule is generally intractable.

Decision

Continuum models weak memory with a dedicated execution-graph fragment and verifies local components against atomic contracts. Distributed models consume those contracts. The initial supported semantics are SC and explicit synchronization; Rust/C11 axiomatic models are solver-backed extensions.

Consequences

- assurance states the memory model explicitly;
- local and distributed verification compose hierarchically;
- Loom traces may seed exploration but do not define semantics;
- runtime refinement of lock-free components requires a weak-memory receipt or an explicit SC assumption.

Document: adr/0033-checked-projection-and-generated-rust-interfaces.md

ADR 0033: Checked Projection May Generate Rust Interfaces

Status

Accepted as a target.

Context

Model/code drift often begins at messages, roles, operation names, and observability. Pure code generation can constrain architecture without proving behavior.

Decision

CML global protocol declarations may project to role-local automata, Rust message types, endpoint traits, asupersync skeletons, instrumentation IDs, and monitors. Projection emits a preservation/compatibility receipt. Generated artifacts establish an interface seam, not full implementation correctness.

Consequences

- global/local choice knowledge must be checked;
- generated APIs must remain idiomatic and overridable behind traits;
- real implementations still require refinement;
- projection failures are model diagnostics, not code-generation errors.

Document: adr/0034-owned-temporal-semantics-with-leanltl-bridge.md

ADR 0034: Own the Temporal Semantics and Bridge to LeanLTL

Status

Accepted.

Context

LeanLTL offers valuable current Lean 4 infrastructure for finite and infinite linear temporal logic. Continuum nevertheless needs stable semantics tied to fairness, stuttering, actions, and proof receipts.

Decision

Continuum defines a minimal temporal semantics in its Lean metatheory and proves bridges to LeanLTL where definitions align. LeanLTL may provide notation, automation, and reusable theorems but does not silently define Continuum product meaning.

Consequences

- dependency upgrades cannot change verdict semantics without an epoch;
- equivalence theorems make reuse explicit;
- temporal proof work is not duplicated unnecessarily;
- unsupported semantic differences remain visible.

Document: adr/0035-proof-receipts-and-axiom-manifests.md

ADR 0035: Strong Claims Emit Proof Receipts and Axiom Manifests

Status

Accepted.

Context

A certificate file or Lean theorem name alone is insufficient to reproduce the trust chain. Tool versions, encodings, transformations, assumptions, and axioms matter.

Decision

Every strongest-assurance result emits a proof receipt containing semantic/proof epochs, closure hashes, transformation chain, certificate and checker hashes, theorem names, imports, and machine-captured axiom output. Receipts are content-addressed and independently checkable.

Consequences

- proof provenance becomes a stable API;
- claims survive engine replacement when semantics and receipts remain valid;
- undocumented axioms fail release gates;
- receipts may be signed but signatures do not replace proof checking.

Document: adr/0036-authoritative-workbench-daemon.md

ADR 0036: Authoritative Workbench Daemon

Status

Accepted.

Context

Multiple interfaces and agents cannot safely maintain independent notions of workspace, task, cache, proof state, and evidence. Process-local sessions make replay and handoff fragile.

Decision

continuumd is the authority for workspace snapshots, Intent Contracts, incremental queries, task lifecycle, artifact publication, evidence status, repair transactions, and Forge archives. Semantic artifacts remain immutable. CLI, LSP, DAP, MCP, SARIF, TUI, and web surfaces are adapters.

Consequences

- one state model and audit trail;
- local embedded deployment remains mandatory;
- daemon correctness and crash recovery become high-priority verification targets;
- adapters cannot promote evidence or own hidden semantic state.

Evidence required

Task/publication model checks, crash/cancellation campaigns, idempotency tests, and adapter conformance.

Document: adr/0037-explicit-content-addressed-handles.md

ADR 0037: Explicit Content-Addressed Handles

Status

Accepted.

Context

Implicit sessions have ambiguous lifetime and sharing semantics. Multi-agent work often needs some state shared and other state isolated.

Decision

All durable workflow state—workspace, intent, task, continuation, proof state, debug branch, repair transaction, Forge archive, and evidence—is addressed by explicit opaque handles. Handles bind to immutable content or versioned transactional state. Authorization is checked separately from possession.

Consequences

- deterministic resume/handoff/caching;
- clients must thread handles explicitly;
- opaque identifiers and lifecycle/retention policy are required;
- stale input becomes a typed error rather than accidental behavior.

Evidence required

Idempotency, stale-handle, concurrent publication, authorization, and resume tests.

Document: adr/0038-machine-contracts-not-terminal-prose.md

ADR 0038: Machine Contracts, Not Terminal Prose

Status

Accepted.

Context

Agents scraping human output are brittle, expensive, and prone to misclassification. Human text also changes more frequently than semantic contracts.

Decision

Every semantic operation has a versioned typed request/result schema. Terminal and UI prose are projections. Exit codes, JSON/CBOR, errors, omissions, epochs, and valid next actions are stable contracts.

Consequences

- additional schema/versioning discipline;
- generated clients become feasible;
- UI copy may evolve independently;
- agents cannot rely on undocumented internal fields.

Evidence required

Golden protocol traces, schema compatibility, fuzzing, and ACI-vs-shell benchmark.

Document: adr/0039-protected-intent-contract.md

ADR 0039: Protected Intent Contract

Status

Accepted.

Context

Formal reward hacking can make all checks pass by changing the question: weaken a property, strengthen assumptions, shrink bounds, remove faults, or hide observations.

Decision

Properties, assumptions, observers, bounds, faults, fairness, trust, assurance, and non-vacuity form a content-addressed Intent Contract. Ordinary repair operations cannot mutate protected fields. Revisions require semantic diff and policy authorization.

Consequences

- intent becomes an input to all evidence identities;
- legitimate requirement changes are more explicit;
- implication/order checking is needed for supported fragments;
- unknown semantic relation blocks automatic promotion.

Evidence required

Adversarial intent-gaming suite and independent policy tests.

Document: adr/0040-typed-context-packs.md

ADR 0040: Typed Context Packs

Status

Accepted.

Context

Raw traces and repository dumps overwhelm humans and agents. Unlabeled summaries may omit the actual cause.

Decision

Continuum compiles bounded Context Packs from causal, proof, source, state, assumption, and correspondence graphs. Packs include guarantee class, omission manifest, expansion handles, replay/evidence references, and exact semantic identity.

Consequences

- context compilation becomes a product subsystem;
- claims such as replay-preserving must be checked;
- compactness is optimized subject to faithfulness;
- packs support human and agent renderings.

Evidence required

Replay-preservation checks, context ablations, human studies, and token/effectiveness benchmarks.

Document: adr/0041-verification-debugger-and-dap.md

ADR 0041: Verification Debugger with DAP Projection

Status

Accepted.

Context

Linear trace viewers cannot expose alternate schedules, causal predecessors, abstractions, or fairness obligations.

Decision

Build a native partial-order verification debugger. Provide semantic stepping, causal reverse, branch comparison, why-enabled/blocked, abstract/concrete/obligation views, and fault branching. Project common operations through DAP for IDE reuse.

Consequences

- debugger works on immutable execution artifacts;
- DAP is insufficient for all semantics, requiring namespaced requests;
- branch handles and lazy values are required;
- debugger results must replay through the semantic engine.

Evidence required

Branch/replay tests and user/agent diagnosis comparisons.

Document: adr/0042-mcp-adapter-not-authority.md

ADR 0042: MCP Is an Adapter, Not Authority

Status

Accepted.

Context

MCP is useful for agent interoperability, but transport/session/tool conventions should not define Continuum semantics or trust.

Decision

Expose a curated MCP adapter over the native protocol. Stateful workflows use explicit Continuum handles. MCP clients cannot promote evidence, bypass capability checks, or own hidden semantic state.

Consequences

- broad agent ecosystem access;
- native protocol remains stable across MCP changes;
- tool results must be bounded and deterministic;
- authorization is enforced below the adapter.

Evidence required

MCP conformance, multi-agent sharing/isolation, prompt-injection, and capability tests.

Document: adr/0043-repair-transactions.md

ADR 0043: Repair Transactions

Status

Accepted.

Context

A patch plus passing tests cannot establish that a concurrency repair preserves intent or generalizes beyond one trace.

Decision

Repairs are versioned transactions containing base failure, hypothesis, changes, semantic/intent diff, exact replay, neighboring exploration, mutation challenge, proof impact, unknowns, and promotion receipt.

Consequences

- autonomous repair gains a hard acceptance boundary;
- evaluation can be resumed and reviewed;
- intent revisions are separate transaction class;
- promotion policy becomes auditable.

Evidence required

Overfit repairs, property-gaming repairs, and genuine repairs in benchmark suite.

Document: adr/0044-incremental-semantic-database.md

ADR 0044: Incremental Semantic Database with Incremental Parity Audit

(The audit was formerly called the clean-build Tribunal; “Tribunal” now refers exclusively to the ADR-0021 corpus oracle harness.)

Status

Accepted.

Context

Interactive verification requires reuse; unsound invalidation can produce stale green results.

Decision

Represent derived computations as content-addressed semantic queries. Dependency/reuse edges are Exact, Validated, Conservative, or Experimental. Clean recomputation continuously audits strong incremental claims and quarantines mismatches.

Consequences

- provenance-rich query graph;
- some interactive results remain provisional;
- promotion may require clean/validated lane;
- invalidation debugging is user-visible.

Evidence required

Random edit sequences, corrupted-edge tests, clean differential results, and simplified Lean model.

Document: adr/0045-progressive-disclosure-with-lossless-expansion.md

ADR 0045: Progressive Disclosure with Lossless Expansion

Status

Accepted.

Context

Formal tools either overwhelm newcomers or hide critical assumptions behind simplified badges.

Decision

Default views are concise, but every summary item links to exact semantic artifacts. Simplification includes omissions, assurance, and expansion paths. Human personas share one evidence model.

Consequences

- more presentation/query work;
- no separate “simple semantics” mode;
- explanations can be empirically evaluated;
- exact evidence remains reachable.

Evidence required

Task studies measuring diagnosis and assurance calibration.

Document: adr/0046-semantic-diff-gates.md

ADR 0046: Semantic and Intent Diff Gates

Status

Accepted.

Context

Text diff cannot reveal changes to reachable behavior, observers, assumptions, proof dependencies, or assurance.

Decision

Every review/repair computes layered semantic diff. Protected intent changes, proof-policy downgrades, new opaque effects, and reduced coverage are explicit policy inputs. Unsupported comparisons return Unknown.

Consequences

- implication/equivalence engines and correspondence provenance are needed;
- review becomes more informative;
- automatic decisions are limited to supported fragments;
- Forge can deduplicate semantically equivalent candidates.

Evidence required

Known semantic-equivalence/change corpus and adversarial review mutations.

Document: adr/0047-forge-sandbox-and-nonvacuity.md

ADR 0047: Forge Sandboxing and Non-Vacuity

Status

Accepted.

Context

Protocol synthesis and agents can satisfy safety by disabling behavior or altering assumptions. Generated code is untrusted.

Decision

Forge runs inside a fixed Intent Contract and typed sketch. Positive/progress scenarios and non-vacuity claims are hard constraints. Candidates are sandboxed and independently verified before archive/promotion.

Consequences

- synthesis tasks require richer intent;
- candidate generators remain outside TCB;
- unrealizability and unknown are distinguished;
- materialization enters a design/repair transaction.

Evidence required

Never-ack/trivial-property mutants, hidden variants, and independent candidate checks.

Document: adr/0048-proof-service-isolation.md

ADR 0048: Proof Service Isolation

Status

Accepted.

Context

Lean environments are version-sensitive and agent-generated proof inputs are untrusted. Large proof workflows need concurrency and cancellation.

Decision

Run proof operations in isolated workers keyed by exact Lean/library/source closure. Workers expose typed goal/proof operations, resource bounds, deterministic checking, axiom manifests, and receipts. Only kernel-checked results reach Proved.

Consequences

- local and remote worker implementations;
- proof context compilation matters;
- environment changes invalidate receipts;
- worker output is untrusted until validated.

Evidence required

Multi-version isolation, malicious inputs, cancellation, and receipt replay.

Document: adr/0049-explicit-budgets-and-resumable-search.md

ADR 0049: Explicit Budgets and Resumable Search

Status

Accepted.

Context

Verification and synthesis can be unbounded. Silent timeouts and abandoned processes are poor human/agent interfaces.

Decision

Every long task accepts typed resource budgets and returns terminal evidence or an explicit continuation. Continuations bind to exact inputs and support deterministic resume/fork under declared semantics.

Consequences

- search engines need serializable committed frontier state;
- budget exhaustion is not pass/fail;
- resource effectiveness can be benchmarked;
- cancellation semantics become foundational.

Evidence required

Resume equivalence, stale epoch rejection, crash recovery, and budget accounting tests.

Document: adr/0050-proof-oriented-model-program-lenses.md

ADR 0050: Proof-Oriented Model/Program Lenses

Status

Accepted.

Context

Automatic synchronization can hide ambiguity and create incorrect abstractions. Manual model/code drift is also costly.

Decision

Represent correspondence as verified/checked bidirectional transformations with consistency relations, complements/provenance, ambiguity conditions, effects, and proof obligations. Reverse updates return alternatives/conflicts when underdetermined.

Consequences

- no silent model/code rewrite;
- generated edits are proposals;
- correspondence becomes inspectable/incremental;
- lens laws are necessary but not sufficient; semantic preservation is required.

Evidence required

Round-trip, ambiguity, drift, and refinement-preservation tests; Lean seed theorems.

Document: adr/0051-assurance-envelope.md

ADR 0051: Assurance Envelope Instead of Verified Badge

Status

Accepted.

Context

A scalar “verified” status hides bounds, faults, fairness, observer, weak-memory, proof, and unknown dimensions.

Decision

Every verdict carries a structured assurance envelope. UI may summarize but cannot omit material dimensions. Promotion policy specifies required envelope.

Consequences

- results become comparable without false total ordering;
- CI policies are explicit;
- users must learn a small assurance vocabulary;
- Inconclusive and Unsupported remain visible.

Evidence required

Schema validation, UI comprehension studies, and policy tests.

Document: adr/0052-multi-agent-evidence-graph.md

ADR 0052: Multi-Agent Evidence Graph

Status

Accepted.

Context

Chat-based swarms lose provenance, duplicate work, and can reach consensus without evidence.

Decision

Durable multi-agent work is stored as typed immutable nodes/edges with status authority. Agents propose; execution/checker services promote. Conflicts and missing obligations are first-class.

Consequences

- orchestration can be model-agnostic;
- work is resumable and auditable;
- graph/context compilers are required;
- chat remains optional coordination only.

Evidence required

Swarm benchmark with conflicting candidates, unauthorized promotion attempts, and task handoff.

Reference Documents: Requests for Comments (RFCs)

Document: rfcs/0001-causal-intermediate-representation.md

RFC 0001: Causal Intermediate Representation

Status: Proposed

Owners: semantics, runtime, verification

Target gate: G0 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22) **Normative vocabulary:** MUST, SHOULD, MAY are interpreted as in RFC 2119.

Summary

Continuum SHALL use a versioned **Causal Intermediate Representation** (CIR) as its normative semantic interchange format. CIR represents executions as finite event structures rather than as total traces. A run is a causally closed configuration of events equipped with causality, conflict, typed effect footprints, observations, and optional view projections.

CIR is neither a Rust AST nor a serialized scheduler log. It is the mathematical boundary between authoring, concrete execution, model exploration, refinement, replay, production observation, and certificate checking.

Motivation

Interleaving traces lose the concurrency that reduction and production-trace validation need. Runtime-specific logs contain incidental details that prevent stable replay and independent checking. A TLA+-style state graph is excellent for some algorithms but forces independent events into arbitrary orders. CIR keeps the partial order primary and derives total orders or states only when required.

Semantic object

A finite prime event-structure fragment is represented by:

$$\mathcal{E} = (E, \preceq, \#, \lambda, \rho, \omega)$$

where:

- E is a finite set of event identities;
- $\preceq$ is a well-founded causal partial order;
- $\#$ is an irreflexive, symmetric, hereditary conflict relation;
- $\lambda : E \rightarrow \text{Label}$ assigns semantic labels;
- $\rho : E \rightarrow \text{Footprint}$ assigns typed resources and effect phases;
- $\omega : E \rightarrow \text{Observation}$ assigns observer-indexed visible facts.

A **configuration** $C \subseteq E$ is causally closed and conflict-free. Configuration extension $C \xrightarrow{e} C \cup \{e\}$ is legal when all causes of e are in C and no event in C conflicts with e .

CIR v0 permits dynamically discovered events and conflict. It does not require materializing the whole event structure before exploration.

Identity

Event identity MUST be semantic, not pointer- or thread-address-based. The canonical identity is:

```
EventId = H(
    cir_semantics_version,
    origin,
    actor_epoch,
    local_operation_index,
    normalized_label,
    normalized_inputs,
    canonical_causal_predecessor_ids
)
```

origin identifies the model action, Rust source site, foreign frontend node, or production probe. Identity construction MUST be deterministic. Hash collisions MUST be resolved by comparing canonical encodings in proof-bearing lanes.

Event schema

Each event contains:

```
pub struct Event {
    pub id: EventId,
    pub origin: Origin,
    pub owner: OwnerId,
    pub epoch: Epoch,
    pub causes: SmallVec<[EventId; 4]>,
    pub conflicts: ConflictDescriptor,
    pub label: Label,
    pub footprint: Footprint,
    pub effect: EffectTransition,
    pub obligations: ObligationDelta,
    pub time: TimeConstraint,
    pub observation: ObservationSet,
    pub fault: Option<FaultEvent>,
    pub provenance: Provenance,
}
```

Label

A label is a stable, typed operation descriptor. It contains an operation family and canonical payload. Payloads MUST use the Continuum value algebra, not arbitrary serde blobs.

Footprint

A footprint is a set of typed access claims:

```
enum AccessMode {
    Observe,
    Read,
    Write,
    Consume,
    Produce,
    Reserve,
    Commit,
    Abort,
    Synchronize,
}

struct Access {
    resource: ResourcePath,
    mode: AccessMode,
    logical_range: Option<RangeDescriptor>,
}
```

Footprints are evidence for independence; they are not automatically trusted. Domain packs define conflict rules and must prove or test their soundness.

Effect transition

Effects use explicit phases:

```
Requested → Reserved → Committed
                ↘ Aborted
```

Some packs add domain-specific phases, such as Submitted, Stable, or Acknowledged. Extensions MUST map to the generic phase algebra.

Obligations

Obligations are linear semantic resources created and discharged by events. Examples include reply obligations, reserved channel capacity, outstanding durable-write acknowledgements, child-region quiescence, and cancellation finalization.

Time

Time is represented by constraints, not only timestamps:

$\text{earliest}(e) \leq \text{occurrence}(e) \leq \text{latest}(e)$
 $\text{occurrence}(e1) + d \leq \text{occurrence}(e2)$

A concrete Lab run may instantiate all event times. A production trace may provide intervals. A timed model may retain symbolic bounds.

Observation

Visibility is indexed by observer:

ObserverId $\rightarrow$ finite set of typed observations

This supports trace refinement, opacity, noninterference, strong observational refinement, and partial telemetry.

Conflict and independence

Two events are independent only if swapping them preserves:

1. enabledness;
2. resulting abstract configuration up to canonical equivalence;
3. all relevant observer projections;
4. obligation ownership and discharge;
5. time/fairness feasibility;
6. fault and durability semantics.

The default footprint rule is conservative. Packs MAY supply a stronger semantic independence oracle. Unsound independence is a soundness defect and MUST be subject to mutation tests and differential execution.

Views

A view is a total or explicitly partial projection:

$$\alpha_v : \text{Configuration} \rightarrow \text{AbstractState}_v \cup \{\text{Unknown}\}$$

and an observation map:

$$\pi_v : \text{Event} \rightarrow \text{AbstractAction}_v^*.$$

Unknown cannot be silently treated as success. A view declares supported configurations and abstraction assumptions.

Stuttering refinement permits an event sequence to map to zero abstract steps. Linearization refinement permits a concrete interval to map to one abstract action when a declared witness event or validated prophecy selects the linearization.

Serialization

CIR SHALL have:

- a canonical binary encoding for hashing, storage, and replay;
- a readable JSON encoding for tooling;
- a deterministic text form for diffs;
- explicit semantic and encoding version numbers;
- unknown-field preservation in forward-compatible readers;
- a schema fingerprint embedded in every crashpack.

Canonicalization includes sorted maps/sets, normalized integer representation, canonical NaNs if floats are enabled, normalized paths, and stable symbol interning.

Validation

The continuum-cir validator MUST check:

- unique event identities;
- acyclic causality;
- causal closure of configurations;
- conflict symmetry and heredity where materialized;
- obligation conservation;

- effect-phase legality;
- resource-path type validity;
- epoch and restart constraints;
- observer-schema validity;
- source-provenance integrity;
- version compatibility.

Performance requirements

CIR is logically rich but hot paths cannot allocate a graph object per event. Implementations SHOULD use interned labels, structure-of-arrays event storage, compressed predecessor sets, persistent configuration bitmaps, and append-only chunked arenas. The wire format is not the in-memory layout.

Security and trust

Production CIR is untrusted input. Parsers MUST be resource-bounded and reject cyclic, oversized, or adversarial structures. Certificates bind to the canonical CIR hash, semantic version, model hash, and assumptions.

Rejected alternatives

- **Total trace as primary:** destroys concurrency and creates false ordering.
- **TLA+-style states only:** unsuitable for incomplete production evidence and runtime causality.
- **Rust MIR as canonical semantics:** too concrete, unstable, and language-specific.
- **OpenTelemetry spans as semantics:** useful transport, insufficient formal contract.
- **Petri nets as sole IR:** excellent mathematical lane, but awkward for rich typed values, observations, and arbitrary abstract transitions.

Open research questions

- Whether stable event structures, occurrence nets, interval pomsets, or a cubical model should become the canonical mathematical presentation after v0.
- Whether conflicts can be represented intensionally without harming certificate checking.
- How much observer-specific independence can be computed compositionally.
- Whether interval events should be first-class or compiled into begin/commit pairs.

Document: [rfcs/0002-controlled-effects-and-domain-packs.md](#)

RFC 0002: Controlled Effects and Semantic Domain Packs

Status: Proposed

Target gate: G0/G1 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

Summary

All nondeterministic or externally visible behavior in a Continuum-controlled program MUST cross a typed effect boundary. Effect implementations are grouped into **domain packs** that provide production, deterministic-Lab, abstract, fault, trace, and refinement semantics for one domain.

A pack is not a mock. It is a semantic module with declared fidelity.

Goals

- Run materially identical protocol logic in production and deterministic exploration.
- Make time, entropy, network, storage, process lifecycle, synchronization, and cancellation explicit.
- Allow packs to be composed without hidden global state.
- Give DPOR and refinement engines typed semantic footprints.
- Prevent the phrase “simulated disk” from concealing an unspecified failure model.

Pack contract

A domain pack exports:

```
pub trait DomainPack {  
    type Command;  
    type Response;
```

```

type Resource;
type AbstractState;
type Fault;
type EventPayload;

fn validate_command(...);
fn production(...);
fn lab_step(...);
fn abstract_step(...);
fn footprint(...);
fn independence(...);
fn observe(...);
fn recover(...);
fn assumptions(...) -> AssumptionSet;
}

```

The actual Rust API may split this trait to avoid monomorphization and object-safety problems. The semantic obligations remain.

Required profiles

Every pack names one or more fidelity profiles:

```

ideal
contractual
platform-qualified
adversarial-envelope

```

- `ideal` is a simple mathematical abstraction.
- `contractual` matches a documented service/API contract.
- `platform-qualified` is backed by measurements and conformance tests for a particular deployment.
- `adversarial-envelope` permits all behavior not forbidden by declared constraints.

A verification result identifies exact profile versions. Profiles are not ordered automatically; a refinement proof or conformance campaign establishes relationships.

Standard packs

Network

Must parameterize at least:

- loss, duplication, reordering;
- partition topology;
- bounded/unbounded delay assumptions;
- connection epochs;
- half-open behavior;
- backpressure and capacity;
- corruption/authentication assumptions;
- DNS/service-discovery behavior when relevant.

Storage

Must distinguish:

- volatile process memory;
- page cache / submitted writes;
- durable stable storage;
- atomicity granularity;
- ordering and barriers;
- torn writes;
- sector/block corruption;
- rename/link/directory durability;
- crash and restart;

- recovery procedure;
- device and filesystem profile.

Process lifecycle

Must model:

- graceful cancellation;
- panic;
- fail-stop crash;
- power loss;
- restart with a new epoch;
- partial cleanup;
- supervisor decisions;
- leaked external effects.

Clock

Must distinguish:

- monotonic local time;
- wall time;
- drift and skew;
- discontinuities;
- uncertainty intervals;
- lease assumptions;
- timer coalescing and delayed wakeup.

Entropy

Must distinguish deterministic pseudo-random choices from cryptographic entropy. Production randomness can be observed as opaque committed values; verification may enumerate a finite abstraction or use symbolic values.

Asupersync integration

Asupersync's Cx, region hierarchy, explicit time/entropy, obligations, reserve/commit primitives, and Lab runtime are the concrete execution foundation. Continuum adapters MUST use public semantic surfaces and MUST NOT duplicate scheduling, cancellation, or quiescence machinery.

A capability call emits a proposed CIR event. The domain pack enriches it with resource paths, effect phases, assumptions, and abstract meaning.

Ambient nondeterminism enforcement

The project SHALL combine:

- API design: protocol crates receive capabilities, not ambient globals;
- lints: deny `std::time::*::now`, ambient RNG, unmanaged spawn, raw socket/file creation in controlled crates;
- dependency manifests: identify foreign code capable of nondeterminism;
- runtime interception where possible;
- replay checks: detect values not derivable from the choice log;
- optional MIR/LLVM audits for host calls;
- explicit escape hatches with an `UnmodeledEffect` event that downgrades assurance.

An unmodeled effect cannot be ignored. It changes the result from `verified` to a typed incomplete claim.

Fault algebra

Faults are typed operations over pack state. They support composition:

```
delay(d) ; duplicate(n) ; crash(epoch)
partition(A,B) || clock_skew(node,+δ)
```

Composition is only commutative when the pack proves independence. Fault campaigns may quantify a budget (e.g., at most two crashes) without implying a real-world probability.

Refinement obligation

For each profile, the pack defines a relation between concrete and abstract histories. In its simplest form:

$$R(c, a) \wedge c \xrightarrow{e} c' \Rightarrow \exists a'. a \xRightarrow{*} a' \wedge R(c', a').$$

For partial telemetry, the pack can emit constraints rather than a unique event.

Pack qualification

A pack is promoted from experimental only after:

- model-based tests against a reference implementation;
- differential testing across production/Lab paths;
- crash-consistency mutation suite;
- independence mutation suite;
- stable replay across supported versions;
- documented unsupported behavior;
- at least one external or independently implemented oracle where feasible.

Rejected alternatives

- A single universal World trait with opaque calls.
- Generic serde payloads and user-written independence functions with no validation.
- Simulators that accidentally promise platform fidelity.
- Build-time feature flags that compile different protocol logic for production and verification.

Document: rfcs/0003-continuum-model-language.md

RFC 0003: Continuum Model Language

Status: Proposed

Target gate: G2 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22) **Working extension:** .ctm

Summary

Continuum SHALL provide a standalone, typed, relational model language usable before implementation. Rust attributes/macros are a second authoring surface, not the sole language.

The language is intentionally closer to a transition-system calculus than to executable Rust. It preserves nondeterminism and arbitrary mathematical abstraction.

Design principles

1. **Relational, not imperative by default.** An action describes a relation between pre- and post-state.
2. **Finite checking without pretending finiteness.** Types can be unbounded; a run configuration selects finite bounds or symbolic domains.
3. **Explicit temporal assumptions.** Fairness, time, and faults are named.
4. **First-class views and refinement.**
5. **No accidental execution order.** Set comprehensions and quantifiers remain mathematical.
6. **Predictable elaboration.** No unrestricted macros or arbitrary host-language execution in trusted elaboration.
7. **Stable semantic IR.** Source syntax can evolve without changing CIR semantics.

Core syntax sketch

```
module ReplicatedRegister
```

```
type Node
type Value
const Nodes: Set[Node]
```

```
state {
  epoch: Node -> Nat
  durable: Node -> Option[(Nat, Value)]
  acked: Set[(Node, Nat, Value)]
```

```

    crashed: Set[Node]
  }

  init {
    forall n in Nodes:
      epoch[n] = 0 &&
      durable[n] = None
    acked = {}
    crashed = {}
  }

  action Propose(n: Node, v: Value) {
    require n notin crashed
    let e = epoch[n] + 1
    next epoch = epoch[n := e]
    next durable = durable
    next acked = acked
  }

  action Sync(n: Node, e: Nat, v: Value) {
    require n notin crashed
    next durable = durable[n := Some((e, v))]
    next acked = acked union {(n, e, v)}
    unchanged epoch, crashed
  }

  invariant UniqueEpochValue {
    forall n1,n2,e,v1,v2:
      (n1,e,v1) in acked &&
      (n2,e,v2) in acked
      => v1 = v2
  }

  fairness weak Sync

```

Exact syntax remains open. The semantics are not.

Type system

Initial types:

- Bool, mathematical integers/naturals, bounded bitvectors;
- finite and symbolic enums;
- tuples, records, tagged unions;
- sets, maps, sequences, multisets, relations;
- uninterpreted sorts;
- opaque external domains with declared equality/canonicalization;
- time and probability types only under extension modules.

Collections are persistent mathematical values. Finite explicit exploration requires enumerability, but the model itself is not syntactically restricted to finite values.

The checker distinguishes:

```

Finite[T]
Symbolic[T]
Opaque[T]
Enumerable[T]
Canonical[T]

```

These are semantic capabilities, not necessarily surface-level traits.

Actions

An action denotes a relation $A(s, p, s')$. The surface supports:

- guarded relational updates;
- choose / existential next values;
- universal branching;
- atomic action composition;
- event emission;
- obligation creation/discharge;
- stuttering;
- action schemas parameterized by values.

Imperative sugar may elaborate to a relation only when deterministic assignment order cannot change meaning.

Properties

State predicates

- invariants;
- inductive invariants;
- state constraints;
- assertion at action boundaries.

Trace and temporal predicates

- LTL-like safety/liveness;
- weak and strong fairness;
- leads-to;
- bounded temporal properties;
- observer-indexed hyperproperties under a dedicated module.

Quantitative predicates

Timed/probabilistic properties are not in the v0 core. They use typed extensions so users cannot accidentally interpret statistical evidence as exhaustive proof.

Configurations and bounds

A model configuration is separate from source:

```
[domains]
Node = [ "a", "b", "c" ]
Value = [0, 1]

[faults]
max_crashes = 2

[assumptions]
fair = [ "Deliver", "Recover" ]
```

Bounds and assumptions are hashed into every result.

Views and refinement

A view declares:

- a source state/configuration;
- a target model;
- an abstraction map or relation;
- visible event mapping;
- stuttering policy;
- prophecy/history variables if needed;
- proof obligations.

```
view RuntimeToProtocol refines Protocol {
  map_state c => {
    committed: c.storage
    .filter(|x| x.phase == Stable)
```

```

    .map(...)
  }

  map_event StorageSync => [Commit]
  map_event Poll | Wake | Reserve => []
}

```

A relation may replace a function when abstraction is nondeterministic or telemetry is incomplete.

Modules and composition

Modules expose typed state fragments, actions, properties, and assumptions. Composition supports shared resources only through explicit coupling declarations. Name-based global-variable merging is forbidden.

The long-term composition model should align with CIR event interfaces and may use assume-guarantee contracts or interface automata.

Execution semantics

A small reference evaluator SHALL define the core. Optimized evaluators, SMT encoders, and foreign frontends are checked against it through differential and metamorphic tests.

The reference evaluator is deterministic except for explicit choices, which are returned to the caller as a finite/symbolic choice frontier.

Foreign compatibility

Initial foreign paths:

- Quint-to-CIR adapter;
- TLA+ semantic-AST adapter using SANY as an oracle;
- trace import/export;
- optional Alloy/SMT-related encodings later.

Foreign adapters are untrusted translators. Their outputs can be compared with reference tools on a conformance corpus.

Non-goals for v0

- Full TLA+ source compatibility.
- General-purpose programming.
- User-defined elaborator macros.
- Dependently typed proofs.
- Silent execution of arbitrary Rust functions inside the model.
- A theorem-prover tactic language.

Open questions

- Unicode/math-like versus Rust-like surface syntax.
- Whether actions should permit explicit event intervals.
- How to expose finite-domain and symmetry declarations ergonomically.
- Whether the language should use refinement types for model well-formedness.
- Whether a minimal core should be mechanized in Lean/Rocq before freezing v1.

Document: rfcs/0004-exploration-dpor-and-unfoldings.md

RFC 0004: Exploration, DPOR, and Unfoldings

Status: Proposed

Target gates: G1/G2/G6 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

Summary

Continuum SHALL implement exploration as a family of algorithms over one semantic choice interface. The baseline is deterministic replay plus conservative source-DPOR. The frontier path adds optimal/parsimonious DPOR, event-structure unfoldings, symbolic complete prefixes, and observer-sensitive reduction.

No reduction may change the claim type unless its independence and cutoff assumptions are explicit.

Choice interface

At a configuration C , an engine asks:

```
pub trait TransitionOracle {  
    fn enabled(&self, c: &Configuration) -> EnabledSet;  
    fn execute(&self, c: &Configuration, choice: Choice)  
        -> Result<Transition, SemanticError>;  
    fn dependence(&self, a: &EnabledEvent, b: &EnabledEvent)  
        -> DependenceEvidence;  
}
```

Choice includes task scheduling, model action parameters, message delivery, timer firing, fault injection, symbolic branch selection, and domain-pack outcomes.

The oracle may be:

- standalone model evaluator;
- asupersync Lab execution;
- replay engine;
- product of a concrete and abstract execution;
- symbolic transition relation.

Baseline exploration

Deterministic campaigns

Seeded schedules/faults provide fast bug finding. Results are sampled, never exhaustive.

Source-DPOR

Version 0 uses vector clocks and a conservative conflict relation. It explores one or more representatives per Mazurkiewicz class, depending on sleep-set precision. Correctness is checked against exhaustive tiny instances.

Stateless versus stateful

Stateless exploration stores schedules/backtracking information and re-executes from the root or snapshot. Stateful exploration stores canonical configurations. Continuum supports both because distributed faults and data nondeterminism may favor different tradeoffs.

Snapshot strategy

Snapshots are semantic checkpoints, not raw process images. A snapshot binds:

- CIR prefix hash;
- runtime/model state snapshot;
- domain-pack versions;
- choice cursor;
- object identity map;
- pending obligations;
- virtual time;
- semantic version.

Restoration MUST be observationally equivalent to replaying the prefix. Snapshot correctness receives a differential campaign and certificate option.

Dependence evidence

```
enum DependenceEvidence {  
    DefinitelyIndependent(ProofRef),  
    DefinitelyDependent(Reason),  
    Unknown,  
}
```


Unknown is dependent. Fast lanes MAY use heuristic independence but must downgrade the claim to unsound/experimental and label it accordingly; CI proof lanes cannot.

Independence is observer-relative. Two events may commute for a safety invariant but not for a latency or fairness property. The property compiler emits an observation footprint that participates in dependence.

Optimal and parsimonious DPOR

The implementation roadmap is:

1. source-DPOR;
2. wakeup-tree optimal DPOR;
3. parsimonious optimal DPOR for polynomial-space exploration where applicable;
4. await/pure-loop-aware reduction;
5. observer-sensitive dependence.

Each step must reproduce exhaustive results on a generated corpus and include adversarial programs constructed to break naïve conflict relations.

Unfolding lane

The unfolding engine builds an occurrence-net/event-structure prefix:

- conditions represent resource/version facts;
- events represent CIR transitions;
- causality comes from consumed/produced conditions;
- conflict comes from competing causes or semantic exclusions;
- cutoff events identify equivalent future behavior.

A finite complete prefix can compactly represent all reachable configurations for suitable finite systems. High-level/symbolic unfoldings are a research lane for rich values.

Causal Cubical Reduction hypothesis

Independent k -event families define k -dimensional cubes. The hypothesis is that a cubical quotient can preserve property-relevant directed homotopy classes more compactly than pairwise trace equivalence.

The experiment SHALL compare:

- explored representatives;
- memory;
- counterexample preservation;
- certificate size;
- preprocessing overhead

against optimal DPOR and unfoldings. It is killed if it does not produce repeatable wins on systems with genuine high-dimensional concurrency.

Fairness and liveness interaction

Safety DPOR is not automatically liveness-preserving. Liveness exploration must use cycle provisos, fairness-aware stubborn/ample sets, or a certificate establishing preservation. The engine refuses to reuse a safety reduction for liveness without such evidence.

Distribution

Distributed exploration partitions by canonical semantic prefix/configuration hash. Work donation includes backtracking obligations, not just frontier states. Deterministic global replay is preserved by stable partition IDs and content-addressed work records.

Distributed checking is postponed until single-node exactness, replay, and certificates are stable.

Outputs

Every campaign emits:

- explored equivalence classes;

- reduction algorithm/version;
- dependence oracle/version;
- complete/incomplete frontier status;
- bounds and assumptions;
- replayable counterexamples;
- optional closed-set or unfolding-prefix certificate;
- semantic coverage metrics.

Rejected alternatives

- A single randomized scheduler marketed as a model checker.
- Blindly importing Loom/Shuttle semantics.
- Treating read/write footprints as a proof of commutativity.
- One global concurrent hash set as the only exploration architecture.

Document: rfcs/0005-certificates-and-independent-kernel.md

RFC 0005: Certificates and the Independent Kernel

Status: Proposed

Target gates: G2/G3 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

Summary

Strong Continuum claims SHALL be backed by machine-checkable evidence. The high-performance engines are outside the trusted computing base whenever practical. A small independent kernel validates semantic well-formedness and certificate families.

The kernel is deliberately boring. Novel mathematics belongs in certificate producers, not in unchecked trust.

Claim envelope

A certificate is valid only relative to:

model hash
CIR semantics version
property hash
domain-pack profile hashes
bounds
assumptions
engine version
certificate schema version

The kernel checks this envelope before content.

Certificate families

Counterexample witness

A counterexample certificate contains a legal initial configuration, legal transitions or causal extensions, and a property violation. Counterexamples are generally easier to check than proofs and are mandatory for reported bugs.

Closed reachable set

For finite safety:

$$Init \subseteq S, \quad Post(S) \subseteq S, \quad S \subseteq P.$$

The certificate may encode states explicitly, with Merkle commitments and transition witnesses, or symbolically with proof-producing solver obligations.

Inductive invariant

A predicate I plus certificates for:

$$Init \Rightarrow I, \quad I \wedge T \Rightarrow I', \quad I \Rightarrow P.$$

SMT proof terms SHOULD use Alethe where supported. SAT subproofs MAY use LRAT. Unsupported solver steps are either independently recomputed by a small decision procedure or leave the claim solver-trusted.

Refinement

For relation $R(c, a)$:

$$Init_C(c) \Rightarrow \exists a. Init_A(a) \wedge R(c, a)$$

and for every concrete step:

$$R(c, a) \wedge T_C(c, c') \Rightarrow \exists a'. T_A^*(a, a') \wedge R(c', a').$$

Certificates may include stutter witnesses, bounded abstract paths, history/prophecy assignments, and observer equivalence.

Liveness

Possible forms:

- accepting-cycle witness for failure;
- SCC/emptiness certificate;
- transition-invariant decomposition;
- ranking/lexicographic ranking function;
- fairness discharge map;
- liveness-to-safety reduction plus safety certificate.

Partial-order closure

A prefix certificate commits to events, cutoffs, and continuation-equivalence obligations. The kernel verifies causality/conflict consistency, cutoff mapping, and property preservation assumptions.

Production conformance

A SAT/SMT solution maps observed events to model events and ordering constraints. The certificate must distinguish fully matched, existentially completed, ambiguous, and impossible traces.

Kernel layering

```
continuum-kernel-core
  canonical values
  CIR validation
  propositional/first-order expression evaluator
  transition relation evaluator
  certificate dispatch
```

```
continuum-kernel-sat
  LRAT checker
```

```
continuum-kernel-smt
  Alethe checker subset
```

```
continuum-kernel-temporal
  graph/SCC/ranking certificates
```

A minimal reference kernel SHOULD avoid async, network, plugins, dynamic loading, and unsafe code. It reads content-addressed artifacts and emits a deterministic verdict.

Independence

The production engine and kernel MUST NOT share optimized evaluator code. They may share generated schema types, but semantics-sensitive functions need independent implementations or mechanically generated definitions from a small formal semantics.

A long-term goal is a mechanized core semantics and extracted/reference kernel in Lean or Rocq, with a native Rust checker cross-validated against it. This is not a gate for v0.

Proof-producing solver policy

Preference order:

1. independently checkable certificate;
2. two independent solvers plus replayable query;
3. one pinned solver with explicit solver-trusted claim;
4. heuristic result marked non-proof.

No CLI may print a bare “verified” without exposing the assurance class and trusted components.

Resource bounds

Certificate checking itself is an attack surface. Formats need:

- size limits;
- streaming validation;
- deterministic resource accounting;
- cycle/recursion bounds;
- hash-agility policy;
- rejection of duplicate/ambiguous encodings.

Kernel tests

- differential tests against engine evaluators;
- hand-built malformed certificates;
- grammar fuzzing;
- mutation testing of each checker rule;
- proof round trips from multiple producers;
- reproducible cross-platform verdicts;
- unsafe audit and minimal dependency closure.

Rejected alternatives

- Trust the Rust engine because Rust is memory-safe.
- Trust an SMT solver’s unsat answer without recording the query.
- Produce certificates only after the rest of the system is complete.
- One opaque binary proof format coupled to a specific solver.

Document: rfcs/0006-production-trace-conformance.md

RFC 0006: Production Partial-Order Conformance

Status: Proposed

Target gate: G5 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

Summary

Continuum SHALL validate production evidence as a partial-order constraint problem. It MUST NOT invent a total event order that telemetry does not establish.

The validator asks whether at least one model execution can explain the observed events under declared instrumentation completeness and clock/causality assumptions.

Observation model

A production record may include:

- stable operation and node/epoch identity;
- parent/span/message correlation;
- start/end or uncertainty interval;
- effect phase;
- request/response payload abstraction;
- durable sequence/fence marker;
- local monotonic sequence;

- vector/hybrid logical clock;
- observer/source;
- sampling and loss metadata.

The import layer translates records to **observation constraints**, not immediately to concrete CIR events.

Conformance question

Given a model M , observations O , and assumptions A , determine whether:

$$\exists \tau \in Behaviors(M). \tau \models_A O.$$

The result is one of:

Conforms

Violates(counterexample/core)

Inconclusive(missing evidence)

Unsupported(feature)

ResourceExhausted(partial evidence)

Inconclusive is not success.

Matching

Observation-to-model matching may be:

- direct by stable semantic event ID;
- keyed by operation family and payload abstraction;
- inferred through constraints;
- one observation to several internal model events;
- several observations to one abstract event;
- absent for stuttering/internal events.

Matching variables and all inferred orders are included in a certificate or explanation.

Ordering constraints

Sources include:

- per-thread/process program order;
- message send-before-receive;
- region/task ownership;
- effect reserve-before-commit;
- storage sequence/barrier;
- logical/vector clocks;
- non-overlapping real-time intervals;
- explicit causal links.

Wall-clock timestamps alone are weak evidence and require uncertainty bounds.

Completeness profiles

Instrumentation declares:

Complete(observer, event family)

LossBounded(observer, family, n)

Sampled(observer, family, p or policy)

BestEffort(observer, family)

Opaque(family)

Some safety properties can be refuted with incomplete traces but cannot be validated. The property compiler determines required observation completeness.

Incremental/streaming validation

The validator maintains a frontier of possible model configurations or a symbolic belief state. It may emit early violations when no completion remains. It must account for out-of-order telemetry and use watermarks/epochs before declaring closure.

Distributed runtime-verification impossibility results mean some properties cannot be decisively monitored under asynchronous faults and incomplete ordering. Continuum must expose those limits rather than paper over them.

Counterexample explanation

A violation report includes:

- minimal unsatisfiable observation core where possible;
- the property and model assumptions;
- orders forced by evidence;
- alternative matches rejected;
- missing event families that would distinguish ambiguity;
- a Lab scenario/replay template when reconstructable.

Privacy and security

Trace abstraction occurs at the source where possible. Payload schemas support redaction and cryptographic commitments. Conformance over commitments may prove equality/identity without exporting secrets, but zero-knowledge proof support is future work.

Trace records are adversarial input. Importers are sandboxed/adapters and cannot affect the kernel.

Production feedback loop

Novel production behaviors that conform but are absent from exploration coverage become seed scenarios. Nonconforming traces become minimized crashpacks. Inconclusive traces drive instrumentation recommendations.

Rejected alternatives

- Sort by timestamp and replay.
- Require global vector clocks everywhere.
- Declare success because no invariant violation appears in sampled logs.
- Treat observability as independent from property semantics.

Document: rfcs/0007-storage-and-crash-antics.md

RFC 0007: Storage, Crash, and Recovery Semantics

Status: Proposed

Target gate: G1 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

Summary

Continuum's storage pack SHALL model durability as a typed transition system, not as a byte map plus random write failure. Crash semantics, atomicity, ordering, persistence barriers, corruption, and recovery are versioned profile data.

State strata

A minimal storage state distinguishes:

application intent
reserved operation
submitted operation
volatile/cache-visible state
device-accepted state
stable state
recovery-visible state

The exact strata depend on the profile. A high-level database pack may instead expose transaction log records, commit indexes, and checkpoints while refining to a lower storage pack.

Operation phases

Example:

```
WriteRequested
  → BufferReserved
  → Submitted
  → CompletedVolatile
  → Stable
  → Acknowledged
```

Cancellation is legal only at declared boundaries. An acknowledgement before the durability level promised by the API is a semantic defect.

Crash

A crash selects a legal projection from pre-crash strata to recovery-visible state according to the profile. It also:

- increments process/node epoch;
- kills or cancels tasks according to crash type;
- invalidates volatile handles;
- preserves or loses in-flight effects according to the profile;
- produces recovery obligations.

The crash choice is explicit and replayable.

Atomicity and ordering

Profiles declare:

- minimum atomic-write unit;
- torn-write possibilities;
- ordering constraints;
- barrier/fsync/fdatasync semantics;
- metadata and directory persistence;
- rename semantics;
- writeback behavior;
- checksum/corruption detection;
- replication/controller caches where modeled.

A profile may be deliberately adversarial, permitting every state not forbidden by the declared contract.

Recovery

Recovery is ordinary controlled code executed under a recovery capability. It emits CIR events and can be canceled/crash again. Recovery invariants are first-class:

- idempotence;
- monotonic durable epoch;
- log-prefix validity;
- no acknowledgement resurrection;
- cleanup/quiescence;
- eventual completion under explicit assumptions.

Layered packs

```
FilesystemProfile
  refines BlockDeviceProfile
```

```
WALProfile
  refines FilesystemProfile
```

```
DatabaseTransactionProfile
  refines WALProfile
```

Continuum does not require all layers in every project. Claims identify the layer and refinement evidence used.

Qualification

Platform-qualified profiles require:

- documentation-derived contract;
- empirical litmus tests;
- fault-injection campaigns;
- cross-version fingerprinting;
- explicit environment (OS, filesystem, mount options, device, cloud service);
- conservative fallback when measurements disagree.

Measurements provide evidence, not universal proof.

First implementation

The first storage pack uses a small abstract append log:

- writes append records;
- sync promotes a prefix to stable;
- crash discards a nondeterministically selected volatile suffix;
- records have checksum validity;
- recovery scans the stable prefix;
- cancellation can occur before reservation, after reservation, after submission, and after stability.

This is enough to expose acknowledgement-before-sync and cleanup defects in the first vertical slice.

Rejected alternatives

- POSIX as one fixed semantics.
- Treat successful write as durable.
- Inject crashes only between test operations.
- Let production and Lab storage adapters use unrelated protocol logic.

Document: rfcs/0008-liveness-fairness-and-progress.md

RFC 0008: Liveness, Fairness, and Progress

Status: Proposed

Target gate: G4 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

Summary

Continuum SHALL treat liveness assumptions as explicit semantic objects. Finite randomized testing is never labeled a liveness proof. Cancellation progress, recovery, delivery, scheduler fairness, and timing assumptions are composed and reported separately.

Property model

Core temporal properties include:

- eventually P;
- always P;
- P leads_to Q;
- recurrence/persistence;
- response and stabilization;
- weak/strong fairness of named actions;
- bounded progress under time assumptions.

The internal representation may compile LTL-like syntax to automata, ranking obligations, or liveness-to-safety transformations.

Fairness scopes

Fairness is attached to action schemas and enabling predicates:

`weak_fair(Action)`: continuously enabled $\Rightarrow$ eventually taken
`strong_fair(Action)`: enabled infinitely often $\Rightarrow$ taken infinitely often

Distributed assumptions are separate:

- eventual message delivery;
- eventual partition healing;
- eventual stable leader;
- bounded clock drift after GST;
- finite crashes;
- fair task polling;
- cooperative cancellation checkpoints.

The result prints them as assumptions, never as implicit defaults.

Cancellation progress

Asupersync's request $\rightarrow$ drain $\rightarrow$ finalize lifecycle becomes a progress protocol. Candidate rankings include:

$$R = (\# \text{ live descendants, } \# \text{ unresolved obligations, remaining cleanup budget, } \# \text{ pending effect phases})$$

with a lexicographic or multiset order. A proof must justify decreases under fair scheduling and identify non-cooperative foreign calls as assumptions or unsupported paths.

Verification lanes

Finite graph

Build the product with a Büchi/parity monitor, find accepting SCCs, and account for fairness. Counterexamples are lassos or partial-order fair cycles.

Liveness to safety

Use prophecy/history/ranking instrumentation and safety engines where sound. The certificate records the transformation.

Ranking synthesis

Use templates, ICE/CEGIS, Horn clauses, and human hints to synthesize ranking functions or transition invariants.

Compositional progress

Components expose progress measures and rely/guarantee conditions. Composition checks that circular waiting assumptions are discharged rather than mutually assumed.

Partial-order reduction

A liveness reduction must preserve relevant cycles and fairness. Property-observer footprints contribute to dependence. The safety DPOR configuration is never reused by default.

Time

Bounded response is a timed property, not ordinary liveness. Virtual time and production clocks map to a common constraint semantics, with uncertainty and fairness explicitly separated.

Diagnostics

A liveness failure should classify:

- genuine fair cycle;
- unfair scheduler artifact;
- violated environment assumption;
- cancellation futurelock/obligation leak;
- insufficient bound;
- unknown due to abstraction.

The explanation includes the recurring causal core and the smallest set of fairness assumptions needed to eliminate it.

Success criteria

G4 requires:

- safety/liveness distinction in result schemas;
- weak/strong fairness semantics tested against TLC/another oracle on a corpus;
- at least one distributed progress proof;
- at least one cancellation-drain ranking certificate;
- liveness counterexample replay;
- no hidden fairness defaults.

Rejected alternatives

- “No failure after N simulated hours” as proof.
- Global scheduler fairness with no action-level semantics.
- Timeouts as a generic substitute for progress.
- Assuming every cancellation-aware task eventually cooperates.

Document: rfcs/0009-views-and-refinement.md

RFC 0009: Zoomable Views and Refinement

Status: Proposed

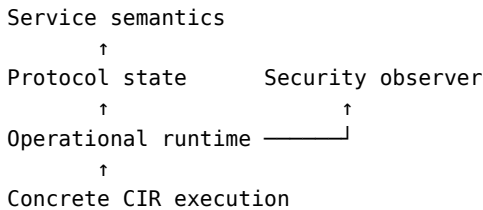
Target gates: G2/G5 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

Summary

Continuum SHALL support a chain of explicit semantic views from abstract service behavior to concrete runtime execution. Each edge is a checkable refinement obligation; no tool may infer correctness merely because two views are generated from related source code.

View graph

Views form a directed acyclic graph, not necessarily a linear stack:



A view defines state abstraction, event projection, observer alphabet, assumptions, and supported domains.

Refinement modes

Trace/stuttering refinement

Each concrete step maps to zero or more abstract steps.

Linearization refinement

Concrete operation intervals map to atomic abstract actions. Linearization witnesses may be explicit events or certified prophecy choices.

Forward simulation

A relation is preserved stepwise.

Backward/progressive simulation

Used where forward simulation is incomplete, especially for strong observational refinement and nondeterministic implementations.

Data refinement

Concrete representation relation plus operation simulation.

Hyper-refinement

Relates sets/distributions of executions for noninterference, randomized algorithms, or adversarial schedulers.

Observation-indexed correctness

Refinement is always relative to an observer set. Hiding an event is not semantically free if it affects timing, fairness, resource exhaustion, or another observer.

```
pub struct ViewContract {
    source: SemanticId,
    target: SemanticId,
    observers: ObserverSet,
    relation: Relation,
    event_map: EventMap,
    fairness_map: FairnessMap,
    assumptions: AssumptionSet,
}
```

Multi-grain checking

A coarse view is cheap but may produce spurious counterexamples. A fine view is expensive. Continuum may move between grains:

1. check coarse model;
2. replay counterexample in next finer view;
3. if infeasible, derive a separating predicate;
4. refine only the affected semantic slice;
5. retain a proof trail.

This is a CEGAR-like process over user-visible views rather than opaque predicates alone.

Abstraction maps from Rust

Rust-derived views may use:

- pure projection functions over snapshots;
- event-derived abstract state;
- ghost state maintained by checked instrumentation;
- relational constraints when state is incomplete.

Projection code is untrusted until checked. A projection can be evaluated differentially, verified with Verus/Creusot, or covered by a refinement certificate.

Compositionality

A component view exposes:

- owned resources;
- imported assumptions;
- exported guarantees;
- visible events;
- interference relation;
- progress obligations.

System composition must prove compatibility and discharge assumptions. Sheaf/gluing techniques are an experimental method for diagnosing incompatible local witnesses, not a default soundness mechanism.

Counterexample lifting and lowering

A violation in an abstract view is replayed in lower views:

```
abstract witness
  → constraint-guided operational scenario
  → Lab execution
  → production probe recommendation
```

A concrete violation is projected upward to locate the highest view whose property fails. The explanation reports where refinement, rather than the top-level invariant, breaks.

Versioning

Each view contract is content-addressed. A source or model change invalidates only dependent claims. Semantic diff reports whether changes affect state mapping, event visibility, fairness, or assumptions.

Rejected alternatives

- A single `fn abstract_state(&Concrete) -> Abstract` with final-state comparison only.
- Generate an abstract model from code and assume it is equivalent.
- One universal refinement relation for every observer and property.
- Hide runtime operations without proving stuttering/observational irrelevance.

Document: rfcs/0010-assurance-results-and-claims.md

RFC 0010: Assurance Results and Claims

Status: Proposed

Target gate: G0 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

Summary

Every Continuum command SHALL emit a typed, machine-readable assurance result. The CLI may summarize it, but cannot collapse distinct evidence classes into a green checkmark.

Result shape

```
pub struct AssuranceResult {
    pub claim: Claim,
    pub verdict: Verdict,
    pub scope: Scope,
    pub assumptions: AssumptionSet,
    pub evidence: Vec<Evidence>,
    pub trusted_components: Vec<ComponentRef>,
    pub limitations: Vec<Limitation>,
    pub reproducer: Option<CrashpackRef>,
}
```

Verdict

Established
Refuted
Inconclusive
Unsupported
ResourceExhausted
EngineError

Established is meaningful only together with evidence and scope.

Evidence classes

Evidence forms a partially ordered set, not a total ladder:

- deterministic example;
- randomized/sample campaign;
- bounded schedule exploration;
- equivalence-class-complete DPOR;
- finite exact reachability;
- symbolic bounded proof;
- inductive invariant proof;
- parameterized proof;
- liveness proof;
- refinement proof;

- production observation;
- external/differential corroboration;
- independently checked certificate.

A production observation is not “higher” or “lower” than an inductive proof; they establish different facts.

Scope

Scope includes:

- model/program hash;
- semantic version;
- domain sizes;
- schedule/fault/time bounds;
- pack profiles;
- properties and observers;
- fairness/environment assumptions;
- instrumentation completeness;
- target architecture/compiler when code-level semantics matter.

Claim examples

For all configurations reachable in the finite domain
Node=3, Value=2, with ≤ 2 fail-stop crashes and the
storage/log-v1 adversarial profile, invariant Agreement holds.
Evidence: exact reachable-set certificate.

Across 10^7 deterministic campaigns selected by PCT and
coverage guidance, no violation was observed.
Evidence: sampled; not exhaustive.

This production trace is inconsistent with all behaviors of
Protocol v12 under telemetry profile complete-send-receive.
Evidence: checked conformance unsat core.

Claim matrix

The repository maintains a claims matrix:

ID	Public wording	Formal predicate	Scope	Required evidence	Current state
----	----------------	------------------	-------	-------------------	---------------

Documentation CI rejects stronger wording than the evidence state permits.

Reproducibility

Every result records:

- exact command and environment;
- random seeds/choice logs;
- pinned toolchain/dependencies;
- solver versions and queries;
- certificate hashes;
- source commit;
- artifact-retention policy.

Human presentation

CLI examples:

REFUTED – replayable causal counterexample
ESTABLISHED (finite exact; certificate checked)
NO BUG FOUND (sampled 10,000,000 executions)
INCONCLUSIVE (production telemetry missing StorageStable)

Colors and checkmarks may not erase the qualifier.

Rejected alternatives

- One confidence score.
- “Passed” for both tests and proofs.
- Hiding solver trust behind a generic verified label.
- Treating resource exhaustion as no bug found.

Document: rfcs/0011-tla-examples-tribunal.md**RFC 0011: TLA+ Examples Compatibility Tribunal****Status:** Proposed**Target gates:** G9 (corpus interaction parity; translated from the Revision 2 corpus gate ladder, docs/26)**Normative corpus:** corpus/tla-examples/validated-examples.csv**Summary**

Continuum SHALL maintain an automated Tribunal that compares native Continuum ports against a pinned TLA+ Examples epoch. The Tribunal measures semantic parity, not textual similarity.

Inputs

Each case directory contains:

```
cases/TV-009-diehard/
  parity.toml
  source.lock
  model.ctm
  model.toml
  correspondence.md
  expected/
    oracle.json
    state-graph.canonical.zst
    shortest-counterexample.json
  lean/
    DieHard.lean
  mutations/
    wrong-capacity.patch
    broken-pour.patch
```

source.lock

Records:

- repository and commit;
- source module paths and SHA-256 hashes;
- configuration files;
- TLA+ tools version and Java runtime;
- optional Apache/TLAPS versions;
- model execution command;
- upstream expected outcome.

parity.toml

Declares:

```
id = "TV-009"
parity = "P2"

[state]
continuum = "State"
tla_vars = ["big", "small"]
projection = "identity"

[actions]
FillBigJug = "FillBigJug"
BigToSmall = "BigToSmall"
```

```
[properties.NotSolved]
kind = "invariant"
expected = "violated"
minimal_depth = 6

[comparison]
mode = "projected-bisimulation"
ignore_stuttering_multiplicity = true
```

Oracle extraction

The Tribunal supports three progressively stronger oracle paths:

1. **Output oracle:** parse TLC/Apalache/TLAPS results and traces.
2. **Graph oracle:** instrument/export finite initial states, successors, action labels, and normalized values.
3. **Semantic-AST oracle:** export SANY-resolved modules into a versioned neutral representation.

The Java tools are quarantined in the Tribunal. Normal Continuum verification never shells out to them.

Canonical value bridge

TLA+ values are converted to CorpusValue:

```
Bool | Int | String | ModelAtom
Tuple([v]) | Record({field: v})
Set(multiset-free canonical members)
Function(canonical finite graph)
Sequence([v])
```

Canonicalization must preserve distinctions relevant to the model. Uninterpreted model values carry stable atom identities scoped to the model configuration, not guessed textual ordering.

Comparison modes

Exact graph isomorphism

Used when state encodings match and auxiliary state is absent.

Projected graph isomorphism

Applies declared state projections before comparison.

Bisimulation

Computes a relation between upstream and native states preserving initiality, observations, and labeled transitions. Strong, weak, or stuttering bisimulation is selected explicitly.

Trace-language comparison

Used when state graph export is impractical. Bounded traces are canonicalized under stuttering and action mapping; hashes are compared and differences minimized.

Verdict-only comparison

Permitted only at P1. It never qualifies as P2+.

Temporal parity

For liveness cases, the Tribunal compares:

- fairness clauses after action mapping;
- accepted/rejected fair lassos;
- SCC witnesses;
- minimal lasso stem/loop where deterministic;
- leads-to obligations;
- deliberate liveness failures.

Search-order differences do not fail parity. Semantic fair-cycle differences do.

Proof parity

A TLAPS proof is not translated line by line. The case identifies theorem statements and semantic dependencies. A P4 case passes when Lean checks the corresponding theorem over the Continuum model and the executable encoding bridge.

Metamorphic tests

The Tribunal automatically transforms native and, where safe, TLA+ inputs:

- alpha-renaming;
- action reordering;
- definition inlining/factoring;
- equivalent comprehensions and map updates;
- explicit stuttering action insertion;
- permutation of symmetric atoms;
- auxiliary-state instrumentation.

Failure triage

Differences are minimized by:

1. model-domain reduction;
2. action-set reduction;
3. expression delta-debugging;
4. trace prefix/causal-core reduction;
5. configuration-key reduction.

The resulting fixture is permanently retained.

CI tiers

- **PR:** 10-second semantic kernel set.
- **Merge:** all Wave 0 plus changed feature families.
- **Nightly:** all 80 validated families at bounded reference configurations.
- **Weekly:** large models, all Lean proofs, mutation suite, multiple worker counts.
- **Epoch qualification:** full upstream update and historical replay.

Acceptance

The RFC is implemented when DieHard, Dining Philosophers, EWD840, Paxos, KeyValueStore, and TCP demonstrate all comparison modes needed by their families and the harness can classify a deliberately introduced semantic divergence.

Document: [rfcs/0012-lean-metattheory-and-reflective-certificates.md](#)

RFC 0012: Lean Metattheory and Reflective Certificates

Status: Proposed

Target gates: G6 (proof service; translated from the Revision 2 proof and verification gates per docs/26 — suffixes retired by plan §22)

Goals

1. Give precise mathematical meaning to Continuum claims.
2. Keep optimized search engines outside the trusted base.
3. Validate large certificates efficiently through reflection.
4. Connect solver-level evidence to original model semantics with verified encodings.
5. Provide reusable theorems for corpus ports and real systems.

Package split

continuum-metattheory-core	Lean core/Std only
continuum-metattheory-mathlib	algebra, order, probability, topology
continuum-certificate-ffi	byte format and parser correspondence
continuum-corpus-proofs	TLA+ example theorem parity
continuum-asupersync-laws	concrete effect/cancellation contracts

Foundational definitions

The first frozen definitions are deliberately conventional:

```
structure TransitionSystem (S) where
  init : S → Prop
  step : S → S → Prop
```

```
abbrev Behavior S := Nat → S
inductive Reachable ...
structure StutteringSimulation ...
```

CIR true-concurrency semantics are related to this interleaving projection rather than replacing it immediately. This permits early proofs with a familiar foundation while keeping the richer event-structure model available.

The theorem ladder**T0 — transition-system safety**

- reachability induction;
- inductive invariant preservation;
- finite closure certificate soundness;
- counterexample path validity.

T1 — stuttering and refinement

- stuttering simulation preserves reachable safety predicates;
- composition of simulations;
- observer projection and hidden-event closure;
- history/auxiliary variable erasure;
- conditions for liveness preservation.

T2 — temporal semantics

- infinite behaviors and stuttering closure;
- LTL without next;
- weak/strong fairness definitions;
- Büchi/Streett acceptance equivalence;
- fair lasso/SCC certificate soundness;
- rank/compassion progress certificates.

T3 — partial-order reduction

- event dependence and commutation diamonds;
- observer-indexed independence monotonicity;
- trace equivalence;
- source-set/DPOR coverage theorem for the supported finite fragment;
- certificate checker soundness.

T4 — symmetry and parameterization

- finite group actions and orbit representatives;
- canonicalization preserves transitions/properties;
- nominal support/equivariance for atom fragments;
- cutoff/quantified-invariant certificate schemas.

T5 — concrete execution

- cancellation phase and obligation state machine;
- reserve/commit/abort laws;
- region closure/quiescence contracts;
- asupersync event adapter refinement into CIR;
- selected protocol implementations refine abstract models.

Certificate architecture

A certificate has three layers:

```
wire bytes
  ↓ parser theorem
well-formed certificate value
  ↓ checker soundness theorem
semantic proposition
```

The parser is not trusted by assertion. Either:

- a verified parser is generated/implemented in Lean and extracted; or
- the Rust parser produces a canonical digest and a second Lean parser consumes the same bytes for proof import.

Certificate families

- path/counterexample;
- finite closure;
- inductive invariant;
- simulation/refinement;
- symmetry quotient;
- DPOR coverage;
- fair SCC/lasso;
- ranking function;
- SAT LRAT;
- pseudo-Boolean VeriPB;
- SMT/Alethe or solver-specific proof;
- MDP/value-iteration bounds with rational enclosures.

Reflection

Large certificates are checked by proved Boolean functions executed as native Lean code. The theorem shape is:

```
theorem check_sound (input : Input) (cert : Cert) :
  check input cert = true → SemanticallyValid input
```

A successful reflected check yields a theorem without materializing every low-level inference as a giant proof term.

Verified encodings

Solver certificates prove facts about encodings. Continuum must also prove:

```
model semantics
  ↔ bounded transition formula
  ↔ bit-blasted/PB/CNF encoding
```

Encoding versions are semantic epochs. A solver proof without an encoding theorem is assurance-capped.

Axiom policy

Every public theorem reports `#print axioms`. Permitted foundations are documented. `sorry`, `admit`, unchecked native axioms, and opaque foreign proof imports are forbidden in release proof packages.

Build strategy

Lean proof compilation is not on the ordinary Rust edit loop. CI uses:

- cached `.olean` artifacts;
- declaration-granular targets;
- proof package partitioning;
- nightly full rebuilds;
- deterministic theorem manifests.

Acceptance

The first accepted milestone imports a Rust-generated finite closure certificate for DieHard, proves `TypeOK` for all reachable states, rejects a malformed closure, and reproduces the same theorem under the pure Lean checker.

Document: rfcs/0013-semantic-triptych.md

RFC 0013: Semantic Triptych and Cross-Path Validation

Status: Proposed

Target gate: G0 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

Problem

Continuum wants one coherent meaning across abstract models, concrete programs, and proofs. A literal single implementation would create circular validation; three unrelated implementations would drift.

Surfaces

Model plane

- CML AST and typed core;
- exact reference evaluator;
- abstract transition/behavior semantics;
- model configurations and finite/symbolic domains.

Program plane

- asupersync runtime and Lab;
- domain-pack operations;
- semantic event journal;
- concrete snapshots and replay.

Proof plane

- Lean definitions;
- certificate schemas/checkers;
- theorem manifests;
- solver encoding proofs.

Shared artifacts

The planes may share declarative artifacts:

- versioned algebraic data schemas;
- operator tables;
- generated serialization fixtures;
- model/value test vectors;
- property IDs and source maps;
- semantic epoch hashes.

They may not all call the same evaluator to justify agreement.

Cross-path matrix

CML reference ↔ optimized Rust evaluator

CML reference ↔ TLA+ corpus oracle

Rust evaluator ↔ SMT/PDR encodings

CIR journal ↔ model/refinement checker

native certificate checker ↔ Lean reflective checker

asupersync adapter ↔ executable primitive conformance models

Each arrow has generated random tests, curated edge cases, and mutation tests.

Semantic-change protocol

Changing semantics requires:

1. ADR/RFC amendment;
2. Lean definition change or proof that behavior is unchanged;
3. reference evaluator update;
4. optimized engine update;

5. corpus differential report;
6. replay migration decision;
7. semantic epoch increment if old artifacts change meaning.

No pull request may change all expected outputs and call the suite green without an explicit semantics review.

Generated finite universes

For operator-level differential testing, a generator constructs small closed universes of values:

- booleans and bounded integers;
- small sets/functions/records/sequences;
- model atoms and permutations;
- nested expressions with definedness constraints.

The same expression/transition fixtures are evaluated by every available plane. This is more effective than waiting for full protocols to expose edge cases.

Undefined and partial behavior

CML does not use host panics to define mathematics. Potentially undefined operations elaborate to:

- a static rejection;
- a proof obligation;
- an explicit Undefined semantic result that invalidates the model;
- never an arbitrary value.

The TLA+ oracle bridge records where upstream semantics are intentionally underspecified or implementation-defined.

Assurance rule

Agreement raises confidence, not theorem level. The proof plane raises theorem level only through checked evidence. Disagreement always lowers confidence until resolved.

Document: rfcs/0014-observer-indexed-dpor.md

RFC 0014: Observer-Indexed DPOR and Causal Reduction

Status: Proposed

Target gates: G2 and G6 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

Objective

Explore one representative of every execution class relevant to the active property contract, while emitting enough evidence to justify the omission of other schedules.

Observation contract

```
struct ObservationContract {
    view: ViewId,
    visible_events: EventPredicate,
    properties: Vec<PropertyId>,
    fairness: Vec<FairnessMonitor>,
    lifecycle: LifecycleSensitivity,
    time: TimeSensitivity,
    durability: DurabilitySensitivity,
    security: Vec<HyperObserver>,
}
```

The contract is content-addressed and included in exploration/cache keys.

Dependence layers

1. **Structural:** same task, causal predecessor, region lifecycle.
2. **Resource:** overlapping read/write/linear resource footprints.
3. **Effect:** network endpoint, storage object, timer, entropy, fault scope.
4. **Observer:** visible order or state projection differs.

5. **Fairness:** swapping changes enabledness/justice/compassion accounting.
6. **Refinement:** event mapping or stuttering classification differs.
7. **Hyperproperty:** cross-run relation observes the order or information flow.

Independence requires all active layers to permit commutation.

Witness

A dynamic witness includes:

configuration digest
left/right event schemas and concrete parameters
enabledness evidence
resource footprint evidence
left-right and right-left successor digests
view equality witness
visible-event equality
obligation/fairness delta equality
semantic rule IDs

The checker can replay the local diamond under reference semantics.

Baseline algorithm

The product baseline is source-DPOR with conservative dependencies. The next promotion target is parsimonious optimal DPOR because it explores one Mazurkiewicz class with polynomial worst-case space for its supported setting. Await-aware handling prevents pure polling/wait loops from dominating exploration and preserves livelock diagnostics.

True-concurrency escalation

When event boundaries and independence are trustworthy, Continuum may build:

- prime event structures;
- occurrence-net/unfolding prefixes;
- interval pomsets;
- higher-dimensional cells for jointly independent events.

These representations are not automatically superior. They compete against the best certified DPOR baseline.

Property preservation tiers

- Tier A: finite state safety predicates.
- Tier B: stutter-invariant LTL without next.
- Tier C: fairness-aware liveness.
- Tier D: hyperproperties and quantitative observers.

A reduction theorem/certificate is specific to a tier. Safety soundness cannot be reused as a liveness claim.

Caching across observers

If observer fine refines coarse, independence under fine is reusable under coarse, while the reverse is not. Continuum maintains an observer lattice and can reuse conservative witnesses downward.

Evaluation

Metrics:

- explored executions/configurations;
- witness-check cost;
- wall time and memory;
- bug depth/time-to-first-counterexample;
- reduction ratio by observer;
- certificate size/check time;
- missed-bug mutation score (must be zero for certified tier).

Acceptance

Demonstrate property-dependent reductions on Dining Philosophers, an audit-sensitive transaction protocol, a cancellation-heavy service, and one consensus model. The fine/coarse observer monotonicity theorem must be checked in Lean before dynamic observer-based pruning raises assurance.

Document: rfcs/0015-temporal-fairness-and-hyperproperties.md

RFC 0015: Temporal, Fairness, and Hyperproperty Semantics

Status: Proposed

Target gates: G3–G4 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

Core behavior model

A behavior is an infinite state/event execution. Finite executions are completed according to an explicit policy:

- stutter forever at terminal state;
- deadlock violation;
- finite-trace property only;
- environment-closed completion.

No engine silently changes this policy.

Temporal language

The initial temporal core supports:

- always P ;
- eventually P ;
- P leads_to Q ;
- recurrence and persistence;
- action occurrence/enabledness;
- weak and strong fairness;
- bounded metric operators under the timed extension.

The theorem-preserving core is LTL without next, because stuttering refinement is central. next is allowed only under an observation granularity that makes each step semantically fixed.

Fairness

Fairness is attached to named action schemas plus a view of variables/events. Each result lists:

- scheduler fairness;
- action weak/strong fairness;
- network delivery fairness;
- crash/recovery restrictions;
- cancellation checkpoint responsiveness;
- environment/client assumptions;
- timer/clock progress.

Fairness monitors become part of CIR and reduction dependence.

Model checking

Finite models use automata-theoretic checking:

1. translate property to a Büchi/generalized Büchi/Streett monitor;
2. construct product on demand;
3. find accepting SCCs/fair lassos;
4. minimize stem and loop under semantic constraints;
5. emit SCC/lasso certificate.

Ranking liveness

For parameterized or symbolic models, Continuum supports proof obligations built from:

- well-founded rankings;
- lexicographic/multiset orders;
- helpful transitions;
- justice/compassion premises;
- modular progress lemmas;
- proof graphs/slices.

Agents may propose rankings. Solvers may synthesize coefficients. Lean or a checked certificate validates the final argument.

Hyperproperties

Some important claims compare executions:

- noninterference;
- observational determinism;
- linearizability/refinement;
- fault transparency;
- constant-time or secrecy observers.

Continuum represents these with explicit product/self-composition or automata over tuples of executions. Strong refinement is required where trace inclusion is insufficient.

Runtime limitation

Incomplete asynchronous production observations can make properties undecidable. Runtime conformance therefore uses four-valued evidence:

Valid | Invalid | Inconclusive | SemanticsMismatch

Inconclusive is not failure and never becomes Valid through timeout.

Assumption provenance

Every liveness theorem contains an assumption object with stable IDs and a partial order of strength. Adding a fairness assumption changes the claim identity.

Corpus requirements

P3 ports must include at least one accepted fair behavior, one unfair behavior excluded by assumptions, and a deliberate liveness mutation. EWD termination cases, mutual exclusion, consensus, and Streamlet-like external cases form the core suite.

Document: rfcs/0016-parameterized-and-nominal-verification.md

RFC 0016: Parameterized, Symmetric, and Nominal Verification

Status: Proposed research/product boundary

Target gate: G6 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

Problem

Checking three nodes is useful bug finding, not a theorem for every cluster size. IDs and fresh names also cause artificial blowups when their concrete identities are irrelevant.

Portfolio

Finite symmetry

Declared finite groups act on states/actions. Canonical representatives and orbit sizes are exact. Certificate witnesses include permutations and canonicalization results.

Symmetry-to-quantification

Finite IC3/PDR clauses are generalized using protocol symmetries into quantified invariants. Cutoff candidates are validated, not assumed.

Counter abstraction

Processes are grouped by local control/data predicates, producing count variables. Soundness obligations define simulation from concrete populations.

Regular/array abstraction

Parameterized arrays/rings can be abstracted to regular languages or string-rewriting systems where applicable.

WSTS

Monotone systems use well-quasi-order coverability and ideals. The engine reports coverability, not exact reachability, unless justified.

Nominal/orbit-finite lane

Atoms represent identities under a permutation action. States are stored by finite support and orbit shape. Fresh allocation becomes binding rather than selection from a fixed numeric bound.

Language annotations

```
type Node : atom[equality]
type Epoch : ordered
symmetry Nodes by all_permutations
fresh RequestId
parameter N : Nat where N >= 1
```

Operations that violate equivariance are rejected or force symmetry refinement.

Quantified invariant certificate

A certificate records:

- invariant formula and quantifier structure;
- finite instances/cutoff evidence used to derive it;
- initialization proof;
- action-local inductiveness obligations;
- implication to target safety property;
- solver proofs or Lean lemmas.

Finite testing can suggest the formula; only the inductive proof establishes the unbounded claim.

Small-model discipline

Continuum reports three distinct results:

- `checked(N = 3)`;
- `cutoff_checked(N ≤ k)` with theorem identifying the cutoff rule;
- `proved(∀ N)` via inductive/parameterized certificate.

UI and APIs must make them visually and structurally different.

Initial targets

- Bakery/mutual exclusion;
- Chang-Roberts ring election;
- German cache coherence;
- quorum protocols/Paxos abstractions;
- sessions with unbounded fresh request IDs;
- EWD self-stabilizing/termination examples.

Document: rfcs/0017-assumption-and-protocol-synthesis-games.md

RFC 0017: Assumption and Protocol Synthesis as Games

Status: Proposed research lane

Target gate: G6 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

System/environment partition

Every nondeterministic choice is classified:

- system-controlled;
- implementation/scheduler-controlled;
- environment-controlled;
- adversarial fault;
- stochastic;
- angelic specification choice.

Conflating these choices makes realizability and probability meaningless.

Game construction

Finite temporal models compile to turn-based or concurrent games with safety, Büchi, generalized Büchi, Streett, or parity objectives.

The engine can return:

- winning system strategy;
- spoiling environment counterstrategy;
- unrealizable core;
- candidate safety restriction;
- candidate fairness assumption;
- strategy implementation sketch.

Assumption grammar

Synthesis is bounded by a declared grammar:

```
never(drop class=control forever)
eventually(deliver m) when sender_alive(m)
weak_fair(schedule task) while obligation_pending(task)
at_most(k, crashes, per=epoch)
clock_drift <= epsilon
```

Candidates are ordered by implication/strength within the grammar. Claims of minimality are relative to this order.

Production realizability

A synthesized assumption must map to:

- domain-pack guarantee;
- deployable mechanism;
- monitorable SLO;
- operator procedure;
- or explicit unverifiable premise.

An assumption that no real network/runtime can guarantee is displayed as a design defect, not a proof success.

Protocol sketch synthesis

CML permits finite holes in guards, updates, quorum thresholds, retry policy, and phase ordering. Counterexample-guided synthesis explores candidates modulo semantic equivalence. Every synthesized candidate is reverified independently and mutation-tested.

Agents can propose grammars and sketches but cannot bypass finite/symbolic proof.

Cancellation application

The game view is especially useful for cancel-correctness:

- environment requests cancellation at adversarial points;
- implementation chooses drain/finalize actions;
- budgets/timeouts constrain progress;
- objective requires no leaked obligations and eventual quiescence under responsiveness assumptions.

Deliverable

A flagship demonstration should start from a liveness-failing replicated service, synthesize the weakest available delivery/scheduling assumption or protocol repair under the chosen grammar, and produce both the counterstrategy and checked repaired model.

Document: rfcs/0018-agent-native-proof-and-repair-loop.md**RFC 0018: Agent-Native Proof and Repair Loop**

Status: Proposed

Target gates: G2 onward (Revision 2 scheme, docs/26; Rev-2 G2 $\approx$ Rev-3 G4 per plan §24 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

Principle

Agents are powerful search and explanation systems. They are not trusted oracles. Continuum exposes structured tasks and checks every result.

Machine-facing artifact graph

```
claim
├─ model/version
├─ assumptions
├─ property AST
├─ search result
├─ counterexample/certificate
├─ source correspondence
├─ proof obligations
└─ reproduction commands
```

Every node has a stable schema and content hash.

Agent operations

- translate prose/design into a draft model;
- port a TLA+ corpus case;
- propose invariants, lemmas, rankings, cutoffs, observers, and abstraction maps;
- classify counterexamples as model/implementation/assumption defects;
- minimize semantic changes;
- propose Rust fixes;
- generate Lean proof attempts;
- search the corpus for analogous proof patterns;
- explain evidence at multiple abstraction levels.

Acceptance pipeline

A proposed repair passes only when:

1. original failure replays;
2. patch changes the intended semantic slice;
3. exact failure no longer occurs;
4. neighboring causal schedules/faults are explored;
5. mutation suite remains sensitive;
6. assumptions/properties did not weaken unexpectedly;
7. required certificate/Lean theorem checks;
8. production/runtime compatibility tests pass.

Counterexample API

Agents receive a minimized causal explanation rather than megabytes of logs:

```
{
  "violated": "AckImpliesDurable",
  "causal_core": ["reserve#18", "cancel#20", "ack#22", "crash#23"],
  "missing_happens_before": ["sync#21 -> ack#22"],
```

```
"abstract_delta": {"acked": "+(n,e,v)", "durable": "unchanged"},
"assumptions_used": ["A-NET-03"],
"replay": "continuum replay cp:..."
}
```

Proof search

Lean goals are emitted with:

- relevant definitions unfolded;
- proof slice/dependency graph;
- candidate lemmas from corpus/search;
- countermodels to failed induction;
- finite-instance evidence marked as heuristic;
- no hidden axioms.

Agent-generated proofs are accepted only by Lean's kernel.

Security

Agent execution is sandboxed. Model/compiler/proof inputs are untrusted. Agents cannot mark claims PROVEN, rewrite evidence ledgers, or weaken policies without an explicit diff and review.

Document: rfcs/0019-corpus-port-manifest.md

RFC 0019: Corpus Port Manifest and Evidence Schema

Status: Proposed

Target gate: G9 (corpus interaction parity; translated from the Revision 2 corpus entry gate per docs/26 — suffixed names retired by plan §22)

Purpose

Make every semantic-equivalence claim inspectable, reproducible, and machine-checkable.

Manifest shape

```
schema = "continuum-corpus-port/v1"
id = "TV-009"
title = "The Die Hard Problem"
source_epoch = "tla-examples@91c22ea..."
parity = "P2"
status = "passing"

[[source.module]]
path = "specifications/DieHard/DieHard.tla"
sha256 = "..."

[[source.model]]
config = "..."
mode = "exhaustive"
expected = "safety-failure"

[native]
model = "model.ctm"
config = "model.toml"
semantics_epoch = "cml/0.2"

[correspondence]
state = "state-map.cir-map"
actions = "action-map.toml"
stuttering = "collapse"

[[claim]]
id = "TypeOK"
kind = "invariant"
```

```
expected = "valid"
evidence = ["closure:sha256:..."]

[[claim]]
id = "NotSolved"
kind = "invariant"
expected = "invalid"
minimal_depth = 6
evidence = ["crashpack:sha256:..."]
```

Required fields by parity

- P1: source, native model, correspondence, expected verdict.
- P2: finite comparison mode, normalized state/action map, oracle artifacts.
- P3: temporal/fairness map and liveness evidence.
- P4: Lean theorem names, axioms report, source theorem correspondence.
- P5: Rust package, binary hash, adapter coverage, refinement evidence.

Status taxonomy

```
untriaged
inventoried
porting
blocked-language
blocked-engine
blocked-proof
oracle-disagreement
passing
intentionally-divergent
unsupported
```

Only passing counts toward release parity. intentionally-divergent must explain why and does not satisfy the 1.0 contract without an explicit scope amendment.

Generated dashboard

The corpus dashboard shows coverage by:

- semantic feature;
- parity level;
- backend;
- proof status;
- mutation score;
- runtime and memory;
- last successful semantic epoch;
- unresolved disagreement.

Document: rfcs/0020-proof-producing-transformations.md

RFC 0020: Proof-Producing Transformations and Optimization Pipeline

Status: Proposed

Target gates: G2–G6 (Revision 2 scheme, docs/26 — not citable without translation to the docs/52 Revision 3 gates per plan §22)

Motivation

CML models undergo elaboration, desugaring, slicing, symmetry reduction, abstraction, bit-blasting, and solver encoding. A proof about the final formula is useless if an earlier transformation changed meaning.

Transformation contract

Every transformation implements:

```
input semantic object
output semantic object
```

mapping/witness
preservation claim
checker or Lean theorem
source provenance map

Claims include:

- equivalence;
- forward simulation;
- backward simulation;
- equisatisfiability;
- safety overapproximation;
- liveness-preserving abstraction;
- property-specific preservation.

“Optimization” is not a semantic category.

Pipeline example

```
CML source
-- elaboration equivalence --> typed core
-- finite instantiation --> finite relation
-- slicing --> property cone
-- symmetry quotient --> orbit graph
-- transition encoding --> SMT
-- bit-blast --> CNF
-- SAT solve --> LRAT
```

The final evidence bundle links every arrow. A failure at any arrow caps the result at exploratory.

Proof artifact economy

Full proof terms are not always practical. Allowed evidence forms:

- local rewrite traces checked by a small normalizer;
- simulation maps checked by finite traversal;
- hash-consed DAG proofs;
- solver certificates checked by reflection;
- theorem IDs for globally proved compiler passes;
- per-instance side conditions.

Translation validation

For complex optimizations where a once-and-for-all compiler proof is expensive, Continuum uses translation validation: each output comes with a witness that the checker validates against the input.

Source mapping

Preservation evidence also carries provenance from transformed variables/actions back to source. Counterexamples and proof failures must remain explainable after aggressive transformations.

Initial proof-producing passes

1. action desugaring;
2. finite-domain instantiation;
3. property cone slicing;
4. exact state canonicalization;
5. finite symmetry quotient;
6. bounded transition unrolling;
7. CNF/PB encoding.

Forbidden pattern

A transformation may not be trusted because “the tests match TLC.” Differential tests are necessary evidence, not a semantic proof.

Document: rfcs/0021-corpus-oracle-protocol.md

RFC 0021: Corpus Oracle Protocol

Summary

Define the reproducible protocol by which native Continuum ports are compared with pinned TLC, Apalache, PlusCal, and TLAPS artifacts without making those tools runtime dependencies.

Required artifacts

- source repository/commit/path hashes;
- toolchain container or lock manifest;
- model configuration and expected result;
- normalized initial/successor/reachable-state facts;
- ITF/CIR trace correspondence;
- semantic relation used for comparison;
- command, exit status, stdout/stderr hashes;
- timeout/resource classification.

Comparison modes

1. exact finite graph;
2. bounded relational successor sampling;
3. shortest failure;
4. liveness/fair-lasso shape;
5. proof theorem intent;
6. mutation agreement.

Security

Foreign tools execute in a sandbox with no network and bounded resources. Oracle output is untrusted data parsed by hardened adapters.

Document: rfcs/0022-weak-memory-local-refinement.md

RFC 0022: Weak-Memory Local Refinement

Summary

Specify a local component-verification lane based on execution graphs and refinement to atomic contracts.

MVP semantics

- SC atomics;
- locks/channels as explicit synchronization;
- program order and reads-from;
- operation call/return events;
- linearization witness.

Extended semantics

- modification order, from-read, synchronizes-with, happens-before;
- Relaxed/Acquire/Release/AcqRel/SeqCst;
- named Rust/compiler/hardware model;
- SAT/SMT candidate execution generation;
- proof-producing solver path.

Composition

Once verified, the component exports an atomic summary used by distributed models. The receipt records model, bounds, memory semantics, and refinement witness.

Document: rfcs/0023-checked-choreographic-projection.md

RFC 0023: Checked Choreographic Projection

Summary

Project CML global protocols to local role automata and generated Rust interfaces.

Outputs

- local role state machines;
- message enums and typed payloads;
- endpoint traits;
- asupersync task skeletons;
- instrumentation labels;
- local monitors;
- projection receipt.

Rejection conditions

Projection fails when a role must choose without knowing the branch, communication assumptions mismatch, cancellation/fault branches are missing, or local composition admits behavior outside the global model.

Non-goal

Generated code does not prove arbitrary user implementation code. It provides a checked interface and refinement target.

Document: rfcs/0024-proof-receipt-format.md

RFC 0024: Proof Receipt Format

Summary

Define the content-addressed record for a strongest-assurance Continuum result.

Fields

- receipt schema version;
- claim and assurance class;
- model/property/assumption/observer hashes;
- semantic, corpus, domain-pack, and proof epochs;
- transformation steps and preservation classes;
- certificate type/hash;
- checker source/binary hash;
- Lean version, theorem names, imports, and axiom manifest;
- foreign evidence references, if any;
- reproduction command;
- optional signatures.

Validation

Schema validation is necessary but insufficient. `continuum proof check` recomputes hashes and invokes the appropriate independent checker.

Document: rfcs/0025-property-directed-abstraction-loop.md

RFC 0025: Property-Directed Abstraction Loop

Summary

Define a CEGAR-style workflow for proposing and validating abstraction maps from concrete CIR/Rust systems to CML models.

Loop

1. derive conservative dependency slice from property/observer;

2. propose predicates and abstract fields;
3. construct abstract transformer;
4. check property/refinement;
5. classify counterexample as concrete or spurious;
6. refine predicates/history/prophecy state;
7. emit semantic diff and obligations;
8. promote only with bounded or theorem receipt.

Agent role

Agents may propose predicates and mappings. They cannot mark a spurious trace or accept an abstraction without checker evidence.

Document: rfcs/0026-continuumd-native-protocol.md

RFC 0026: continuumd Native Protocol

Status

Draft for implementation.

Target gate: G1 (workbench identity and lifecycle), G2 (agent-computer interface) **Owners:** daemon/protocol leads
Normative language: MUST/SHOULD/MAY per RFC 2119.

Summary

The single authoritative request/result protocol. CLI, Cargo, LSP, DAP, MCP, SARIF, TUI, and web clients are projections of this protocol and MUST NOT define semantics of their own (ADR-0036/0038/0042).

IDL and versioning

- The protocol is defined in one machine-readable IDL file (checked into the repo; generated Rust/TS/JSON-schema clients derive from it). Prose in this RFC summarizes the IDL; the IDL is normative once it exists.
- `protocol_version` (e.g. "3.0") is negotiated at connection open: the client sends its supported range; the daemon selects the highest common version or rejects with `ProtocolVersionUnsupported`. Protocol, semantic, intent, evidence, proof, and corpus epochs are versioned independently (docs/12 §7) and MUST NOT be conflated.
- Within a major protocol version, servers MUST ignore unknown optional request fields and MUST NOT emit fields the negotiated version does not define. Evidence artifacts are never “best-effort decoded” across breaking epochs (docs/09 T13): unknown breaking epoch $\Rightarrow$ typed rejection.
- Handles remain valid across daemon upgrades within a major version; continuations additionally pin engine and semantic epochs and MUST be rejected with `ContinuationEpochMismatch` on mismatch (never silently re-run).

Request envelope

Field	Type	Req	Notes
protocol_version	string	yes	negotiated
request_id	string	yes	client-unique, for tracing
idempotency_key	string	yes for mutations	see below
actor	string	yes	e.g. agent:repairer-1, human:bob
capability	cap_* handle	yes	checked below the adapter, independently of handle possession
operation	namespace.verb	yes	from the operation registry (plan §10.2)
snapshot	ws_* or null	yes	explicit null when unused; the field name is snapshot everywhere (schemas included)
intent	in_* or null	yes	explicit null when unused
arguments	object	yes	per-operation schema from the IDL
budget	object	for long ops	wall/cpu/memory/states/solver/proof/tokens/candidates
output_policy	object	optional	max bytes/tokens/nodes, audience
trace	object	optional	W3C/OTel context propagation

Idempotency: a mutation replayed with the same `idempotency_key` and byte-identical canonical request **MUST** return the same task/artifact identity; the same key with a different request **MUST** be rejected (`IdempotencyKeyReused`). Keys are scoped per actor and **MUST** be honored for at least the retention window declared in server capabilities (default 24h).

Result envelope

Field	Type	Notes
request_id	string	echo
status	enum	ok, error, task_started, task_suspended
verdict	typed	operation-specific; never bare prose
error	typed	from the taxonomy below; MAY carry recovery = list of allowed operations with pre-filled arguments (never free-form commands)
assurance	envelope	required on every semantic verdict (plan B11); every dimension names a producer or reads Unsupported
artifacts	handle list	typed prefixes per plan §4.4
task / continuation	task_* / cont_*	for long operations
omissions	manifest	INV-007
warnings	typed list	
cost	object	actual spend per budget dimension
epochs	object	semantic/engine/proof epochs the result is pinned to
next_operations	list	allowed operations from this state — the safe recovery/discovery surface (plan §0.2)

Task lifecycle

Created → Running → Suspended | Completed | Failed | Cancelled (plan §4.1).

- Task-starting operations (verification.start, model.check, forge.create, and peers — there is no generic task.start) are idempotent under idempotency keys; task.status is monotonic; task.cancel triggers request→drain→finalize and MUST leave either committed partial evidence plus a valid continuation, or nothing published (INV-009, B19).
- task.resume validates continuation identity, snapshot, and epochs; stale inputs are rejected (StaleSnapshot, ContinuationEpochMismatch). Resume MAY add evidence; it MUST NOT replace prior artifacts under the same identity.
- task.subscribe streams progress events over the same connection; events are hints — committed artifacts and task.status are authoritative.

Pagination

List-returning operations accept page_size and an opaque page_token, and return next_page_token. Ordering MUST be deterministic (content identity or explicitly declared sort key); two identical requests against the same snapshot return identical pages.

Transport, encoding, authentication

- Local IPC (unix socket / named pipe) first; authenticated HTTP/QUIC for shared/remote modes.
- Canonical JSON for debugging; CBOR with the same canonical field order for performance. One encoding per connection, negotiated.
- Remote mode requires an identity model; capabilities (cap_*) are minted, scoped, delegated, and revoked via daemon operations recorded in the audit log (plan §4.5). Possession of an artifact handle never implies authorization (ADR-0037).

Error taxonomy

Stable codes (plan §10.3): StaleSnapshot, UnsupportedSemanticFeature, IntentMutationDenied, InsufficientEvidence, BudgetExhausted, ContinuationEpochMismatch, AmbiguousCorrespondence, UntrustedDomainBoundary, CertificateRejected, ReplayDiverged, CapabilityDenied, PolicyGateFailed; plus protocol-level ProtocolVersionUnsupported, IdempotencyKeyReused, MalformedRequest. BudgetExhausted is never a semantic verdict (docs/49); it carries the continuation when one exists.

Rejected alternatives

- **Session-scoped implicit state.** Rejected (INV-002): a dropped connection must not change meaning.
- **Adapter-defined semantics.** Rejected (ADR-0042): MCP/LSP/DAP translate; they do not decide.
- **Free-form recovery suggestions in errors.** Rejected: recovery is a list of typed allowed operations to prevent injection through error text.

Open questions

- IDL technology choice (custom vs. an existing schema language) — decide before PR 5 freezes anything.
- Capability delegation depth and expiry defaults for multi-agent handoff (with RFC 0027).

Acceptance

Golden request/response traces pinned per protocol version; malformed-input fuzzing on both encodings; idempotency replay tests; restart/resume with epoch mismatches; cancellation at every instrumented phase; deterministic pagination; adapter parity (same operation through CLI/MCP/LSP yields identical artifacts).

Document: rfc/0027-agent-tool-protocol.md

RFC 0027: Agent Tool Protocol

Status

Draft for implementation.

Target gate: G2 (agent-computer interface) **Owners:** ACI leads **Normative language:** MUST/SHOULD/MAY per RFC 2119.

Summary

The typed operation surface agents use, layered over RFC 0026. No agent workflow requires terminal parsing, cursor positions, or session reconstruction (plan B2, INV-003).

Operation registry

The registry is the plan §10.2 list; each entry carries a minimum authority level. Signatures live in the RFC 0026 IDL.

Continuum: Master Implementation Plan & Comprehensive Dossier

Namespace	Operations	Min authority
workspace	create / fork / diff / seal	propose
intent	get / diff	read
intent	propose_revision	revise-intent
intent	accept / reject	revise-intent
intent	lock	revise-intent
verification	start / result / await	execute
model	check / explore / compare	execute
program	extract / run / replay	execute
refinement	check / explain	execute
proof	goal / attempt / check / slice	execute
correspondence	bind	propose
correspondence	status / drift	read
debug	open / state / enabled / step_event / step_abstract / reverse_causal / branch / compare / why_enabled / why_blocked / export	execute
context	compile / expand	read
failure	explain / minimize / branch	execute
repair	begin / apply / attach / evaluate / resume / review	propose/execute
repair	promote / reject	promote
observe	ingest	execute
observe	classify / result	read
forge	create / step / archive / materialize	execute
task	status / cancel / resume / subscribe	read/execute
evidence	get / query / verify	read
query	explain_reuse / explain_invalid / clean_compare	read
benchmark	run	execute

correspondence.bind creates or updates a §16 correspondence link at patch-proposal authority; correspondence.status and correspondence.drift are read-only inspections. observe.ingest additionally requires the production-trace capability (plan §18.2; capture-time contract in plan §18.4); observe.classify and observe.result are read-only. Both families are registered ahead of their producing subsystems (the Phase C read-only observation lane and Phase F production partial-order evidence); until those ship, calls MAY fail with the typed UnsupportedSemanticFeature error.

Authority levels and capabilities

Five levels — read < propose < execute < revise-intent < promote — map onto the plan §18.2 capability set. A cap_* capability names its actor, level, resource scope (snapshots/intents/artifact classes), expiry, and delegation allowance. Agent-facing installations MUST omit promote and revise-intent by default; intent.accept is the only transition from Proposed to protected status and, with intent.lock, is audit-recorded and never present in default agent capability profiles (plan §5.4); promotion is a service decision gated by RFC 0032, never an agent assertion (INV-015, B12). Calls above the capability's level fail with CapabilityDenied before any semantic work runs.

Context policy

- Requests carry `output_policy` (max bytes/tokens/nodes, audience). Token counts are advisory and tokenizer-relative; **byte budgets are the enforced contract** (token counts differ per model and are recorded with the tokenizer id when reported).
- Default failure result: one Context Pack (RFC 0028) plus `next_operations`. Everything else is reachable by expansion, never by dumping.
- Expansion is relation-based over the evidence/causal/proof/source graphs. The initial relation vocabulary: `causal_predecessors`, `causal_successors`, `conflicts_with`, `same_owner`, `property_automaton_step`, `proof_dependency`, `source_span`, `assumption_uses`, `abstraction_of`, `refinement_of`, `alternate_branch`. Each expansion returns a new immutable pack referencing its parent.

Handoff

A coordinator hands agents: shared snapshot, intent, and evidence-graph read scope; isolated `dbg_*`, `ps_*`, and Forge candidate scopes (plan §10.5). Handles thread through tool calls; no session affinity exists — any client holding the handles and a valid capability can continue the work.

Safety

- Source, logs, model text, and production payloads are data. They MUST never be interpolated into tool descriptions, error text, or `next_operations` (INV-016).
- Result schemas always include omissions and typed uncertainty; hiding uncertainty to save tokens is prohibited (docs/34 anti-goal).
- All privileged calls are audited with actor, capability, inputs, and decision (plan §18.5).

Evaluation

Baseline ladder (research/33): raw shell agent → typed ACI → ACI + Context Packs → ACI + evidence graph, with identical base models and budgets. Metrics: task success, invalid-operation rate, tokens/bytes, expensive failures, stale-handle recovery. The protocol MUST NOT freeze until ablation shows the typed surface beats disciplined shell use (G0-DX-10); if it does not, the ACI is redesigned, not excused.

Rejected alternatives

- **Prose-first results with a JSON flag.** Rejected: prose is a projection (INV-003).
- **Session-scoped agent state.** Rejected: breaks resumability and multi-agent handoff (B4).
- **Token-denominated enforcement.** Rejected: token counts are model-relative; bytes are objective.

Open questions

- Expansion-relation completeness for proof workers (with RFC 0035).
- Capability delegation depth for sub-agent spawning (with RFC 0026).

Acceptance

Registry/authority table enforced in tests for every operation; prompt-injection corpus cannot trigger privileged operations (docs/52 G2); ablation benchmark report attached to the freeze decision; two subagents sharing snapshot/intent but isolated debugger/proof handles complete PR 27's scenario without session coupling.

Document: `rfcs/0028-context-pack-format.md`

RFC 0028: Context Pack Format and Compiler

Status

Draft for implementation.

Target gate: G2 (agent-computer interface); replay-preservation obligation feeds G6 **Owners:** context/explanation leads **Normative language:** MUST/SHOULD/MAY per RFC 2119. **Normative schema:** `../schemas/context-pack.schema.json`.

Summary

A Context Pack is a bounded, typed, property-directed compilation of the evidence graph for one question (plan §6). It is the default unit agents and humans receive; expansion, not dumping, reaches everything else (INV-007).

Artifact

Per the schema: pack identity (ctx_*, the plan §4.4 handle prefix), schema_version, target question, snapshot + intent identities, pinned semantic epoch, typed verdict, assurance envelope (every B11 dimension present or typed Unsupported), selected items (events, state deltas, obligation/resource flow, order constraints, source/model/proof references, assumptions, counterfactuals, heuristic repair surfaces), per-pack guarantee set, omission manifest, expansion queries, evidence references, replay handle (crash_*) and optional debugger handle (dbg_*), canonical content hash (ADR-0013), and content budget. The semantic-epoch field is what ReplayPreserving is pinned to; a pack without it cannot claim that guarantee. Packs are immutable; context.expand creates a child pack referencing its parent.

Guarantee classes

guarantees is a closed enum; multiple MAY apply, and every claimed guarantee MUST be checkable:

Guarantee	Claim	Checked by
ReplayPreserving	the selected core replays to the same verdict under the pinned epoch	replay execution (INV-006)
PropertyPreserving	the slice preserves the property automaton's verdict-relevant structure	independent monitor
ProofRelevant	items lie on the proof dependency slice of the named obligations	proof-slice check
HeuristicRelevant	ranked-relevant only; no semantic claim	none — MUST be structurally distinct, never mixed into a preserved core
OneMinimal / CardinalityMinimal / CausallyMinimal / ValueMinimal / OwnerMinimal / FaultMinimal / ExplanationMinimal	which minimality was actually achieved (docs/38)	minimizer transcript
CausallyClosed	selection is downward-closed under the causal order	closure check
CounterfactualUnderNamedModel	counterfactual items are valid under a named causality model	model named in the item; "cause" language is prohibited otherwise (plan §12.2)

The pack MUST record which minimality class was achieved rather than implying the strongest; "minimal" without a class is prohibited output.

Compiler pipeline

Ten stages (plan §6.3 plus root selection), each producing an auditable intermediate:

1. root selection from property/evidence handles;
2. backward causal slicing (downward closure over the CIR);
3. property-automaton relevance filtering;
4. static/dynamic dependence join for source spans;
5. proof-dependency slicing for obligations;
6. observer projection;
7. abstraction/refinement correspondence mapping (selected concrete items link to their abstract counterparts);
8. minimal unsatisfied core / correction-set analysis where a solver artifact exists;
9. heuristic ranking of optional context (information-gain or configured ranker) — outputs are HeuristicRelevant only;
10. budget packing.

Stages 1–8 produce guarantee-bearing content; stage 9 never upgrades an item's guarantee. Redaction policy (plan §18.4) is applied **before** slicing; redactions appear in the omission manifest, and a redacted pack cannot support claims requiring hidden data.

Budget packing

The enforced budget is bytes (tokens are advisory, recorded with tokenizer id). Packing MUST be lexicographic: (1) every item required by a claimed guarantee (if these alone exceed budget, the compiler MUST drop the guarantee or fail — never silently truncate a guaranteed core); (2) highest-utility optional items; (3) omission-manifest completeness is non-negotiable and reserved before optional content.

Omission manifest

Counts omitted items by kind and relation, with reason (budget, redaction, unsupported, heuristic-cutoff, slice-irrelevant — the last for items provably outside the property-directed slice), expandability, and the expansion query that retrieves them. An empty manifest asserts completeness and is checkable.

Validation

- Replay-preservation check on every pack claiming it (this is plan §15.3 theorem 3’s finite statement; the Lean model covers finite causal graphs).
- Single-item removal tests on minimality claims.
- Independent property monitor on PropertyPreserving.
- Schema validation of every emitted pack in CI — spike and engine outputs included (the R2/R3 spikes’ pack dicts MUST be brought under this schema or labeled non-conformant fixtures).

Rejected alternatives

- **One “relevance score” per item.** Rejected: conflates checked guarantees with heuristics — the precise failure INV-007 exists to prevent.
- **Token-budget enforcement.** Rejected: model-relative (see RFC 0027).
- **Mutable packs updated in place.** Rejected: breaks INV-009 and caching.

Open questions

- Ranking-function choice and its evaluation (research/32; kill criterion applies).
- Cross-pack deduplication for multi-agent fan-out.

Acceptance

The synthetic 200-event case plus at least three non-synthetic failures (real exploration output, not hand-built traces) compile to packs whose guarantees all pass their checkers; guarantee-violation mutants (drop a core event, reorder, relabel heuristic as preserved) are rejected; omission manifests reconcile exactly against the unsliced graph.

Document: [rfcs/0029-causal-verification-debugger.md](#)

RFC 0029: Causal Verification Debugger

Status

Draft.

Model

Debugger handle identifies immutable execution branch and selected causally closed configuration.

API

open, state, enabled, step_event, step_abstract, reverse_causal, branch, compare, why_enabled, why_blocked, export.

Frontier

Each enabled event includes label, owner, effect, guard derivation, observer impact, conflicts, and estimated branch novelty. Selection must be replay-checked.

Reverse

Reverse removes one or a set of maximal events while preserving causal closure. Ambiguity returns choices.

DAP

Define mapping and custom namespaced requests; native API remains normative.

Acceptance

Known race branch, liveness cycle, abstract stuttering, fault insertion, and round-trip export/replay.

Document: rfcs/0030-incremental-semantic-query-engine.md**RFC 0030: Incremental Semantic Query Engine****Status**

Draft for implementation.

Target gate: G5 (incremental trust) **Owners:** incremental/database leads **Normative language:** MUST/SHOULD/MAY per RFC 2119.

Summary

All derived artifacts are memoized queries over immutable inputs (plan §9). Reuse must be fast for interactive work and auditable for trust: every reuse edge is classed, and the Incremental Parity Audit (renamed from “clean-build Tribunal”; plan §9.5) continuously compares incremental against clean recomputation (INV-010, B9).

Query key

(function_id, function_version, canonical_input_identities, semantic_epoch, proof_epoch, strategy_config).

- A source hash alone is never a sufficient key (plan §9.2).
- **Budget rule:** budget belongs to execution identity, not output identity, for any query whose result is budget-independent when it completes (parse, elaborate, property_automaton, diffs). For budget-truncated searches (explore, proof search, synthesis), the committed artifact records the budget actually consumed and its coverage frontier; two runs of the same key with different budgets produce *comparable*, *monotone* artifacts related by frontier inclusion, and a continuation — never two conflicting “results” under one identity (INV-009, B18).

Edge classes

Class	Meaning	Reuse rule
Exact	output is a pure function of named inputs	reuse by content identity
Validated	reuse carries a checker witness (e.g., translation validation)	reuse after witness check
Conservative	invalidation may over-approximate but MUST NOT miss changes under stated assumptions	reuse; audited
Experimental	speed-up only	result cannot support strong finality until clean validation

Semantic dependency reasons

Every recorded dependency carries one of the nine typed reasons (plan §9.4): reads-type, reads-value, unfolds-definition, selects-instance-or-profile, observes-event-family, relies-on-assumption-fairness-bound, uses-abstraction-component, depends-on-lemma-or-checker, consumes-encoding-epoch-or-correspondence. These reasons drive precise invalidation and power the explain API.

Independence between an edit and a query is a tri-state (DefinitelyIndependent(witness), DefinitelyDependent(reason), Unknown), and **Unknown is dependent** (RFC 0004’s rule generalized). Heuristic independence is permitted only on Experimental edges.

Persistence and crash safety

CAS stores outputs; the query index maps keys to content. Transactions MUST commit output before index; a crash leaves unreachable content eligible for GC, never a stale index entry (this is the operational content of INV-017; see plan §4.5). The index format is versioned; an index verifier (fsck) ships with the daemon and runs on recovery.

Incremental Parity Audit

- **Sampling policy:** a configurable fraction of interactive queries (default 1 in 64, uniformly by key hash) plus every promotion-relevant query at promotion time is recomputed clean; CI additionally runs deterministic full-clean sweeps nightly.
- **Compared artifacts:** verdict, canonical state-graph digest, counterexample class, certificate result, context-slice soundness, semantic diff, proof axiom manifest (plan §9.5).
- **On mismatch:** publish a mismatch evidence node; quarantine the implicated edge class + query implementation version (its reuse drops to `Experimental` until cleared); run the invalidation minimizer to produce the smallest input delta reproducing the divergence; surface per the release-blocker doctrine — a stale green result is a blocker, not a bug ticket (docs/42).
- The audit's own overhead is measured; the Phase C exit budget for it is part of the docs/34 latency accounting.

Explain API

`query.explain_reuse(key)` — why this result was reused (edges, classes, witnesses).
`query.explain_invalidation(key, edit)` — why this re-ran (the dependency reasons hit).
`query.clean_compare(key)` — run and diff a clean recomputation now. All three are ordinary read operations (plan §10.2).

Formal model

A simplified Lean model (per ADR-0044) MUST state and check: exact-reuse soundness (equal keys $\Rightarrow$ equal results) and conservative-closure soundness (the computed invalidation cone contains the true dependency cone) for the finite model of the query graph. This is plan §15.3 theorem 5's home; the existing seed (lean/Continuum/Interaction/Incremental.lean) states the shape but not the computed-cone side and MUST be extended.

Rejected alternatives

- **File-granularity-only invalidation.** Retained as the *fallback*, not the design: research/27's kill criterion drops sub-file granularity if capture proves untrustworthy, and the fallback MUST remain a supported configuration.
- **Trusting engine-reported dependencies without audit.** Rejected: B9 — incremental reuse is dangerous exactly when invalidation is silently unsound.

Open questions

- Cache eviction/retention interaction with the GC policy of plan §4.5.
- Stable reuse identities for nondeterministic engines (research/27 kill criterion) — candidate answer: canonicalize by committed frontier, not by execution.

Acceptance

Random edit traces across model/property/correspondence/proof/context queries with clean parity; intentional dependency bugs (broken edges) are caught and quarantined by the audit (PR 24's exit); property-only edits do not rebuild extraction; model-action edits invalidate reachable graphs and dependent packs (PR 23's exit); crash-mid-publication leaves no stale index entries under fault injection.

Document: rfcs/0031-semantic-and-intent-diff.md

RFC 0031: Semantic and Intent Diff

Status

Draft for implementation.

Target gate: G3 (intent integrity) **Owners:** intent/diff leads **Normative language:** MUST/SHOULD/MAY per RFC 2119. **Normative schema:** ../schemas/semantic-diff.schema.json.

Summary

The diff engine is the anti-reward-hacking core (plan B1, INV-001, INV-011, G0-DX-02). Given two snapshots and two intent versions, it classifies every change, computes the evidence impact set, and produces the policy verdict that gates ordinary versus privileged promotion. The diff artifact this RFC defines carries the `diff_*` handle prefix (plan §4.4 registers it); the normative schema pattern-enforces `^diff_` identities.

Inputs

Two workspace snapshots, before/after Intent Contract identities, the declared semantic fragments from the intent's scope, and the requested assurance for the classification itself.

Classification lattice

Every intent field change is classified with one relation from the closed set:

unchanged	strengthened	weakened
expanded	contracted	
refined	coarsened	(observers)
merged	split	(abstraction maps)
upgraded	downgraded	(assurance / proof policy)
added	removed	(assumptions, faults, fairness, non-vacuity)
incomparable	unsupported	unknown

Directional relations are defined per field by these orders:

- **Properties (per fragment):** in Finite, exact language inclusion over the bounded universe; $P \text{ strengthened } P'$ iff $\text{behaviors}(P') \subseteq \text{behaviors}(P)$ with a checked witness. In Symbolic, an implication obligation is discharged by SMT with a checked certificate or by Lean; otherwise unknown.
- **Bounds:** componentwise partial order on (values, nodes, faults, depth); a decrease in any component with no increase elsewhere is contracted; mixed changes are incomparable. no-decrease blocks contracted and blocks incomparable pending review.
- **Assurance:** total order observed < sampled < bounded < validated < proved; movement down is downgraded and is what no-downgrade blocks. Checker requirements (independent_checker, clean_recompute) turning off is also downgraded.
- **Observers:** refined iff the new observer distinguishes at least the old projections/events; dropping an event family or coarsening a projection is coarsened.
- **Abstraction maps:** the intent's abstraction_maps group binds content identities of the §16 correspondence/abstraction maps the claims are stated against; mapping distinct concrete values to one abstract value is merged, the reverse is split, and both directions are privileged.
- **Scope:** no directional order is defined; any scope change (components, abstraction level, declared fragments) on a protected contract is a semantic change and blocks ordinary promotion pending review.
- **Assumptions/faults/fairness/non-vacuity:** set membership per classified item (added/removed), with strengthened/weakened for edits to an item's expression evaluated in its fragment. Adding an assumption or fairness constraint is environment-strengthening and therefore protected; removing a fault or a non-vacuity behavior is protected.
- **Trust boundaries:** growth of the opaque set is expanded and is what no-expansion blocks.
- **Security policy:** weakening a data classification, removing a redaction class, or relaxing a capability requirement is weakened and protected; the reverse direction is strengthened.
- **Completion policy / nondeterminism classes:** any change is a semantic change; cross-policy relations are incomparable (there is no soundness order among completion policies).

Naming note: plan §5.3's prose speaks of "bound increase/decrease" and "fault envelope expansion/contraction"; this RFC's relation assignments are normative — bounds classify as expanded/contracted, faults as added/removed per classified item. The prose phrases are informal aliases for these relations.

Completeness guarantee

This RFC is the normative home of the plan §5.3 diff guarantee. Within the intent's declared supported fragments, every property weakening, assumption strengthening, bound decrease, observer coarsening, fault removal, fairness addition or removal, and assurance downgrade MUST be classified as a privileged intent change — the seven G3 dimensions. Outside declared fragments the classification is unsupported, never a guessed direction.

Fail-closed rule

- Where implication is decidable or solver-checkable in the declared fragments, Continuum MUST prove the direction and attach the witness/obligation to the change record.
- Otherwise the relation is unknown (undecidable/unattempted) or unsupported (outside declared fragments) — these are distinct (INV-008).

- unknown, unsupported, and incomparable on a protected field **MUST** be treated exactly as a confirmed protected change: ordinary promotion is blocked pending review (plan §5.3) — the incomparable rule stated for bounds above holds for every protected field. The diff never guesses an ordering.

Program semantic diff

Program-side changes are classified along these axes: effects (new/removed/moved effect sites), atomicity (split/merged critical regions), task ownership, cancellation structure (checkpoints, obligations), durability ordering, observer publication points, and correspondence (which links of §16's graph a change touches). Each axis yields typed change records with source locations; these feed the impact set and the repair gates, not the intent policy.

Impact set

The diff **MUST** emit invalidated / reused / unknown evidence sets computed against the incremental database's dependency classes (RFC 0030). Evidence whose dependency on a changed field is unknown goes to unknown, never to reused.

Policy verdict

The verdict (allow | review | block | unknown) is computed from the per-field relations and the intent's policy verbs. It **MUST** name the reasons per field. The promotion endpoint re-computes the verdict at promotion time; cached client verdicts are never trusted (RFC 0032).

Rejected alternatives

- **Token/set comparison of property strings.** Rejected as the production mechanism: defeated by renaming and rewriting (it survives only as the R3 spike baseline).
- **Best-effort ordering for undecidable cases.** Rejected: a guessed “weakened/strengthened” is exactly the false confidence the system exists to prevent.
- **One global strength order across fields.** Rejected: bounds are partial, assurance is total, completion policies are unordered — collapsing them loses the distinctions the locks need.

Open questions

- Property normalization/equivalence beyond Finite (shared with RFC 0037).
- Which SMT fragments earn strengthened/weakened with CHECKED_CERTIFICATE versus obligation-only.

Acceptance

- Mutation corpus with known strengthened/weakened/incomparable/unknown relations per field, including renames and formula rewrites that **MUST NOT** classify unchanged.
- All G0-DX-02 gaming patches classify as protected; a source-only guard repair classifies as program change only.
- Fail-closed tests: protected-field unknown/unsupported/incomparable blocks ordinary promotion.
- Impact-set soundness: no invalidated-in-truth evidence lands in reused on the acceptance corpus.

Document: rfc0032-repair-transaction-protocol.md

RFC 0032: Repair Transaction Protocol

Status

Draft for implementation.

Target gate: G4 (causal debugging and real repair) **Owners:** repair/workbench leads **Normative language:** MUST/SHOULD/MAY per RFC 2119. **Normative schema:** ../schemas/repair-transaction.schema.json.

Summary

A repair is a transaction: an immutable proposal plus accumulating evidence, promoted only when the policy gates close (plan §8, B8, INV-011).

Operations

repair.begin / apply / attach / evaluate / resume / review / promote / reject (plan §10.2).

- begin(failure: crash_*, base: ws_*, intent: in_*) → rt_* v1. The base triple is fixed for the transaction's lifetime.

Continuum: Master Implementation Plan & Comprehensive Dossier

- `apply(patch, hypothesis)` → new version with sealed candidate workspace. Patches apply to the declared base snapshot only; hidden edits (candidate workspace differing from `base+patch`) fail gate `patch_application`.
- `attach(evidence)` — agents and services append typed evidence nodes; nothing is edited in place (docs/41: swarm workers append, they do not edit one mutable document).
- `evaluate` — runs the gate campaign for the transaction's `gate_profile`; returns a new version with gate statuses.
- `resume(cont_*)` — continues a budget-suspended evaluation monotonically.
- `review` — produces the reviewer projection (plan §8.5) and records the review decision as evidence.
- `promote / reject` — terminal. `promote` re-computes the policy verdict server-side at promotion time from current evidence; it MUST NOT trust any cached client verdict.

Every operation returns a new immutable transaction version; prior versions remain addressable (plan §8.1).

Gates

The twelve gates (plan §8.2), by schema id:

#	Gate id	Claim
1	<code>base_replay</code>	the original failure replays on the base snapshot
2	<code>patch_application</code>	patch applies cleanly; no hidden edits
3	<code>intent_integrity</code>	RFC 0031 classifies no protected change (Unknown blocks)
4	<code>exact_regression</code>	the exact failure no longer occurs
5	<code>neighborhood</code>	neighboring schedules/faults/values explored
6	<code>property_mutation</code>	property mutations still fail where expected
7	<code>defect_mutants</code>	known defect mutants remain detected
8	<code>refinement_coverage</code>	refinement coverage not reduced unexpectedly
9	<code>certificate_rebuild</code>	invalidated certificates/proofs rebuilt
10	<code>incremental_parity</code>	clean and incremental results agree
11	<code>code_and_security</code>	tests, static verification, security gates pass
12	<code>receipt_generation</code>	promotion receipt composed and verified

Gate profiles are phase-staged (plan §21): phase-b enforces 1–8, 11–12; phase-c adds 10; phase-d/default enforce all twelve. Gates outside the active profile MUST appear with status `not_yet_enforced` — present and visibly unenforced, never omitted or passed. A promoted transaction MUST list all twelve gates with no pending/fail/inconclusive (schema-enforced).

Neighborhood construction

Property-directed and budgeted, drawn from the eight strategies of plan §8.3: alternate enabled events at causal decisions; fault insertion/removal around the repaired window; cancellation at adjacent checkpoints; value/name permutations; message duplication/loss/delay changes; schedule perturbations within the trace-class boundary; abstraction-map generated variants; hidden corpus-style mutations. The receipt records which strategies ran and their coverage counts — silent caps are prohibited.

Reclassification as intent revision

If gate 3 detects a protected (or Unknown) intent change, the transaction blocks. It MAY be explicitly reclassified as an intent revision: this routes the intent delta through RFC 0037's revision procedure (review path, new `in_*`, evidence

invalidation), records the reclassification decision as evidence, and re-bases the transaction on the accepted revision. Reclassification is never implicit (INV-011).

Concurrency

Multiple transactions MAY share a base. Promotion is serialized per base lineage: the first promotion advances the lineage; a second promotion against the stale base fails `StaleSnapshot` and MUST be re-based and re-evaluated (no auto-merge of semantic evidence).

Receipt

Composed at `receipt_generation`: intent identity; before/after snapshots; semantic and intent diffs (the intent diff travels as the `intent_changes` set inside the semantic-diff artifact, RFC 0031 — one artifact, referenced as `semantic_diff` in the schema); replay, neighborhood, mutation, refinement, certificate, and parity results with coverage; cumulative cost ledger (plan §8.6); unknowns; policy decision; gate profile. Receipts are canonically encoded and checkable by reference (`evidence.verify` re-fetches and verifies every referenced artifact and accepts no client-declared status).

Signing: the daemon's receipt service holds the signing keys; agents never do (INV-015). Signatures bind identity and authorship; they never substitute for proof checking (ADR-0035).

Rejected alternatives

- **Mutable transaction document.** Rejected: destroys auditability and multi-agent append semantics.
- **Client-computed promotion verdicts.** Rejected: promotion is the single most attackable decision; it is recomputed server-side.
- **Omitting unenforced gates.** Rejected: a Phase B receipt must be structurally distinguishable from a Phase D receipt, or phase staging becomes silent scope-hiding.

Open questions

- Policy DSL surface for per-property/branch gate requirements (declarative TOML per plan §5.4 vs. embedded policy engine).
- Merge assistance for re-basing a blocked transaction across an accepted intent revision.

Acceptance

The seven failure modes (docs/41) each map to a failing gate on the mutation corpus: exact-overfit → neighborhood; intent gaming → `intent_integrity`; verifier/instrumentation gaming → `property_mutation/defect_mutants`; stale proof → `certificate_rebuild`; availability collapse → non-vacuity within `property_mutation`; opaque escape → `intent_integrity` (trust-boundary expansion); abstraction gaming (concrete bad states merged) → `intent_integrity` (RFC 0031 classifies the abstraction map merged). A hard-coded exact-trace repair fails; the semantic guard repair promotes (PR 21's exit); forged or agent-signed receipts are rejected by `evidence.verify` (PR 22's exit).

Document: `rfcs/0033-continuum-forge.md`

RFC 0033: Continuum Forge

Status

Research implementation RFC.

Problem

Intent + typed sketch/holes + constraints + positive scenarios + objectives + diversity descriptors + assurance + budget.

Candidate

Algorithm, invariant, abstraction, ranking/fairness, implementation correspondence, cost vector, evidence.

Loop

Generate, type/fragment check, non-vacuity screen, verify, generalize counterexample, refine component, archive.

Engines

Enumeration, SMT/SyGuS/CHC, IC3/PDR, games, evolutionary/QD, LLM proposals, retrieval. Each adapter emits candidates only.

Archive

Behavior descriptor cell plus objective dominance and evidence status. Semantic equivalence suppresses duplicates when established.

Materialization

Creates a design transaction with model, asupersync skeleton, correspondence, scenarios, and proof obligations.

Acceptance

Known rediscovery tasks, hidden variants, finite unrealizability, and one behaviorally diverse optimization task.

Document: rfcs/0034-continuum-bench.md

RFC 0034: ContinuumBench Task and Grader Contract

Status

Draft.

Bundle

Public snapshot/intent/task/capabilities/budget; hidden variants/mutations/grader; required evidence class.

Submission

Final snapshot/artifacts, evidence graph root, operation trace, resource report, and optional explanation. Natural-language-only submissions are invalid for artifact tasks.

Grader order

Intent integrity, security, semantic correctness, evidence validity, hidden generalization, cost, explanation.

Isolation

Hidden grader is inaccessible to agents. Source hashes and semantic clones are screened across splits.

Reporting

Vector metrics and Pareto fronts; no single leaderboard hides cost/assurance.

Acceptance

Reproducible grader, mutation sensitivity, leakage audit, baseline agents/humans.

Document: rfcs/0035-isolated-lean-proof-service.md

RFC 0035: Isolated Lean Proof Service

Status

Draft.

Worker key

Lean version, library closure, source snapshot, options, resource policy.

Operations

Strict check, goal extraction, proof-state step, metadata/axioms, lemma extraction, deterministic rewrite/repair, receipt.

Isolation

No ambient network/credentials; bounded CPU/memory/output; per-request workspace; cancel/hard-kill; canonical diagnostics.

Context Pack

The service accepts exact goal plus curated declarations/proof slice and can return missing-dependency requests.

Receipt

Statement/environment/proof hashes, theorem, imports, axiom output, kernel/tool version, worker artifact.

Acceptance

Multi-version concurrency, malicious source, cancellation, deterministic replay, and zero unapproved axioms for seed theorems.

Document: rfcs/0036-proof-oriented-correspondence.md

RFC 0036: Proof-Oriented Model/Program Correspondence

Status

Draft.

Structure

Forward projection, optional reverse proposal, consistency relation, complement/provenance, ambiguity predicate, effects, obligations, evidence status.

Reverse update

Returns zero/one/many candidate changes. Zero gives conflict; many gives alternatives and distinguishing obligations. It never chooses by hidden heuristic.

Drift

Extraction changes trigger impacted correspondence edges and refinement obligations.

Proof

Lean seed laws for simplified lenses; production correspondence uses translation validation and bounded refinement evidence.

Acceptance

Unambiguous round-trip, ambiguous field aggregation, action split/merge, observer leakage, and stale map cases.

Document: rfcs/0037-intent-contract.md

RFC 0037: Intent Contract Schema and Policy

Status

Draft for implementation.

Target gate: G3 (intent integrity) **Owners:** intent/workbench leads **Normative language:** MUST/SHOULD/MAY per RFC 2119. **Normative schema:** ../schemas/intent-contract.schema.json. Where this document and the JSON schema disagree, the schema is corrected or this RFC is amended by an explicit revision; neither drifts silently.

Summary

The Intent Contract is the protected statement of what a system must mean. It is the root object of plan.md B1/INV-001: ordinary tasks cannot mutate it, repairs are evaluated against it, and every change to it is classified by RFC 0031 before any evidence survives the revision.

Storage and custody

- Intent Contracts MUST be stored and versioned only in the daemon's intent registry, outside every writable or forkable workspace snapshot (plan §4.2).
- A workspace snapshot carries the intent's content identity (in_*) as a reference. workspace.fork MUST preserve the binding by identity.
- Rebinding a snapshot lineage to a different intent is a privileged operation that MUST produce an intent diff (RFC 0031) and MUST invalidate dependent evidence.

Fields

The contract consists of these field groups (types in the schema):

Continuum: Master Implementation Plan & Comprehensive Dossier

Group	Content	Notes
claims	property AST + stable semantic id + kind (safety, liveness, refinement, hyperproperty, security, performance) + observer	at least one claim required
assumptions	classified expressions (environment, scheduler, timing, storage, network, trust), each MAY declare a domain-pack fidelity_profile (ideal, contractual, platform-qualified, adversarial-envelope; RFC 0002 – profiles are NOT automatically ordered)	timing assumptions are declarative until ADR-0016 lanes ship
observers	event families + state/knowledge/security projections (the four projection kinds of plan §5.2)	the unit reductions are justified against (INV-013)
abstraction_maps	content identities of the §16 correspondence/abstraction maps the claims are stated against	merged/split changes are privileged (RFC 0031)
scope	components, abstraction level, declared semantic fragments (Finite, Symbolic, Temporal, Probabilistic, Theorem, Runtime; ADR-0025)	fragments bound what the diff can classify (see RFC 0031)
trust_boundaries	trusted and opaque components/effects	expansion of the opaque set is a protected change (“opaque escape”, docs/41)
bounds	values, nodes, faults, depth	componentwise partial order; see RFC 0031
fault_model	enabled fault classes + pack profiles	
fairness	weak/strong constraints per action	protected: fairness strengthening is the canonical gaming vector (docs/50)
completion_policy	stutter-forever, deadlock-violation, finite-trace-only, closed (RFC 0015)	no engine may silently change it
nondeterminism	typed choice classes per site (demonic, angelic, scheduler, probabilistic, timed, epistemic)	probabilistic/timed/epistemic declarative until ADR-0016
assurance	minimum evidence class + checker requirements	total order: observed < sampled < bounded < validated < proved
optimization	hard constraints, soft objectives, non-vacuity behaviors	INV-012
security_policy	data classification, redaction classes, capability requirements	first-class per plan §0.1/§5.2; weakening is a protected change
policy	per-field change verb	see below

Canonical identity

- The `in_*` identity MUST derive from a canonical encoding of the semantic content: sorted keys, normalized property ASTs (alpha-renamed binders, normalized associativity/commutativity where the fragment defines it), and no comment/whitespace/label content.
- `name` and human labels are metadata and MUST NOT affect identity.
- Encoding MUST be byte-deterministic across platforms and releases within a schema version; golden identity vectors are part of the acceptance suite.
- In certified lanes, identity collisions MUST be resolved by canonical comparison (ADR-0013).

Change policy

Each protected field carries one verb:

Verb	Meaning
unlocked	ordinary edits allowed
proposal-only	agents may propose; humans accept
review	any change requires named-reviewer approval
locked	no change without policy amendment
no-decrease	changes classified as decrease (bounds) are blocked
no-removal	removals (faults, non-vacuity behaviors) are blocked
no-downgrade	assurance downgrades are blocked
no-expansion	growth of the set (opaque boundaries) is blocked

The verb set is closed. A proof-gated acceptance verb (`proof-required`) is a candidate future addition and requires an explicit revision of this RFC (see Open questions); plan §5.4’s policy examples use only closed-set verbs.

- The protected set is: properties, assumptions, observers, bounds, faults, **fairness**, **trust boundaries**, **non-vacuity**, **completion policy**, assurance, **security policy**, **optimization**, **scope**, **nondeterminism**, and **abstraction maps** (ADR-0039; all fifteen MUST be lockable — plan §5.4).
- Directional verbs (`no-decrease`, `no-downgrade`, `no-expansion`) are only enforceable where RFC 0031 defines the order for that field. Where the order is undefined or the change is outside declared fragments, the change classifies Unknown and MUST be treated as blocked pending review (fail closed, plan §5.3).
- The default agent repair capability profile MUST deny every protected change; a change slipped into a repair reclassifies the transaction as an intent revision (RFC 0032, INV-011).

Drafting and acceptance

- `cargo continuum init` MAY propose draft contracts from templates, property libraries, and observed effect footprints. Drafts enter the registry at evidence status Proposed and gain INV-001 protection only on explicit human acceptance (plan §3.1).
- The Proposed → protected transition is performed only by the `intent.accept` operation (RFC 0027): it requires the `revise-intent` capability, produces an audit record, and is never performed implicitly by `init`, `check`, or any repair operation (plan §5.4). `intent.lock` edits the §5.4 policy table under the same capability.
- Continuum MUST NOT silently promote an inferred intent to protected status, and MUST NOT claim a generated model is the intended abstraction (docs/37).

Revision procedure

1. A revision is submitted against a named base contract version.
2. RFC 0031 classifies every field change; undecidable cases yield Unknown.
3. Policy verbs are evaluated against the classification; any blocked or Unknown protected change requires the review path.
4. On acceptance, dependent evidence is invalidated per the impact set, a new `in_*` identity is issued, and the supersession edge is recorded in the evidence graph.

Rejected alternatives

- **Intent embedded in the workspace snapshot.** Rejected: forkable containers cannot protect their contents (research/33 principle 1; the fork-with-modified-intent gaming path).
- **Free-form policy strings.** Rejected: directional enforcement requires a closed verb set with defined orders.
- **Identity from source text.** Rejected: comment/formatting changes must not invalidate evidence; renames must not evade the diff.

Open questions

- Property-AST normalization rules per fragment beyond Finite (owned jointly with RFC 0031).
- Whether a proof-gated acceptance verb (proof-required) joins the closed verb set; adding it requires a revision of this RFC, and plan §5.4's examples use only closed-set verbs until then.

(Resolved: security_policy is a first-class field group, carried by the schema, lockable per plan §5.4, and diffed per RFC 0031's security-policy order.)

Acceptance

- Round-trip schema validation; deterministic identity across platforms (golden vectors).
- All G0-DX-02 gaming mutations classify as protected changes; a source-only guard repair does not.
- Policy-verb enforcement tests for all fifteen protected fields, including Unknown-fails-closed.
- Adversarial suite: renaming, formatting, comment, and label changes MUST NOT change identity; semantic changes MUST.

Document: rfcs/0038-multi-agent-evidence-graph.md

RFC 0038: Multi-Agent Evidence Graph

Status

Draft for implementation.

Target gate: G2 (agent-computer interface); promotion authority feeds every gate's evidence discipline **Owners:** evidence/workbench leads **Normative language:** MUST/SHOULD/MAY per RFC 2119. **Normative schemas:** ../schemas/evidence-graph-node.schema.json and ../schemas/evidence-graph-edge.schema.json carry the normative plan §11 content — node kinds, edge types, the §11.4 status lattice, typed Inconclusive reasons (INV-008), validation_basis (checked-certificate vs trusted-solver), and parameterization (checked / cutoff_checked / proved_universal). This RFC summarizes; the schemas decide.

Nodes/edges

Typed immutable candidates, claims, failures, proofs, patches, runs, receipts, conflicts, and decisions, connected by the 13 plan §11.3 edge types: SUPPORTS, REFUTES, DEPENDS_ON, REFINES, EXPLAINS, REPAIRS, INVALIDATES, GENERALIZES, COUNTEREXAMPLE_T0, CHECKED_BY, DERIVED_FROM, CONFLICTS_WITH, SUPERSEDES. Edges name checker/evidence when applicable; the machine encoding is schemas/evidence-graph-edge.schema.json (nodes: schemas/evidence-graph-node.schema.json).

Authority

Actor capabilities control node creation; status promotion is service-restricted. Confidence is metadata. The status lattice is exactly plan §11.4 (Proposed, Observed, Sampled, Bounded, Validated, Proved, Refuted, Inconclusive, Superseded — there is no draft status). Only trusted services promote into Validated or Proved; Sampled and Bounded promotions name the producing engine's service identity; every Inconclusive carries a typed INV-008 reason. Agent votes or confidence never change status.

Queries

Missing obligations, conflicting candidates, proof frontier, repair frontier, semantic duplicates, provenance, and task generation.

Whiteboard compiler

Converts structured notes to proposed nodes; rejects nonexistent references and status claims without evidence.

Acceptance

Parallel swarm with conflicting abstractions and adversarial false consensus.

Document: rfcs/0039-explanation-engine.md

RFC 0039: Explanation Engine

Status

Draft.

Inputs

Evidence root, question, audience, guarantee, observer, and budget.

Pipeline

Normalize, slice, minimize, compute deltas/order, find contrastive branch, map sources/proofs, compile context.

Explanation types

Trace, causal, contrastive, logical, correction, instrumentation, and assurance.

Guarantees

Replay/property preservation and minimality class are explicit. Causal language requires declared causality semantics.

Evaluation

Independent replay/monitor plus human/agent studies.

Acceptance

Durability, deadlock, liveness/fairness, refinement, and insufficient-telemetry examples.

Document: rfcs/0040-protocol-adapters.md

RFC 0040: LSP, DAP, MCP, SARIF, and CLI Adapters

Status

Draft.

Rule

Adapters are stateless projections over explicit handles and native operations. They cannot create semantic truth or hide unsupported fields.

LSP

Overlays become workspace snapshots; diagnostics carry evidence handles.

DAP

Common debugger mapping plus namespaced partial-order requests.

MCP

Curated bounded tools/resources with explicit handles and capability checks.

SARIF

Stable rule/source/code-flow projection with evidence URIs; not proof format.

CLI

Stable exit codes, JSON, concise prose, non-TTY determinism.

Acceptance

Cross-adapter parity: same request inputs resolve to same authoritative artifacts.

Reference Documents: Research Programs & Notes

Document: research/01-true-concurrency.md

Research Note 01: True Concurrency as the Native Verification Object

Claim class: design hypothesis

Relevant sources: [S15]–[S24], [S17], [S21]

Thesis

Most model checkers linearize concurrent activity immediately and spend the rest of their lives recovering the independence they discarded. Continuum should invert that architecture: a finite partial-order execution is primary, while an interleaving is one linear extension used for execution or presentation.

This is not merely representational taste. It affects:

- reduction power;
- production-trace ingestion;
- compositionality;
- counterexample stability;
- fairness reasoning;
- certificate size;
- semantic debugging.

Mathematical candidates

Mazurkiewicz traces

Given an alphabet Σ and independence relation I , quotient words by adjacent swaps of independent letters. This is the foundation of DPOR and a practical v0 model. Its weakness is that independence can depend on state, observers, faults, and effect phases.

Prime/stable event structures

Events carry causal predecessors and conflict. Configurations are conflict-free downward-closed sets. They naturally represent branching and concurrency, but rich data may make conflict and enabling intensional.

Occurrence nets and Petri-net unfoldings

Unfoldings separate causal histories and can produce finite complete prefixes under cutoffs. They offer strong reduction for asynchronous systems and a possible proof-carrying closure artifact.

Interval pomsets

Operations in real systems extend over time. Interval pomsets can retain overlap without forcing an arbitrary linearization and align with production telemetry. Higher-dimensional automata research increasingly uses interval pomsets as accepted languages.

Higher-dimensional automata

Independent transitions span cubes: two commuting actions form a square, three form a cube, and so on. Directed paths correspond to executions; directed homotopies identify schedules that differ only by deformation through independent operations. Recent Kleene and Myhill–Nerode results suggest automata/language minimization principles exist beyond interleavings.

Proposed semantic tower

CIR event structure

- └ linear extension → executable replay
- └ configuration graph → explicit-state model checking
- └ occurrence prefix → unfolding engine
- └ interval pomset → production conformance
- └ cubical complex → experimental high-dimensional reduction

Each projection has a checkable preservation obligation.

Novel proposal: Causal Cubical Reduction

For a property observer O , define an observer-relative independence relation I_O . Build cubical cells for jointly enabled sets of pairwise independent events, but retain faces that interact with:

- property observations;
- fairness enabling;
- obligation flow;
- cancellation phases;
- time constraints;
- durability barriers.

Compute a directed quotient that preserves O -relevant reachability and selected cycle classes. The intended win is to collapse entire families of schedules with high concurrency dimension rather than discover pairwise swaps incrementally.

Why this might work

Distributed runtimes often exhibit bursts of independent activity across nodes, shards, or keys. Pairwise DPOR records many races/backtracking choices even when a larger commuting family exists. A cubical representation makes the k -way independence explicit.

Why it might fail

- Building cells may cost more than exploring schedules.
- State-dependent independence can fracture cubes.
- Directed homotopy preservation may be too weak for liveness.
- Rich data and faults can make the complex enormous.
- Certificate checking may become harder than the saved exploration.

Falsification experiment

Corpus:

- independent per-shard requests;
- actor systems with mailbox-local work;
- replicated protocols with message fan-out;
- cancellation trees;
- key-value transactions with disjoint footprints;
- adversarial cases with mostly dependent events.

Compare against source-DPOR, optimal DPOR, parsimonious ODPOR, and unfolding prefixes. Report:

- maximal executions/relevant classes explored;
- event/cell count;
- wall time;
- peak memory;
- counterexample latency;
- certificate bytes;
- false independence defects found by mutation.

Kill the lane unless it yields at least an order-of-magnitude reduction on a non-artificial subset without a serious regression on dependent workloads.

Novel proposal: Observer-Sensitive Independence

Traditional dependence asks whether transitions commute in full concrete state. Continuum properties often observe only a projection. Define independence relative to:

$$Obs_P = \text{property state} \cup \text{fairness enabling} \cup \text{assumption monitors} .$$

Events may be equivalent for one property and dependent for another. This can produce property-directed reduction while remaining sound if the observation abstraction is complete.

The hard part is generating and checking the property footprint. A certificate should include a proof that swapping the events preserves the observer projection and future enabled observer-relevant behavior.

Counterexample geometry

A causal counterexample should be a minimal bad configuration, not merely a short word. Minimization objectives can include:

- event count;
- number of owners/nodes;
- number of faults;
- causal width;
- obligation-flow complexity;
- observer-visible events;
- semantic edit distance to a passing execution.

Computing a minimal causal core can use delta debugging over downward-closed configurations plus SAT/SMT constraints. Geodesic linearization then produces a readable replay with few owner switches.

Topological coverage

Betti numbers or persistent homology over a commuting-diamond/cubical complex may indicate unexplored concurrency structure. This is a heuristic, not assurance. It may help schedule selection by prioritizing traces that add new cycles or fill unobserved cells.

The benchmark must compare it against simpler novelty metrics: event-pair coverage, happens-before edge coverage, state novelty, and random/PCT scheduling. If topology does not improve bug yield per CPU, delete it.

Deliverables

1. Formal CIR-to-event-structure semantics.
2. Reference enumerator for tiny event structures.
3. Source-DPOR over CIR.
4. Unfolding prototype.
5. Cubical prototype behind an experimental feature.
6. Cross-engine counterexample equivalence tests.
7. Certificate format for partial-order closure.

Document: research/02-symbolic-inductive-and-parameterized.md

Research Note 02: Symbolic, Inductive, and Parameterized Verification

Claim class: implementation and research program

Relevant sources: [S18]–[S20], [S25]–[S29], [S61], [S71A]

Thesis

Explicit exploration is essential but not sufficient. Continuum should expose semantic structure once and support several infinite-state and parameterized reasoning lanes:

- decision diagrams and saturation;
- SMT bounded checking;
- CHC/PDR/IC3;
- symmetry-aware quantified induction;
- cutoff discovery;
- abstract interpretation and CEGAR;
- well-structured transition systems;
- compositional recomposition.

The frontier is not one magic solver. It is a portfolio with shared obligations, common counterexamples, and certificates.

Partitioned transition interface

LTSmin's PINS architecture demonstrates the leverage of separating a language frontend from transition-group structure. Continuum should expose:

state variables / symbolic fields
 transition groups
 read dependencies
 may-write dependencies
 must-write dependencies
 guards
 action labels
 symmetry actions

CIR footprints give much of this information natively. This supports symbolic relational products, saturation, static dependency matrices, and compositional decomposition.

Decision diagrams

For finite but huge structured state spaces:

- BDDs suit Boolean structure;
- MDDs suit finite-domain variables;
- saturation exploits locality by applying transition groups near their topmost affected variable;
- parallel decision-diagram packages provide multicore execution.

The variable-order problem is central. Continuum can derive candidate orders from causal/resource graphs, view dependencies, and separator decompositions. A learned selector may choose among explicit, MDD, SAT-BMC, and PDR lanes, but learned choice only affects performance, never soundness.

SMT bounded model checking

Compile model/CIR transitions to formulas:

$$Init(s_0) \wedge \bigwedge_{i < k} T(s_i, s_{i+1}) \wedge \neg P(s_k).$$

Use incremental solving, symmetry-breaking, partial-order constraints, and unsat-core-guided bound refinement. For richer values, use arrays, algebraic datatypes, bitvectors, and finite sets conservatively.

An unsat at bound k is bounded evidence only unless transformed into induction.

PDR/IC3 and CHCs

PDR incrementally constructs inductive clauses blocking bad states. For quantified distributed protocols, symmetry-aware generalization can lift finite-instance facts into first-order invariants. Continuum should:

1. extract a relational transition system from .ctm;
2. preserve sorts and symmetry groups;
3. run finite-instance PDR;
4. generalize clauses through orbit representatives and quantified templates;
5. validate the candidate invariant on larger instances and with an independent checker;
6. emit an inductive-invariant certificate.

Spacer/GSpacer-style global guidance can help avoid locally attractive but globally useless generalizations.

Novel proposal: Orbit-Lifted Reachability Grammar

Recent cutoff work derives quantified reachability formulas from symmetry-aware finite exploration. Continuum can generalize this into a grammar:

- enumerate small instances modulo symmetry;
- synthesize a minimum description of reachable orbit patterns;
- infer quantification shapes and cardinality thresholds;
- test stabilization on the next sizes;
- use the formula as both an invariant candidate and a cutoff hypothesis;
- attempt induction over domain extension.

The output is never called a cutoff proof until induction/checking establishes it. Failed stabilization still yields useful predicates for CEGAR.

WSTS lane

Some unbounded systems are monotone under a well-quasi-order:

- lossy channels;
- coverability abstractions;
- multisets of indistinguishable processes;
- monotone resource counts.

A domain/model can declare an order $\preceq$, and Continuum checks monotonicity obligations. Upward-closed sets are represented by finite bases/ideals. This can prove coverability for unbounded populations where finite model checking cannot.

Rust implementations rarely satisfy monotonicity directly. The technique belongs primarily at an abstract view, with refinement carrying the result downward.

Abstract interpretation

Every view can define a Galois connection or sound abstraction:

$$\alpha : C \rightarrow A, \quad \gamma : A \rightarrow \mathcal{P}(C).$$

Continuum should support abstract domains for:

- intervals/congruences;
- cardinalities;
- set membership summaries;
- queue/channel shapes;
- ownership/obligation counts;
- epochs and monotone logs;
- topology/partition summaries.

The abstract interpreter can overapproximate reachability. Spurious counterexamples drive refinement. Proof obligations state sound transfer, not merely test agreement.

Recomposition

Traditional component decomposition can lose cross-component relations. Recomposition-style analysis suggests dynamically grouping variables/components around the current property or counterexample. CIR resource hypergraphs offer a natural source for candidate partitions and separators.

Novel hypothesis: choose decomposition using a weighted hypergraph whose edges combine transition footprints, obligation transfer, and observer coupling. Recompose only the causal cone needed to refute/prove a property.

Portfolio scheduling

A verification task emits static/dynamic features:

- domain sizes and symmetry;
- transition locality;
- estimated branching;
- queue bounds;
- formula theories;
- causal width;
- observed explicit-state growth;
- invariant vocabulary.

An algorithm selector allocates budget across engines. The selector's decision and fallback policy are recorded. Sound engines retain independent verdicts; a timeout is not a vote.

Required experiments

- Paxos/Raft-style parameterized safety.
- Mutual exclusion/token protocols.
- Sharded key-value protocols with symmetry.

- Lossy-channel examples for WSTS.
- Storage recovery with unbounded log abstraction.
- Benchmarks where each engine is known to fail.
- Cross-validation against Apalache, TLC, Ivy/IC3PO-related artifacts, LTSmin, and standalone SMT solvers where practical.

Kill criteria

- Quantified generalization that cannot be independently validated.
- Learned engine selection that does not beat a simple schedule on held-out tasks.
- WSTS support that requires users to encode more mathematics than a direct external tool.
- Symbolic encodings whose counterexamples cannot replay in the reference semantics.

Document: `research/03-refinement-composition-and-sheaves.md`

Research Note 03: Refinement, Composition, and Local-to-Global Reasoning

Claim class: core design plus experimental mathematics

Relevant sources: [S14], [S24], [S32]–[S37], [S53], [S54], [S71A]

Thesis

Continuum’s central product claim is not “the simulator found no bugs.” It is that guarantees established at useful abstract views apply to the implementation under explicit refinement relations.

The system needs ordinary engineering paths for stuttering/trace refinement and research lanes for strong observational refinement, multi-grain abstraction, compositional separation logic, and sheaf-style local-to-global consistency.

Refinement ladder

```
abstract service
  ↓ data/trace refinement
distributed protocol
  ↓ stuttering/linearization refinement
operational model
  ↓ event/concrete refinement
asupersync execution
  ↓ observational conformance
production evidence
```

Each arrow can fail independently. Diagnostics should locate the highest broken edge.

Multi-grain specifications

The EuroSys 2025 ZooKeeper work demonstrates the practical value of multiple specification grains for connecting implementation behavior to models. Continuum should make grain boundaries first-class:

- a coarse model for global safety;
- a protocol model for message/state logic;
- an operational model for retries, queues, timers, and storage;
- a runtime view for cancellation and task structure.

A counterexample is replayed downward. If it becomes infeasible, the system learns or requests a refinement predicate at the smallest necessary grain.

Strong observational refinement

Trace inclusion may be insufficient for randomized clients, adversarial schedulers, security properties, or hyperproperties. Strong observational refinement asks that replacement preserve all client observations under scheduler interaction. Progressive simulations provide proof principles for such properties.

Continuum should not implement this in v0, but its observer-indexed CIR and view contracts must avoid ruling it out.

History and prophecy

Forward simulation sometimes cannot predict which abstract step a concrete concurrent operation will realize. Controlled history and prophecy variables can make the relation inductive. The system should:

- make prophecy explicit in the view contract;
- constrain prophecy choices;
- include assignments in certificates;
- never execute prophecy in production code;
- minimize prophecy scope in explanations.

Separation and resources

Aneris, Grove, Perennial, and Trillium show that resource-oriented separation logics can verify realistic distributed, concurrent, and crash-safe systems. Continuum will not recreate Iris initially. It can borrow the engineering principle:

compositional proof requires ownership of semantic resources, not merely disjoint Rust memory.

CIR resources and obligations can serve as a lightweight dynamic/semantic resource algebra. Long-term adapters may discharge local obligations with Verus or separation-logic proofs and expose summarized contracts to the global model checker.

Novel proposal: Sheaf Refinement

Suppose each subsystem or observer has a local model and local witness that a trace fragment conforms. Overlaps impose compatibility constraints. Construct a presheaf:

- base space: causal/resource cover of the execution;
- sections: local abstract-state/refinement witnesses;
- restrictions: projection to overlaps.

A global refinement witness is a compatible global section. Failure to glue indicates an inconsistency that no pairwise local checker may expose. Cohomological obstructions may summarize incompatible cycles of assumptions.

Concrete use cases

- sharded services with cross-shard transactions;
- distributed obligation ownership;
- independent node-local reconstructions of one protocol state;
- trace fragments from partial telemetry;
- composed domain packs whose local abstractions disagree at interfaces.

Boundary between theorem and metaphor

The sheaf lane becomes real only when:

1. the cover and restriction maps are mechanically defined;
2. global sections correspond to actual refinement witnesses;
3. nonzero obstruction has a sound interpretation;
4. the method outperforms or diagnoses better than a direct CSP/SAT formulation.

Until then, it is experimental.

Novel proposal: Semantic Blame via Minimal Ungluable Covers

When conformance fails, find a smallest subcover whose local witnesses cannot be glued. Report the corresponding components, overlap resources, and assumptions. This could yield much better diagnostics than one enormous SMT unsat core.

Benchmark against standard minimal unsat cores and causal slicing.

Assume-guarantee contracts

Components export:

assumptions over imported events/resources
guarantees over owned events/resources
safety invariants
progress guarantees

observation alphabet
fault envelope

Composition checks:

- ownership/resource compatibility;
- guarantee implies peer assumption;
- no circular ungrounded liveness assumptions;
- compatible fairness;
- refinement on shared observations.

The default must be conservative. Automated circular assume-guarantee inference is a research lane.

Proof reuse and semantic diff

Content-address each model declaration, view mapping, property, and pack profile. A change analysis tracks:

- read/type/value dependencies;
- event visibility;
- fairness dependence;
- abstraction mapping;
- resource ownership.

Only claims whose semantic cone changed are invalidated. This mirrors proof/build incrementality without assuming file-level boundaries.

Deliverables

1. Stuttering refinement checker for finite models.
2. Linearization witness support.
3. Multi-grain CEGAR prototype.
4. Component contract schema.
5. Tiny sheaf-gluing prototype on sharded examples.
6. Cross-validation with a direct SAT encoding.
7. Strong-refinement research prototype for one concurrent object.

Document: research/04-liveness-progress-and-fairness.md

Research Note 04: Liveness, Progress, and Fairness at Scale

Claim class: research and implementation program

Relevant sources: [S30], [S31], [S31A], [S55]

Problem

Safety counterexamples have finite bad prefixes. Liveness failures are infinite behaviors, typically represented by cycles, and their validity depends on fairness and environment assumptions. Distributed systems add crashes, partitions, retries, cancellation, and partial synchrony, making “eventually” dangerously ambiguous.

Continuum must make progress properties explicit enough to prove and diagnose, while avoiding an unreadable temporal-logic research language.

Semantic decomposition

A progress claim is a tuple:

trigger
goal
environment assumptions
scheduler/action fairness
fault budget or eventual-stability assumption
time model
observer

Example:

Every accepted request eventually receives a response provided:

- a majority remains alive after some finite time,
- network links among that majority eventually deliver,
- stable storage eventually completes,
- enabled request-processing tasks are weakly fair,
- cancellation is not requested.

Cancellation requires a separate goal: if requested, the request must drain to a terminal outcome under cooperative-path assumptions.

Ranking functions

Recent work demonstrates substantial automation for distributed-protocol liveness using ranking functions. Continuum can synthesize:

- natural-number rankings;
- lexicographic tuples;
- multisets;
- ordinal-shaped templates;
- phase-indexed rankings;
- transition invariants.

For asupersync, region/obligation state provides unusually strong ranking features. A drain proof might use a multiset of outstanding obligation depths and cleanup phases rather than elapsed time.

Novel proposal: Obligation-Flow Ordinals

Associate each obligation with a phase rank and region depth. Define a multiset order:

$$\mathcal{R}(C) = \{\{\text{phase } (o), \text{depth } (o), \text{budget } (o)\} \mid o \in O\}$$

Cancellation transitions must either decrease this multiset under a well-founded extension or make a fairness-eligible step that will. Spawning new cleanup obligations is allowed only if their ordinal mass is bounded by the consumed parent obligation.

This could turn structured-concurrency lifecycle rules into machine-checkable global progress arguments.

The idea is killed if ordinary lexicographic counts suffice with equal automation and clarity.

Liveness-to-safety

Several approaches reduce progress checking to repeated safety or induction queries. Continuum should support transformations whose proof artifacts are visible:

- monitor construction;
- loop/fair-cycle detection;
- k-liveness style counters;
- ranking certificates;
- recursive safety chains.

The transformed system is versioned and replayable. Users can inspect which fairness constraints eliminated a cycle.

Fair partial orders

An interleaving lasso is often a poor explanation. A fair cycle can be represented as a recurring partial-order motif plus a schedule/fairness witness. Research question: can unfoldings or directed topology identify recurrent event-structure components without enumerating all linearizations?

A sound v0 still uses finite graph/SCC algorithms. Partial-order liveness is frontier work.

Quantitative progress

Latency SLOs are not ordinary liveness. Under probabilistic/timed profiles, Continuum may establish:

- worst-case bound;
- probability of deadline violation;

- expected termination;
- almost-sure termination.

These claim types are incomparable. Statistical simulation does not prove a tail bound unless the sampling/model assumptions justify it.

Assumption mining

When a counterexample is unfair, the tool can search for a minimal set of fairness/environment assumptions that excludes it. This is diagnostic, not permission to automatically strengthen the specification. Proposed assumptions appear as a diff and require human acceptance.

Testing the liveness engine

Mutation classes:

- permanently enabled action starved;
- action enabled infinitely often but discontinuously;
- retry resets progress ranking;
- cancellation creates obligation cycles;
- crash/recovery livelock;
- timer continually postponed;
- fairness accidentally applied to a disabled action;
- hidden production effect blocks quiescence.

Differential corpus against TLC and other temporal model checkers is mandatory.

Exit criteria

Liveness is credible only when:

- assumptions are explicit in source and results;
- fair-cycle witnesses replay;
- rankings are independently checked;
- cancellation progress is covered;
- property-directed POR is proven preserving or disabled;
- time and probability claims cannot be confused with qualitative liveness.

Document: research/05-runtime-conformance-and-observability.md

Research Note 05: Runtime Conformance Under Partial Observation

Claim class: core research program

Relevant sources: [S12]–[S14], [S68]

Thesis

Production telemetry should be treated as a constraint on possible executions, not a total trace. Continuum's native partial-order semantics can unify:

- implementation trace validation;
- production runtime verification;
- failure reproduction;
- model coverage feedback;
- instrumentation synthesis.

This is one of the clearest ways Continuum can exceed TLA+ and conventional DST systems.

State of the art

MODIST showed transparent model checking of unmodified distributed systems. More recent work validates program traces against TLA+ specifications and uses multi-grained specifications to bridge implementation/model gaps. OmniLink explores trace validation for unmodified concurrent systems using semantic meanings and event intervals rather than requiring a single exact order.

These systems validate the need. Continuum’s opportunity is to make the semantics and instrumentation native rather than an after-the-fact adapter.

Core satisfiability problem

Let O be observed records and M the model. Build variables for:

- observation-to-model event matches;
- unobserved internal events;
- event ordering;
- abstract state;
- fault/restart epochs;
- time within uncertainty intervals.

Solve:

$$M(\tau) \wedge Match(O, \tau) \wedge Order(O, \tau) \wedge Completeness(O)$$

for a model behavior τ .

An unsatisfiable result is a violation only when the completeness/ordering assumptions support it. Otherwise the result may be inconclusive.

Impossibility boundaries

Recent theory of asynchronous fault-tolerant runtime verification shows that some real-time-order-sensitive properties cannot be decisively monitored under weak asynchronous evidence. Continuum should encode a **monitorability analysis**:

```
property
+ observation model
+ failure model
→ refutable? confirmable? only inconclusive?
```

A monitorability verdict is included before processing production data.

Instrumentation synthesis

Given a property and current telemetry schema:

1. compute the observer/event dependencies;
2. find ambiguous model behaviors indistinguishable under telemetry;
3. synthesize candidate probes/correlation IDs/fences;
4. rank them by information gain, overhead, and privacy cost;
5. verify that the new observation set distinguishes the target class.

This is an active-learning/test-generation problem over model executions.

Novel proposal: Causal Information Budget

Define a measure of how much the telemetry reduces the model belief state—not Shannon entropy by default, because no probability distribution may exist. Candidate measures:

- number of surviving symmetry orbits;
- antichain width of possible configurations;
- logical formula size/prime implicants;
- distinguishability partition refinement;
- worst-case adversarial ambiguity.

Use this to choose instrumentation and to report why a trace is inconclusive.

From production to Lab

For a violating or suspicious trace:

- retain observed order/interval constraints;
- synthesize missing choices/faults;

- minimize the causal explanation;
- generate a deterministic Lab scenario;
- replay against the exact program version;
- if reproduction fails, classify divergence (instrumentation, environment, adapter, model, nondeterminism leak).

Not every production trace is reproducible in a simplified pack profile. The tool must say so.

Runtime overhead

Semantic instrumentation should support tiers:

- always-on stable IDs and lifecycle;
- sampled payload abstraction;
- incident-mode complete event families;
- hardware/eBPF/adapter-assisted external observation where needed.

Source-generated semantic probes beat generic tracing because they know event families and effect phases. Open-Telemetry compatibility is a transport concern, not the semantic model.

Security and privacy

The conformance engine handles sensitive operational data. Requirements:

- local abstraction/redaction;
- typed privacy classification;
- cryptographic commitments for selected values;
- access-controlled crashpacks;
- deterministic secret scrubbing;
- no solver query exfiltration;
- eventually, privacy-preserving or zero-knowledge evidence for cross-organization protocol conformance.

Evaluation

- known production incidents encoded as traces;
- injected telemetry loss and clock uncertainty;
- unmodified versus semantically instrumented variants;
- comparison with timestamp sorting, trace-to-TLA validation, and direct replay;
- overhead and ambiguity reduction;
- quality of synthesized instrumentation;
- success rate producing faithful Lab reproductions.

Promotion and kill criteria (draft, pending ratification)

Per the research README contract, this lane declares its soundness boundary, baseline, measurable promotion criterion, and kill condition.

Soundness boundary. Verdicts are sound only relative to the declared observation and failure model; the monitorability analysis above is run first, and properties classified as only-Inconclusive are reported as such, never as passes or violations.

Baseline. The lane's own comparators from the evaluation plan: timestamp sorting, trace-to-TLA validation, and direct replay.

Promotion criterion (draft).

- faithful Lab reproduction of $\geq 70\%$ of the curated known-incident corpus under injected telemetry loss and clock uncertainty;
- always-on (tier-1) instrumentation overhead $\leq 1\%$;
- measurable ambiguity reduction per synthesized probe set: $\geq 2\times$ reduction in surviving symmetry orbits, measured with the Causal Information Budget measures above.

Kill condition. The majority of target properties on two real systems are monitorable only as Inconclusive, or the overhead budgets cannot be met.

These thresholds are drafts registered in plan §24.5 (“pending lane-owner ratification”). An unratified threshold may not survive Phase A; until ratified or revised it blocks this lane’s promotion.

Document: research/06-timed-probabilistic-and-hyperproperties.md

Research Note 06: Timed, Probabilistic, and Hyperproperty Extensions

Claim class: future extension

Relevant sources: [S50]–[S56], [S76]

Principle

Qualitative nondeterminism, real time, probability, and adversarial choice are different mathematical structures. Continuum must not combine them in an untyped choose operation.

Typed choice algebra

Proposed effect distinctions:

demonic choose	all outcomes must satisfy
angelic choose	witness exists
probabilistic sample	measure/distribution specified
scheduler choose	governed by scheduler/adversary model
timed delay	constrained by clocks/invariants
epistemic unknown	evidence incomplete, not a runtime choice

Mixed systems may form stochastic games or Markov decision processes. Result semantics identify which choices are optimized/adversarial.

Time

Timed extensions can compile to:

- timed automata/zones;
- difference-bound matrices;
- parametric timed automata;
- bounded SMT constraints;
- virtual-time concrete exploration.

Clock types need explicit semantics:

- monotonic;
- wall;
- logical/vector/hybrid;
- uncertainty interval;
- drift-bounded;
- resettable.

A lease proof must state its clock and synchrony assumptions.

Probability

Probabilistic packs can model:

- randomized algorithms;
- failure-rate models;
- network loss distributions;
- workload distributions.

Claims include:

- reachability probability;
- expected reward/cost;
- almost-sure safety/progress;
- percentile/quantile under assumptions;
- adversarial scheduler bounds.

Statistical model checking is sampled statistical evidence, not exact probabilistic model checking.

Storm/PRISM interoperability is preferable to implementing every numerical backend initially. Certificates for numerical results are a research problem; interval bounds and independently checked linear programs are possible paths.

Hyperproperties

Many security and robustness properties relate multiple executions:

- noninterference;
- observational determinism;
- opacity;
- differential privacy;
- strong linearizability/refinement;
- scheduler robustness.

Self-composition or HyperLTL-style automata can reduce some finite hyperproperties to ordinary model checking. Observer-indexed CIR and strong refinement contracts are designed to support this.

Novel proposal: Adversary-Parametric Refinement

Model scheduler, fault injector, and environment as typed adversaries. A refinement claim quantifies over an adversary class:

$$\forall A \in \mathcal{A}_C. \exists B \in \mathcal{A}_A. \text{Obs}(\text{Exec}_C, A) \preceq \text{Obs}(\text{Exec}_A, B).$$

Stronger versions require strategy-preserving mappings. This makes hidden scheduler assumptions explicit and connects strong observational refinement to probabilistic games.

Timed multiparty protocols

Timed multiparty session types can statically validate local communication against global timing protocols. Continuum could import a session/choreography contract as:

- a domain-pack interface;
- an action-enabling constraint;
- a source of monitor events;
- a refinement target.

This offers cheap local guarantees before global state-space exploration.

Boundaries

Initial Continuum MUST NOT advertise timed/probabilistic proof support. It should only preserve typed events and assumptions needed for future adapters. The G6 research gate requires comparison with UPPAAL, IMITATOR, Storm, and PRISM.

Experiments

- lease protocol under clock drift and partition;
- randomized leader election;
- retry/backoff with probabilistic loss and adversarial scheduling;
- noninterference of tenant requests;
- strong refinement of a concurrent queue;
- timed session protocol implementation.

Kill criteria

- One mixed semantics whose results users cannot interpret.
- Numerical answers without error bounds.
- Probability inferred from arbitrary DST schedule sampling.
- Hyperproperty support that destroys replay/explanation quality without compelling use cases.

Document: research/07-alien-mathematics.md

Research Note 07: Alien-Mathematics Portfolio

Status: exploratory; none of these ideas are product claims

Rule: mathematics survives only by producing a sound algorithm, smaller evidence, better diagnostics, or measurable verification power.

1. Directed topology and concurrency geometry

Idea

Executions are directed paths through a state/configuration space. Independent operations create higher-dimensional cells, and schedules related by directed homotopy represent the same causal behavior.

Potential value

- stronger partial-order quotienting;
- liveness classes insensitive to local commutations;
- geometric counterexample normalization;
- concurrency-dimension metrics.

Required theorem

A quotient/reduction must preserve the selected safety/temporal property and produce a checkable witness.

Kill condition

No consistent win over optimal DPOR/unfoldings on systems with high causal width.

2. Sheaves and cohomological obstruction

Idea

Local model/refinement witnesses form sections over components or causal regions. Compatibility on overlaps is restriction equality. A global proof is a global section; cohomology can expose obstruction.

Potential value

- modular conformance from partial telemetry;
- detecting inconsistent local assumptions;
- cross-shard protocol diagnosis;
- local-to-global proof composition.

Required theorem

For the selected class, global sections correspond to valid global refinement witnesses. Obstruction must be sound, not merely correlated.

Kill condition

A direct SAT/CSP formulation is simpler, faster, and equally diagnostic.

3. Homological schedule coverage

Idea

Construct a filtration of observed commuting cells; persistent homology tracks stable holes/components as exploration grows.

Potential value

Prioritize schedules that reveal new concurrency topology and detect missing commutation faces.

Caveat

Coverage is not correctness. Homology can be an attractive dashboard with no bug-finding value.

Kill condition

No held-out improvement over pair/event/state coverage.

4. Category-theoretic semantics

Idea

Domain packs are effectful transition systems; composition is monoidal; views are morphisms; refinement certificates compose. String diagrams may expose dataflow and independence.

Potential value

- principled pack composition;
- reusable proof combinators;
- compositional probabilistic/timed semantics;
- semantic optimizer correctness.

Discipline

The implementation API uses ordinary Rust types. Category theory belongs in the specification/proofs unless it materially simplifies code.

5. Coalgebra and coinduction

Idea

Potentially infinite behaviors are coalgebras. Bisimulation and coinduction support reactive equivalence, minimization, and liveness/stream reasoning.

Potential value

- canonical behavior quotients;
- incremental conformance;
- coinductive protocol contracts;
- infinite-state symbolic representations.

Candidate experiment

Use partition refinement/bisimulation minimization on observer-projected transition systems before liveness checking.

6. Domain theory and resumable partial computation

Idea

Executions under budgets produce increasing approximations. A verification campaign is a monotone computation in an information order, capable of checkpointing and resuming without changing meaning.

Potential value

- mathematically clean ResourceExhausted evidence;
- distributed/resumable search;
- anytime assurance;
- merging partial exploration artifacts.

Concrete proposal

Define an evidence domain where partial results join only when semantic envelopes match. A completed proof is a maximal element; sampled and exhaustive artifacts remain distinct branches rather than one scalar confidence.

7. Linear logic and obligation semantics

Idea

Runtime obligations are linear resources: they cannot be duplicated or silently discarded. Reserve/commit/abort and cancellation drain have natural session/linear interpretations.

Potential value

- static/dynamic obligation conservation;
- compositional cancellation proof;
- protocol-capability synthesis;
- failure explanations as unconsumed proof resources.

Candidate implementation

A lightweight affine/linear ghost calculus embedded in view contracts, with runtime events as evidence of resource transitions.

8. Ordinals and termination

Idea

Distributed recovery and cancellation may need lexicographic/multiset/ordinal rankings beyond a single natural number.

Potential value

- proof of nested cleanup;
- phase-changing retry protocols;
- parameterized progress.

Discipline

Use the weakest ranking domain that works. Exotic ordinals are not a badge.

9. Game semantics

Idea

The system, scheduler, network, faults, and environment are players with different powers. Verification asks for winning strategies under adversary classes.

Potential value

- fault-tolerant synthesis;
- robust refinement;
- explicit scheduler assumptions;
- controller generation.

Candidate lane

Compile a finite model to a parity/safety game and synthesize a recovery or scheduling policy, then refine it into an asupersync supervisor.

10. Information geometry / active experiment design

Idea

Select faults, schedules, or production probes that maximally distinguish competing semantic hypotheses.

Potential value

- pack qualification;
- instrumentation synthesis;
- model debugging;
- high-value test generation.

Soundness boundary

Experiment selection is heuristic. Only resulting checked evidence affects assurance.

11. Supervisory control and runtime enforcement

Idea

Ramadge–Wonham-style supervisory control synthesizes the maximally permissive controller that disables controllable events to maintain safety.

Potential value

Continuum could generate a guard/supervisor around an implementation when full correctness is not yet proven, while preserving as much behavior as possible.

Hard question

Which effects are genuinely controllable without violating liveness or changing the contract?

12. Choreographies and global types

Idea

A global protocol can project to node-local implementations. Projection correctness removes classes of communication mismatch before model checking.

Potential value

- generated typed channels;
- compatibility with zoomable views;
- reduction of exploration space;
- explicit timed/session obligations.

13. Proof repair as constrained synthesis

Idea

A failing refinement or invariant creates a synthesis problem over restricted patches: guards, retry placement, commit ordering, obligation discharge, or invariant strengthening.

Safety rule

Generated patches are candidates only. They must pass replay, neighboring exploration, regression properties, and proof/certificate gates.

Research governance

Every lane receives:

- a formal statement;
- baseline algorithms;
- public benchmark corpus;
- predeclared success metric;
- implementation budget;
- falsification/kill criterion;
- result classification (OBSERVED, HYPOTHESIS, PROVEN);
- no effect on stable semantics until an ADR promotes it.

Alien mathematics is valuable precisely when it stops being decorative.

Document: research/08-agentic-verification.md

Research Note 08: Proof-Gated Agentic Verification

Claim class: research program

Relevant sources: [S69], [S70], [S70A]

Thesis

Continuum should be the verification substrate on which coding agents can safely iterate:

```
discover counterexample
→ localize causal/refinement failure
→ propose patch or invariant
→ replay exact failure
→ explore neighboring equivalence classes
→ discharge proof obligations
→ independently check certificate
```

The agent is never trusted. It is a search heuristic whose output crosses deterministic evidence gates.

Machine-legible artifacts

Every failure package contains:

manifest.json
claim.json
causal-trace.cir
minimal-core.cir
linear-replay.cir
state-diff.json
obligation-flow.json
property.json
assumptions.json
replay.toml
explanation.md

Stable schemas let an agent reason without scraping terminal prose.

Causal diagnosis

The explanation engine computes:

- backward causal cone from the violation;
- minimal fault/event core;
- first broken view/refinement edge;
- obligation/resource imbalance;
- effect-phase mismatch;
- competing passing execution;
- source/provenance sites;
- candidate repair templates.

This produces a better repair problem than a 50,000-step interleaving.

Patch search spaces

Initially restrict synthesis to auditable transformations:

- move acknowledgement after commit/sync;
- add/strengthen guard;
- add epoch validation;
- make operation idempotent;
- add cancellation drain/finalizer;
- reorder independent-looking effects;
- bind task to a region;
- strengthen an abstraction predicate;
- add a missing fairness assumption as a specification proposal, not code.

Unrestricted code generation remains possible but receives no special trust.

Invariant synthesis

Agents can propose invariants using:

- counterexample states;
- reachable-state samples;
- templates from model types;
- symmetry;
- unsat cores;
- IC3 clauses;
- natural-language design intent.

Every invariant is checked for initiation, consecution, and property implication. “Plausible” invariants do not enter the claims matrix.

Proof search orchestration

The agent may select engines, decompose goals, request lemmas, and generate Verus/Lean/SMT annotations. Proof objects and deterministic rechecks remain the authority.

Research on AutoVerus and later agent-based Verus systems indicates meaningful automation potential, but evaluations can overfit benchmark styles. Continuum should maintain hidden mutation and semantic-shift sets.

Anti-reward-hacking controls

- hidden mutants and metamorphic transformations;
- separate model and implementation agents where possible;
- no access to expected counterexample seed in evaluation;
- certificate and replay checks isolated from the agent;
- detect assumption strengthening or property weakening;
- semantic diff of every accepted patch;
- adversarial prompt/data contamination tests;
- bounded tool permissions.

Novel proposal: Neighborhood Closure Gate

Fixing one schedule can merely move the bug. After replay passes, automatically explore a neighborhood defined by:

- causal swaps around the original core;
- nearby fault placements;
- alternative linearization witnesses;
- small data perturbations;
- one additional cancellation/crash;
- symmetry-renamed instances.

A patch is not accepted until this closure campaign and the original proof obligation pass.

Novel proposal: Proof-Carrying Repair

A repair submission contains:

```
patch
failure replay result
semantic diff
new/changed assumptions
certificate(s)
coverage delta
remaining unsupported obligations
```

Review focuses on contract changes and evidence, not agent confidence.

Evaluation

- known distributed/concurrency bugs;
- hidden mutants;
- novel model/implementation drift;
- cancellation and durability defects;
- invariant synthesis tasks;
- time-to-certificate;
- false-fix rate;
- property weakening rate;
- human review effort.

Product boundary

Agent features must remain optional. Core Continuum is deterministic and usable without an LLM. No proof claim depends on a model service being available or honest.

Document: research/09-cancellation-and-obligation-calculus.md

Research Note 09: Cancellation and Obligation Calculus

Claim class: core research program

Foundation: asupersync lifecycle semantics

Motivation

Most formal models treat cancellation as task disappearance or a Boolean flag. Real async software can be canceled between reservation and commitment, while holding capacity, locks, reply duties, durable-write intents, or child tasks. This is a major source of bugs and an under-modeled semantic dimension.

Asupersync makes cancellation explicit: request $\rightarrow$ drain $\rightarrow$ finalize, with region ownership and obligations. Continuum should turn that engineering discipline into a formal calculus.

Core objects

Let:

- R be a tree of regions;
- T tasks owned by regions;
- O linear obligations;
- E effect instances with phases;
- B cleanup budgets/capabilities.

State tracks:

owner(o) $\in$ Task $\cup$ Region $\cup$ External
 phase(o)
 discharge condition
 transfer history
 deadline/budget where applicable

Obligations cannot be duplicated. Dropping requires an explicit abort/discharge transition.

Effect protocol

A two-phase effect:

$$Idle \rightarrow Reserved(o) \rightarrow Committed$$

or

$$Reserved(o) \rightarrow Aborted.$$

Cancellation request prevents new ordinary work under policy, initiates drains, and eventually requires every owned obligation to be discharged/transferred or explicitly classified as external/non-cooperative.

Safety invariants

- no orphan tasks;
- region closure implies no live descendants;
- every obligation has exactly one owner;
- committed effects are not aborted;
- reserved effects do not vanish;
- finalization runs at most once;
- reply/ack obligations are not duplicated;
- epoch changes invalidate stale capabilities.

Progress

Under cooperative polling and finite cleanup creation:

- cancellation request eventually reaches terminal outcome;
- drain phase does not create unbounded obligation mass;
- region tree reaches quiescence.

Potential ranking:

$$(\text{region phase, } \text{multiset}(\text{obligation phase/depth}), \# \text{ live tasks, budget})$$

ordered lexicographically/multiset-wise.

Static surface

Possible annotation:

```
#[continuum::effect]
async fn send_reply(
    cx: &Cx,
    permit: ReplyPermit,
    value: Reply,
) -> Outcome<(), Error>
where
    permit: consumes ReplyObligation;
```

The initial implementation should infer events from asupersync primitives and use runtime checking. A future static checker/Verus integration can prove obligation conservation locally.

Capability security

Cx and tokens express authority. Continuum records capability creation, delegation, use, and revocation as resource events. This supports:

- least-authority audits;
- cross-task ownership;
- cancellation-safe delegation;
- security views where unauthorized effects violate refinement.

Foreign calls

An FFI/blocking operation may be non-cooperative. The model must choose one:

- bounded, cancel-aware adapter;
- isolated external obligation with watchdog/recovery;
- process-level kill;
- unsupported path.

No universal cancellation guarantee is inferred.

Novel proposal: Session-Typed Cancellation

Treat operation lifecycle as a local session:

```
request . (commit . end @ cancel . drain . finalize . end)
```

Global cancellation across task trees becomes a choreography. Projection can generate local protocol states and monitor transitions. This may prevent mismatched parent/child shutdown logic.

Mutation corpus

- cancel between reserve and commit;
- winner returns before loser drains;
- child spawned during region close;
- reply obligation lost on panic;
- finalizer performs unbounded retry;
- stable write acknowledged before sync;
- obligation transferred to crashed epoch;
- timeout implemented as silent future drop;
- external call never returns;
- cancellation reason overwritten.

Promotion and kill criteria (draft, pending ratification)

Per the research README contract, this lane declares its soundness boundary, baseline, measurable promotion criterion, and kill condition.

Soundness boundary. No universal cancellation guarantee is inferred: foreign/blocking calls follow one of the declared non-cooperative paths above, and claims hold only for code using the modeled asupersync primitives.

Baseline. Runtime checking only (the initial implementation path above: events inferred from asupersync primitives, checked at runtime, with no static obligation calculus).

Promotion criterion (draft).

- all 10 mutants of the mutation corpus above (10/10) detected, with zero false alarms on the correct implementation;
- a machine-checked drain-ranking certificate for the replicated register.

Kill condition. Invariant annotations become pervasive in real code (the static surface is too invasive to adopt), or the calculus cannot express asupersync’s actual cancellation semantics.

These thresholds are drafts registered in plan §24.5 (“pending lane-owner ratification”). An unratified threshold may not survive Phase A; until ratified or revised it blocks this lane’s promotion.

Deliverables

1. Formal small-step calculus.
2. CIR mapping.
3. executable reference model.
4. invariant and liveness properties.
5. asupersync differential corpus.
6. drain ranking certificate.
7. optional session/choreography prototype.
8. paper-quality semantics if the theory proves genuinely novel.

Document: research/10-combinatorial-geometry-of-concurrency.md

Research Note 10: Combinatorial Geometry of Concurrency

Claim class: experimental algorithm-design program

Relevant ideas: trace monoids, event-structure domains, median graphs, CAT(0) cube complexes, antimatroids/greoids, distributive lattices

Thesis

The configuration space of concurrency is often not an arbitrary graph. In important fragments it has rigid combinatorial geometry. Exploiting that geometry may yield faster canonicalization, counterexample minimization, decomposition, exact coverage measures, and incremental verification.

This track asks a narrow question:

Which real Continuum configuration spaces fall into recognizable geometric/combinatorial classes, and what algorithms become available when they do?

1. Configuration ideals and distributive lattices

For a fixed causal poset with no conflict, configurations are order ideals. They form a finite distributive lattice:

```
meet = intersection
join = union
join-irreducibles = individual causal events
```

Consequences:

- canonical coordinates by event/hyperplane;
- cheap joins/meets for merging partial executions;
- Birkhoff-style representation;
- antichain/frontier representation;
- exact causal slicing.

With conflict, the family is no longer one distributive lattice globally, but may decompose into compatible domains.

Engineering proposal

Represent a configuration by its maximal-event antichain plus a persistent ideal index when causality is stable. Measure whether this reduces storage versus full bitsets/sets on unfolding workloads.

2. Median graphs and partial cubes

Domains of event structures are closely related to median graphs/CAT(0) cube complexes in established concurrency theory. In a median graph, three configurations have a unique median lying on pairwise shortest paths. Hyperplanes/ Θ -classes provide coordinates.

Potential Continuum uses:

- median of several failing/passing configurations as a central diagnostic state;
- hyperplane cuts as semantic event dimensions;
- convex/gated subgraphs for component decomposition;
- linear-time or near-linear distance/median algorithms;
- partial-cube embeddings for compact configuration identity;
- separator-guided work partitioning.

Novel hypothesis: Hyperplane-Blame Decomposition

Map each semantic event/resource dimension to a configuration-space hyperplane. A property violation region and initial region may be separated by a small hyperplane set. Use that set as a candidate minimal semantic blame slice and as a learned abstraction vocabulary.

Compare against ordinary backward slicing, unsat cores, and feature importance from PDR clauses.

3. Trace monoids and Cartier–Foata theory

Under a fixed independence alphabet, executions form a trace monoid. Cartier–Foata normal form groups maximally parallel layers. The clique automaton and Möbius polynomial encode growth.

Potential uses:

- stable canonical traces;
- exact counting of equivalence classes by length;
- estimating explosion before exploration;
- schedule-space coverage that counts causal classes rather than interleavings;
- uniform/random trace sampling rather than biased word sampling;
- parallel-depth/width statistics.

Asupersync already uses Foata-style canonicalization concepts. Continuum can make the algebra explicit in CIR tooling.

Novel proposal: Trace-Growth Forecasting

For a locally stable independence graph, estimate the growth series of trace classes and compare it with observed branching. Use mismatch to detect state-dependent conflicts or missing semantic footprints. Feed the estimate into engine/partition selection.

This is heuristic unless the independence relation is globally fixed.

4. Antimatroids and greedoids

Feasible event prefixes in some monotone systems may form accessible set systems, antimatroids, or greedoids. Antimatroids admit greedy constructions and convex-geometry duals.

Potential uses:

- greedy shortest/most-readable legal linearization;
- incremental feasible-prefix maintenance;
- canonical extreme-event sets;
- test-generation order with guaranteed accessibility.

General event structures with conflict will not be antimatroids. The value is in recognizing restricted substructures—cleanup phases, monotone recovery, or one fixed conflict branch.

Novel proposal: Drain Antimatroids

Hypothesize that valid cancellation-drain prefixes for a well-structured region, after the cancellation choice is fixed, form an antimatroid: prefixes are accessible and unions of compatible drain prefixes remain feasible. If true, greedy algorithms could find canonical cleanup orders and reason compositionally about quiescence.

Falsify with generated cancellation protocols; do not force the property.

5. Heaps of pieces

Viennot-style heaps give a geometric representation of trace-monoid elements: dependent pieces stack, independent pieces commute. A CIR trace can be visualized as a heap whose contact graph is dependence.

Potential value:

- compact human explanation;
- canonical layering;
- incremental hash;
- schedule mutation by moving exposed pieces;
- causal minimization preserving feasibility.

This may be a better user-facing representation than raw DAGs for medium traces.

6. Möbius inversion and causal attribution

Incidence algebras over posets support Möbius inversion. Given cumulative measurements over configurations, inversion can recover marginal contributions.

Possible applications:

- attribute aggregate latency/resource deltas to causal events;
- separate repeated/overlapping observer effects;
- incremental state reconstruction;
- exact inclusion-exclusion over causal cones.

This is only valid where the measured quantity obeys the required additive relation. It should not become generic “causal inference.”

7. Non-positive curvature as a tractability signal

CAT(0)/median geometry informally means independent choices fit together without pathological positive curvature. A tractability detector might measure local cube completion:

if every observed commuting square/cube closes consistently,
the configuration region may admit median/partial-cube methods.

Failure to complete a cube is diagnostically useful: it reveals hidden conflict, state-dependent independence, observer sensitivity, or missing causality.

8. Experiments

Recognition

On generated and real CIR prefixes, test:

- distributive-lattice laws;
- median uniqueness;
- partial-cube embedding;
- antimatroid accessibility/union closure;
- fixed versus state-dependent trace independence.

Algorithms

Compare:

- bitset configuration representation versus antichain/hyperplane coordinates;
- BFS shortest counterexample versus median/convex methods;
- standard causal slicing versus hyperplane blame;

- random interleaving sampling versus trace-class sampling;
- ordinary replay presentation versus heap/Foata presentation.

Negative corpus

- disabling/conflict that breaks lattice joins;
- dynamic resource aliases;
- fault events whose commutation depends on timing;
- observer-sensitive noncommutation;
- non-monotone recovery.

9. Governance

These structures are recognized properties of a semantic fragment, never assumed globally. Recognition either has a proof/certificate or remains an optimization guarded by runtime validation and fallback.

The goal is not to rename model checking with geometric language. The goal is to discover exploitable structure that ordinary graph treatment leaves on the floor.

Document: research/11-tla-examples-corpus-gap-analysis.md

Research 11: TLA+ Examples Corpus Gap Analysis

Research question

What capabilities must Continuum possess before it can honestly replace the practical role of TLA+ across a heterogeneous example corpus?

Corpus role

The TLA+ Examples repository describes itself as a comprehensive example library, a corpus for tool development/testing, and a case-study collection. Its manifests and configs explicitly exercise:

- PlusCal and proofs;
- action composition;
- exhaustive, simulation, trace generation and symbolic modes;
- safety, liveness, deadlock and assumption failures;
- SYMMETRY, VIEW, ALIAS, CONSTRAINT, and DEADLOCK configuration features.

Source: <https://github.com/tlaplus/Examples>

Gap 1 — mathematical value semantics

Rust-oriented model DSLs often start with structs, enums and collections. TLA+ examples routinely treat sets and total functions as first-class mathematical values, quantify over them, construct powersets, and use model values independent of implementation representation.

Continuum response:

- persistent canonical value algebra;
- finite and symbolic capabilities;
- row-polymorphic records and tagged sums;
- functions represented extensionally in finite lanes;
- explicit definedness;
- typed escape for heterogeneous mathematical data where needed.

Risk: an overly strict type system can silently remove legitimate behaviors and make parity appear easier.

Gap 2 — action formulas, not commands

Primed variables, existential next-state values, UNCHANGED, enabledness, and stuttering are relational. An imperative DSL can encode them only with care.

Continuum response:

- relational core;
- procedural surface lowered with proof-producing transformation;

- frame inference checked against explicit write sets;
- first-class action labels for fairness/refinement.

Gap 3 – infinite behaviors

A DST explores finite runs. TLA+ specs denote infinite behaviors, often closed under stuttering. Fairness changes which infinite behaviors count.

Continuum response:

- behavior semantics independent of the runtime;
- explicit terminal completion policy;
- Büchi/Streett checking and ranking proofs;
- fairness assumptions in result identity;
- Lean semantics for preservation.

Gap 4 – model configuration is semantic

Constants, overrides, constraints, symmetry, views and deadlock settings materially change a model. Treating config as CLI decoration makes results unreproducible.

Continuum response:

- typed, hashed model configuration;
- proof-producing finite instantiation;
- view/alias as observation contracts;
- model constraints separated from specification assumptions;
- exact config in every evidence artifact.

Gap 5 – refinement

Several corpus cases are explicitly about refinement, auxiliary variables or multi-grain specifications. Trace equality is inadequate.

Continuum response:

- relational/stuttering refinement edges;
- history/prophecy variables;
- observer/event maps;
- composition theorem;
- strong refinement lane for hyperproperties.

Gap 6 – theorem proving

TLAPS-bearing examples contain claims beyond finite model checking. A Rust checker alone cannot replace that role.

Continuum response:

- Lean theorem parity;
- finite certificates imported by reflection;
- reusable protocol mathematics;
- no line-by-line TLAPS emulation requirement.

Gap 7 – source language metafeatures

Modules, instantiation, recursive operators, higher-order operators, level checking and nested definitions appear in corpus stress cases.

Continuum response:

- predictable typed module system;
- no unrestricted host macros in trusted elaboration;
- semantic feature waves;
- optional TLA frontend after native semantics stabilize.

Gap 8 – domain diversity

The corpus covers puzzles, shared memory, distributed protocols, cache coherence, storage, TCP, files, B-trees, probability and cyber-physical scheduling.

Continuum response:

- domain packs only for concrete/runtime semantics;
- abstract models remain domain-independent mathematics;
- specialized timed/probabilistic fragments rather than forcing everything through generic events.

Conclusion

Corpus parity changes the target from “excellent Rust DST” to “verification environment with DST as one interpretation.” The largest technical risks are temporal semantics, theorem/refinement parity, and maintaining a rich mathematical language without losing predictable execution.

Document: research/12-lean-reflection-and-proof-certificates.md

Research 12: Lean Reflection and Proof Certificates

Thesis

Continuum should separate finding evidence from trusting evidence. Modern certificate import work in Lean 4 suggests this can scale beyond toy proofs.

Relevant work

Lean4Lean

Mario Carneiro’s Lean4Lean provides an independent Lean 4 typechecker written in Lean and reports verification of mathlib with performance within tens of percent of the reference checker.

<https://arxiv.org/abs/2403.14064>

Relevance: proof-checker diversity and a path toward validating Continuum’s Lean artifacts with an independent implementation.

LRAT-Catcher

LRAT-Catcher imports SAT LRAT certificates using Lean’s formally verified checker through reflection, including cube-and-conquer composition.

<https://arxiv.org/abs/2607.00815>

Relevance: huge finite-state/CNF proofs need not become gigantic explicit proof terms.

PBLean

PBLean imports VeriPB pseudo-Boolean certificates with a fully proved Boolean checker and verified encodings.

<https://arxiv.org/abs/2602.08692>

Relevance: cardinality, quorum, counting, optimization and finite combinatorics often encode more naturally in PB than CNF.

Architectural inference

The scalable pattern is:

```
untrusted search/solver
→ compact certificate
→ proved executable checker
→ theorem
```

But there are two trust gaps:

1. certificate checker soundness;
2. encoding from Continuum model/property to solver formula.

The second is frequently omitted in verification tooling and must be first-class.

Certificate composition

Large Continuum results should be partitioned:

- state-space shards;
- per-action invariant obligations;
- per-cube SAT results;
- SCC components;
- refinement partitions;
- per-domain-pack assumptions.

A root certificate proves coverage/composition. This aligns with parallel search and incremental proof rebuilding.

Native checker versus Lean checker

Routine workflow:

Rust explorer → Rust small checker → assurance result

High-assurance/release workflow:

same certificate → Lean reflective checker → theorem manifest

The Rust checker should eventually be proven equivalent to the Lean checker or generated from a shared verified specification. Until then, cross-checking catches implementation errors.

What to formalize first

1. finite transition systems and closure;
2. canonical value/state digest correspondence;
3. bounded path witnesses;
4. stuttering simulation;
5. SCC/fair-lasso certificate;
6. CNF/PB bounded-unrolling encoding;
7. finite symmetry quotient.

DPOR and nominal symmetry come later because their preservation theorems are harder and their interfaces should be informed by working baseline engines.

Performance questions

- Can certificate parsing/checking be streamed?
- Can Merkle-partitioned closure be checked without loading all states?
- What checker-to-search time ratio is acceptable?
- How stable are theorem packages across Lean patch releases?
- Does reflection retain reproducible diagnostics on malformed evidence?

Novel proposal: theorem receipts

Every PROVEN result emits a receipt:

```
model hash
semantics epoch
property theorem name
assumption theorem names
certificate hash
checker theorem/version
encoding theorem/version
Lean environment hash
axioms report
```

This is a durable, composable proof provenance object rather than a console message.

Document: research/13-observer-indexed-independence.md**Research 13: Observer-Indexed Independence****Problem**

Classic partial-order reduction defines independence using transition enabledness and commutation on concrete states. Continuum has richer observations:

- abstract views;
- temporal properties;
- fairness eligibility;
- obligation/cancellation lifecycle;
- durability phases;
- security/hyperproperty observers;
- audit order.

Two events can be independent for one claim and dependent for another.

Prior art

Context-sensitive independence has shown that observers can yield exponentially stronger reductions than context-insensitive relations.

A practical baseline is Parsimonious Optimal DPOR, which explores one Mazurkiewicz class with polynomial worst-case memory in its supported sequentially consistent setting: <https://arxiv.org/abs/2405.11128>

Await-aware optimal DPOR avoids executions dominated by pure waiting and can diagnose livelocks: <https://arxiv.org/abs/2208.09259>

Formal proposal

Let $O : \text{Exec} \rightarrow \text{Obs}$ be an observer. Events e, f are independent at configuration C when:

1. both orders are executable under the required diamond condition;
2. resulting configurations are equivalent under O ;
3. emitted observations agree modulo the observer's equivalence;
4. continuation languages are equivalent for the supported property class.

Write $e \text{ } I_O(C) \text{ } f$.

Observer order

$O_{\text{fine}} \preceq O_{\text{coarse}}$ when equality under O_{fine} implies equality under O_{coarse} .

Then the desired monotonicity is:

$$e \text{ } I_{\text{fine}} \text{ } f \Rightarrow e \text{ } I_{\text{coarse}} \text{ } f$$

This creates a Galois-flavored relationship:

more abstraction $\Leftrightarrow$ more possible independence

The exact adjunction should be investigated, not asserted prematurely.

Observer components

An observer is a product of selected components. Product refinement makes the lattice explicit:

$\text{StateView} \times \text{VisibleEvents} \times \text{Fairness} \times \text{Obligations} \times \text{Time} \times \text{Security}$

Adding a component refines the observer and can only remove independence.

Certificates

A local commutation witness can be checked by replaying both orders under reference semantics. Global DPOR coverage still needs source/backtracking evidence.

Possible certificate decomposition:

- local diamond witnesses;

- dependence graph;
- source-set coverage per prefix;
- observer lattice hashes;
- reduction theorem ID.

Risks

- continuation equivalence is as hard as verification;
- local commutation may be insufficient for liveness/fairness;
- dynamic witness cost may exceed savings;
- property changes invalidate caches;
- hyperproperties can observe correlations invisible to single-run state.

Product strategy

Start conservative:

1. static resource/effect conflicts;
2. view-equality local diamonds for finite safety;
3. observer monotonicity theorem;
4. only then temporal/fairness-aware independence;
5. hyperproperty reductions remain separate until proved.

Kill criteria

- local commutation (view-equality diamonds) proves insufficient to certify reduction for the observer-sensitive class on real protocols, leaving only continuation-equivalence checks that are as hard as verification itself;
- certificate/checker overhead makes observer-indexed reduction slower than conservative unreduced exploration on the target corpus (the docs/31 bar: median $\geq 5\times$ on the observer-sensitive class, checker overhead $< 20\%$, zero mutation loss);
- observer or property changes invalidate cached independence so frequently that reuse never pays for its witness cost.

Fairness/liveness-aware independence is explicitly out of scope for this lane: it is deferred to the liveness-preserving reduction lane (plan §24.5; research/04), and negative liveness results do not kill this lane.

Executable spike

spikes/observer_independence.py demonstrates the core phenomenon: independent final-state writes become dependent under an audit-order observer.

Document: research/14-nominal-sets-and-orbit-finite-verification.md

Research 14: Nominal Sets and Orbit-Finite Verification

Why revisit nominal mathematics

Modern systems continuously create names: request IDs, sessions, object keys, trace IDs, transaction IDs, leases, generations and nonces. Model checkers usually bound them to a tiny finite set. This creates duplicate states distinguished only by renaming and makes freshness awkward.

Nominal sets treat names as atoms acted on by permutations. Finiteness is replaced by **orbit-finiteness**: finitely many equivalence classes under renaming, each element having finite support.

Relevant theory

- Nominal automata and coalgebraic semantics: <https://arxiv.org/abs/2202.06546>
- Nominal Büchi automata with name allocation and decidable language inclusion: <https://doi.org/10.4230/LIPIcs.CONCUR.2021.4>
- Alternating nominal automata with name allocation, LICS 2025: <https://arxiv.org/abs/2408.03658>
- Orbit-finite-dimensional vector spaces and weighted register automata: <https://doi.org/10.46298/theoretcs.24.13>

Continuum fragment

```
type RequestId : atom[equality]
type Node : atom[equality]
fresh rid : RequestId
```

A state such as:

$\text{pending} = \{r1 \mapsto \text{Waiting}, r2 \mapsto \text{Done}\}$

is represented by support and equality pattern, not concrete integers. Renaming $r1, r2$ yields the same orbit.

Equivariance obligation

A transition/property is eligible only if it commutes with atom permutations:

$\text{step}(\pi \cdot s, \pi \cdot s') \Leftrightarrow \text{step}(s, s')$

$\text{property}(\pi \cdot s) \Leftrightarrow \text{property}(s)$

Operations such as hashing raw IDs, comparing allocation order, leaking textual IDs, or using numeric arithmetic violate equality symmetry. Continuum must reject or refine the symmetry.

Ordered atoms

Epochs and sequence numbers need order, not just equality. Data symmetries can model ordered atoms, but products may have more orbits and complexity rises. The lane should distinguish:

- equality atoms;
- total-order atoms;
- tree/path structured names;
- opaque externally observed names.

Interaction with runtime

Concrete UUIDs map to abstract atoms through an allocation table in the refinement view. Production traces need only preserve equality/freshness relationships unless the property observes representation.

This could dramatically reduce traces containing thousands of unique request IDs.

Interaction with parameterized verification

Orbit-finite representation and symmetry-to-quantification are complementary:

- nominal methods quotient names within a behavior;
- quantified invariants prove properties across unbounded populations.

A successful finite orbit model can supply invariant candidates and support patterns to IC3/PDR.

First experiment

Model an idempotent request service with:

- unbounded fresh request IDs;
- retries and duplicate delivery;
- cancellation;
- deduplication table garbage collection;
- property: one committed result per request atom.

Compare:

1. concrete bounded IDs;
2. finite permutation symmetry;
3. nominal orbit exploration.

Promote only if nominal exploration changes the practical scaling frontier.

Document: research/15-games-and-assumption-synthesis.md

Research 15: Games and Assumption Synthesis

Problem

A liveness property often fails because the environment can drop every message, crash every leader, never schedule a task, or cancel at pathological points. Engineers add fairness assumptions manually until the property passes. This hides the actual contract.

Foundational direction

Environment-assumption synthesis formulates the problem as a game and seeks safety/liveness restrictions making a specification realizable. Minimal fairness assumption selection is generally hard, so “weakest” must be scoped to a grammar/order.

Foundational source: <https://arxiv.org/abs/0805.4167>

Recent protocol synthesis work shows counterexample-guided sketching can synthesize substantial TLA+ distributed protocols and exploit semantic equivalence reduction: <https://arxiv.org/abs/2405.07807> <https://arxiv.org/abs/2501.14585>

Continuum opportunity

Continuum already classifies nondeterminism and owns domain-pack fault semantics. This enables a practical game boundary:

System player: protocol actions/repair choices

Environment player: clients, network, crashes, cancellation timing

Scheduler player: task/action scheduling under declared control

Chance: probabilistic delays/faults where explicitly modeled

Outputs

Counterstrategy

A finite-state strategy explaining how the environment defeats the property. This is far more actionable than one lasso because it describes a family of failures.

Assumption candidate

Examples:

- eventually deliver control messages between live nodes;
- do not crash more than f nodes per epoch;
- weakly fairly poll a task while it owns an obligation;
- clock uncertainty remains below lease margin;
- cancellation checkpoints are reached within a responsiveness rank.

Protocol repair

For sketched guards/updates, synthesize an implementation strategy or prove unrealizability under current architecture.

Novel proposal: operational assumption type

Every assumption has four interpretations:

logical formula

simulator adversary restriction

production monitor/SLO

deployment mechanism or human procedure

A proof that relies on an assumption with no operational interpretation is flagged as non-deployable.

Assumption strength lattice

Within a grammar, implication creates a partial order. Continuum reports Pareto-minimal candidates by:

- logical strength;
- implementation cost;
- observability;
- availability impact;
- security implications.

Cancellation game

Cancellation is naturally adversarial: it may arrive at any checkpoint. The system must drain and finalize within budgets. A ranking/parity game can synthesize where checkpoints, reservation boundaries or cleanup obligations must exist.

Risks

- state explosion in parity/Streett games;
- grammars encode the answer;
- synthesized assumptions are too strong or unmonitable;
- multiple environment players create imperfect-information games;
- chance and adversarial choices must not be conflated.

The first product output should be counterstrategies; assumption synthesis follows once users trust the game semantics.

Document: research/16-causal-abstract-interpretation.md

Research 16: Causal Abstract Interpretation

Motivation

Classical abstract interpretation approximates sets of program states using lattices and sound transfer functions. Distributed executions also have causal structure. Collapsing them immediately to global states can lose independence and create enormous cross-products.

Proposal

Define abstract domains over **configurations**—causally closed sets of events—rather than only total states.

Let:

- C be concrete event configurations;
- A be an abstract domain;
- $\alpha : P(C) \rightarrow A, \gamma : A \rightarrow P(C)$ form a Galois connection or sound approximation;
- transfer adds enabled events modulo conflict/causality.

Abstract elements can summarize:

- which obligations may exist;
- quorum knowledge/support sets;
- durable/volatile phase frontiers;
- causal reachability of acknowledgements;
- message provenance;
- cancellation region topology;
- partial-order width/depth.

Why causal domains may help

Two systems with the same global variable valuation can differ in causal history relevant to:

- which messages can still arrive;
- whether an acknowledgement depends on sync;
- which obligation owner must finalize;
- whether events are concurrent or ordered;
- production trace conformance.

Causal abstraction can retain exactly those facts without retaining every schedule.

Product of domains

A Continuum analysis could use a reduced product:

```
state interval/domain
× happens-before summary
× obligation linearity domain
× durability phase domain
× quorum knowledge domain
```

Reduction operators exchange facts between components.

Widening and acceleration

Loops/retries generate unbounded event histories. Candidate widenings:

- collapse repeated independent event layers via Foata normal forms;
- summarize monotone message/knowledge accumulation;
- accelerate idempotent retry cycles;
- use well-quasi-order ideals for obligation/message multisets;
- widen causal intervals while preserving forbidden order patterns.

Relation to partial-order reduction

POR chooses representative concrete executions. Causal abstract interpretation overapproximates families. They can compose:

1. POR reduces redundant orderings;
2. abstraction merges semantically similar configurations;
3. CEGAR refines when a counterexample is spurious.

The preservation story is harder than state-only CEGAR and must be formalized per observer/property.

Falsifiable experiment

Use a retrying replicated write protocol where message multiplicity and causal provenance dominate. Compare:

- explicit global states;
- DPOR only;
- state abstract interpretation;
- causal abstract interpretation.

Success requires fewer states/configurations and fewer spurious counterexamples, not merely a novel formalism.

Document: research/17-proof-producing-reductions-and-translation-validation.md

Research 17: Proof-Producing Reductions and Translation Validation

Threat model

An unsound optimizer in a verifier is worse than a crash: it manufactures confidence. High-risk passes include:

- property slicing;
- symmetry quotienting;
- partial-order reduction;
- abstraction;
- module composition;
- finite-domain instantiation;
- bit-blasting and solver encoding.

Two proof strategies

Verified transformation

Prove once in Lean that implementation function `transform` preserves a relation.

Advantages: compact per-instance evidence.

Cost: implementation verification and maintenance.

Translation validation

For each instance, transformation emits a witness; a small checker verifies preservation.

Advantages: optimized implementation may change.

Cost: per-instance certificate and checker design.

Continuum should prefer translation validation early and prove stable checkers/semantics.

Property slicing witness

Given property-variable roots, emit a dependency graph showing every omitted variable/action cannot affect:

- property value;
- enabledness of retained actions;

- fairness monitor;
- observer output;
- hidden action refinement.

A simple syntactic cone is safe but conservative. Semantic slicing needs proof obligations.

Symmetry witness

For each canonical state:

- representative;
- permutation mapping original to representative;
- proof action relation equivariant;
- property invariant under action.

Group generators can compress evidence.

DPOR witness

A difficult open engineering problem: produce compact global evidence that source/backtracking choices cover every relevant trace class. Candidate structure:

- exploration prefix tree;
- per-prefix source set;
- dependence relation digest;
- race/backtracking witnesses;
- local commutation diamonds;
- sleep/source-set closure proof.

The certificate checker should not re-explore all schedules, or the reduction buys nothing.

Solver encoding witness

Each lowering step uses a verified normal form and emits mapping tables. SAT/PB proof import then proves the final formula, while Lean theorems connect formula validity to model safety.

Incremental certificates

Content-addressed subproofs allow unchanged partitions/actions to be reused after edits. This is essential for agent loops.

Research metric

Measure:

- certificate/search size ratio;
- check/search time ratio;
- incremental reuse rate;
- malformed-certificate detection;
- semantic mutation detection;
- proof maintenance across optimizer changes.

Document: [research/18-coalgebra-modal-logic-and-behavioral-interfaces.md](#)

Research 18: Coalgebra, Modal Logic, and Behavioral Interfaces

Why coalgebra

Transition systems, automata, probabilistic systems and labeled processes can often be viewed as coalgebras: state is characterized by observations and possible next behavior. This perspective is useful for Continuum because refinement, bisimulation, minimization and modal properties are central across several backend types.

Practical leverage

Generic behavioral equivalence

Rather than hard-code one bisimulation algorithm, a coalgebraic interface can identify:

- observation functor;
- branching type (nondeterministic, probabilistic, weighted);
- transition labels;
- relation lifting.

This may unify finite parity checks, nominal automata and domain-pack component summaries.

Modal property extraction

Hennessy–Milner-style logics characterize behavioral equivalence under conditions. Counterexample explanations can be generated as distinguishing formulas:

```
state A and B differ because
  after visible Commit
  A may reach Durable while B cannot
```

This is potentially better than raw state diffs for refinement failures.

Compositional interfaces

Components expose behavior through ports/actions. Interface automata, modal transition systems and assume/guarantee contracts can fit a coalgebraic layer, while CIR supplies concrete causal events.

Interaction with nominal methods

Coalgebraic semantics for nominal automata already exists: <https://arxiv.org/abs/2202.06546>

This makes coalgebra a plausible organizing theory for the orbit-finite lane rather than an abstract decoration.

Interaction with CML

CML modules can elaborate to behavioral interfaces:

```
required actions/assumptions
provided actions/guarantees
visible observations
hidden internal transitions
refinement relation
```

Composition checks compatibility and produces proof obligations.

Novel proposal: distinguishing-property counterexamples

When refinement/bisimulation fails, compute a small modal formula accepted by one side and rejected by the other. Attach it to the causal witness and source map.

This could make semantic discrepancies substantially more legible to humans and agents.

Risks

- generic abstraction harms performance;
- coalgebraic generality may not cover fairness/hyperproperties cleanly;
- users do not care about the theory unless explanations improve;
- category-heavy APIs can become inaccessible.

Keep coalgebra internal unless it produces reusable algorithms or superior diagnostics.

Document: research/19-semiring-provenance-and-weighted-exploration.md

Research 19: Semiring Provenance and Weighted Exploration

Observation

Many graph/model analyses differ only in how alternative paths and sequential steps combine.

Analysis	Alternative combine	Sequential combine
reachability	OR	AND
path count	+	×
shortest cost	min	+
fault provenance	+	× symbolic monomials
reliability	probability sum*	multiplication*

The starred probabilistic case requires care around dependence and adversarial nondeterminism.

Proposal

Use typed weighted-transition APIs where algebraic laws are explicit and checked. A transition carries labels/weights; analysis evaluates path/configuration expressions in a selected algebra.

Provenance polynomial

Assign variables to semantic causes:

```
c_cancel_17
f_disk_loss
m_deliver_42
```

A property violation obtains a polynomial/expression describing combinations sufficient for reachability. Simplification can reveal:

- minimal fault sets;
- common causal factors;
- alternative independent witnesses;
- sensitivity to assumptions.

This is richer than one shortest counterexample.

Causal-class counting

Trace monoids and Foata normal forms suggest counting equivalence classes rather than linear schedules. Weighted automata over orbit-finite sets suggest a route for names and counts together.

Quantitative optimization

Tropical or lexicographic weights can minimize:

- semantic events;
- context switches;
- crashed nodes;
- distinct faults;
- elapsed virtual time;
- abstraction-visible steps.

A product algebra yields a canonical “smallest useful counterexample” objective.

Probability warning

Probabilistic choice, scheduler nondeterminism and adversarial faults require MDP/stochastic-game semantics. A naive probability semiring can double-count dependent paths or choose an unjustified scheduler. Continuum only uses weighted algebra where the semantic conditions are explicit.

Experiment

Implement one generic acyclic dynamic-programming kernel for:

1. reachability;
2. shortest causal witness;
3. number of equivalence-class representatives;
4. minimal fault-set provenance.

Compare code complexity and performance with specialized implementations. Promote only if the abstraction pays rent.

Document: research/20-rust-concurrency-verification-frontier-2026.md

Research 20: Rust Concurrency Verification Frontier, 2026

Layering principle

No current tool spans abstract distributed protocol semantics, concrete async runtime schedules, unsafe Rust, memory models, liveness and production conformance. Continuum should integrate by proof boundaries, not pretend to dominate every layer.

Concrete schedule/runtime tools

- Loom: controlled atomics/synchronization and schedule exploration.
- Shuttle: scalable randomized/deterministic schedule testing.
- asupersync Lab: deterministic runtime, cancellation/obligation semantics, virtual time, traces, DPOR-oriented infra-structure.
- Stateright: executable Rust state/actor models and consistency checks.

Continuum's distinctive role is abstract/refinement/proof integration above asupersync.

Rust program verification

Verus

Rust-like verification with SMT, linear ghost state and transition-system patterns. Useful for local functional/concurrency contracts and selected implementation components.

<https://verus-lang.github.io/verus/>

Kani

Bit-precise bounded model checking for Rust code. Useful for finite unsafe/data-structure obligations and serialization/checker code.

<https://model-checking.github.io/kani/>

Thrust

Prophecy-based refinement types for Rust, aimed at modular verification of ownership and mutation patterns.

<https://doi.org/10.1145/3729333>

RustMC

Research on model checking compiled Rust concurrency and FFI behavior broadens coverage below source-level abstractions.

<https://arxiv.org/abs/2502.06293>

Agentic proof work

Benchmarks such as VeruSAGE show growing interest in agents producing verified Rust proofs. Continuum can provide higher-level protocol goals and concrete counterexamples, while code verifiers discharge local obligations.

Boundary contract

A Continuum result identifies layers:

Protocol model: proved/checked

Runtime refinement: checked under controlled effects

Local component contract: imported theorem or assumption

Unsafe/FFI/memory model: verified by X or unverified

Production observation: valid/invalid/inconclusive

No green badge flattens these distinctions.

Proposed interoperability

- generate Kani harnesses from finite action/domain constraints;

- generate Verus-style transition invariants for local state machines;
- consume code-verifier theorem receipts as domain-pack assumptions;
- use Loom for primitive implementations not fully controlled by asupersync;
- import Miri/sanitizer/fuzzer evidence as testing evidence, not proof.

Challenge to the vision

“Same code in model and production” is only true inside controlled boundaries. Dependencies, FFI, allocator behavior, OS semantics, and hardware memory models remain contracts or separate verification tasks. Continuum succeeds by making these boundaries explicit and composable.

Document: [research/21-choreographies-and-session-types.md](#)

Choreographies, Multiparty Session Types, and Verified Projection

Question

Can Continuum derive useful Rust interfaces and runtime monitors from a global protocol model without pretending generated code proves the implementation correct?

Relevant ideas

A choreography describes communication globally. Projection derives one local behavior per role. Multiparty session types and communicating automata study when these projections are compatible, deadlock-free, and faithful to the global description.

This is directly relevant to Continuum because CML already names roles, messages, actions, causality, and fairness. The same declarations can generate:

- Rust message enums and codecs;
- typed role/endpoint APIs;
- asupersync task skeletons;
- instrumentation event IDs;
- local runtime monitors;
- model-based scenario generators;
- refinement obligations connecting generated interfaces to the global model.

Proposed architecture

```
CML global model
  ↓ checked projection
Role automata / local contracts
  ├── Rust types and endpoint traits
  ├── asupersync skeletons
  ├── production monitors
  └── refinement obligations
```

The generated artifacts constrain the implementation surface but do not dictate internal architecture. A role implementation may batch, pipeline, retry, persist, or use auxiliary tasks provided its visible behavior refines the role contract.

Projection obligations

A successful projection receipt should establish:

1. every global communication has compatible send/receive projections;
2. local choices are known to the role that must make them, or are communicated before divergence;
3. channel assumptions match the selected domain pack;
4. projection does not introduce deadlock absent from the global model;
5. local traces compose into global traces modulo hidden/stuttering actions;
6. cancellation and crash branches are represented explicitly;
7. monitor verdicts are sound for the observation contract.

Rust API sketch

```
#[continuum::role("Leader")]
trait LeaderEndpoint {
```

```

async fn send_append(
    &mut self,
    cx: &Cx,
    follower: NodeId,
    msg: AppendEntries,
) -> Outcome<AppendReply, RpcError>;
}

```

The generated trait identifies semantic effects and event labels. The implementation remains ordinary Rust.

Why this is not code generation theater

The value is not boilerplate reduction alone. Generated interfaces create a stable seam where:

- the model and implementation share names and types;
- instrumentation is complete by construction for covered operations;
- domain-pack assumptions are visible;
- agents receive precise missing-case diagnostics;
- refinement can reason over an explicit local contract rather than arbitrary code.

Research experiments

1. Project a two-phase commit model to coordinator/participant role automata.
2. Generate asupersync endpoint traits and a monitor.
3. Implement a correct and mutant coordinator.
4. Check local monitor behavior and global refinement.
5. Compare developer friction with handwritten message plumbing.

Promotion criterion

Promote when projection catches real interface drift, generated APIs remain idiomatic, and a proof or translation validator establishes trace preservation. Kill or narrow the feature if projection forces unnatural code structure or if generated monitors cannot handle operational refinements.

Primary references

- sound/complete projection and projection failure diagnostics: [S109];
- generalized asynchronous projection: [S110];
- mechanized choreographic programming and Pirouette: [S111]–[S112];
- multiparty asynchronous session types and event-structure semantics: [S113]–[S114].

Document: research/22-weak-memory-execution-graphs.md

Weak Memory as Execution Graphs

Problem

Distributed protocol verification and local lock-free verification operate at different scales. Treating every local atomic operation as sequentially consistent can miss real Rust executions; exploring the full memory model inside every distributed schedule is intractable.

Proposed separation

Continuum uses hierarchical verification:

```

local concurrent component
  weak-memory execution graph
    ↓ refines
atomic component contract
  ↓ used by
protocol/distributed model

```

The local lane establishes that a queue, channel, log buffer, or synchronization primitive refines an atomic contract under a named Rust/compiler/hardware memory model. The distributed lane consumes the contract.

Execution graph

A candidate execution contains:

- events and thread/task identity;
- program order (po);
- reads-from (rf);
- modification/coherence order (mo);
- from-read (fr);
- synchronizes-with (sw);
- happens-before (hb);
- dependency edges and fences;
- atomic ordering annotations.

A memory model is a set of acyclicity/irreflexivity and consistency constraints over these relations.

Engine strategy

1. Instrument or lower a restricted Rust concurrency IR.
2. Generate candidate event graphs symbolically.
3. Use SAT/SMT to solve reads-from and coherence choices.
4. Import proof-producing SAT/PB evidence where possible.
5. Map externally visible operations to the atomic component model.
6. Check linearization/refinement.

Relationship to Loom

Loom remains valuable for controlled schedule exploration and for testing Continuum internals. Continuum's weak-memory lane should not claim Loom's approximation is a complete C11/Rust semantics. The lane may consume Loom traces as bug seeds while independently checking the selected axiomatic model.

State-space control

- verify local components separately;
- collapse verified operations to atomic summaries;
- use observer-indexed event slicing;
- exploit coherence-class symmetry;
- bound data values through abstraction;
- use CEGAR when a weak-memory witness is spurious at the protocol level.

Corpus pressure

The Disruptor, lock-free, and shared-memory examples can initially be ported under SC semantics. Runtime refinement exemplars must state whether SC, Rust atomics, compiler lowering, or hardware behavior has been justified.

Promotion criterion

Promote beyond SC when the lane reproduces known litmus tests, finds or excludes mutants in real Rust components, and emits independently checkable solver evidence. Never merge weak-memory and distributed scheduling into one uncontrolled Cartesian product.

Primary references

- RustMC as a current compiled-Rust/GenMC direction: [S81];
- weak-memory formalism taxonomy and execution-graph vocabulary: [S115];
- Loom remains an engineering baseline, not the normative weak-memory semantics: [S12].

Document: research/23-compositional-proof-algebra.md

A Compositional Algebra of Models, Refinements, and Certificates

Motivation

Large systems cannot be verified as one flat transition graph. Continuum needs a principled way to compose components, assumptions, observers, refinement edges, and evidence without turning the architecture into ad hoc glue.

Candidate mathematical structure

Treat:

- models/components as objects;
- refinements as vertical morphisms;
- interface compositions or wiring as horizontal morphisms;
- commuting preservation arguments as 2-cells.

This suggests a double-category or bicategorical organization. The user need never see the terminology; the implementation benefits from explicit laws:

- identity refinement;
- refinement composition;
- monotonicity under compatible composition;
- assumption discharge;
- observer projection compatibility;
- certificate composition.

Practical artifact

A refinement graph edge contains:

```
source model
target model
state/event relation
assumptions guaranteed/required
hidden actions
observer mapping
certificate type
semantic epoch
```

A composition checker verifies that adjacent edges agree on interfaces and that assumptions are discharged.

Assume-guarantee rule

For components A and B:

```
Environment(B) ⊨ Assume(A)
Environment(A) ⊨ Assume(B)
A ⊨ Guarantee(A)
B ⊨ Guarantee(B)
```

A || B ⊨ GlobalProperty

The exact rule depends on trace, event-structure, or game semantics. Continuum should encode the chosen rule explicitly rather than use “compositional” as an adjective.

Certificate algebra

Certificates should compose without rebuilding a monolithic proof:

- invariant certificates combine through shared interface invariants;
- refinement receipts compose transitively;
- assumption-game strategies compose with environment monitors;
- solver theorems become lemmas in Lean;
- local production conformance can glue into a global witness when overlaps agree.

Sheaf connection

The local-to-global gluing problem resembles sheaf consistency: local sections over overlapping subsystems must agree on intersections. A failed gluing attempt may yield a minimal incompatible cover, useful for diagnosis.

Promotion criterion

Adopt the algebra only where it produces executable validation rules, smaller proofs, or better failure localization. Avoid exposing category-theoretic vocabulary in ordinary UX unless it directly clarifies a problem.

Primary references

- three-dimensional refinement algebra and proof/compilation composition: [S85];
- local-to-global/sheaf consistency research: [S37]–[S39];
- modular decidable verification and compositional protocol reasoning: [S27], [S31].

Document: research/24-automatic-abstraction-and-property-directed-modeling.md

Automatic Abstraction and Property-Directed Modeling

Goal

Reduce the cost of creating and maintaining abstraction maps without allowing an agent or heuristic to invent a model that merely agrees with observed tests.

Inputs

- concrete CIR executions;
- Rust types and effect signatures;
- candidate abstract state fields;
- active properties/observers;
- counterexamples and proof failures;
- existing model/refinement graph;
- corpus-derived modeling patterns.

Candidate techniques

Predicate abstraction

Infer predicates from guards, assertions, property atoms, and counterexample interpolants. Use CEGAR to refine when an abstract counterexample is spurious.

Abstract interpretation

Define a Galois connection between concrete state/configuration domains and abstract domains. Derive sound abstract transformers rather than fitting traces.

Interpretation reduction

Search for a small interpretation from a concrete transition system into a known abstract protocol template. Validate each candidate by SMT or exhaustive checking.

Invariant inference

Use PDR/IC3, CHCs, symmetry-to-quantification, and agent-proposed lemmas. Every accepted invariant is checked independently.

Causal feature selection

Use observer/property dependency and event footprints to identify which concrete facts can affect the property. This yields a conservative initial abstraction.

Protocol-template mining

Recognize reusable patterns: quorum certificates, epochs, leases, two-phase commit, monotonic logs, ownership tokens, and retry loops. Templates propose—not assert—abstraction fields and invariants.

Anti-overfitting rules

A candidate abstraction must survive:

- held-out schedules and faults;
- semantic mutants;
- neighboring causal classes;
- stronger bounds;
- differential reference execution;
- proof of simulation or a clearly bounded evidence class.

Agreement on a finite trace set is not refinement.

Agent workflow

1. propose abstraction variables and mapping;
2. generate proof obligations;
3. classify failed obligations;
4. request missing ghost/history/prophecy state;
5. verify candidate under bounded and symbolic lanes;
6. emit semantic diff for human review;
7. promote only with receipt.

Product outcome

The ideal UX is not “AI generated your formal model.” It is:

Continuum found that the property depends only on epoch, durable index, and acknowledgement set; here is the proposed abstraction, the executions it collapses, the proof obligations, and the one obligation still open.

Primary references

- invariant inference and proof slicing: [S116], [S120];
- protocol synthesis and interpretation reduction: [S117]–[S118];
- symmetry-to-quantification and automatic cutoff discovery: [S119], [S123];
- regular abstractions and ranking-function liveness: [S122], [S124];
- 2026 neuro-symbolic IC3 synthesis, treated as an untrusted proposal engine whose outputs require proof checking: [S125].

Document: research/25-agent-computer-interfaces-for-formal-systems.md

Agent–Computer Interfaces for Formal Systems

Claim class: design hypothesis

Question

What interface lets a coding/proof agent use a verification environment reliably and efficiently without granting it authority over truth?

State of the art

SWE-agent demonstrates that interface design materially changes coding-agent effectiveness. Pantograph argues that human-oriented Lean LSP interaction burdens machine users with cursor/text-state mechanics and instead exposes proof-state operations. LeanDojo, AXLE, OProver, LAMP, and related systems reinforce the value of compiler/kernel feedback, retrieval, explicit proof context, and planner/worker/verifier separation.

Primary references:

- <https://arxiv.org/abs/2405.15793>
- <https://arxiv.org/abs/2410.16429>
- <https://arxiv.org/abs/2606.26442>
- <https://arxiv.org/abs/2605.17283>
- <https://arxiv.org/abs/2606.28841>

Continuum hypothesis

A formal-systems ACI should expose **semantic state**, not filesystem/editor state:

workspace snapshot
intent contract
verification task
proof/debug state
counterexample/evidence graph
repair/synthesis transaction

The agent should never need to infer these from process lifetime, cursor position, logs, or chat memory.

Interface calculus

Treat the ACI as a labeled transition system:

$\text{AgentState} \times \text{ToolOperation} \rightarrow \text{AgentState} \times \text{Observation}$

We can evaluate:

- validity: operation preconditions explicit;
- sufficiency: all benchmark tasks expressible;
- minimality: redundant operations removed;
- observability: necessary distinctions visible;
- safety: authority not encoded in prose;
- compositionality: handles can be passed among subagents;
- resumability: connection loss does not alter task state.

A possible Lean formalization defines capability-preserving traces and proves that no sequence of unprivileged operations reaches `EvidenceStatus.Proved` without a checker event.

Novel proposal: semantic action grammar

Ship a machine-readable workflow grammar describing legal operation sequences and state transitions. An agent can validate a plan before calls; orchestrators can synthesize workflows; invalid operations become low-cost local feedback.

Example:

```
FailureAvailable
├─ compile_context → ContextAvailable
├─ open_debug → DebugAvailable
└─ begin_repair → RepairDraft
```

```
RepairDraft
└─ apply_patch → RepairApplied
```

```
RepairApplied
└─ evaluate → RepairReady | RepairBlocked | RepairInconclusive
```

This is not a rigid UI wizard: advanced clients may call any operation whose preconditions hold.

Novel proposal: observation bisimulation

Two ACI designs are equivalent for a task class if they expose observations sufficient to distinguish all semantic states requiring different correct next actions. This suggests a rigorous way to detect underpowered or excessively verbose interfaces:

- underpowered: semantically distinct states are observation-equivalent but require different actions;
- verbose: observations differ without affecting any valid task policy.

Finite benchmark fragments can compute/approximate a minimal quotient of interface observations.

Experiments

1. Native handles vs shell/CLI on identical agent/model.
2. Raw trace vs Context Pack.
3. Cursor/source proof API vs proof-state handle API.
4. Implicit session vs explicit workspace/debug/proof handles in multi-agent tasks.
5. Free-form errors vs typed recovery actions.
6. Static one-shot context vs expandable graph context.

Metrics: success, tokens, calls, invalid operations, stale-state errors, time, expensive failures, intent violations.

Kill criteria

- typed API does not improve effectiveness/cost;
- task grammar constrains legitimate strategies without reducing errors;
- Context Pack expansion still requires repository-scale reconstruction;
- handle threading produces more failures than session-based state under realistic clients.

The result may still justify a thinner API, but “agent-native” cannot remain a slogan.

Document: research/26-causal-and-contrastive-counterexample-explanations.md

Causal and Contrastive Counterexample Explanations

Claim class: research hypothesis

Problem

Model checkers produce witnesses, but a witness is not automatically an explanation. The counterexample-explanation literature reports heavy use of traces, minimization, visualization, localization, and logic-specific techniques, while industrial studies show raw verification output remains difficult to interpret.

References:

- <https://arxiv.org/abs/2201.03061>
- <https://arxiv.org/abs/2304.08950>
- <https://doi.org/10.1007/s10664-023-10353-4>

Continuum setting

Continuum has richer data than traditional state traces:

- event causality/conflict;
- concrete and abstract states;
- observer/property automata;
- asupersync obligations/cancellation;
- source correspondence;
- fault/durability phases;
- alternate branches.

This permits explanations that distinguish causal mechanism from incidental schedule order.

Formal explanation object

For execution structure E, property P, and observer O, an explanation candidate X includes selected events, conditions, and interventions. Possible guarantees:

```
Witness(E, P)
Closed(X)
Replay(X, semantics) refutes P
Minimal_k(X)
Contrast(X, safe_execution)
ActualCause_M(X, P)
```

The causality model M must be named. Halpern–Pearl-style counterfactual causation, structural causal models, event-structure causality, and minimal correction sets answer different questions.

Novel proposal: layered causal certificates

A Context Pack can include a certificate chain:

1. selected events form a valid causally closed configuration;
2. replay/partial execution reaches a violating monitor state;
3. omitted events commute or are observer-irrelevant under a checked relation;
4. a counterfactual intervention leads to a safe branch;
5. claimed minimality class checked by deletion/solver proof.

This makes explanation an evidence object rather than narration.

Novel proposal: explanation lattice

Explanations form a partial order by information and abstraction:

```
raw execution
  ≥ causal slice
  ≥ abstract transition slice
```

- ≥ source-local mechanism
- ≥ one-sentence contrast

But no single chain fits all users. Define a lattice where joins combine complementary explanations and meets find common causal kernels. Agents can request the least explanation sufficient to choose among available repair actions.

Novel proposal: causal responsibility over obligations

For cancellation/durability bugs, events alone may mislead. Model linear obligations/resources and compute responsibility for an unresolved or prematurely discharged obligation. A finalizer that publishes a reply can be the responsible transition even if cancellation “triggered” the path.

Active contrast generation

Find a nearest safe execution under a declared distance:

schedule-choice edits
fault edits
causal graph edits
abstract state edits
owner switches

Then compute the minimal relevant difference. Distance sensitivity should be visible; multiple Pareto contrasts may be better than one arbitrary nearest trace.

Experiments

- durability/cancellation;
- deadlock;
- fair/unfair liveness cycle;
- refinement mismatch;
- weak-memory litmus test;
- insufficient production telemetry.

Compare raw trace, minimized trace, causal core, state delta, and contrastive explanation with humans and agents.

Success

- explanation preserves/refutes property as claimed;
- improved diagnosis and repair;
- lower false-confidence rate;
- small Context Packs without omitting mechanism;
- explanations stable under irrelevant commuting events.

Promotion threshold (draft, pending ratification)

Registered in plan §24.5: a replay-preserving causal core $\leq 10\%$ of trace length on real (non-synthetic) failures. The registered kill condition is the existing one below: minimization cost dominates verification. This is a draft pending lane-owner ratification; an unratified threshold may not survive Phase A.

The neighboring exploration-reduction threshold (research/01: $\geq 10\times$ reduction in explored classes, without regression on dependent workloads) governs a different lane — DPOR-style exploration reduction, not causal minimization — and must not be conflated with this one; plan §24.5 now carries them as separate register rows.

Kill criteria

- causal claims too sensitive to arbitrary modeling choices;
- minimization cost dominates verification;
- users/agents perform no better than with simple state deltas;
- proof certificates become larger than useful evidence without improving trust.

Document: research/27-incremental-verification-and-proof-reuse.md

Incremental Verification and Proof Reuse

Claim class: research and implementation program

Question

How can Continuum provide interactive feedback while preventing unsound cache reuse from becoming a false proof?

Relevant foundations

- self-adjusting computation and demand-driven incremental queries;
- Salsa-style red/green query evaluation;
- incremental SAT/SMT and proof certification;
- incremental model checking;
- differential dataflow;
- proof dependency graphs and proof retrieval.

Selected references:

- <https://salsa-rs.github.io/salsa/>
- <https://doi.org/10.29007/pdcc>
- <https://easychair.org/publications/paper/TbPs>
- <https://arxiv.org/abs/2401.13244>

Core difficulty

A source edit can be local while reachability impact is global. Conversely, a comment or proof presentation change may be semantically irrelevant. File timestamps and module dependency alone are too coarse.

Semantic dependency graph

Edges carry a reason:

READS_TYPE
READS_VALUE
UNFOLDS
SELECTS_PROFILE
OBSERVES_EVENT
USES_ASSUMPTION
USES_FAIRNESS
USES_BOUND
USES_ABSTRACTION
USES_LEMMA
USES_ENCODING_EPOCH

Only Complete/Conservative edge classes may drive trusted invalidation. Experimental dynamic provenance can accelerate previews but must not certify final results.

Novel proposal: proof-carrying cache entries

A cache entry includes one of:

- exact content-function identity;
- translation-validation witness;
- certificate proving result for input digest;
- conservative dependency theorem instance;
- clean-comparison receipt.

Cache lookup returns both value and reuse evidence. Assurance of the consuming result is the meet/composition of dependency assurance.

Novel proposal: invalidation counterexamples

When incremental and clean results differ, minimize the edit/query graph to produce:

smallest edit sequence
smallest missing/incorrect dependency edge
first divergent query
semantic result difference

This treats the build/verifier engine as another system Continuum can debug.

Novel proposal: change-action algebra

Model edits as typed actions with commutation and impact composition. Independent edits may reuse proofs/caches in either order. This connects incremental computation to true-concurrency semantics and may permit partial-order reduction over edit/test campaigns.

Proof program

Lean model:

- query graph with declared dependencies;
- valid closure condition;
- deterministic query functions;
- theorem: unchanged transitive inputs imply reusable output;
- theorem: conservative over-approximation may over-invalidate but not under-invalidate.

The production engine remains more complex; the theorem defines the target contract.

Experiments

- property edit;
- action guard edit;
- abstraction-map edit;
- domain-profile edit;
- proof lemma edit;
- source refactor with semantic equivalence;
- concurrent identical tasks;
- crash during publication;
- intentionally missing edge.

Metrics

latency, recomputation, cache size, clean mismatch, evidence freshness, debugging time, promotion overhead.

Kill criteria

- dependency capture cannot be made trustworthy below file/module granularity;
- promotion requires nearly full recomputation on every change;
- certificate/provenance overhead dominates useful savings;
- nondeterministic engines prevent stable reuse identities.

Document: research/28-agentic-proof-repair-and-context-compilation.md

Agentic Proof Repair and Context Compilation

Claim class: research hypothesis

State of the art

Pantograph exposes machine-oriented Lean proof states. AXLE targets scalable isolated Lean utilities. OProver and LAMP report gains from compiler feedback, retrieved verified proofs, explicit domain context, and multi-agent decomposition. APOLLO and related proof-repair systems explore iterative repair after library/source evolution.

References:

- <https://arxiv.org/abs/2410.16429>
- <https://arxiv.org/abs/2606.26442>
- <https://arxiv.org/abs/2605.17283>
- <https://arxiv.org/abs/2606.28841>

Continuum opportunity

Proof repair is coupled to model/program changes. Continuum can provide context unavailable to generic theorem provers:

- semantic diff;
- changed abstraction/refinement edge;

- finite countermodel to failed invariant;
- execution witness;
- proof dependency slice;
- corpus analogues;
- protected theorem statement/axioms.

Proof Context Pack

goal + local context
protected theorem statement
relevant definitions/theorems
proof dependency slice
changed semantic edges
failed attempts and Lean diagnostics
finite countermodels / induction failures
candidate auxiliary lemmas
allowed tactics/axioms
version and budget

Novel proposal: proof blueprint graph

Represent a proof as semantic obligations rather than one tactic script:

init establishes invariant
step action A preserves invariant
step action B preserves invariant
invariant implies property
ranking decreases on progress action
fairness discharges infinite stutter

Each node may have Lean theorem(s), solver certificate, or finite evidence. Agents repair the smallest invalid blueprint nodes. Presentation proof can be regenerated.

Novel proposal: countermodel-directed lemma synthesis

When an induction attempt fails, extract model states/transitions satisfying current hypothesis but violating preservation. Synthesize predicates that separate them from reachable states, rank them by generalization/corpus analogues, and send as lemma candidates. Finite evidence is clearly heuristic until theorem proved.

Novel proposal: version-parametric proof receipts

A proof receipt stores theorem statement/environment hashes and can be replayed across compatible toolchain epochs only after declaration-level semantic equality checks. This avoids both needless rebuild and unsafe “same source text” assumptions.

Experiments

- changed invariant definition;
- action split requiring new case lemma;
- renamed/restructured source with same theorem;
- stronger property;
- changed abstraction;
- library upgrade;
- malicious theorem weakening;
- proof agent given full repo vs Context Pack.

Metrics

kernel success, attempts, tokens, wall time, relevant lemma recall, invented theorem changes, axiom integrity, proof size, human review burden.

Kill criteria

- Context Pack loses critical lemmas too often;
- proof-state API does not outperform source/LSP loop;
- automated repair frequently proposes theorem weakening;

- proof blueprint abstraction adds maintenance without improving localization/reuse.

Document: research/29-protocol-invariant-ranking-and-abstraction-cosynthesis.md

Protocol, Invariant, Ranking, and Abstraction Co-Synthesis

Claim class: research hypothesis

Motivation

Synthesizing a protocol alone often produces candidates that are hard to prove. Synthesizing an invariant for a fixed bad protocol is futile. Liveness may require ranking functions or fairness structure; implementation refinement may require auxiliary state or a better abstraction.

Candidate tuple

$C = (\text{Protocol}, \text{Invariant}, \text{Abstraction}, \text{Ranking}, \text{Fairness}, \text{AuxiliaryState})$

Correctness constraints couple components:

$\text{Init} \Rightarrow \text{Inv}$
 $\text{Inv} \wedge \text{Step} \Rightarrow \text{Inv}'$
 $\text{Inv} \Rightarrow \text{Safety}$
 $\text{ConcreteStep} \Rightarrow \text{AbstractStep} \vee \text{Stutter}$
 Ranking decreases/progress under fairness

Related work

Protocol synthesis systems such as Scythe and interpretation reduction, invariant systems such as IC3PO/Endive/DistAI, CHC/SyGuS synthesis, and ranking-function verification provide specialized pieces.

References:

- <https://arxiv.org/abs/2501.14585>
- <https://arxiv.org/abs/2103.14831>
- <https://arxiv.org/abs/2108.08796>
- <https://sygus-org.github.io/>

Novel proposal: coupled counterexample typing

Normalize verifier feedback into:

InitFailure
 SafetyTrace
 InductionCounterexample
 RefinementMismatch
 FairCycle
 RankingViolation
 UnrealizableEnvironment
 ImplementationConstraintViolation

Each type updates only relevant candidate components while preserving learned constraints for others.

Novel proposal: proof-complexity objective

Include proof cost as an optimization objective:

- number/size of invariant clauses;
- quantifier alternation;
- auxiliary state;
- ranking dimension;
- Lean proof dependency size;
- certificate checking cost.

A slightly slower protocol with a dramatically simpler proof may be preferable.

Novel proposal: abstraction as search control

Start with a coarse abstraction. Spurious counterexamples drive refinement. But allow protocol changes that make a simpler abstraction sound. This turns CEGAR into co-design rather than one-way model refinement.

Search architecture

- typed grammar enumeration/LLM proposals;
- interpretation-class pruning;
- IC3/PDR for invariant hints;
- CHC/ranking synthesis;
- game solver for environment assumptions;
- quality-diversity archive;
- independent verifier/Lean for acceptance.

Benchmarks

- acknowledgement/durability guard;
- mutual exclusion;
- lock service with cancellation;
- two-phase commit recovery;
- reliable broadcast variants;
- quorum register;
- small leader election;
- dynamic resource allocator.

Success

- rediscover known algorithms;
- solve tasks that separated synthesis cannot;
- produce smaller proof artifacts;
- generalize to hidden parameter/value variants;
- expose unrealizability rather than time out.

Kill criteria

- joint space overwhelms gains from coupled feedback;
- proof complexity objective biases toward trivial/slow designs;
- abstractions overfit finite bounds;
- agent proposals dominate and cannot be reproduced by structured search.

Document: research/30-quality-diversity-for-verified-algorithm-discovery.md

Quality-Diversity for Verified Algorithm Discovery

Claim class: research hypothesis

Problem

Optimization usually returns one best candidate under a scalar objective. For algorithm invention, this destroys structural diversity and may hide qualitatively different solutions.

Proposal

Use quality-diversity (QD) search where archive cells are defined by semantic/architectural descriptors and candidates enter only after required verification evidence.

Possible descriptors:

- message rounds/count;
- quorum intersection geometry;
- stable-write count;
- state bytes;
- concurrency width;
- recovery/cancellation strategy;

- fairness strength;
- failure-detector reliance;
- proof size/quantifiers;
- refinement stuttering depth.

Verified archive

Each cell stores:

candidate artifact
behavior descriptor evidence
objective vector
assurance envelope
counterexample history
proof/certificate receipts
semantic equivalence links

Unverified candidates may inhabit a provisional archive but cannot dominate verified cells in reports.

Novel proposal: counterexample morphology as diversity

Characterize candidates by the shape of near-failing counterexamples or eliminated interpretation classes. Two correct protocols with similar cost but different vulnerability boundaries may be valuable alternatives.

Novel proposal: homological/causal descriptors

Experimental descriptors from execution geometry:

- trace-class width;
- cube/higher-dimensional concurrency counts;
- causal bottleneck hyperplanes;
- Betti-like features of independence complexes.

These must prove practical value versus simpler descriptors or be deleted.

Search

Combine MAP-Elites/novelty search with typed mutation/crossover and LLM transformations. Every mutation is a semantic proposal; invalid syntax/fragment candidates are filtered cheaply, then staged verification.

Evaluation

- number of behaviorally distinct verified candidates;
- rediscovery of known families;
- Pareto improvements;
- hidden variant robustness;
- human expert rating of conceptual novelty;
- archive stability under descriptor changes;
- proof cost.

Risks

- descriptor gaming;
- syntactic diversity mislabeled semantic;
- verification budget spread too thin;
- huge archives with no insight;
- novelty mistaken for usefulness.

Kill criteria

If QD does not discover useful alternatives beyond multi-objective Pareto search on known benchmarks, keep only a lightweight semantic-diversity archive.

Document: research/31-proof-oriented-bidirectional-transformations.md

Proof-Oriented Bidirectional Transformations

Claim class: research hypothesis

Question

Can Continuum reduce model/program drift without silently inventing incorrect synchronization?

Background

Bidirectional transformations and lenses maintain consistency between views. Verified systems such as KBX and formally verified lens languages provide useful foundations; effectful lenses extend the theory to non-cancellable effects. Software-model synchronization work also emphasizes provenance and conflict.

References:

- <https://arxiv.org/abs/2404.18771>
- <https://doi.org/10.1145/3747523>
- <https://doi.org/10.1145/2847538.2847544>

Continuum requirements

Model/program correspondence is not ordinary data synchronization:

- one abstract action may represent many concrete steps;
- concrete internal steps may stutter;
- abstraction may deliberately discard detail;
- source edits can add effects or observer leaks;
- reverse changes are underdetermined;
- proof obligations and intent constrain acceptable synchronization.

Correspondence object

```
get : Concrete → Abstract
put? : Concrete × AbstractEdit → Set ConcreteEdit
consistent : Concrete × Abstract → Prop
complement : provenance needed for reconstruction
obligations : generated semantic claims
ambiguity : conditions yielding multiple/no edits
effects : synchronization side effects and failures
```

Novel proposal: proof-oriented lens law

Beyond round-trip laws, require:

```
if put(c, edit) = c'
and edit preserves protected abstract intent
then generated obligations establish
  consistency(c', edited_abstract)
  and relevant refinement/property preservation.
```

The tool may return a candidate plus obligations instead of claiming the law automatically.

Novel proposal: conflict as a first-class proof goal

When synchronization is ambiguous, return a compact conflict:

```
Abstract field durable may map to:
  disk.stable
  journal.committed
  replica.quorum_acked
```

Distinguishing obligation:
which event is externally authoritative under observer 0?

Agents/humans resolve by intent or evidence, not heuristic selection.

Change provenance

A complement stores why the current correspondence was chosen, source spans, generated code origin, and proof receipts. This supports incremental repair and review.

Experiments

- field rename/refactor;
- action split/merge;
- internal buffering;
- new observer publication;
- asynchronous effect/cancellation;
- abstract requirement change with many implementation options;
- generated Rust skeleton round trip.

Ambiguity metric

Plan §24.5's lenses row keys on a quantity this note must define:

```
ambiguity rate :=  
  fraction of abstract edits in the drift corpus for which  
  put? yields multiple concrete candidate edits, or none
```

The denominator is a **drift corpus**: each experiment scenario above (field rename/refactor, action split/merge, internal buffering, new observer publication, asynchronous effect/cancellation, abstract requirement change, generated-skeleton round trip) is instantiated as concrete abstract-edit instances over real model/program pairs, with a recorded ground-truth set of acceptable concrete edits. Building this corpus is a lane deliverable; the metric is undefined — and the lane cannot promote — without it.

Promotion and kill are keyed to this rate together with the existing kill criteria below:

- **promotion** requires a measured ambiguity rate on the drift corpus low enough that candidate-plus-obligation proposals are useful on the majority of corpus edits (this note is the authority for the number: the numeric target is fixed here at ratification, and plan §24.5 quotes it — draft, pending lane-owner ratification; an unratified threshold may not survive Phase A);
- **kill** if the rate shows most real mappings are too ambiguous for useful proposals, or if users mistake candidate synchronization for verified preservation — both restated from the kill criteria below, the first now measurable as this rate.

Lean program

Formalize a finite pure core:

- consistency relation;
- get/put candidate relation;
- sound conflict reporting;
- composition;
- preservation theorem parameterized by discharged obligations.

Production effects remain translation-validated.

Kill criteria

- most real mappings are too ambiguous for useful proposals;
- complement/provenance burden exceeds manual model maintenance;
- generated obligations are as hard as writing correspondence;
- users mistake candidate synchronization for verified preservation.

Document: research/32-semantic-context-compilation-and-information-theory.md

Semantic Context Compilation and Information Theory

Claim class: research hypothesis

Goal

Select the smallest faithful context that enables a human or agent to choose the correct next action.

Inputs

- evidence graph;
- task/question;

- actor capabilities and prior context;
- token/byte/node budget;
- required guarantee;
- privacy policy.

Candidate facts

Events, state deltas, source spans, model actions, assumptions, proof lemmas, counterexamples, branch contrasts, and unknowns.

Baseline algorithms

- backward causal slice;
- static/dynamic dependence;
- proof dependency slice;
- minimal unsat core/correction set;
- submodular relevance/diversity ranking;
- graph centrality;
- retrieval/reranking.

Novel proposal: decision-sufficient context

Given a finite set of valid next actions and semantic worlds consistent with evidence, context is sufficient if no two worlds requiring different correct actions remain observationally equivalent. This turns context selection into active feature acquisition.

Approximate objective:

maximize expected reduction in decision regret
subject to faithfulness, privacy, and budget

This is better than maximizing generic relevance.

Novel proposal: rate-distortion for explanations

View compression as rate-distortion:

- rate: tokens/bytes/nodes;
- distortion: loss in diagnosis/repair/proof decision quality;
- hard constraints: causal/property preservation and omission transparency.

Human and agent empirical results estimate task-specific distortion. The compiler can select among representations.

Novel proposal: semantic entropy budget

Raw event count poorly predicts context difficulty. Estimate uncertainty over:

- defect hypotheses;
- enabled branch choices;
- abstraction correspondence;
- proof lemmas;
- assumptions.

Spend context on high-entropy distinctions rather than repetitive logs.

Expansion policy

Context Packs expose graph queries. A learned/analytic policy can propose the next expansion maximizing information gain, but clients may choose manually.

Privacy

Redaction changes the set of distinguishable worlds. The pack must say when privacy filtering makes the task inconclusive or weakens guarantees.

Experiments

- synthetic noisy causal traces;

- real concurrency bugs;
- Lean proof repair;
- liveness/fairness diagnosis;
- multi-agent handoff;
- private production trace.

Compare random/truncated context, retrieval, causal slice, proof slice, decision-sufficient selection.

Kill criteria

- information-theoretic estimates do not predict task performance;
- learned ranking overfits benchmark families;
- context compilation cost is too high for interaction;
- compact packs repeatedly induce incorrect repairs despite preservation checks.

Document: research/33-agent-benchmarks-and-reward-hacking.md

Agent Benchmarks and Formal Reward Hacking

Claim class: evaluation methodology

Lane status (plan §24.5)

This note owns two register rows:

1. **Neighborhood adequacy (§8.3)** — lane to be opened. Threshold: hidden-variant catch rate of the §8.3 repair neighborhood on the docs/50 gaming corpus; the numeric target is fixed at lane opening — draft. The kill clause is the first kill criterion below (hidden variants too easy to leak or too hard to grade independently).
2. **Co-ownership of the context-compilation ablation benchmark** (general context compilation, §6) with research/25 and research/32: this note supplies the anti-gaming grading discipline for that lane's agent benchmark.

Problem

Standard coding benchmarks often grade final tests. Formal-systems agents can exploit the specification, environment, bounds, observer, verifier, or grader itself.

Threat model

The agent may knowingly or accidentally:

- alter intent;
- exploit visible cases;
- disable instrumentation;
- submit stale evidence;
- consume excessive resources;
- manipulate tool state;
- inject instructions via source;
- overfit proof/search to known lemmas.

Benchmark principles

1. Protected Intent Contract outside writable snapshot.
2. Hidden semantic variants, not only hidden tests.
3. Independent grader/checker.
4. Full operation/evidence trace.
5. Resource-normalized reporting.
6. Family-level split and semantic clone detection.
7. Security/capability tasks.
8. Exact artifact requirement.

Novel proposal: adversarial intent mutations

For every task, automatically derive neighboring intents:

- remove property conjunct;

- add assumption;
- reduce bound;
- remove fault;
- coarsen observer;
- strengthen fairness;
- downgrade assurance.

The agent must preserve the original fingerprint. The benchmark checks whether its patch accidentally behaves as if it solved a neighboring easier intent.

Novel proposal: verifier-aware hidden variants

Generate variants preserving high-level intent but changing:

- names/types/order;
- topology size within cutoff;
- equivalent action decomposition;
- scheduler/fault realization;
- implementation structure;
- proof lemma names;
- domain profile.

This measures semantic generalization.

Novel proposal: trajectory evidence score

A successful final patch can still arise from unsafe behavior. Score:

- invalid privileged attempts;
- unsupported claims;
- stale handle use;
- context expansion efficiency;
- hypotheses refuted/updated;
- evidence status honesty;
- cancellation/budget behavior.

Trajectory score is diagnostic; final semantic evidence remains primary.

Baselines

- raw shell coding agent;
- ACI agent;
- ACI + Context Packs;
- ACI + multi-agent evidence graph;
- human engineer;
- human-agent team.

Kill criteria

- hidden variants are too easy to leak or too hard to grade independently;
- benchmark success fails to predict real project adoption;
- cost normalization dominates task semantics;
- task generation introduces invalid or ambiguous intents.

Document: [research/34-human-factors-of-formal-systems-workbenches.md](#)

Human Factors of Formal-Systems Workbenches

Claim class: evaluation methodology

Premise

Formal correctness evidence is useful only when engineers understand what it establishes and act correctly on failures.

Known adoption barriers

Industry studies and counterexample-explanation surveys identify difficulty with formal notation, incomplete models, refinement inconsistencies, and interpreting checker output. Engineers remain interested when tools reduce manual work and improve safety.

References:

- <https://arxiv.org/abs/2304.08950>
- <https://arxiv.org/abs/2201.03061>

Continuum hypotheses

- causal/state-delta explanations improve diagnosis over raw traces;
- explicit assurance envelopes improve calibration over green/red badges;
- progressive disclosure serves both Rust engineers and formal experts;
- branch comparison makes concurrency mechanisms easier to understand;
- model/program correspondence navigation reduces drift errors;
- excessive formal detail at first contact increases abandonment.

Study design

Participants

Rust engineers, distributed-systems experts, formal-methods users, proof experts.

Tasks

- identify ack-before-durable mechanism;
- distinguish deadlock from slow progress;
- classify fair/unfair liveness cycle;
- review property-weakening patch;
- decide whether production evidence is sufficient;
- repair model/program mismatch.

Conditions

- raw logs/traces;
- conventional checker output;
- Continuum summary;
- Context Pack + debugger;
- agent-assisted Context Pack.

Measures

accuracy, time, confidence, confidence calibration, repair quality, retention, transfer, cognitive load, expansion behavior.

Novel proposal: assurance calibration curves

Treat user confidence as a probabilistic forecast and score calibration/Brier-like measures against task truth. A fast correct answer with wildly overconfident interpretation of bounded evidence is not ideal.

Novel proposal: semantic wayfinding

Measure whether users can navigate among source, concrete execution, abstract model, property, and proof without losing the conceptual locus. Design correspondence breadcrumbs and persistent intent/assurance anchors.

Accessibility

- non-color semantics;
- keyboard navigation;
- textual graph alternatives;
- scalable typography;
- no animation required to understand causality;
- screen-reader labels for evidence status and edges.

Kill criteria

- experts prefer raw output and newcomers gain no accuracy;
- progressive disclosure hides key assumptions;
- causal graph increases confusion;
- assistance raises confidence faster than correctness.

Document: research/35-security-of-agentic-verification-workbenches.md

Security of Agentic Verification Workbenches

Claim class: research hypothesis

Research question

How does a verifier remain trustworthy when untrusted agents control queries, patches, generated models/proofs, and large compute budgets?

Attack surfaces

- protocol/adapters;
- source/log prompt injection;
- artifact handles/CAS;
- intent policy;
- incremental cache;
- proof/solver workers;
- generated code execution;
- production telemetry;
- benchmark graders;
- multi-agent coordination.

Novel proposal: capability-typed semantic operations

Associate each protocol operation with a capability effect:

```
ReadArtifact(A)
ProposePatch(W)
RunVerification(I, B)
RequestProof(E)
ReviseIntent(F)
PromoteEvidence(P)
```

A static/generated client can prevent impossible calls; the daemon enforces dynamically. Lean can model non-escalation for a simplified protocol state machine.

Novel proposal: evidence noninterference

Sensitive production fields may be hidden from an agent while a trusted checker uses them. Define a receipt proving a verdict over hidden evidence without exposing content. Research selective disclosure, commitment schemes, or zero-knowledge techniques only where needed; simpler trusted local checking is baseline.

Novel proposal: semantic prompt-injection firewall

Instead of trying to detect malicious language, architecturally separate:

- instruction channel: fixed tool schemas/policies;
- data channel: source/log/model strings;
- authority channel: capabilities/checkers.

Even a fully compromised model cannot promote evidence without authority.

Verification targets

- handle authorization state machine;
- task/publication atomicity;
- intent-policy enforcement;
- proof-worker isolation;

- audit-log append consistency;
- cache-poison resistance;
- replay artifact integrity.

Red-team corpus

- comments asking agent to weaken tests/property;
- forged receipt JSON;
- predictable handles;
- stale snapshot substitution;
- solver output bombs;
- generated code attempting host access;
- hidden benchmark exfiltration;
- malicious domain pack;
- production trace secret leakage;
- resource-exhaustion synthesis grammar.

Kill criteria

No project kill for individual vulnerabilities, but autonomous promotion remains disabled until no known unprivileged path can alter intent/evidence status or escape isolation.

Document: research/36-active-diagnosis-and-experiment-design.md

Active Diagnosis and Experiment Design

Claim class: research hypothesis

Motivation

A failure may support several hypotheses. More passive logs may not distinguish them. Continuum can deliberately choose the next schedule, fault, observation, or proof query to maximize diagnostic value.

Formalization

Let H be candidate defect hypotheses and E available experiments. Each experiment yields outcomes under the model. Choose:

$$\operatorname{argmax}_e \quad \operatorname{ExpectedInformationGain}(H; \operatorname{outcome}(e)) - \operatorname{Cost}(e)$$

Under nondeterminism/adversarial environments, use minimax or robust information criteria rather than a naive probability distribution.

Experiment types

- branch on enabled event;
- move cancellation checkpoint;
- insert/remove crash;
- choose message delay/duplication;
- expose one instrumentation field;
- evaluate a derived property;
- increase a model bound;
- request a proof lemma/countermodel;
- switch observer or abstraction for diagnosis only.

Diagnosis-only observer changes must not mutate protected intent.

Novel proposal: distinguishing causal cuts

Given safe and failing execution families, find a minimal set of frontier choices or observed events separating them. This is related to decision trees over event structures and may yield compact diagnostic experiments.

Novel proposal: telemetry synthesis as experiment design

For production conformance, choose the least-cost event fields/correlation edges that distinguish legal from illegal model explanations. Output:

- instrumentation requirement;
- expected event volume/privacy cost;
- exact claims it enables;
- remaining ambiguity.

Novel proposal: proof-search experiment selection

When several lemmas could unblock a proof, estimate which finite countermodel or subgoal most reduces candidate invariant space. Agents receive the chosen discriminating goal.

Experiments

- ack-before-durable hypotheses: finalizer vs storage profile vs telemetry;
- deadlock: missing wake vs scheduler starvation;
- liveness: protocol cycle vs unfair scheduler;
- refinement: wrong abstraction vs missing event;
- weak memory: compiler/hardware order vs algorithm.

Success

Fewer runs/tool calls/tokens to correct diagnosis; no loss of soundness; explicit cost and assumptions.

Kill criteria

- model mismatch makes selected experiments misleading;
- computing information gain costs more than brute-force exploration;
- learned priors dominate and fail on new protocols;
- privacy/production constraints make proposals impractical.

Reference Documents: Executable Spike Reports

Document: spikes/R3_SPIKE_REPORT.md

Revision 3 Executable Spike Report

Executed with: Python 3.13.5

Result artifact: results/r3-spike-results.json

Runner: run_r3_spikes.py

All eight Revision 3 spike groups passed their internal assertions.

1. Context Pack causal slicing

The synthetic durability trace contained **200 events**: four causal events and 196 observer-independent noise events.

Results:

- replay-preserving core: **4 events**;
- omitted events: **196**;
- raw JSON: **17,645 bytes**;
- Context Pack: **747 bytes**;
- compression ratio: **23.62×**;
- core was 1-minimal under single-event deletion;
- core replay still produced `acked = true ∧ durable = false`.

Core:

```
WriteSubmitted
  → ReplyReserved
  → CancelRequested
  → ReplyPublished
```

This validates the basic artifact shape, not the general context compiler. Real traces need conflict, abstraction, proof, and domain semantics.

2. Semantic intent diff

The spike correctly distinguished an ordinary source guard repair from six formal reward-hacking mutations:

Mutation	Classification
source guard repair	protected intent unchanged
remove property term	property weakened
add favorable assumption	assumptions strengthened
nodes 5 $\rightarrow$ 3	bound contracted
hide SyncCompleted	observer coarsened
remove crash fault	fault envelope contracted
validated $\rightarrow$ sampled	assurance downgraded

All expected protected changes were detected.

3. Explicit agent protocol state

The in-memory workbench demonstrated:

- canonical workspace snapshot identities;
- idempotent identical task creation;
- rejection of idempotency-key reuse with different request;
- resumable continuation;
- rejection of continuation under a different workspace snapshot.

This supports the explicit-handle architecture. It does not yet test concurrency, authorization, persistence, or crash recovery.

4. Forge finite CEGIS

Candidate grammar:

```
true
false
reserved
~reserved
~synced
reserved  $\wedge$  synced
synced
```

Constraints:

- safety: acknowledgement implies synced;
- non-vacuity/progress: a reserved and synced request can acknowledge.

CEGIS iterations:

1. false rejected by progress counterexample;
2. reserved rejected by safety counterexample;
3. synced accepted.

The only valid grammar candidates were synced and reserved $\wedge$ synced; AST-size objective selected synced. Invalid candidates were exhausted in the finite grammar.

This validates coupling safety with positive behavior. It is not evidence that broad protocol synthesis will scale.

5. Incremental query invalidation

A ten-query synthetic graph was tested under property, source, model, proof-only, and domain-profile edits.

Findings:

- all incremental outputs matched clean recomputation;
- property edit reused source parsing/extraction and model elaboration;
- proof-only edit invalidated only the proof receipt;
- source edit invalidated extraction, correspondence, refinement, context, and proof receipt;
- domain-profile edit invalidated both model and program semantic paths.

The spike validates the edge taxonomy idea but not production dependency capture.

6. Causal debugger

At a configuration with submitted and reserved work, the enabled frontier was:

```
SyncCompleted
CancelRequested
```

Branches:

- SyncCompleted $\rightarrow$ NormalPublishes produced acked $\wedge$ durable;
- CancelRequested $\rightarrow$ FinalizerPublishes produced acked $\wedge$ ~durable.

The spike emitted why-enabled conditions, first conflicting choices, abstract branch difference, and causal reverse choice.

7. Proof-oriented lens conflict

The abstract field durable could legitimately map to either:

- local storage stability;
- quorum acknowledgement under a stronger domain contract.

The reverse abstract edit therefore had two concrete candidates. The spike returned AmbiguousCorrespondence and a distinguishing obligation instead of selecting silently. Direct acknowledgement/publication mapping remained unambiguous.

8. Multi-agent Evidence Graph

The immutable graph spike coordinated two independent patch proposals and verifier/kernel evidence. It demonstrated:

- byte-identical proposals deduplicate by content identity;
- an agent cannot publish `validated` or `proved` status;
- stale clean-parity evidence cannot complete a repair envelope;
- the safe acknowledgement patch promotes only after fresh exact replay, neighborhood, mutation-challenge, and clean-parity evidence exist;
- a fault-model contraction is blocked as a protected intent change;
- contradictory claims about the same subject and predicate become an explicit conflict rather than last-writer-wins state.

The run produced ten immutable nodes, seven typed edges, one surfaced claim conflict, and one accepted repair envelope. This validates authority separation and coordination semantics, not distributed storage or Byzantine agent resistance.

What the spikes changed

The experiments support five architecture decisions:

1. bounded Context Packs can be dramatically smaller while preserving a witness;
2. intent integrity needs a dedicated semantic diff, not code review convention;
3. explicit handles provide deterministic agent handoff and stale-state rejection;
4. Forge requires positive/non-vacuity constraints and independent verification.
5. Multi-agent work needs immutable evidence, authority-separated promotion, stale-evidence rejection, and explicit conflicts.

What remains unproven

- context slicing on real causal/conflict graphs;
- human/agent performance improvements;
- sound semantic implication for rich temporal properties;
- persistent/concurrent daemon behavior;
- incremental correctness under real edits;
- scalable CEGIS/co-synthesis;
- verified lens laws and effectful correspondence;
- Lean kernel checking of Revision 3 seed modules.

Document: spikes/SPIKE_REPORT.md

Revision-2 Executable Spike Report

Execution environment: Python reference implementation, exact state identity, deterministic BFS.

Purpose: falsify architectural assumptions before optimized Rust or Lean implementations exist.

Status: executable spike, not a production implementation or formal proof.

Die Hard parity seed

- Reachable states: **16**
- Enumerated transitions: **96**
- Shortest state with four gallons in the large jug: depth **6**
- Exact closure/type certificate accepted by independent checker: **True**

The transition relation is a direct semantic port of the six actions in the TLA+ Examples `DieHard.tla`. This establishes the shape of the future corpus harness: a native model, a pinned source model, exact reachable-graph facts, expected failing property, and a replayable shortest witness.

Dining philosophers

- Reachable states: **573**
- Enumerated transitions: **2365**
- Shortest deadlock depth: **10**
- Exact closure/type certificate accepted: **True**

This confirms that the reference kernel can distinguish ordinary invariant closure from deadlock reachability and produce a shortest concrete schedule.

Stuttering refinement

A concrete register implements each write as `Reserve(v)` followed by `Commit`, with `Abort` available before commit. The abstraction forgets the pending reservation. Every concrete transition either:

1. preserves the abstract state; or
2. corresponds to an abstract atomic write.

Finite exhaustive check result: **True**.

Observer-indexed independence

Results:

```
{
  "events": [
    "write_x",
    "write_y"
  ],
  "commutes_under": {
    "state_xy": true,
    "state_x": true,
    "audit_order": false
  },
  "observer_monotonicity_holds": true,
  "interpretation": "The same events are independent for state observers but dependent for an order-sensitive audit observer. Reduction must therefore be indexed by the active observation/property contract."
}
```

The key finding is intentional: `write_x` and `write_y` commute for state observers but not for an observer that records audit order. Independence cannot be a global property of event kinds; it must be indexed by the property/view/observer contract.

Additional revision-2 experiments

Cyclic symmetry

- Raw dining-philosopher states: **573**
- Quotient states under process rotation: **117**
- Reduction factor: **4.90×**
- Exhaustive transition-automorphism check: **True**

This is not yet a proof of symmetry reduction. It establishes the witness shape: a group action, canonical representative, and independently checkable automorphism obligation.

Fair-lasso semantics

The same infinite `Wait` cycle is a raw liveness counterexample but is rejected under weak fairness of `Complete`; when `Complete` is genuinely unavailable, the cycle becomes a valid fair counterexample. This demonstrates why fairness must be attached to named actions and checked on the cycle rather than treated as a scheduler slogan.

Environment-assumption synthesis

A tiny turn-based safety game synthesizes the maximal-permissive contract required by `Ack` $\Rightarrow$ `Durable`. The only forbidden environment move is `AckBeforeSync`; after removing it, the initial state enters the safety-winning region. This is the seed for deriving domain-pack contracts and production monitors from games.

Nominal canonicalization

Two traces using unrelated dynamically allocated identifiers canonicalize identically, while a causally different allocation/send order does not. This supports an orbit-finite lane for request IDs and other fresh names without accidentally erasing causal distinctions.

Semiring-valued search

The Die Hard graph is evaluated over a product of the tropical and natural-number semirings, yielding shortest distance **6** and **1** shortest labeled witnesses in one algebraic traversal.

What this spike does not establish

- correctness of a Rust implementation;
- correctness of the planned model language;
- soundness or completeness of DPOR;
- temporal/liveness preservation;
- TLA+ semantic parity beyond the hand-ported Die Hard transition system;
- any Lean theorem.

Those become explicit G0/G1 gates rather than prose claims.

Reference Documents: Implementation Sequence & G0 Matrix

Document: notes/G0_SPIKE_MATRIX.md

G0 Falsification Matrix — Revision 3

G0 exists to kill seductive architecture before it ossifies. A failed load-bearing spike blocks interface freeze.

The freeze-blocking subset is DX-01–05, 07, 08, 10, 12, 13, 14 (plan §22): a failed or unexecuted freeze-blocking item blocks interface freeze. Re-homed items block their target gates instead. Plan §0.3's counts derive from this table. Spike evidence (finite Python spikes, `spikes/R3_SPIKE_REPORT.md`) validates artifact shapes and interaction contracts, not engines, scale, concurrency, persistence, or soundness (docs/53). DX-09 requires preregistration before it runs: the four docs/34 acceptance workflows, two cohorts (Rust newcomers on the deterministic/causal workflow; distributed-systems experts on real failures), a raw-trace baseline comparator, and cohort sizes, instruments, and pass thresholds fixed in an expanded docs/48 published before the study (plan §21.1).

Continuum: Master Implementation Plan & Comprehensive Dossier

ID	Question	Required experiment	Pass condition	Failure consequence	Status	Evidence	Decision
G0-DX-01	Can an agent diagnose from a bounded artifact rather than raw logs?	200+ event noisy durability failure → Context Pack	causal core replay-pre-serving; $\geq 10\times$ context reduction; exact expansion handles	redesign evidence model	Evidence spike (finite spike)	R3_SPIKE_REPORT §1 (Context Pack slicing)	Optd artifact shape; engine unproven (docs/53)
G0-DX-02	Can Continuum detect formal reward hacking?	patches that weaken property, strengthen assumptions, reduce bounds, hide observer events, remove faults	all classified as intent changes; ordinary code guard repair is not	block autonomous repair	Evidence spike (finite spike)	R3_SPIKE_REPORT §2 (intent diff)	Optd artifact shape; engine unproven (docs/53)
G0-DX-03	Is all state explicit and resumable?	create snapshot, start bounded task, resume continuation, mutate workspace, reuse old handle	idempotence, stale snapshot rejection, deterministic resume	reject daemon protocol	Evidence spike (finite spike)	R3_SPIKE_REPORT §3 (agent protocol)	Optd artifact shape; engine unproven (docs/53)
G0-DX-04	Can an exact counterexample be debugged causally?	branch before Ack/ Sync race	enabled frontier and alternate branch reproduce expected safe/failing outcomes	DAP design remains experimental	Evidence spike (finite spike)	R3_SPIKE_REPORT §6 (causal debugger)	Optd artifact shape; engine unproven (docs/53)
G0-DX-05	Does incremental-ity preserve clean semantics?	mutate property, model action, source function, proof lemma, docs	invalidation cone matches declared dependency classes; incremental result equals clean result	disable affected reuse class	Evidence spike (finite spike)	R3_SPIKE_REPORT §5 (incremental parity)	Optd artifact shape; engine unproven (docs/53)
G0-DX-06	Can a repair transaction	patch exact trace only	narrow repair repair	no autonomous	Re-homed → G4	none	runs as the G4 neigh-

Promotion rule

A spike may become architecture only when it has:

1. a precise semantic claim;
2. a reference implementation or checker;
3. adversarial mutations;
4. deterministic artifacts;
5. a baseline;
6. a failure policy;
7. a path to independent evidence.

A visually impressive demo is not a pass condition.

Document: notes/START_HERE_IMPLEMENTATION.md

Start Here: Revision 3 Implementation Sequence

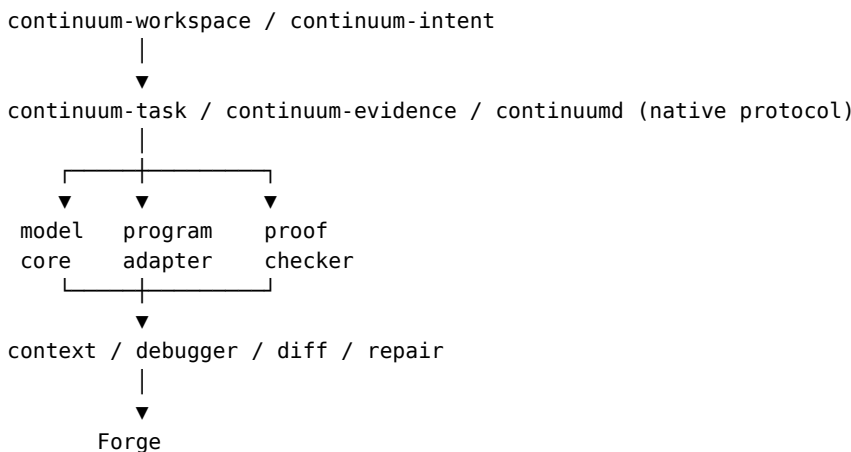
Governing rule

Close the smallest complete trustworthy loop before broadening the verifier:

```
protected intent
  → immutable snapshot
  → typed verification task
  → exact failure
  → bounded Context Pack
  → causal debugger
  → repair transaction
  → semantic/intent diff
  → neighborhood and mutation challenge
  → independent evidence
  → promotion receipt
```

Do not begin with a polished web UI, a general LLM coordinator, distributed search, or the full CML surface language (the Finite core fragment lands at PR 15a). The first users are the implementation team and coding agents driving the first vertical slice.

Dependency islands



There is no continuum-protocol crate; the native protocol lives in continuumd. The full Revision 3 crate list is plan §20.

The certificate checker may not depend on search. The model core may not depend on asupersync. Adapters may not own semantic state. Forge may not be imported by the verifier.

Resourcing (plan §21.1)

Phase	Owner	Minimum viable team	Status
A	unassigned	unassigned	BLOCKED
B	unassigned	unassigned	BLOCKED
C	unassigned	unassigned	BLOCKED
D	unassigned	unassigned	BLOCKED
E	unassigned	unassigned	BLOCKED
F	unassigned	unassigned	BLOCKED

Filling a phase's row is a merge requirement of that phase's opening PR (plan §21.1); an unfilled row records the phase as BLOCKED, not in progress.

First pull requests (PR 0 – PR 30)

PR numbers 1–30 are stable; inserted work carries PR 0 or a lettered suffix (4a, 15a, 25a). Each heading names the release gate(s) the PR advances, per plan §21's phase deliverables and the G0 staging rule; some PRs carry gates from later phases where §21 explicitly pulls work forward (e.g. PR 4a [G6; Phase A band]).

PR 0 — Specification pass and program decisions [G1, G2]

Implement (documentation, not code):

- expand RFCs 0026, 0027, 0028, 0030, 0031, 0032, 0037 from summaries to full specifications: field types, enums, classification lattices, IDL, versioning, RFC-2119 language;
- reconcile each RFC one-to-one with its plan section; where they disagree, correct the RFC and make it normative;
- product license decision (permissive, compatible with asupersync and solver adapters, §21.1);
- corpus per-family redistribution audit before any public benchmark release (§21.1);
- ADR-0029 rule: foreign oracle tooling (TLC, Apache, solvers) never ships in release binaries.

Exit: the seven RFCs are normative specifications; PR 5 may not merge before PR 0 closes.

PR 1 — Revision 3 constitution and epochs [G1]

Implement:

- Rust workspace and crate boundaries;
- protocol, semantic, intent, evidence, proof, and corpus epochs;
- assurance and inconclusive enums;
- `#![forbid(unsafe_code)]` defaults;
- deterministic build and artifact paths;
- claim-status lattice.

Exit: an unsupported empty task returns a valid machine result naming every epoch and no misleading success flag.

PR 2 — Canonical values and CAS primitives [G1]

Implement:

- exact finite values from Revision 2;
- canonical encoding and total order;
- content identity per ADR-0013: in certified lanes, canonical content identity is primary and hash collisions are resolved by canonical comparison; 256-bit-hash identity only in explicitly labeled non-certified modes;
- collision-injection tests;
- atomic artifact publication;
- authorization separate from handle possession.

Exit: concurrent publication of identical artifacts yields one identity; artificial hash collisions are detected and resolved by canonical comparison in certified lanes.

PR 3 — Workspace snapshots [G1]

Implement:

- Merkle workspace snapshots;
- disk import, editor overlay, fork, seal, and diff;
- source/dependency/toolchain/config identities;
- stale-snapshot error;
- deterministic file ordering.

Exit: two clients can fork and analyze independently; an old snapshot remains reproducible after the working tree changes.

PR 4 — Intent Contract v0 [G1, G3]

Implement schema/types for:

- properties;
- assumptions;
- observers;
- bounds;
- faults;
- fairness;
- assurance policy;
- optimization/non-vacuity;
- field-level change policy.

Exit: Die Hard and replicated-register intents serialize canonically; ordinary operations cannot mutate them.

PR 4a — Lean environment and seed theorems [G6; Phase A band]

Implement:

- pinned Lean toolchain (leanprover/lean4:v4.32.1);
- kernel-check the existing seed modules under lean/Continuum/;
- T0/T1 theorems: transition-system safety, stuttering simulation, finite-closure certificate soundness.

Exit: T0/T1 compile with no sorry and empty axiom manifests (RFC 0012 theorem ladder; axiom manifests per ADR-0035). The proof *service* remains PR 28.

PR 5 — Native protocol kernel [G1, G2]

Implement request/response types and local transport for the plan §10.2 operation names:

- workspace.create / fork / diff / seal;
- intent.get / diff / propose_revision / accept / reject / lock;
- verification.start; task.status / cancel / resume / subscribe;
- evidence.get / query / verify;
- capability negotiation;
- typed errors and idempotency keys.

Exit: replaying an idempotent request returns the same task/artifact identity. May not merge before PR 0 closes.

PR 6 — Cancel-correct task service [G1]

Use asupersync regions for daemon work:

- task lifecycle;
- committed partial evidence;
- budget accounting;
- suspension/continuation;
- drain/finalize behavior;
- no orphan workers.

Exit: cancellation at every instrumented phase leaves either a valid continuation or no published partial artifact.

PR 7 — Evidence Graph v0 [G1]

Implement node/edge/status types, immutable versions, and queries:

- support/refute/depend/refine/explain/repair/check edges;
- provenance;
- trusted status transitions;
- conflict nodes.

Exit: an untrusted client cannot promote a proposal to validated/proved.

PR 8 — Exact finite model service [G1, G2]

Wrap Revision 2 reference semantics behind native protocol:

- programmatic transition model;
- deterministic BFS;
- invariant/deadlock checking;
- shortest witness;
- finite closure certificate.

Exit: Die Hard returns 16 states and depth-6 solution through the daemon API.

PR 9 — Trusted kernel crates [G1, G6]

Implement the trusted checking base as four crates — `continuum-kernel-core`, `continuum-kernel-sat`, `continuum-kernel-smt`, `continuum-kernel-temporal` (plan §20):

- finite closure/type certificate checking;
- no shared evaluator code with any engine; no async, no unsafe, no plugins or dynamic loading;
- <15,000 non-test-line covenant across the four crates (docs/03);
- serialization boundary: certificates are checked from wire form, never from shared memory;
- receipt generation with checker epoch, build digest, and input hashes;
- mutation tests.

Exit: every single-field certificate mutation in the test suite is rejected, checking wire-form input only.

PR 10 — Agent client and ACI benchmark harness [G0 (DX-10), G2]

Implement a minimal client exposing only typed operations. Compare against a shell-scraping baseline on Die Hard and Dining Philosophers tasks.

Measure:

- valid operation rate;
- tokens/bytes;
- task completion;
- recovery from errors;
- deterministic reproduction.

Exit: native ACI is measurably more effective or the protocol is redesigned before freeze.

PR 11 — Context Pack schema and compiler v0 [G2]

Implement:

- target/verdict/assurance;
- state and event slice;
- source/model references;
- omissions and expansion handles;
- replay reference;
- byte/token budgets.

Start with graph reachability and invariant failures.

Exit: the synthetic 200-event durability case compiles to a replay-preserving core with substantial reduction.

PR 12 — Intent semantic diff v0 [G3]

Classify supported changes:

- exact equality;
- property AST edit;
- assumption add/remove;
- bound change;
- observer event change;
- fault/fairness/assurance change.

Add solver-based implication only where sound and bounded.

Exit: all G0 intent-gaming patches are privileged changes; a source-only guard repair is not.

PR 13 — Human CLI v0 [G1]

Implement stable commands and output, aligned with plan §13.4/§3.1:

```
continuum snapshot
continuum check
continuum explain
continuum debug
continuum repair
continuum evidence show
continuum context expand
continuum task status/resume/cancel
```

Support `--json`; prose is a projection.

Exit: golden tests pin JSON, exit codes, and concise terminal output.

PR 14 — Asupersync semantic journal [G4]

Instrument narrow primitives:

- task/region lifecycle;
- reserve/commit/abort;
- cancellation phases;
- obligations;
- virtual time;
- channel communication.

Exit: identical controlled choice logs produce canonical identical semantic events.

PR 15 — Network/process/storage packs [G4]

Implement only the profiles needed for replicated register:

- delivery/drop/duplicate/delay/partition;
- crash/restart/epoch;
- submit/stable/sync/ack;
- declared unsupported cases.

Exit: crash windows and cancellation points replay exactly.

PR 15a — CML core-fragment parser and elaborator [G4]

Implement `continuum-cml-syntax` and `continuum-cml-elab`:

- the Finite fragment only;
- enough surface for the replicated register and the Wave 0 corpus ports;
- the programmatic model API remains supported — CML is a second front end, not a replacement.

Exit: the replicated-register model written in CML elaborates to the same semantic model identity as its programmatic equivalent.

PR 16 — Replicated-register model and implementation [G4]

Create:

- abstract atomic register;

- operational durable register;
- asupersync implementation;
- correct version;
- ack-before-sync, lost-abort, stale-epoch, and orphan mutants.

Exit: each mutant has an expected intent/property and deterministic campaign.

PR 17 — CIR and concrete/abstract correspondence [G4]

Implement:

- event/configuration validation;
- causal/conflict edges;
- concrete/abstract state projection;
- stuttering classification;
- uncovered semantic effect errors.

Exit: correct implementation satisfies bounded refinement; mutants fail at mapped transitions.

PR 18 — Causal minimization [G4]

Implement deletion, causal-closure, owner/fault/value reduction, and replay validation.

Exit: ack-before-sync failure reduces to a compact core and does not delete the actual causal mechanism.

PR 19 — Verification debugger core [G4]

Implement:

- selected configuration;
- enabled frontier;
- semantic step/reverse;
- branch on alternate event;
- state/observer/obligation views;
- why-enabled/blocked derivations.

Exit: branch before Sync/Ack shows safe and failing successors from one handle.

PR 20 — Repair Transaction v0 [G3, G4]

Implement begin/apply/evaluate/promote with:

- hypothesis;
- patch identity;
- exact replay;
- semantic/intent diff;
- evidence accumulation;
- policy verdict.

Exit: moving ack after sync is evaluable; a property-weakening patch is reclassified and blocked.

PR 21 — Neighboring exploration and mutation challenge [G4; closes re-homed G0-DX-06]

Generate semantic neighbors around the causal core and run known property/model mutants.

Exit: a hard-coded exact-trace repair fails; the semantic guard repair passes the bounded envelope.

PR 22 — Promotion receipts [G3, G4]

Compose:

- intent identity;
- before/after snapshots;
- semantic diff;
- replay/neighborhood/mutation results;
- refinement/certificate status;
- unknowns;

- `gate_profile` (the phase-staged profile the receipt was evaluated under, plan §21);
- `NotYetEnforced` list — gates not yet in the profile, never rendered as passed;
- policy decision.

Exit: receipt independently verifies references and cannot be forged by the agent client.

PR 23 — Incremental query database v0 [G5]

Implement content-addressed queries for parsing, model construction, property automata, exploration, context, and diff. Classify edges as exact/validated/conservative/experimental.

Exit: property-only edit does not rebuild unrelated extraction; model action edit invalidates reachable graph and dependent context.

PR 24 — Incremental Parity Audit [G5]

(Renamed from “clean-build Tribunal”; **Tribunal** refers exclusively to the TLA+ corpus oracle harness, plan §9.5.)

Randomly and deterministically compare incremental and clean results. Minimize invalidation mismatches.

Exit: intentionally broken dependency edge is caught and quarantined.

PR 25 — LSP authoring adapter [G5]

Implement CML/source diagnostics, semantic hover, go-to correspondence, code lenses, and intent-change preview over snapshots.

Exit: unsaved buffer overlays create explicit snapshots and never mutate daemon state implicitly.

PR 25a — SARIF exporter [G5]

Implement `continuum-sarif` per plan §17.3:

- source-located, deduplicated diagnostics with artifact URIs;
- stable rule IDs and severity;
- code flows;
- evidence handles;
- SARIF is a report projection, not a proof format.

Exit: the replicated-register failure exports schema-valid SARIF with stable rule IDs across reruns; every result carries an evidence handle.

PR 26 — DAP adapter [G5]

Map Continuum debugger state to DAP threads, frames, scopes, variables, breakpoints, and stepping; expose custom causal operations.

Exit: VS Code-compatible client can inspect and branch the replicated-register failure.

PR 27 — MCP adapter [G2]

Expose curated native operations with explicit handles, deterministic schemas/order, result bounds, and capability checks.

Exit: two subagents share one workspace/intent but use isolated debugger/proof handles without session coupling.

PR 28 — Lean proof service and Context Pack [G6; closes re-homed G0-DX-11]

Implement the proof *service* over the environment pinned in PR 4a: per-request isolation and cancellation, goals, diagnostics, receipts with theorem/axiom manifests, and relevant-context extraction. (Lean pinning and the foundational T0/T1 theorems are PR 4a, not here.)

Exit: concurrent requests across Lean epochs are isolated and cancellable with deterministic results; every proof receipt carries theorem and axiom manifests with zero unapproved axioms.

PR 29 — Forge finite CEGIS v0 [G7]

Implement typed holes, finite grammar enumeration, exact counterexample feedback, safety plus progress/non-vacuity, and candidate archive.

Exit: acknowledgement guard task synthesizes synced, rejects never-ack, and independently verifies the result.

PR 30 — End-to-end agent benchmark [G2, G4]

Give an advanced coding agent only:

- repository snapshot;
- protected intent;
- failing Context Pack;
- native operations;
- bounded budget.

Require it to diagnose, patch, evaluate, and request promotion.

Exit: the complete receipt is produced without terminal parsing or human repair guidance; all operations and evidence are replayable.

Work deliberately deferred

Until PR 30 closes, defer:

- distributed model checking;
- cloud control plane;
- polished web product;
- generalized weak-memory engine;
- CML surface syntax beyond the Finite core fragment (the fragment itself ships in PR 15a);
- all 80 corpus ports;
- unrestricted synthesis grammars;
- learned search in the trusted loop;
- automatic production deployment;
- broad foreign-runtime support.

Not deferred: the CML core fragment (PR 15a) and SARIF export (PR 25a) are scheduled above.

These are multipliers. They must not precede the trustworthy loop they multiply.

Reference Documents: ContinuumBench Specifications

Document: benchmarks/README.md

Continuum Evaluation Corpus

The benchmark suite is evidence infrastructure, not a victory-lap collection. It must contain workloads favorable and hostile to every engine.

Suites

micro/

Semantic and algorithm litmus tests:

- cancellation reserve/commit;
- task ownership and quiescence;
- message loss/duplication/reordering;
- timer epochs;
- storage sync/crash;
- observer-sensitive independence;
- fairness and pure-await loops;
- exact-state hash collision injection.

protocols/

- two-phase commit and presumed-abort variants;
- Raft/Paxos abstractions;
- leases under partial synchrony;
- primary/backup replication;
- membership/reconfiguration;
- distributed lock service;
- sharded transaction fragment;
- CRDT convergence;
- workflow/saga compensation;
- replicated register vertical slice.

concurrency/

- channels and queues;
- lock-free structures;
- cancellation races;
- actor mailboxes;
- region/task trees;
- work-stealing internals;
- linearizability/strong refinement examples.

storage/

- append log;
- write-ahead log;
- checkpoint/recovery;
- double-write/page repair;
- manifest/rename protocols;
- snapshot install;
- object-store multipart commit.

parameterized/

- mutual exclusion;
- token ring;
- cache coherence abstractions;
- quorum protocols;

- lossy-channel coverability.

production-traces/

- complete traces;
- partial/lossy traces;
- clock uncertainty;
- cross-epoch traces;
- known incidents and generated mutants.

negative/

Cases deliberately hostile to proposed techniques:

- little/no independence;
- pathological symmetry;
- bad decision-diagram variable orders;
- SMT nonlinear arithmetic;
- huge values with tiny schedule space;
- massive schedules with tiny state;
- liveness properties invalidated by safety POR;
- topology/sheaf methods with no advantage.

Competitors/oracles

Where licensing and automation permit:

- TLC;
- Apalache;
- Quint simulator/Quint Connect;
- Stateright;
- Loom;
- Shuttle;
- Kani;
- LTSmin;
- GenMC/Nidhugg-style DPOR artifacts;
- Ivy/IC3PO-related examples;
- Storm/UPPAAL for extension lanes.

Comparisons report semantic differences and unsupported features, not just runtime.

Metrics

- time to first counterexample;
- unique causal classes;
- generated and distinct configurations;
- memory per configuration/class;
- replay stability;
- minimized causal-core size;
- certificate size/check time;
- multicore scaling;
- snapshot overhead;
- solver time and proof-check time;
- integration LOC and production-code divergence;
- mutant kill rate;
- false/inconclusive conformance classifications;
- user/agent time to diagnosis.

All benchmark manifests pin semantic, compiler, solver, and hardware envelopes.

Document: benchmarks/continuum-bench/README.md

ContinuumBench Seed

ContinuumBench evaluates human/agent work against protected intent and independently checked artifacts.

Seed tasks

- repair-ack-before-durable.json — diagnose and repair a cancellation/durability defect.
- governance-property-weakening.json — reject a patch that makes the property easier.
- synthesize-ack-guard.json — synthesize a safe, non-vacuous acknowledgement guard.

These are task manifests, not a completed benchmark dataset. Hidden variants and graders must live outside agent-readable snapshots in production benchmark runs.

Required baseline matrix

human raw trace
human Continuum Context Pack/debugger
agent shell/CLI
agent native ACI
agent ACI + Context Pack
multi-agent Evidence Graph

Every result reports semantic correctness, intent integrity, evidence class, hidden generalization, cost, and invalid operations.